



JOI Open 2014 Day1 Factories



概要

- 辺にコストがある木が与えられる
- 次のクエリをたくさん処理せよ
 - disjoint な頂点集合 X , Y が与えられるので, X に含まれる点から Y に含まれる点まで移動する時の距離の最小値を求めよ

小課題 1

- $N, Q \leq 5000$
- クエリごとに $O(N)$ が許される
- 木 DP
- (その点から最も近い X, Y 中の点までの距離) を返しながら DP をすると答えが出せる
- あるいは, Dijkstra を実行しても $O(QN \log N)$ で, 間に合う

小課題 2

- 各 $S, T \leq 20$
- クエリごと $O(ST)$ が許される
- すべての点对を試せば良い
- 根付き木にして, 各点の根からの距離を求めておき, LCA を $O(1)$ で求められる準備をしておくと, 2 点間の距離が $O(1)$ で求められる

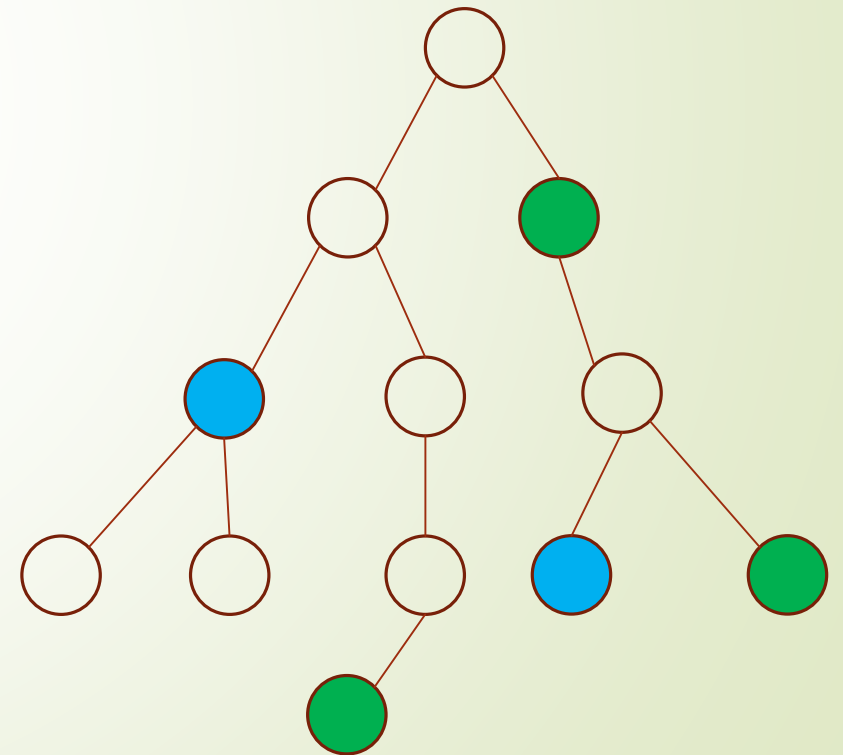


満点解法

- S, T の和がそれぞれ 1,000,000 以下である
- 入力サイズの線形より速くは解けない
- $O(S+T)$ とか $O((S+T) \log N)$ くらいで解ければよさそう

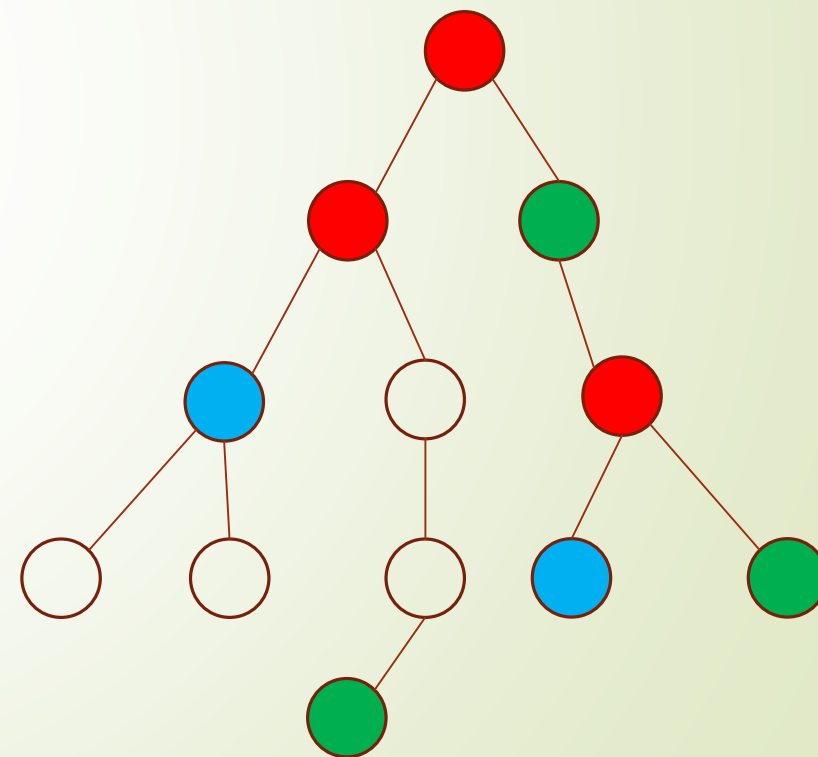
観察

- 木 DP をするときに, N 個の頂点すべて見ないといけない?
- X の点を青で, Y の点を緑で表す

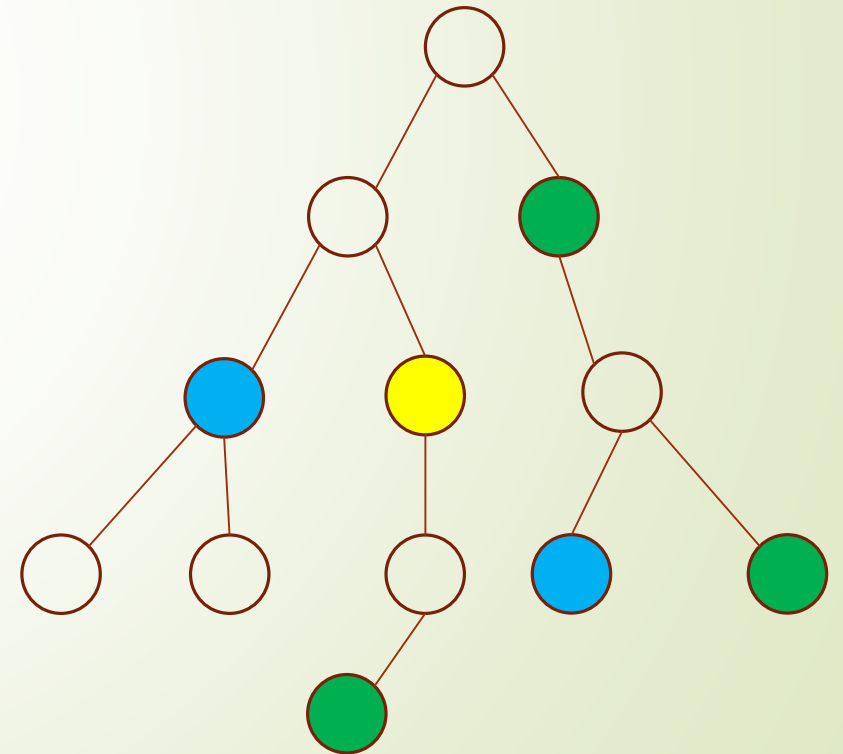


観察

- 木 DP をしたいだけだったら、必要な頂点は
 - X, Y に最初から含まれる点
 - 最初から含まれる点の LCA となる点
- だけ (右図では、赤い頂点)
- 他の点を調べても、
 - 下に X, Y に含まれる点がない
 - 下から来た値をそのまま受け流す
- のどちらかで、無意味

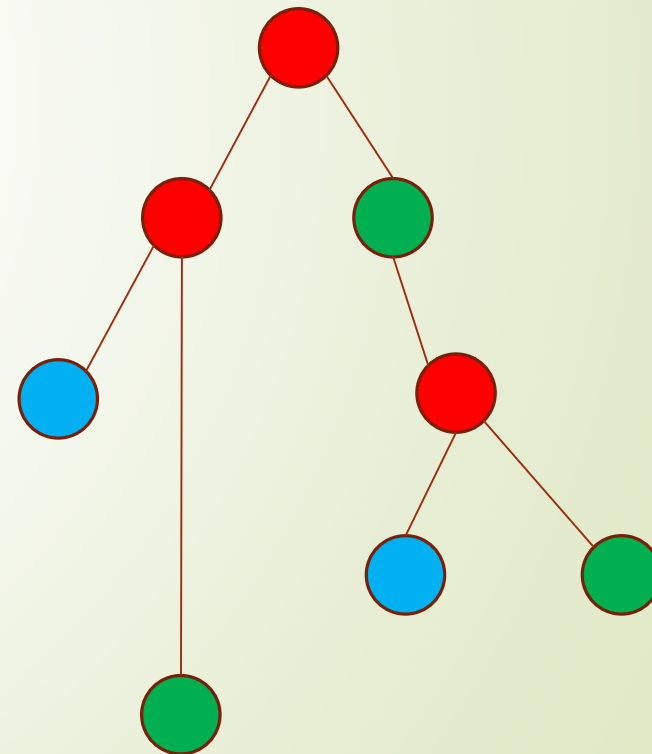


-



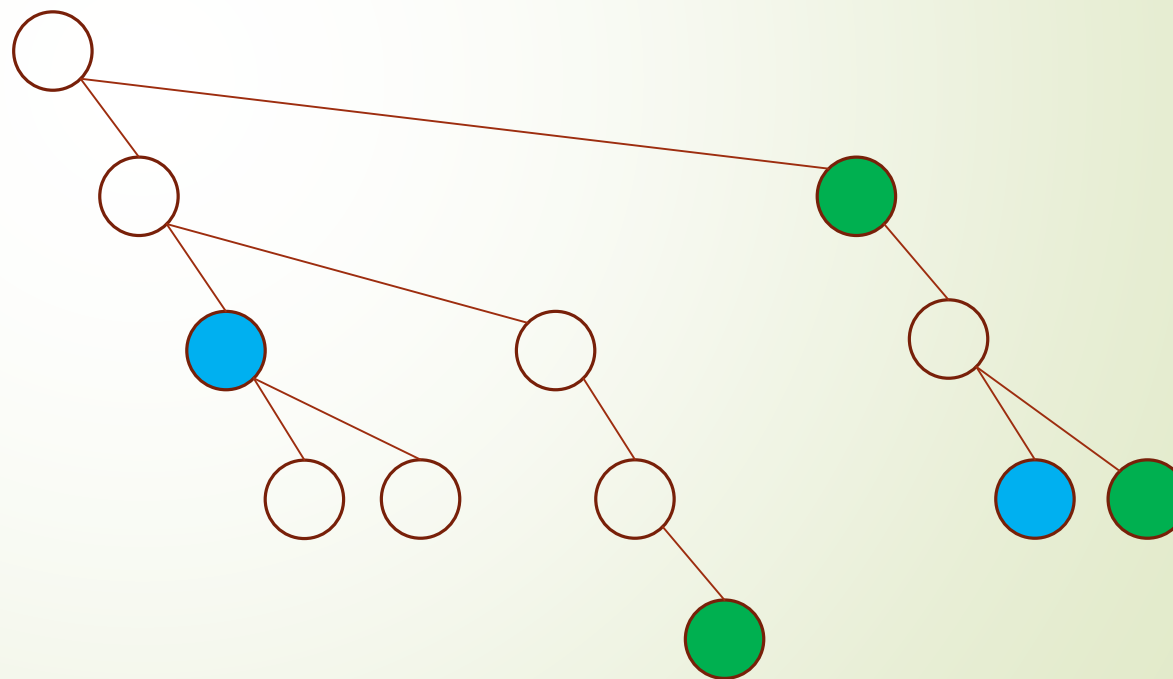
観察

- 結局，右のグラフについて DP すればよくなる
- 点をつぶしたとき，辺の長さは単に和を求めればよい



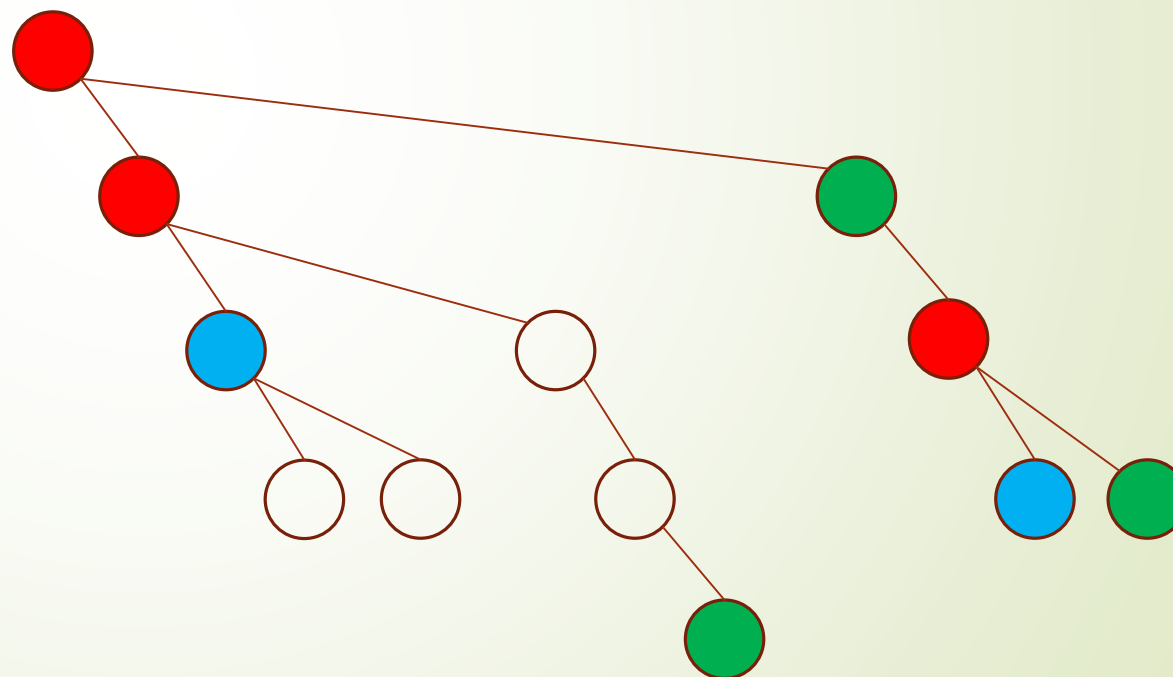
木の必要な部分を抽出

- まず、木に DFS 順序を与える



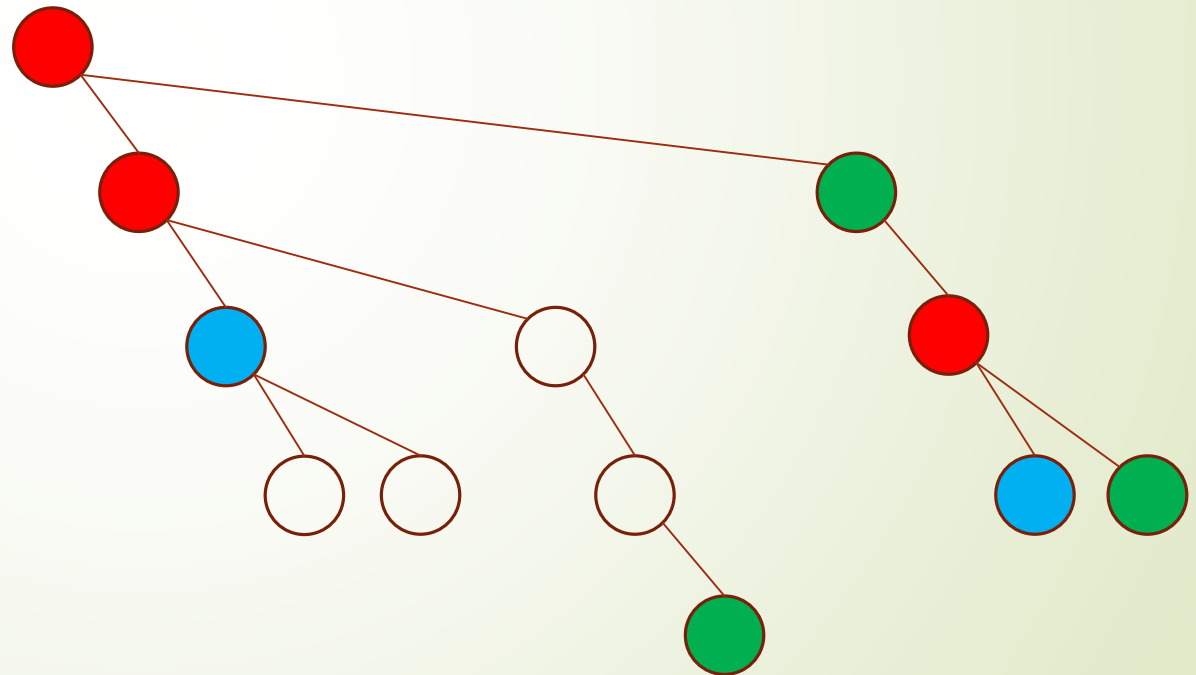
木の必要な部分を抽出

- 実は、この順序において隣り合う点同士の LCA だけ見れば十分



なぜか？

- この順序において「隣り合わない」点同士の LCA も、ある「隣り合う」点同士の LCA として現れることを言えばよい



なぜか？

- ▶ 2点 P, Q の LCA を考える (R とする)
- ▶ R が P か Q と一致する場合は省略します
- ▶ 列の上で $P = S_1, \dots, S_k = Q$ がこの順で隣り合っているとする
- ▶ R の子で、子孫に S_i を含むものを S_i' とする
- ▶ S_i' と S_{i+1}' が異なるなら、 S_i' と S_{i+1}' の LCA は R (あるいはもっと上) になる
- ▶ 一方、 S_i' は R の子孫であるから、このとき LCA は R になる
- ▶ LCA の定義から、 P' と Q' は異なる

なぜか？

- LCA の定義から, $P' = S_1'$ と $Q' = S_k'$ は異なる
- よって, $S_1', \dots, S_k'$ がすべて一致するということはない
- ゆえに, DFS 列上で隣接する 2 点で, それらの LCA が R になるものが存在

点の数の評価

- 最初与えられる点の数は $S + T$ 個
- 新たに必要になる点の数は、高々 $S + T - 1$ 個
- よって、DP で必要になる点の数は全部で $O(S+T)$ 個
- 新たな点を含めた木の構造がわかってしまえば、 $O(S+T)$ で DP が行える

「新しい木」の作り方の例

- まず、最初の点たちを X, Y 由来区別なく DFS 順序でソート
- 隣り合う点同士の LCA を求める
- RMQ などを用いて、LCA 列のある範囲でどこが最も根に近いかが求められるようにする

「新しい木」の作り方の例

- 点たちのある範囲において木を構築する方法
- RMQ を用いて、その範囲でどこが根になるかを LCA 列の中で求める
- そのような点は複数回現れるかもしれないが、どこを選んでもよい
- その点の左側、右側について再帰的に木を構築する
- 左側、右側の根と、現在の根を結べば終わり
- 辺の長さは、元の木における「根からの距離」の差を用いる
- 同じ点が複数回現れるかもしれないが、結局長さ 0 の辺で結ばれるだけなので無問題

計算量

- 初期化
 - DFS 木の作成に $O(N)$
 - LCA を $O(\log N)$ で求めるための準備に $O(N)$
- クエリ (S, T の和をそれぞれ $Ssum, Tsum$ とする)
 - DFS 順序に並び替えるのに $O((S + T) \log (S + T))$
 - LCA をすべて求めるのに $O(S + T)$
 - LCA 列上での RMQ に, 準備とクエリ合わせて $O((S + T) \log (S + T))$
- 結局, $O((N + Ssum + Tsum) \log N)$
- 満点が得られる

総和占い Fortune Telling 2

今西 健介(@japlj)

問題概要



問題

- 両面に整数 A_i, B_i が書かれたカードが N 枚ある
 - 最初は A_i が書かれた面が見えている
- 「見えている整数が T_j 以下のカードを裏返す」という操作を K 回行う
- すべての操作後に見えている整数の合計値は？

満点制約

- $1 \leq N, K \leq 200,000$

小課題 1 (4点)



制約

- $1 \leq N \leq 1,000$
- $1 \leq K \leq 1,000$

解法

- 各操作をそのまま実装する
 - 1 回の操作の計算量は $O(N)$
 - 合計で $O(NK)$

小課題 2 (31点)



制約

- $1 \leq N \leq 40,000$
- $1 \leq K \leq 40,000$

解法

当然、小課題 1 のままでは解けないので

何らかの工夫が必要

→操作について詳しく考えてみよう

考察、その前に



仮定

- ・ここから先では全てのカードで $A_i \leq B_i$ と仮定
 - ・説明のしやすさ、分かりやすさのため

仮定？

- ・じゃあ $B_i < A_i$ なカードはどうなるの？
- ・ A_i, B_i を交換し、最初 B_i の面が見えていると思う
 - ・面倒そうな方法に聞こえるけど、これから説明する解法ではこう考えておいたほうが楽

「操作」は何をしているか



操作

- 見えている整数が T_j 以下のカードを裏返す

カード視点で見る

- $T_j < A_i$ のとき $\rightarrow$ 必ず裏返されない
- $A_i \leq T_j < B_i$ のとき $\rightarrow A_i$ の面が上なら裏返される
- $B_i \leq T_j$ のとき $\rightarrow$ 必ず裏返される

A_i, B_i と T_j の大小関係によって操作の内容が決まる

大小関係による分類



カードの同一視

- (A_i, B_i) がそれぞれ $(3, 6)$, $(2, 5)$ の 2 枚のカード
- それらに対する 3 回の操作 $T = (4, 9, 1)$
- 両方のカードに対し、操作の内容は全て同じ！

操作	カード	カード
4	$3 \leq 4 < 6$	$2 \leq 4 < 5$
9	$6 \leq 9$	$5 \leq 9$
1	$1 < 3$	$1 < 2$

→ A_i が上なら裏返す

→ 裏返す

→ 裏返さない

大小関係による分類



カードの同一視

- 同一視できるカードたちについてはそれぞれに対して操作の内容を考える必要がない
 - そのうち 1 枚に対して考えれば、後は同じ
- カードを分類することで計算量が削減できる？

カードの種類数

- T は K 個の値からなる
- 大小関係で分類してもまだ $O(K^2)$ 通りもある

分類してうまくいく場合



カードの種類数を抑える

- K (操作の回数) が小さければ、種類数も小さい
- 操作をいくつかの区間に分けて考えよう！
 - ・ 各区間の中では操作の回数は少ない
 - ・ その区間の操作たちをカードの分類によって高速処理

具体的に

- 1区間に B 個の操作を行うとする
 - ・ B の値をうまく決めると……？

バケットサイズ



計算量

- 各区間には B 個の操作がある
- カードを $O(B^2)$ 種類に分類できる
- カードの処理はまとめて $O(B^3)$ でできる
- よって、各区間の計算量は $O(N + B^3)$
- 区間の個数は K / B 個
- 全体の計算量は $O(NK / B + B^2K)$ となる
 - $B = K^{1/3}$ とおけば $O(NK^{2/3} + K^{5/3})$

分類処理や
実際に裏返す処理などに
 $O(N)$ にかかることに注意

これで小課題 2 が解ける！

小課題 3 (65点)



制約

- $1 \leq N \leq 200,000$
- $1 \leq K \leq 200,000$

解法

操作についてさらに考察を深める！

「操作」は何をしているか(再)



操作

- 見えている整数が T_j 以下のカードを裏返す

カード視点で見る

- $T_j < A_i$ のとき $\rightarrow$ 必ず裏返されない
- $A_i \leq T_j < B_i$ のとき $\rightarrow A_i$ の面が上なら裏返される
- $B_i \leq T_j$ のとき $\rightarrow$ 必ず裏返される

A_i, B_i と T_j の大小関係によって操作の内容が決まる

「操作」は何をしているか

操作

- 見えている整数が T_j 以下のカードを裏返す

カード視点で見る

- $T_j < A_i$ のとき $\rightarrow$ 必ず裏返されない
- $A_i \leq T_j < B_i$ のとき $\rightarrow A_i$ の面が上なら裏返される
- $B_i \leq T_j$ のとき $\rightarrow$ 必ず裏返される

これは考えなくていい

とは一体……？

何も考えずに裏返せばよい

「操作」は何をしているか

操作

- 見えている整数が T_j 以下のカードを裏返す

カード視点で見る

- $T_j < A_i$ のとき → 必ず裏返されない
- $A_i \leq T_j < B_i$ のとき → 操作後、 B_i の面が上を向く
- $B_i \leq T_j$ のとき → 必ず裏返される

これは考えなくていい

こういうことだ！

何も考えずに裏返せばよい

操作の性質



重要な事実

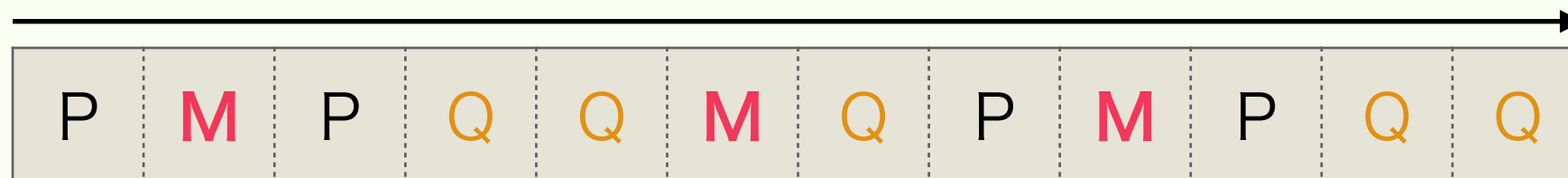
- $A_i \leq T_j < B_i$ なる操作 T_j を行ったあと、
上を向く面は**以前の状態によらず B_i の面**になる
 - ・ これは「**大きい方を上に向ける操作**」と言える
- 大きい方を上に向ける操作を行うと、
それ**以前に行われた操作のことは忘れてもよい！**
 - ・ この操作によってカードの状態がリセットされる

1 枚のカードから見た操作列



操作の分類

- カードから見れば操作は 3 種類に分類できる
 - 何もしない(P)、裏返す(Q)、大きい方を上に向ける(M)
- 例として以下の様な操作列を考える



1 枚のカードから見た操作列

操作の分類

- カードから見れば操作は 3 種類に分類できる
 - 何もしない(P)、裏返す(Q)、大きい方を上に向ける(M)
- 例として以下の様な操作列を考える



実質行う操作は
これらだけ

ここの操作を行った時点で
それ以前の操作はなかったことにしてよい

解法アウトライン



解法の方針

- 各カードごとに以下のように処理する: 最後の
- $A_i \leq T_j < B_i$ なる 最も後ろの T_j を探す 「大きい方を上にする」操作
- 見つけた T_j 以降で $B_i \leq T_k$ なる k の 個数 を求める
- これによりカード i の最終的な向きがわかる
 - 大きい方を上に向ける操作がないときに注意

実装

- そのまま書くだけでは $O(NK)$ なので、工夫が必要

実装



実装方針

- 色々な方法がありますが、たとえば
 - T_j を座標圧縮し、位置 T_j に値 j を書いておく
 - A_i, B_i に挟まれる区間の中の最大値をとってくる
- といった感じで
 - segtree, BIT, RMQ 等の応用的な使い方

細かい実装の差にもよるが

$O((N+K) \log (N+K))$ や

$O((N+K) \log^2 (N+K))$ で満点

実装詳細

- segtree 等についての説明は省きます
 - 他に解説している資料・本がいろいろあります

Space Pirate 解説

問題概要

- 関数グラフ f がある
- 頂点 A の行き先を $f(A)$ から頂点 B に置き換える
- スタート地点(頂点1)から K 歩進む
- (A,B) を色々動かしたときに、 K 歩進んだ先はどのようなになるでしょうか。数えてください。

小課題1 (10点)

- $N \leq 100$

小課題1 (10点)

- $N \leq 100$
- 全ての(A, B)を試し、それぞれの行き先を調べる

小課題1 (10点)

- $N \leq 100$
- 全ての(A, B)を試し、それぞれの行き先を調べる
- Kが大きいのでループを検出してうまく処理する

小課題1,2 (10+37点)

- $N \leq 3000$

小課題1,2 (10+37点)

- $N \leq 3000$
- 何かしらのオーダーを減らせばよい

小課題1,2 (10+37点)

- $N \leq 3000$
- 各(A,B)について $O(\log n)$ で調べられればよい

小課題1,2 (10+37点) – 考察

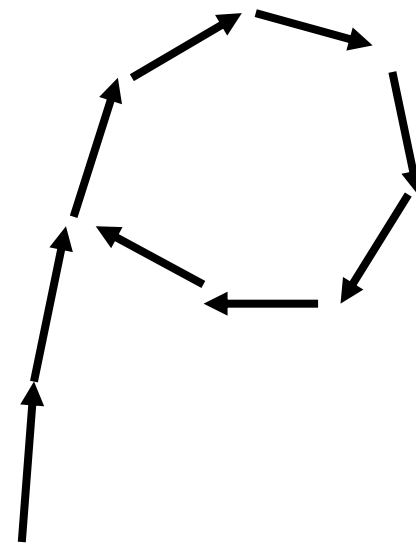
- $N \leq K$

小課題1,2 (10+37点) – 考察

- $N \leq K \rightarrow K$ 歩進んだ先は閉路上にある

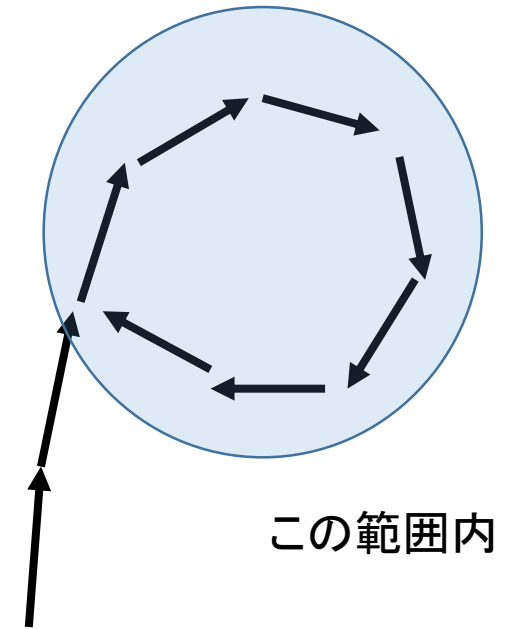
小課題1,2 (10+37点) – 考察

- $N \leq K \rightarrow K$ 歩進んだ先は閉路上にある



小課題1,2 (10+37点) – 考察

- $N \leq K \rightarrow K$ 歩進んだ先は閉路上にある



小課題1,2 (10+37点) – 事前計算

- 以下を事前計算

小課題1,2 (10+37点) – 事前計算

- 以下を事前計算
 - 元のグラフで、各点から 2^i 進んだ先はどの点か？ (doubling)
 - 各連結成分について根を一つ決めておく
 - 正確には木になっていないが、一箇所を切り開いて木と見なす
 - 各点について、その点が所属する連結成分の根
 - 各連結成分について、その連結成分に存在する閉路の大きさ

小課題1,2 (10+37点) – 事前計算

- doublingにより、ある点から元のグラフ上で x 歩進んだ先は高速に求められる

小課題1,2 (10+37点) – 場合分け

- 以下のように場合分け

小課題1,2 (10+37点) – 場合分け

- 以下のように場合分け
 - 頂点1から頂点Aに到達できない場合
→ 全て同じ点に到達する。到達先は簡単にわかる
 - 頂点1から頂点Aに到達可能な場合
→ さらに場合分け

小課題1,2 (10+37点) – 場合分け

- 頂点1から頂点Aに到達可能な場合
 - 閉路上の到達可能な点を一つ選び、Cとおく
 - 頂点1からCに初到達するまでの歩数を計算
 - Cからの残り歩数を閉路長で割り、余りをとる
 - この余りの数だけ進んだ点が、到達地点

小課題1,2 (10+37点) – 場合分け

- 以下のように場合分け
 - 頂点Bが頂点Aの子孫である場合
→もとの閉路とは無関係の閉路ができる ($C=B$ とおく)
 - 頂点Bが頂点Aの子孫ではなく、頂点Aがもとの閉路上にある場合
→もとの閉路の一部が変更されたような閉路ができる ($C=B$ とおく)
 - 頂点Bが頂点Aの子孫ではなく、頂点Aがもとの閉路上にない場合
→もとの閉路が使われる

小課題1,2 (10+37点) – 場合分け

- 以下のように場合分け
 - 頂点Bが頂点Aの子孫である場合
→もとの閉路とは無関係の閉路ができる ($C=B$ とおく)
 - 頂点Bが頂点Aの子孫ではなく、頂点Aがもとの閉路上にある場合
→もとの閉路の一部が変更されたような閉路ができる ($C=B$ とおく)
 - 頂点Bが頂点Aの子孫ではなく、頂点Aがもとの閉路上にない場合
→もとの閉路が使われる
- $C=B$ とおくことで、もとのグラフのdoublingを使って辿っても問題ない
(一般には、もとのグラフ上を辿るのと変更後のグラフ上を辿るのでは異なる頂点に行き着く)

小課題1,2 (10+37点)

- 以上のアルゴリズムで $O(n^2 \log n)$ が実現された

小課題3 (33点)

- 各頂点の行き先は互いに異なる

小課題3 (33点)

- 各頂点の行き先は互いに異なる
- →グラフは閉路が何個か集まったものになる

小課題3 (33点)

- 頂点1の連結成分の大きさをMとする。
- Aが頂点1とは異なる連結成分にあるとき → 一括処理
- Aが頂点1と同じ連結成分にあるとき
 - Bが頂点1とは異なる連結成分にあるとき
 - Bが頂点1と同じ連結成分にあるとき

小課題3 (33点)

- Aが頂点1と同じ連結成分にあり、Bが頂点1とは異なる連結成分にあるとき
 - この場合をまとめて処理すると、結果として頂点1とは異なる連結成分にある各頂点に $N - M$ を足すのと同等であることがわかる。

小課題3 (33点)

- Aが頂点1と同じ連結成分にあり、Bが頂点1と同じ連結成分にあるとき
 - この結果できる閉路の大きさは1以上M以下。この大きさLごとに処理する
 - Lを固定すると、到達先としてありえる頂点はM/Lより少し多いくらいしかない
 - したがって処理の回数は
$$\frac{M}{1} + \frac{M}{2} + \dots + \frac{M}{M-1} + \frac{M}{M}$$
程度となる。これは、 $O(M \log M)$ である。各処理を $O(1)$ で行えばよい。

小課題3 (33点)

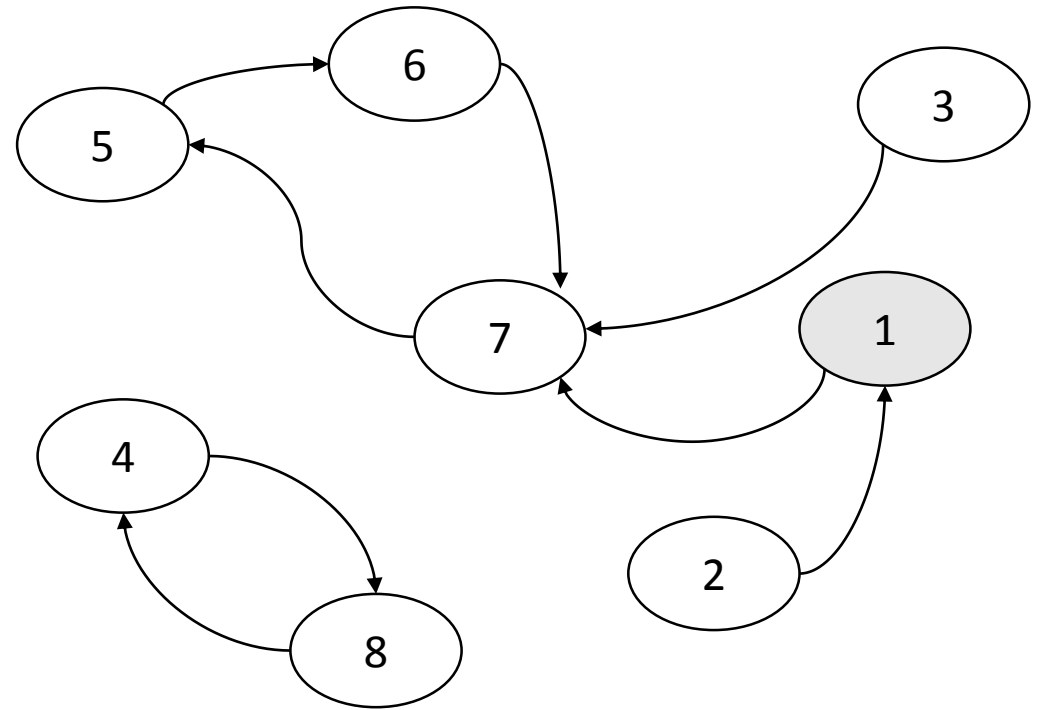
- 以上により、 $O(n \log n)$ で処理できた

満点解法

- グラフを例示しつつ説明

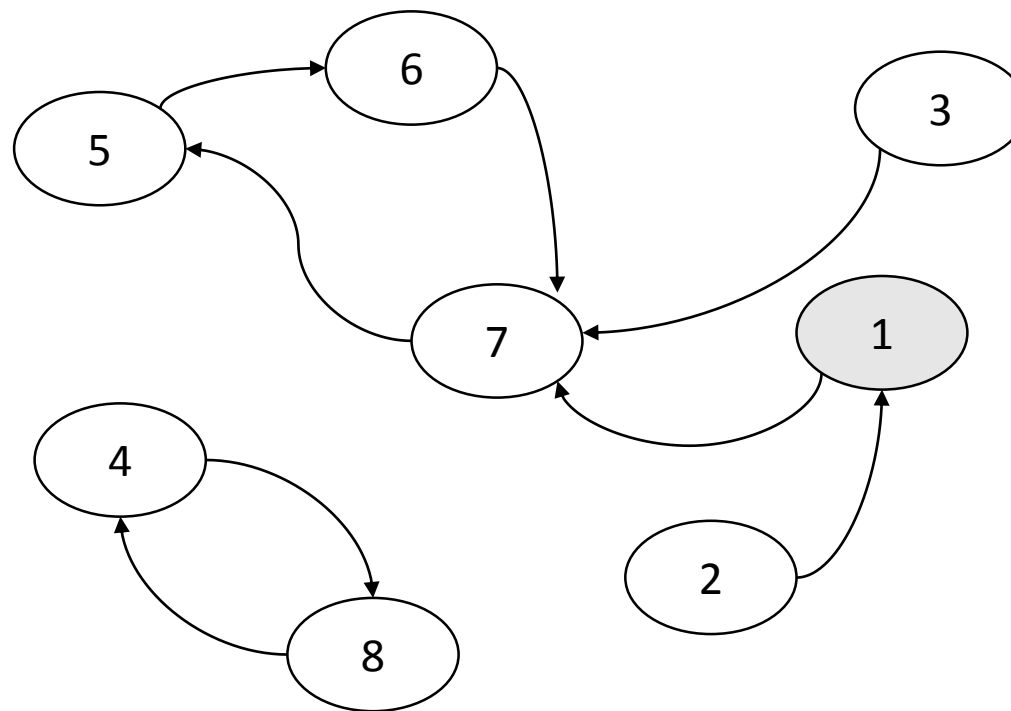
満点解法

- このグラフは $N=8$, $K=10$



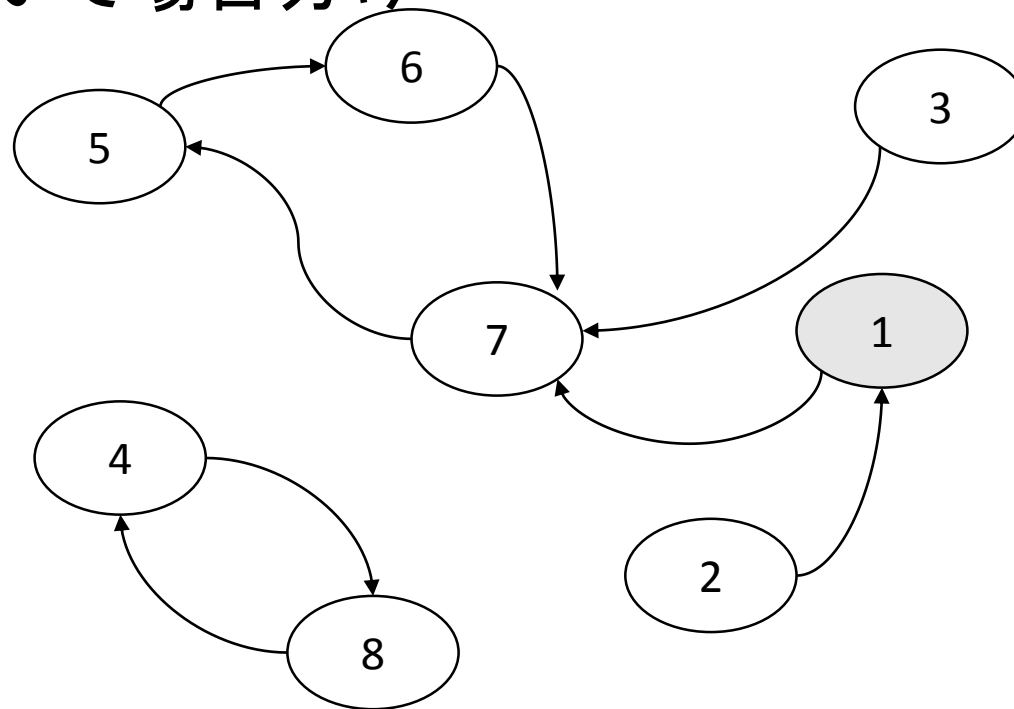
満点解法

- 行き先が変更される頂点をA, その頂点の新しい行き先をBとする



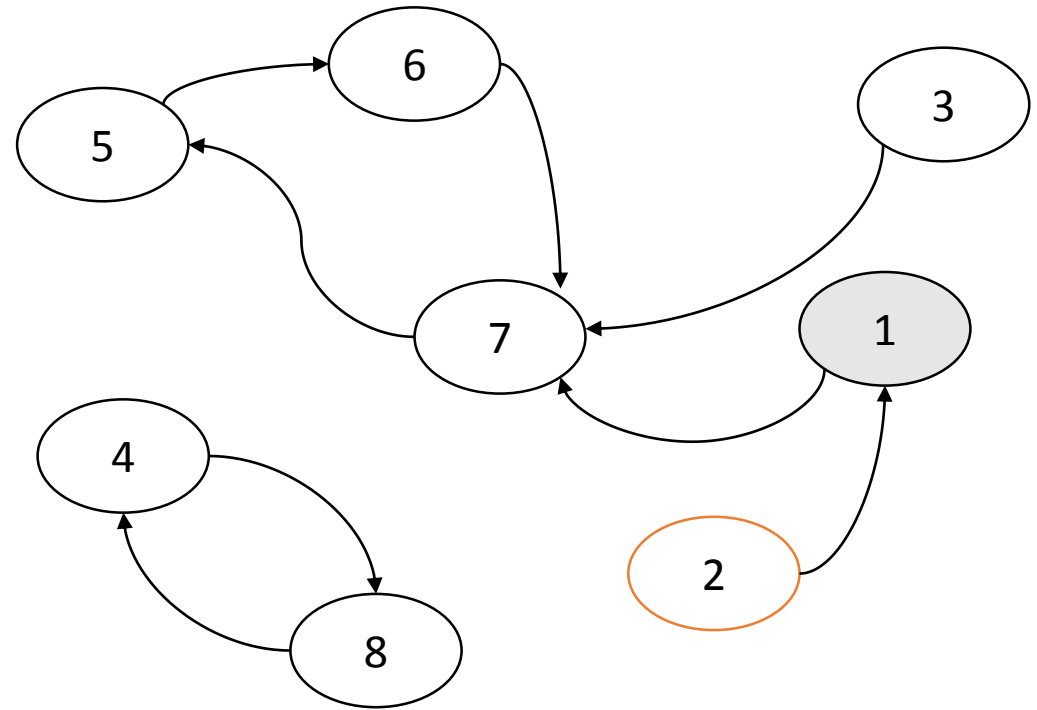
満点解法

- 行き先が変更される頂点をA, その頂点の新しい行き先をBとする
- そもそも頂点1からAに到達できるかで場合分け



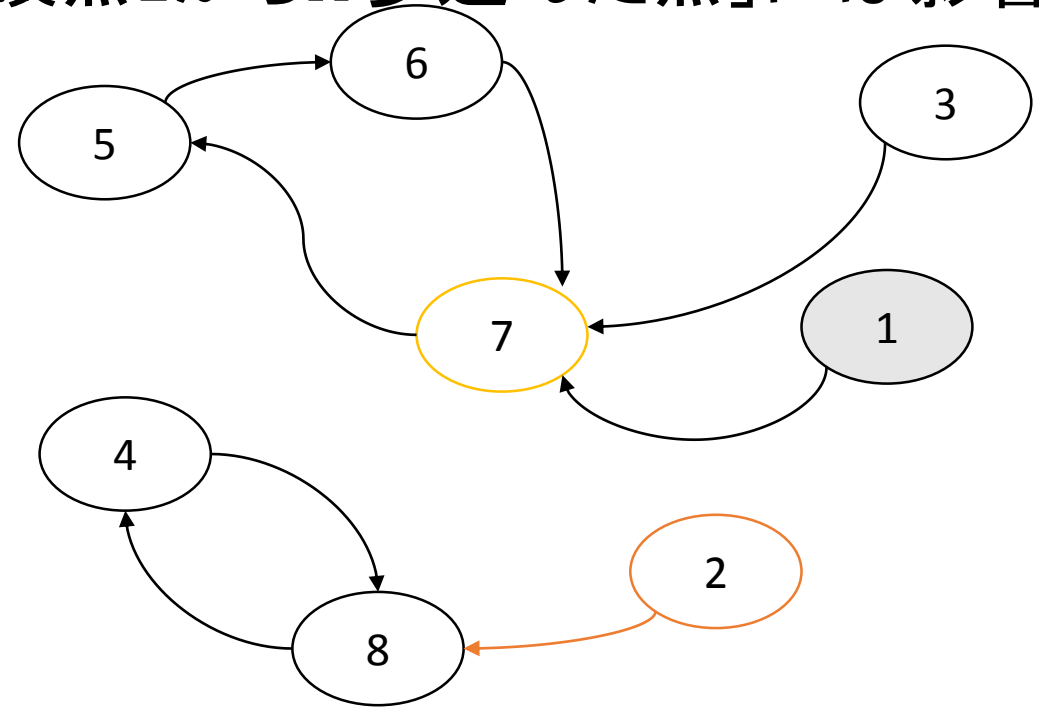
満点解法 – 場合分け(1)

- 頂点1からAに到達できない場合



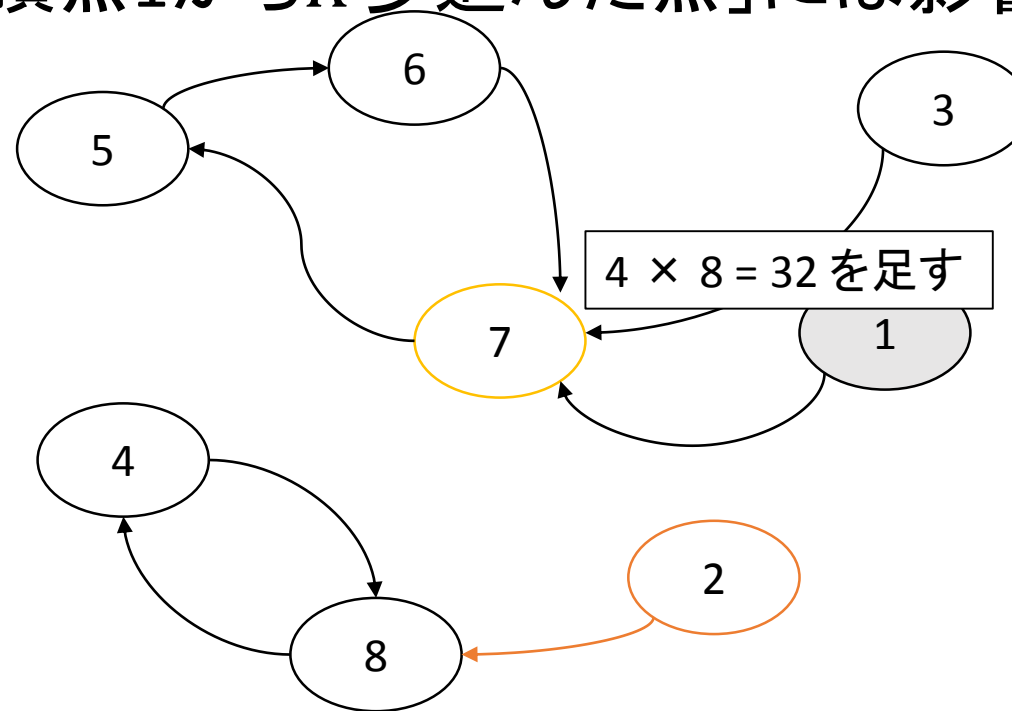
満点解法 – 場合分け(1)

- 頂点1からAに到達できない場合
- このように行き先が変わっても、「頂点1から K 歩進んだ点」には影響しない



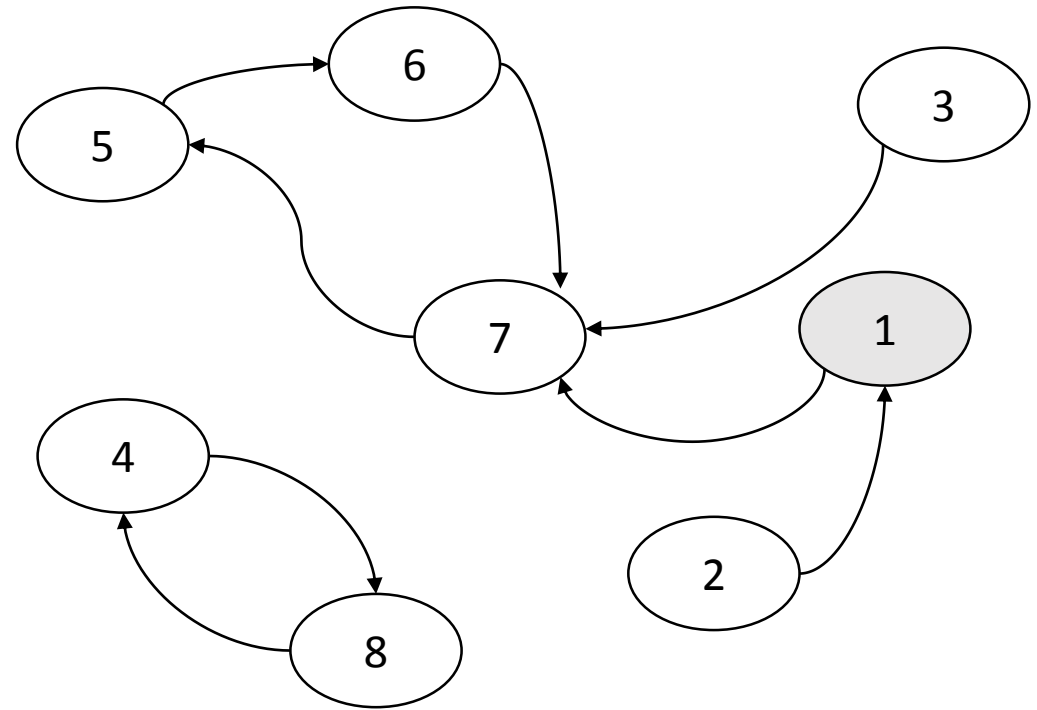
満点解法 – 場合分け(1)

- 頂点1からAに到達できない場合
- このように行き先が変わっても、「頂点1から K 歩進んだ点」には影響しない
- (頂点1から到達不能な点の個数)
 $\times N$
を1から K 歩進んだ先の点に足す



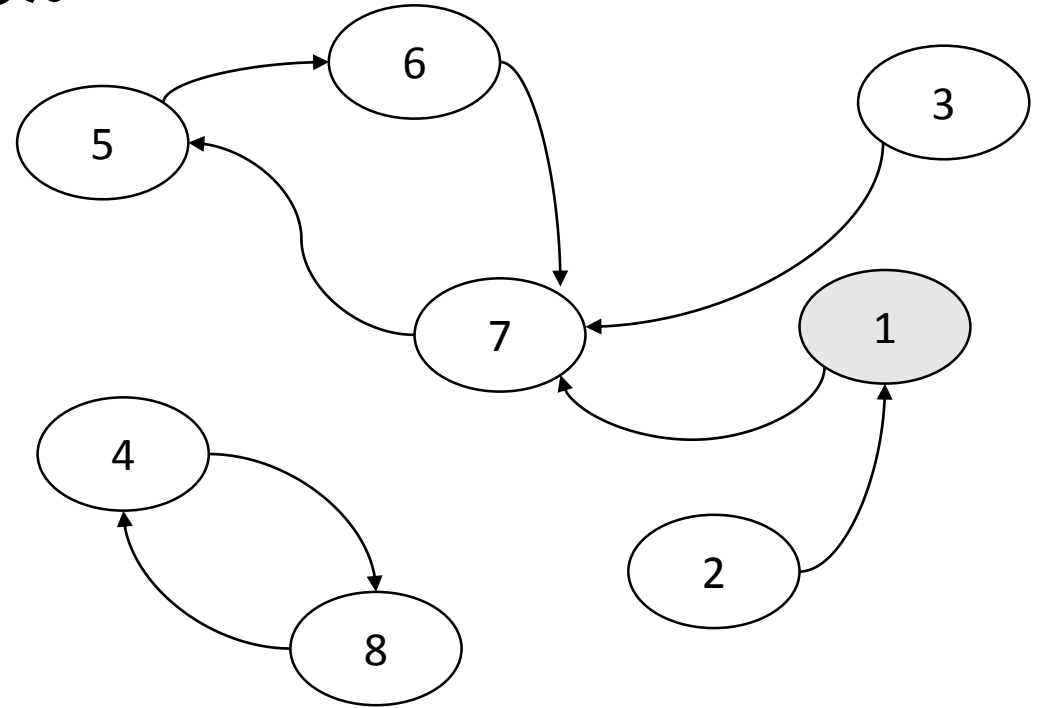
満点解法

- 以下、Aが頂点1から到達可能な場合を調べる



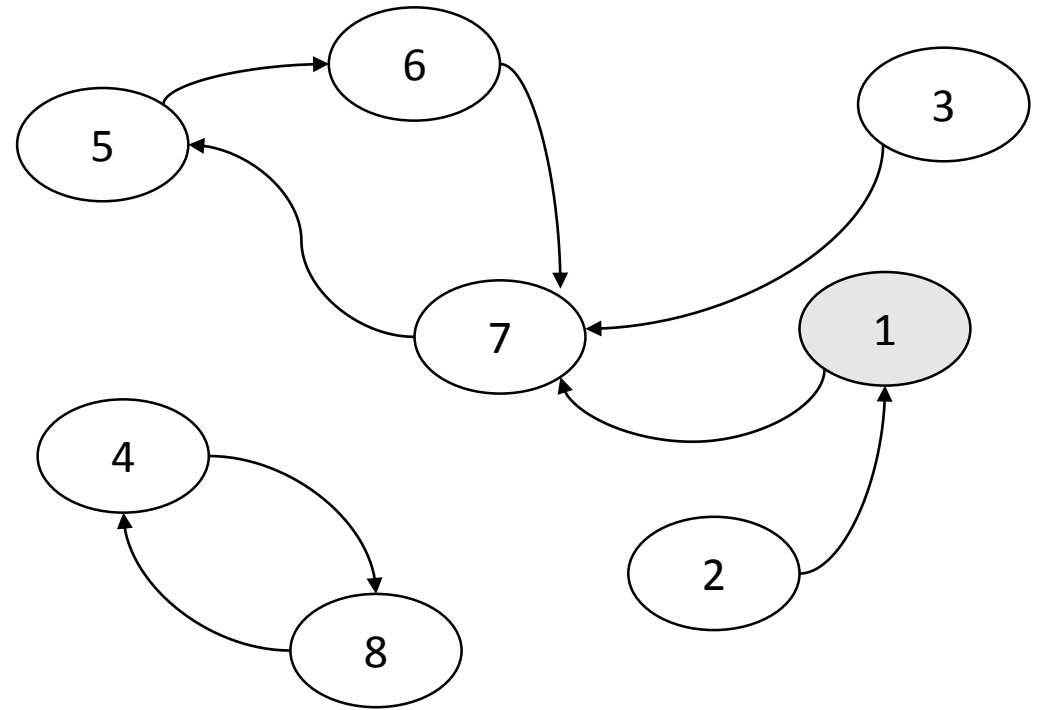
満点解法

- 以下、Aが頂点1から到達可能な場合を調べる
- Bは頂点1と同じ連結成分に属するか？



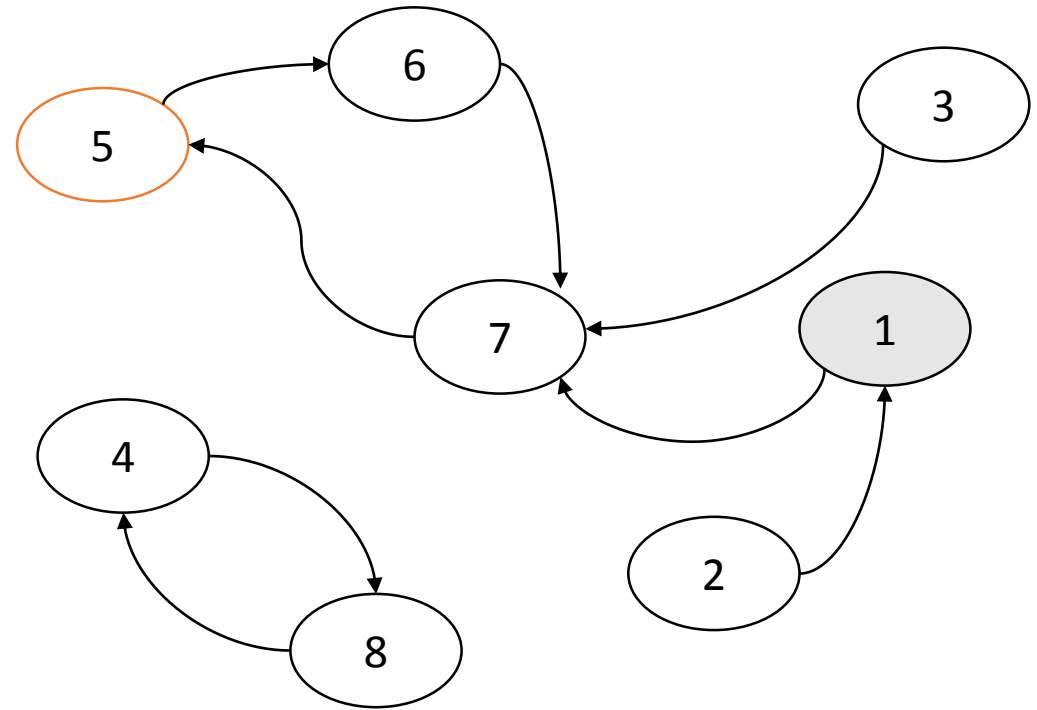
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合



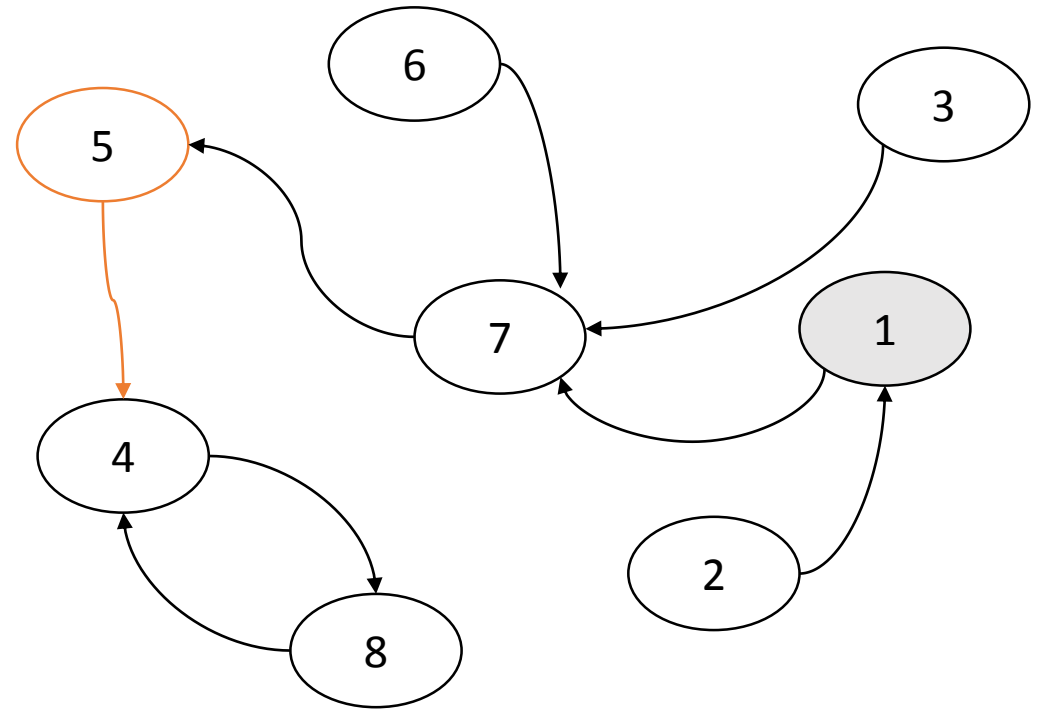
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合



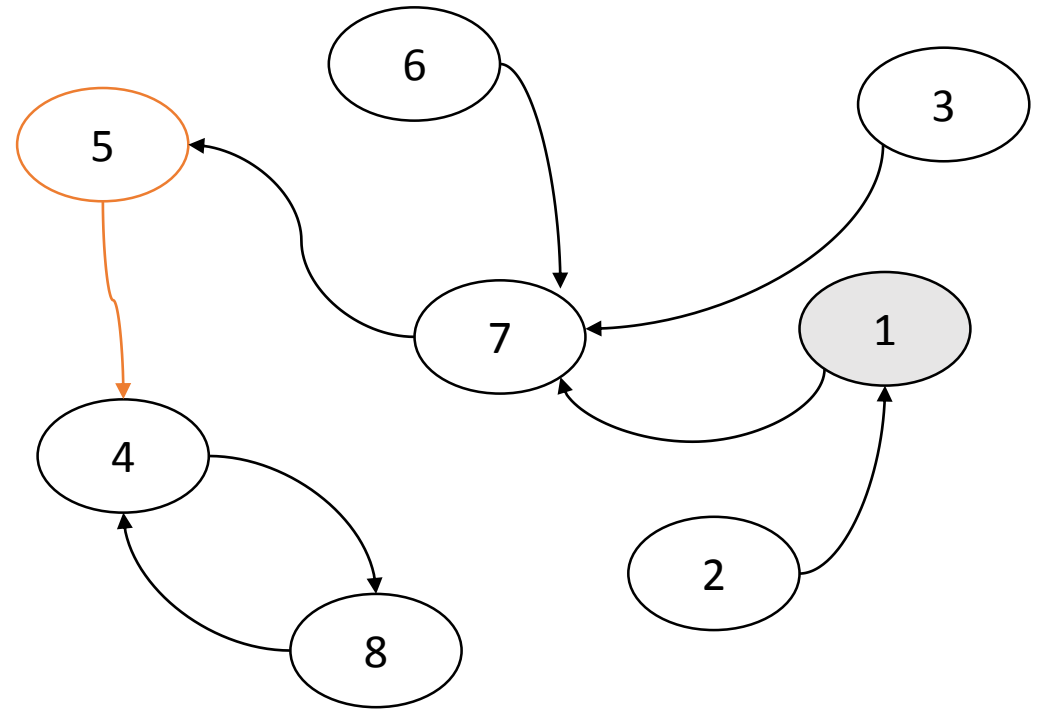
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合



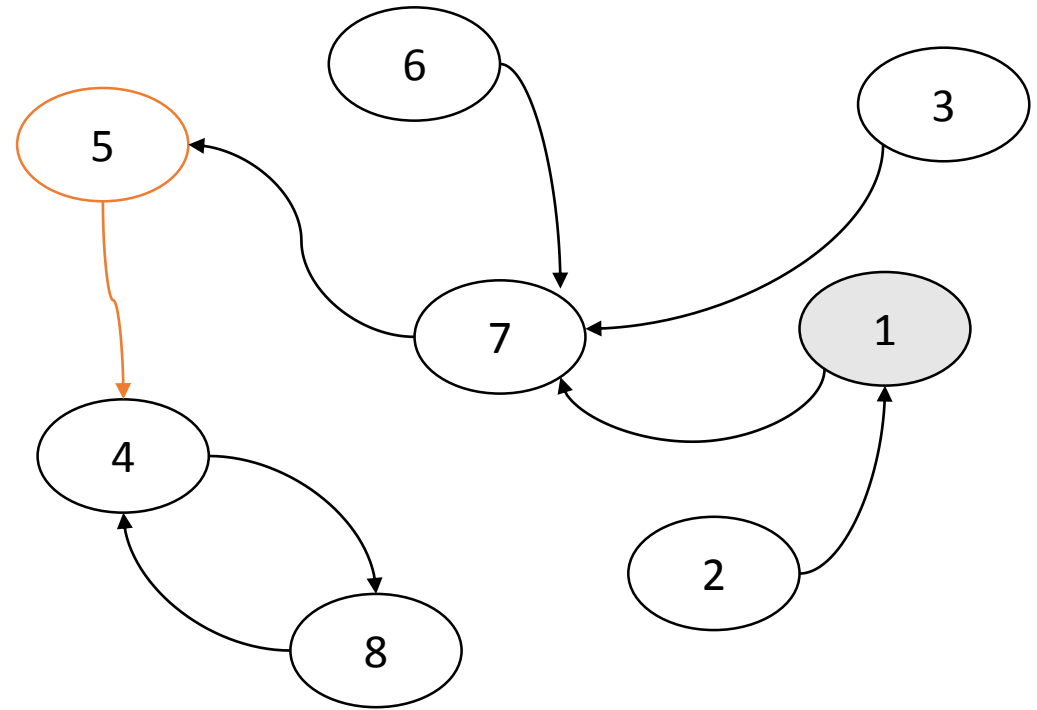
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合
- Bを固定して考える



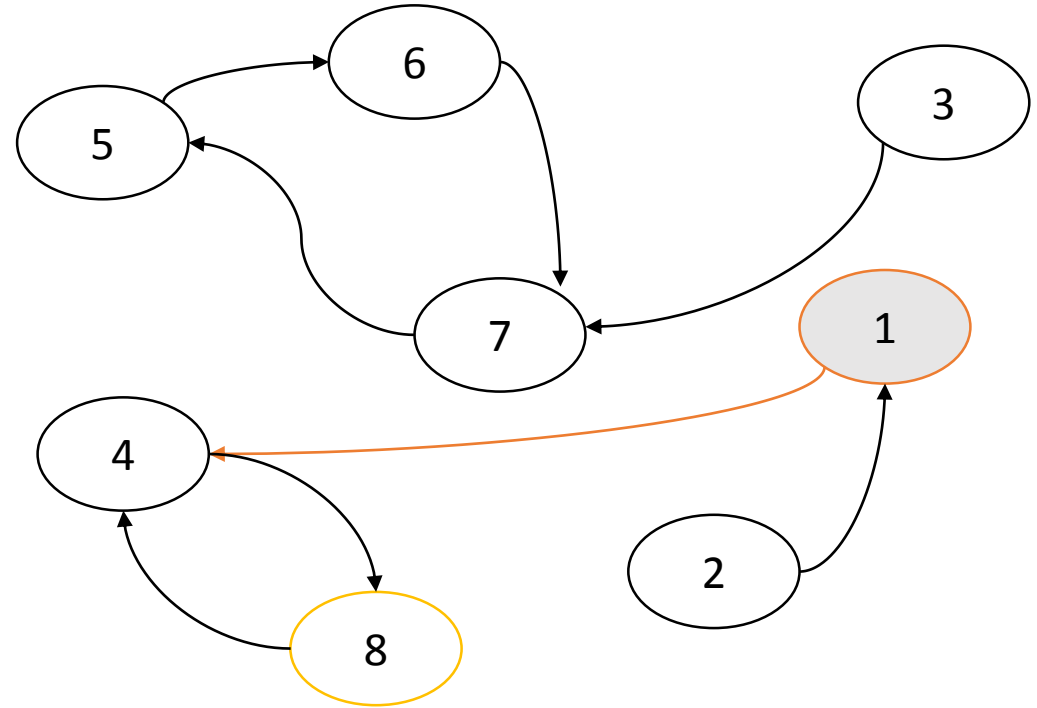
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合
- Bを固定して考える
- →ループの長さは一定



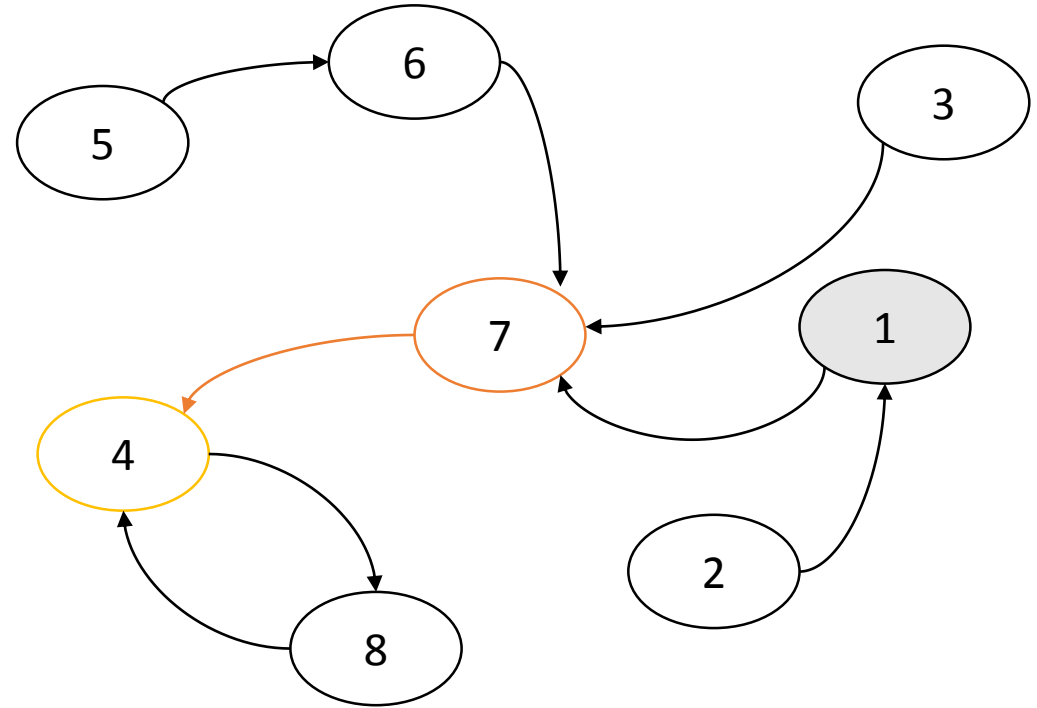
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合
- Bを固定して考える
- →ループの長さは一定
- Aを頂点1から順番に動かすと、到達先は隣に移動していく



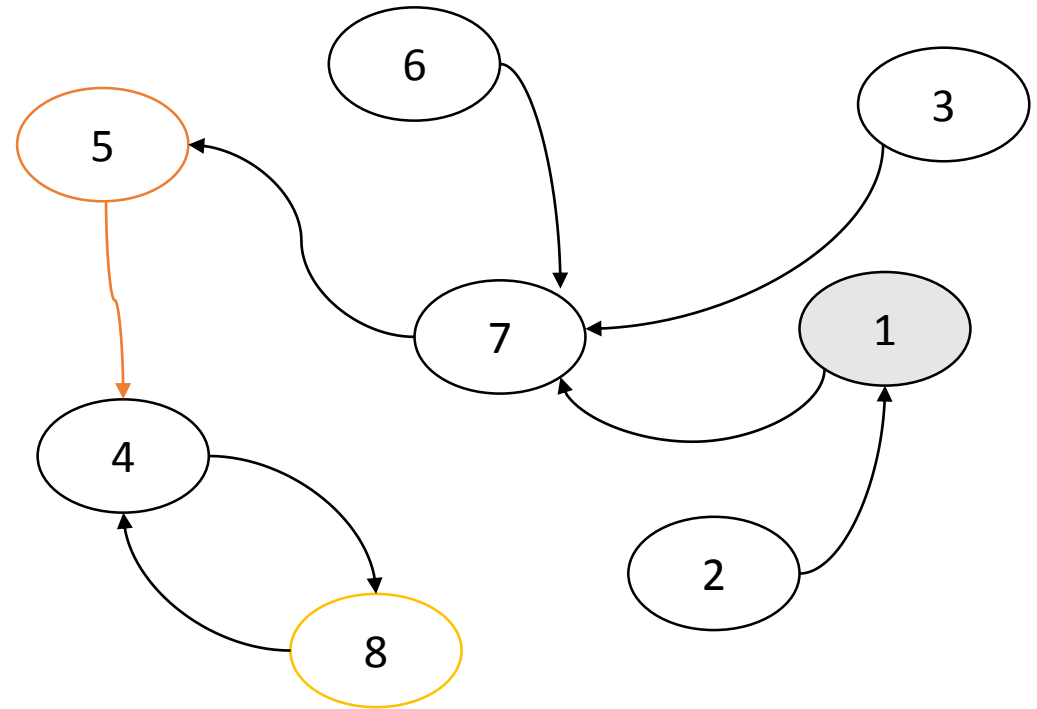
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合
- Bを固定して考える
- →ループの長さは一定
- Aを頂点1から順番に動かすと、到達先は隣に移動していく



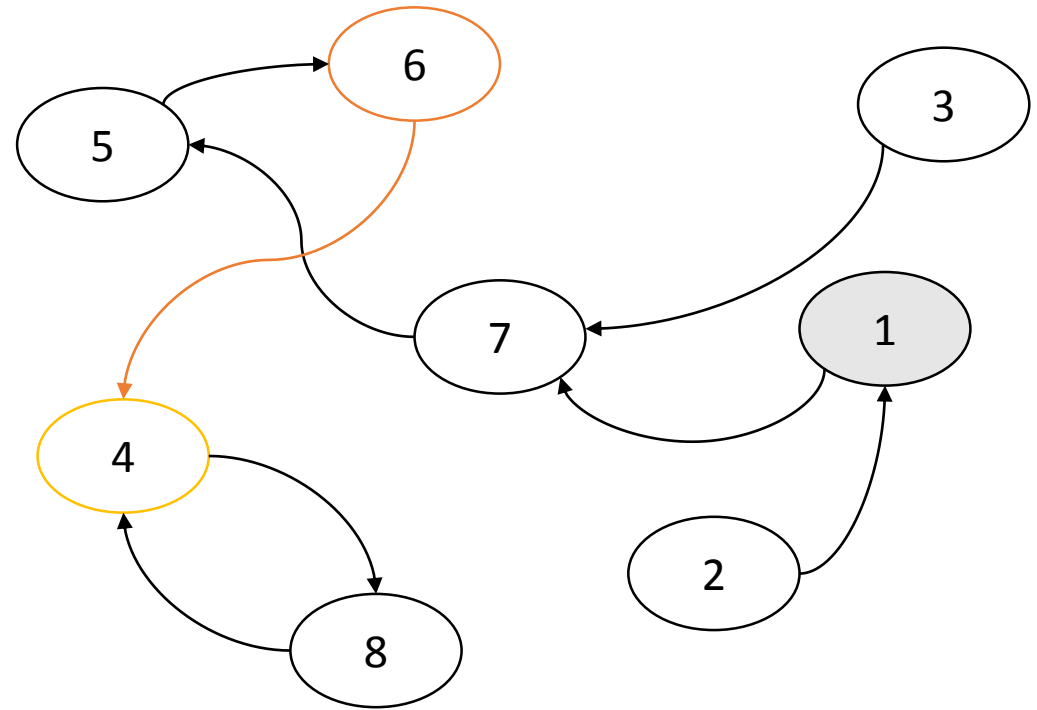
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合
- Bを固定して考える
- →ループの長さは一定
- Aを頂点1から順番に動かすと、到達先は隣に移動していく



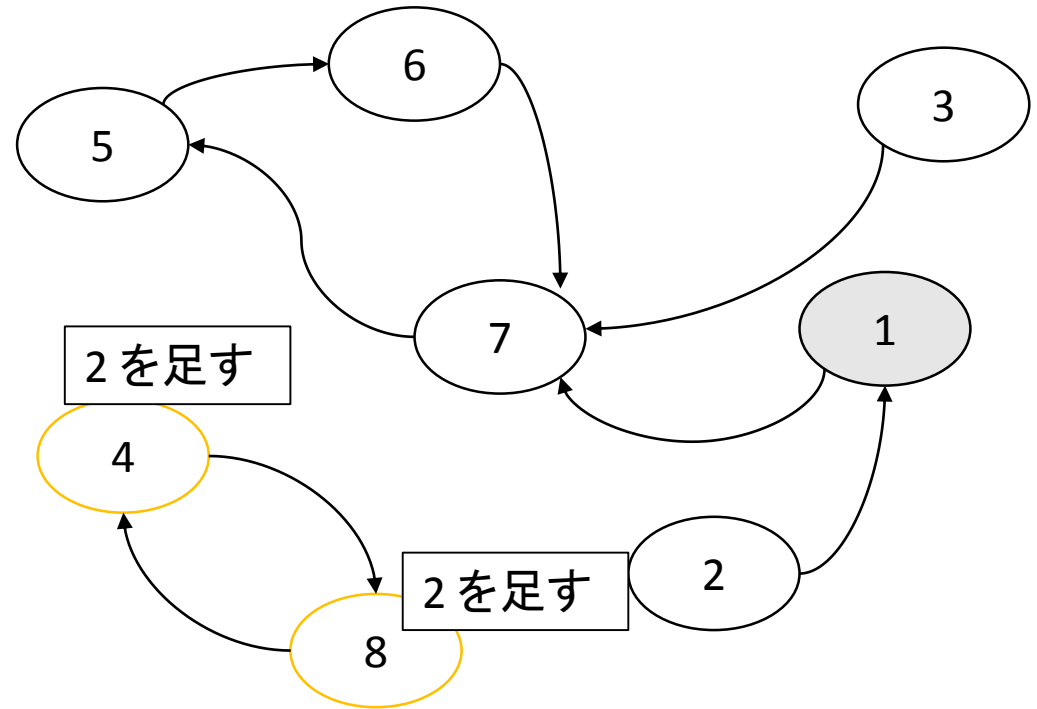
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合
- Bを固定して考える
- →ループの長さは一定
- Aを頂点1から順番に動かすと、到達先は隣に移動していく



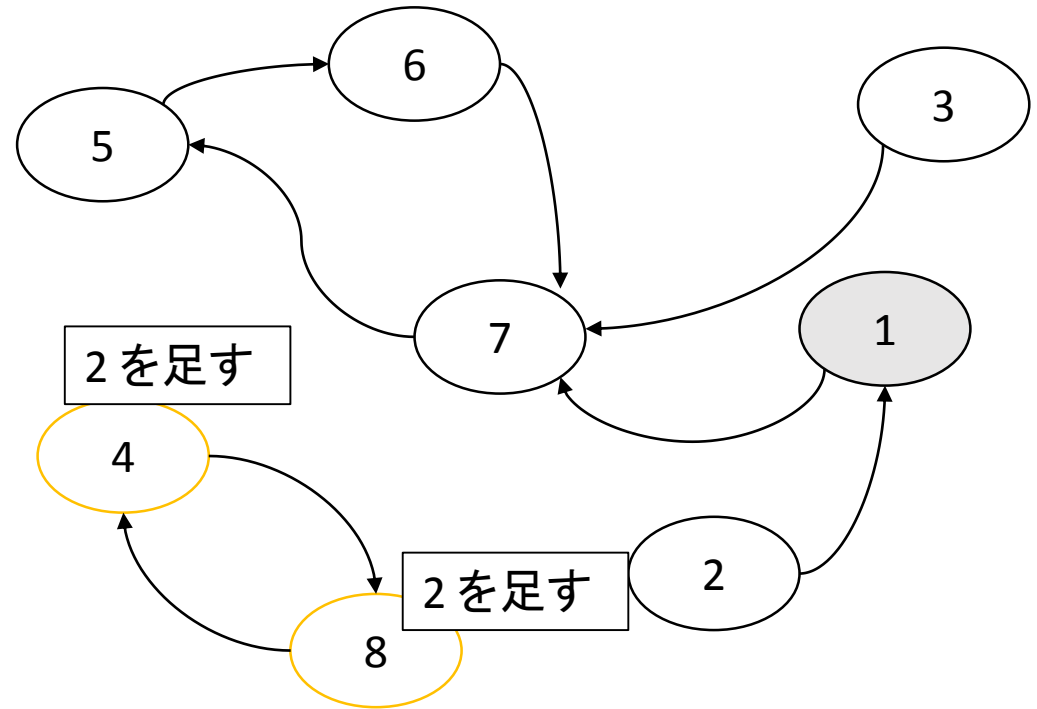
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合
- Bを固定して考える
- →ループの長さは一定
- Aを頂点1から順番に動かすと、到達先は隣に移動していく
- 連続した区間に1を足す→所謂imos法でできる(ループに注意)



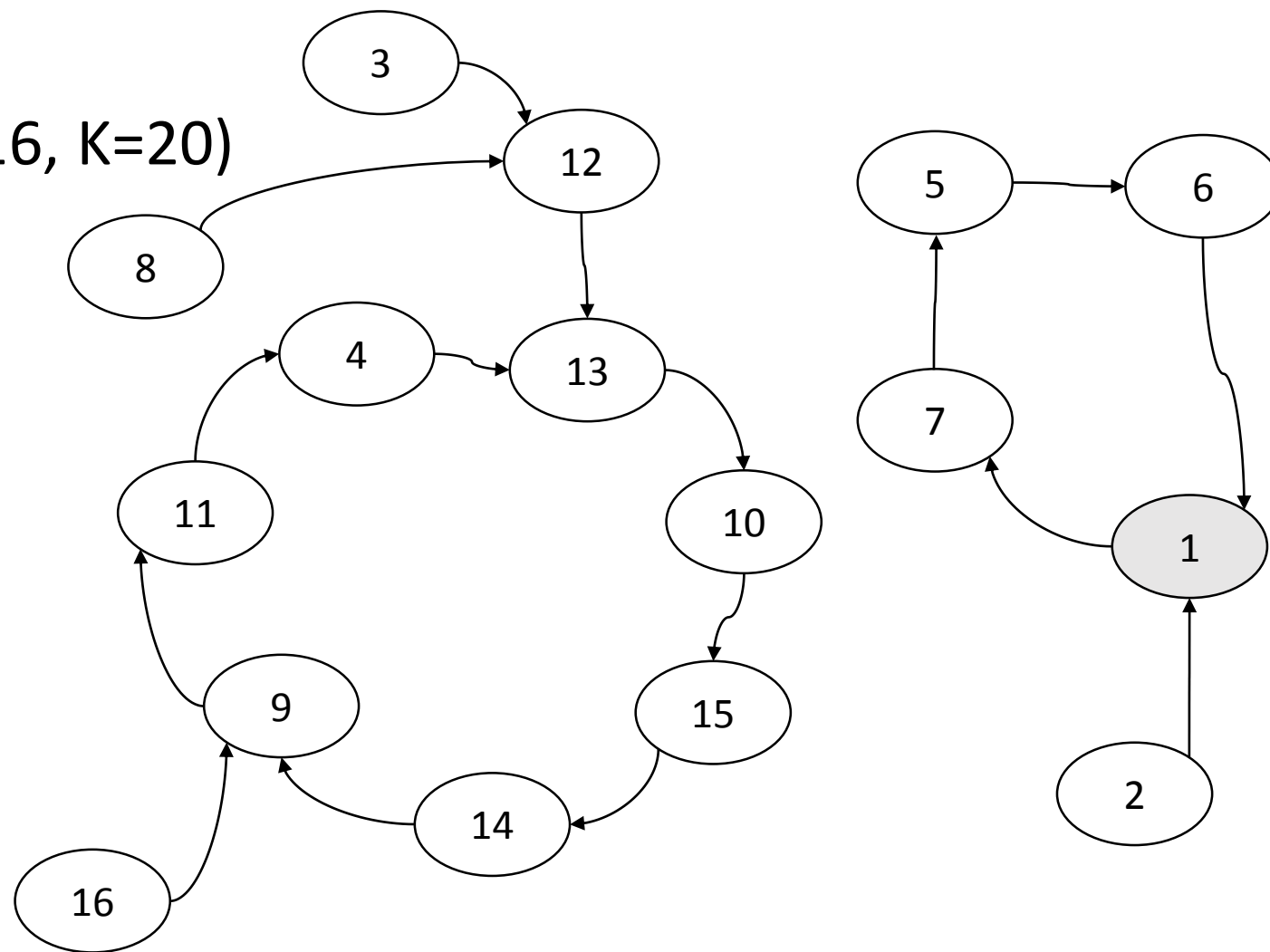
満点解法 – 場合分け(2)

- Bが頂点1とは異なる連結成分に属する場合
- Bを固定して考える
- →ループの長さは一定
- Aを頂点1から順番に動かすと、到達先は隣に移動していく
- 連続した区間に1を足す→所謂imos法でできる(ループに注意。この場合2周している)



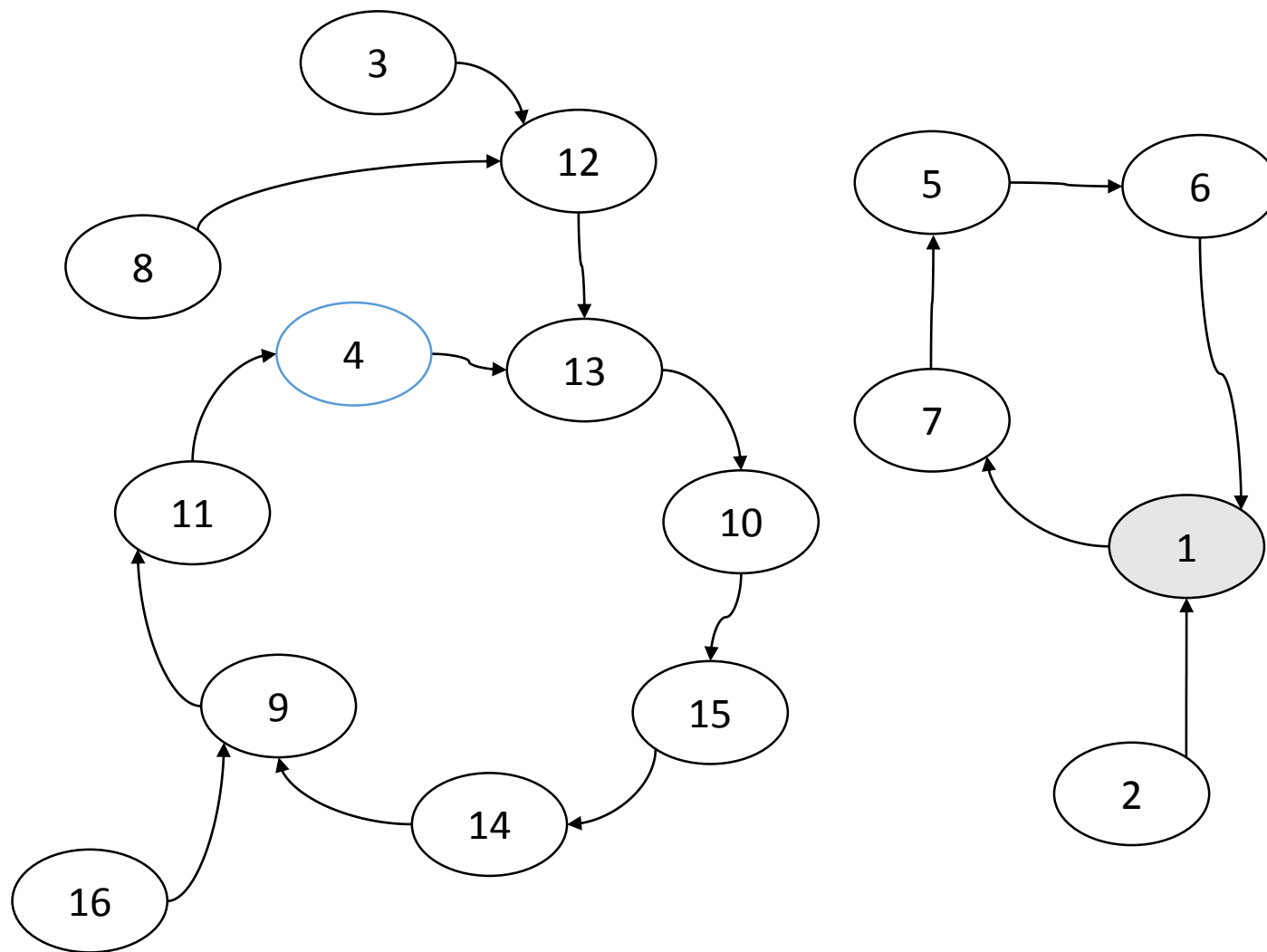
満点解法 – 場合分け(2)

- 別のグラフで再説明($N=16$, $K=20$)



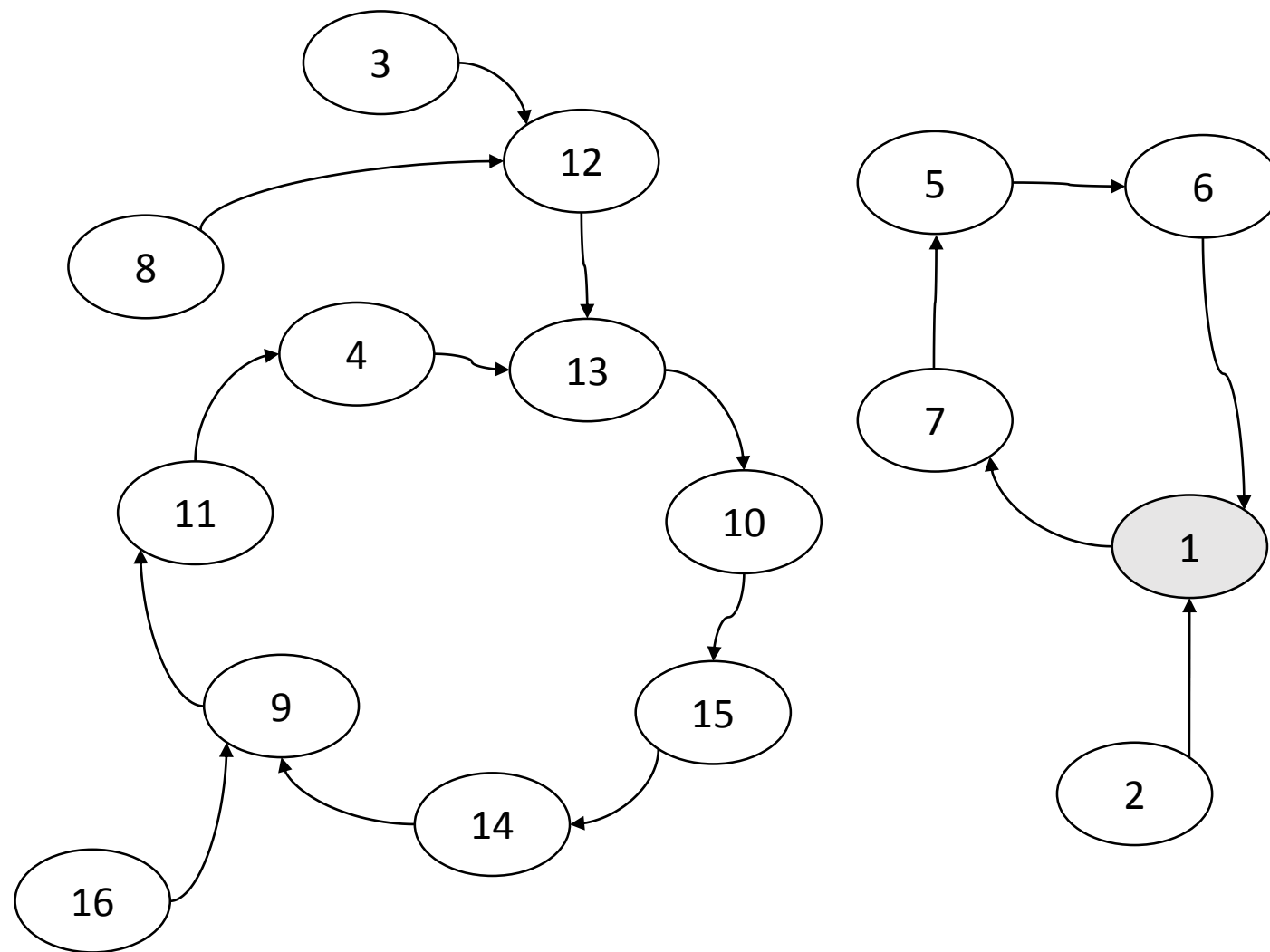
満点解法 – 場合分け(2)

- 頂点4をこの連結成分の「根」としておく



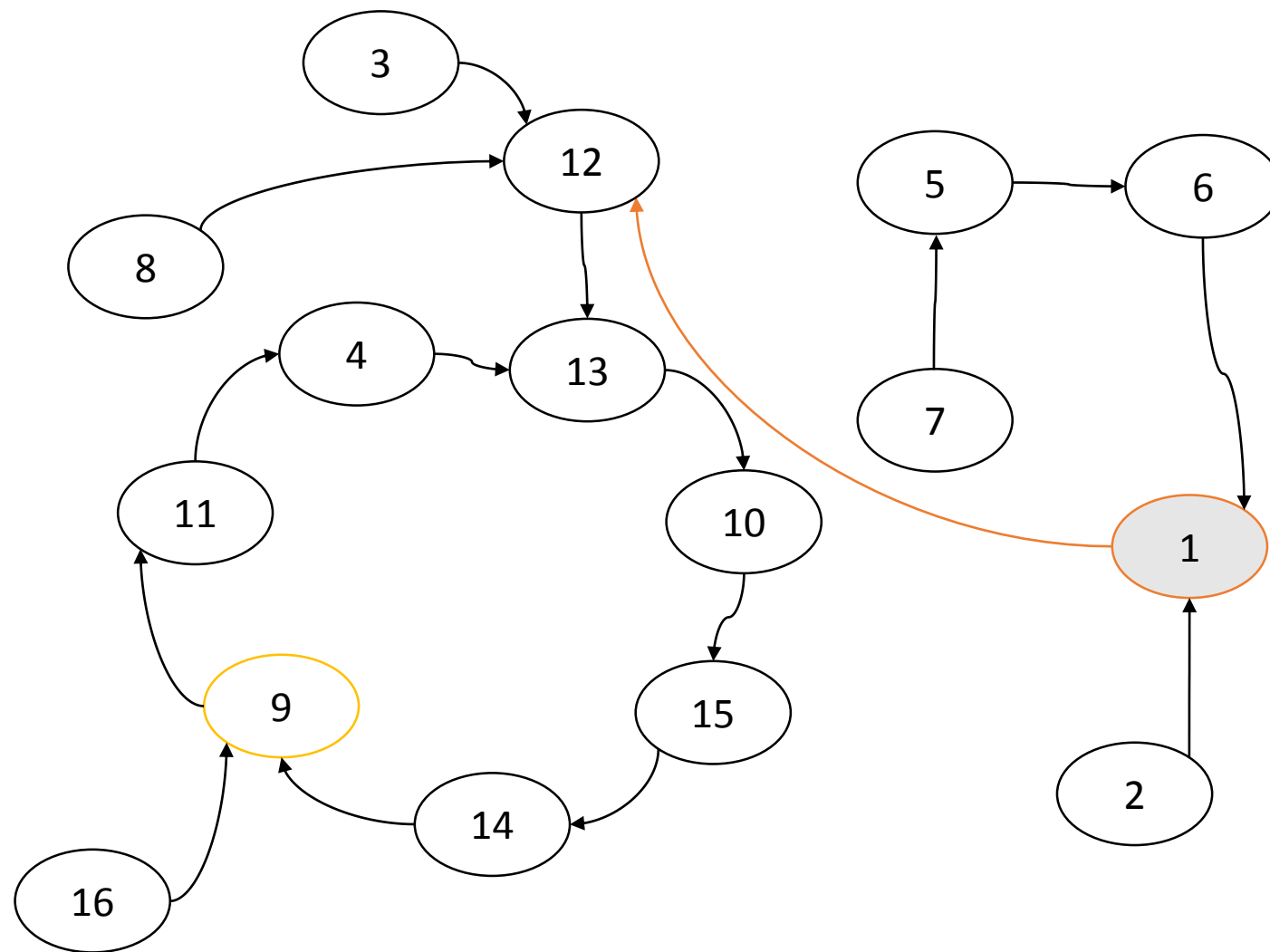
満点解法 – 場合分け(2)

- B=12の場合を考える



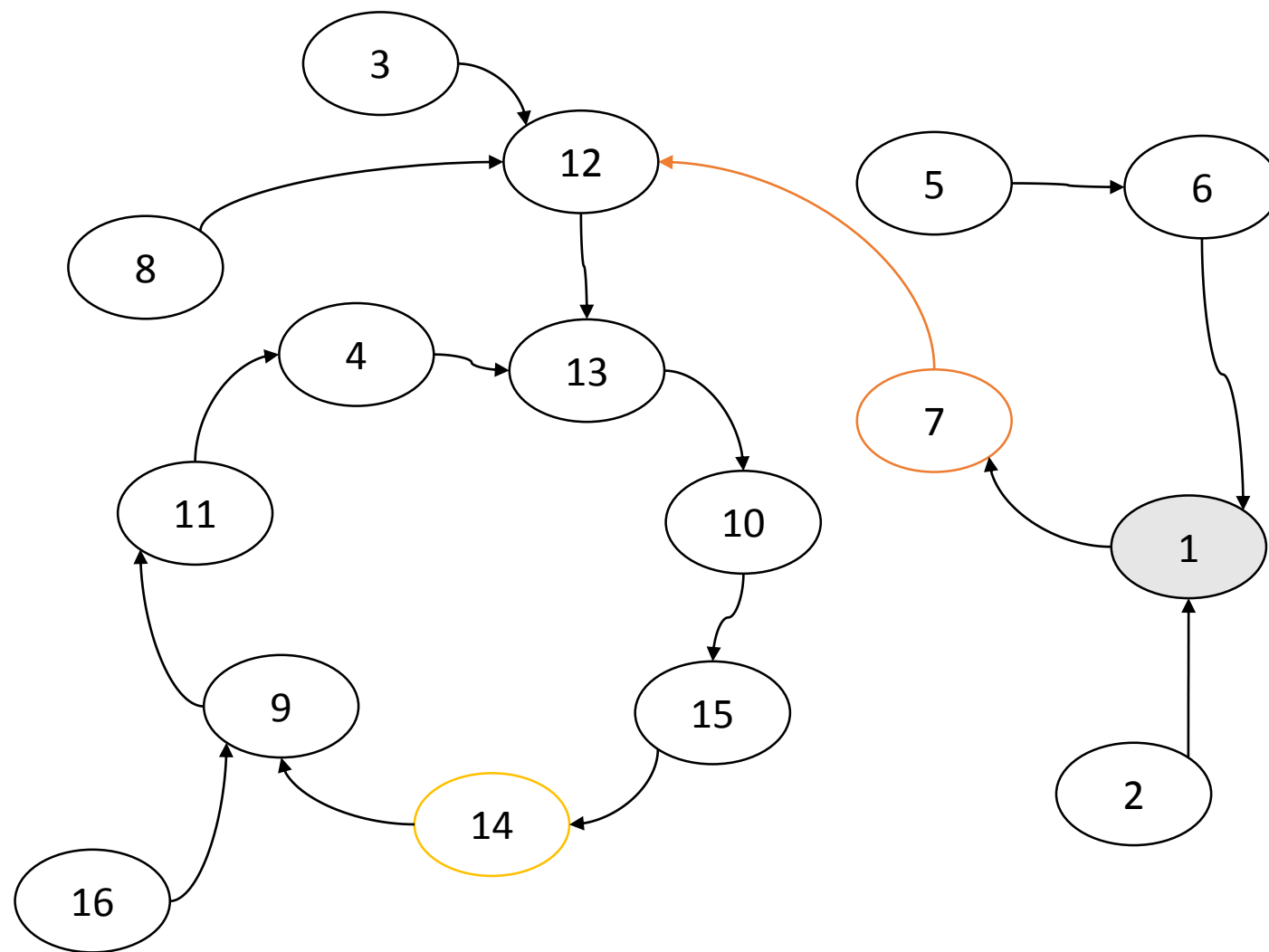
満点解法 – 場合分け(2)

- B=12の場合を考える



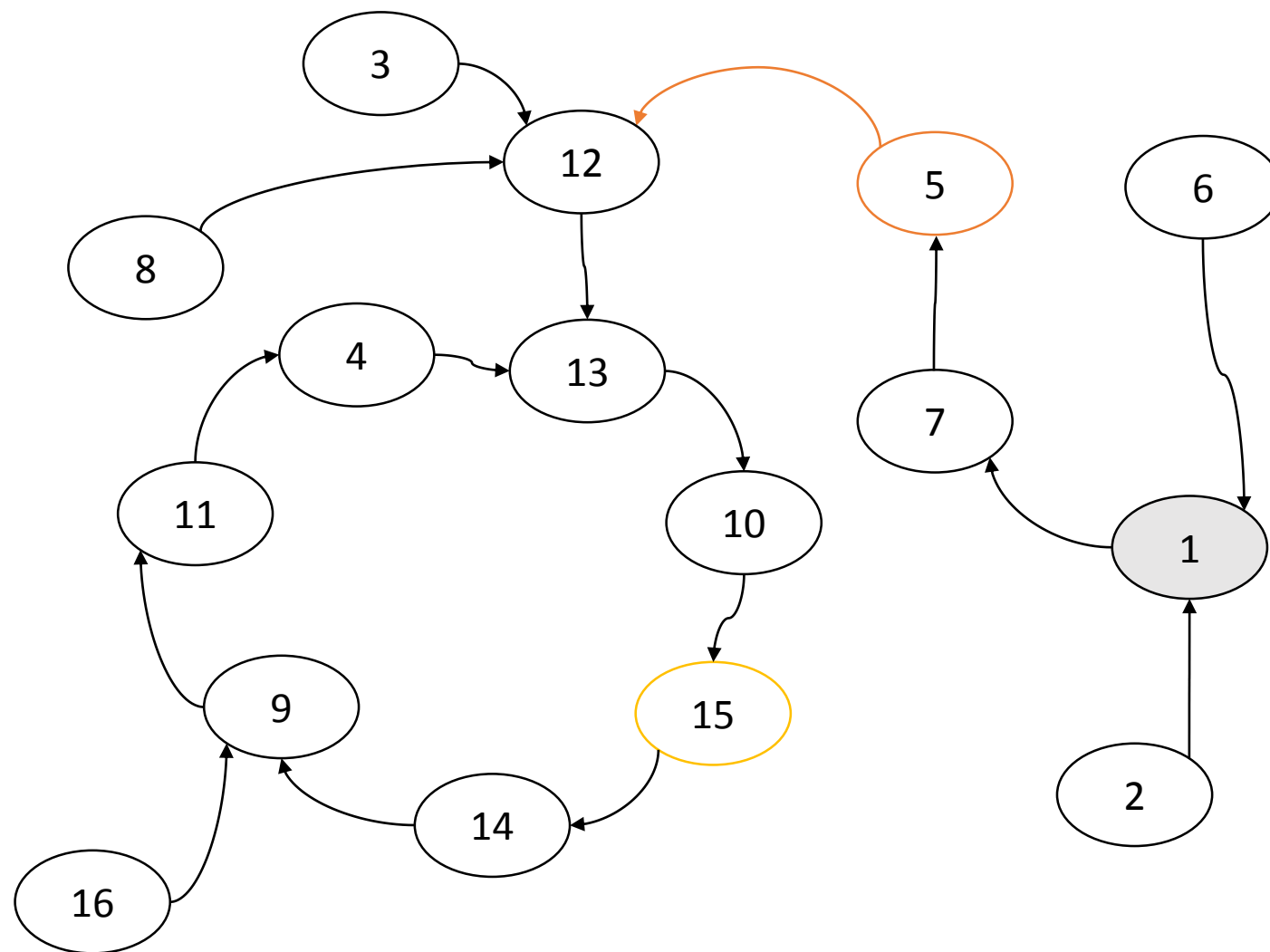
満点解法 – 場合分け(2)

- B=12の場合を考える



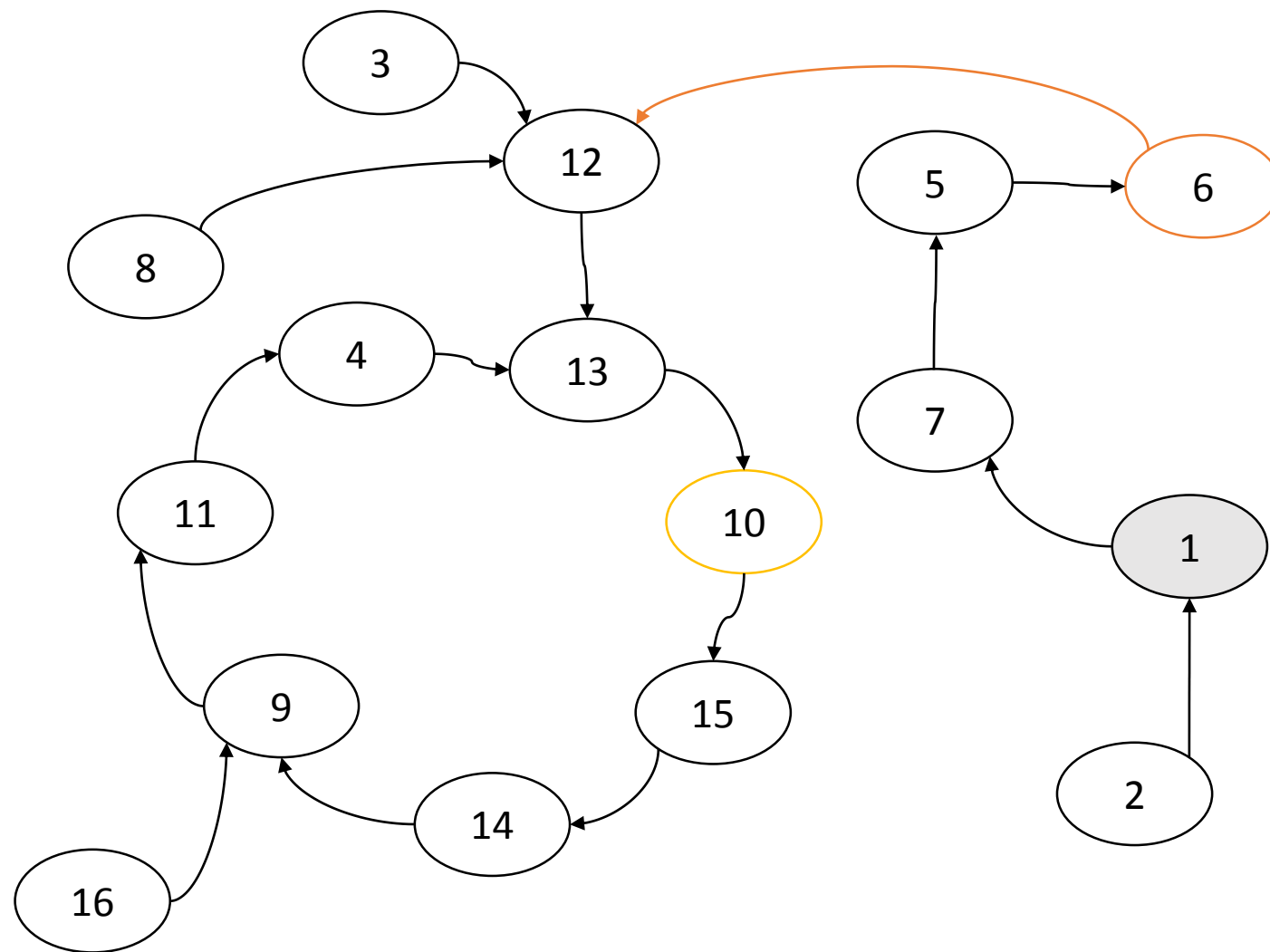
満点解法 – 場合分け(2)

- B=12の場合を考える



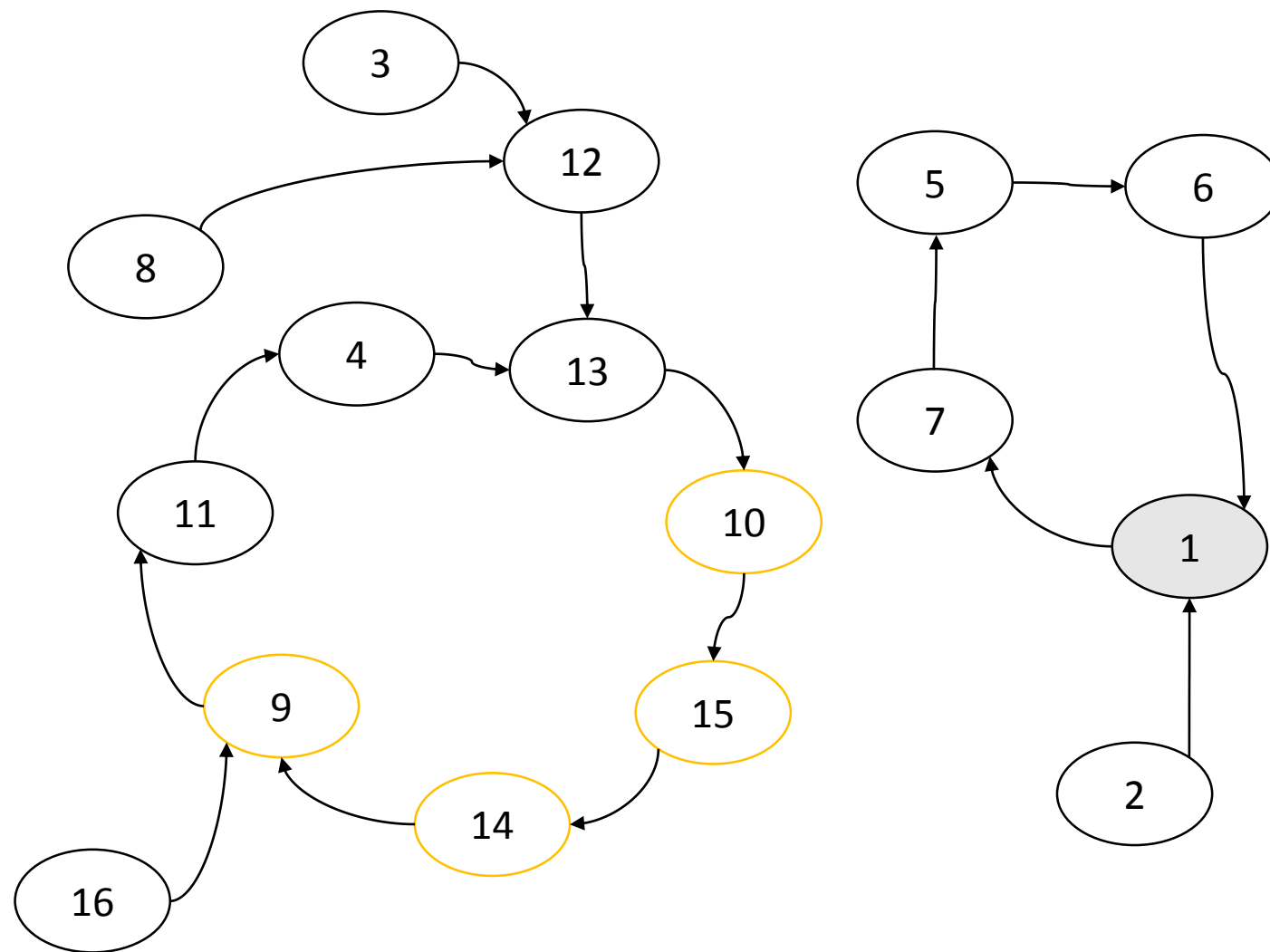
満点解法 – 場合分け(2)

- B=12の場合を考える



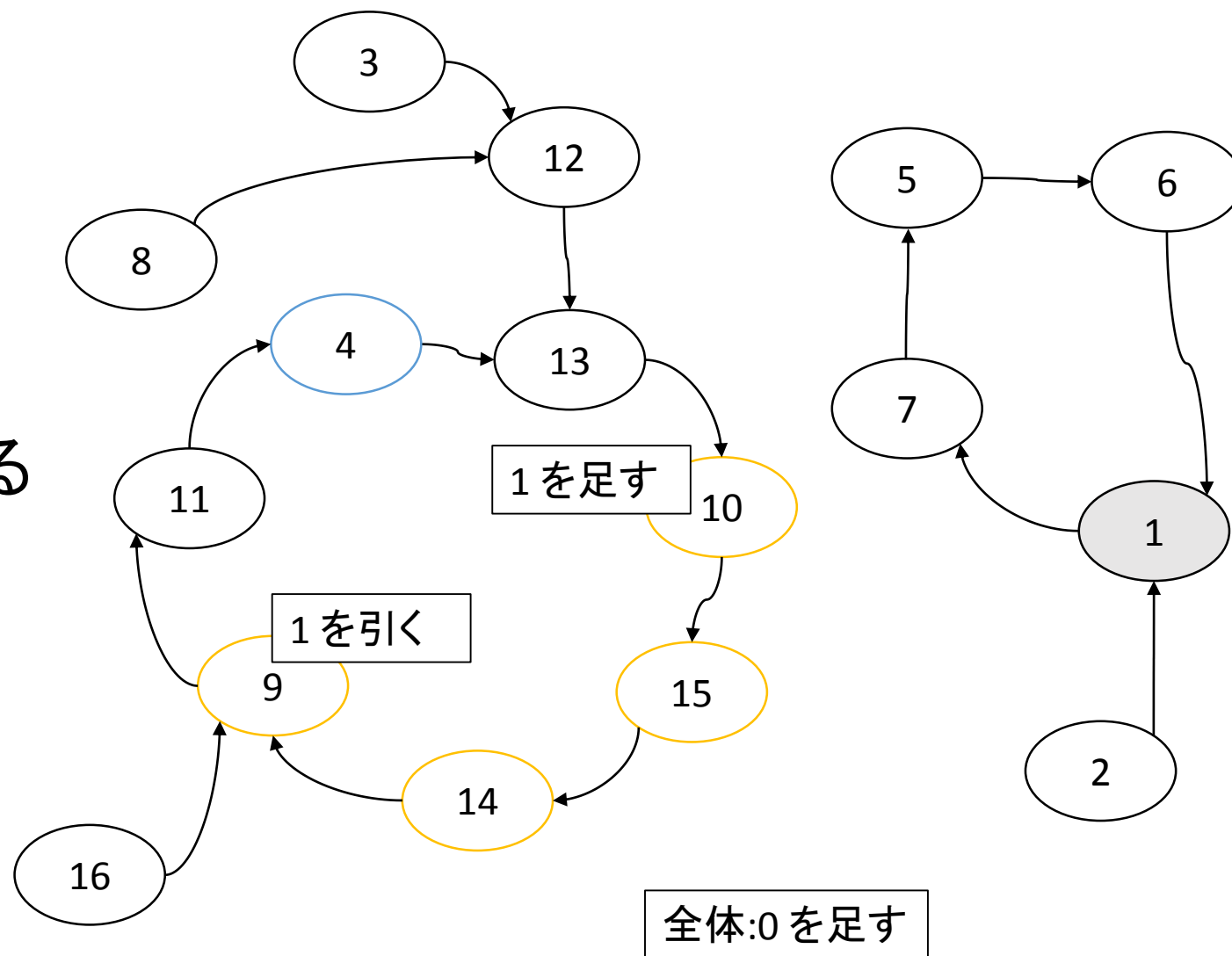
満点解法 – 場合分け(2)

- B=12の場合を考える
- 頂点10,15,14,9に1ずつ足せばよいことがわかった



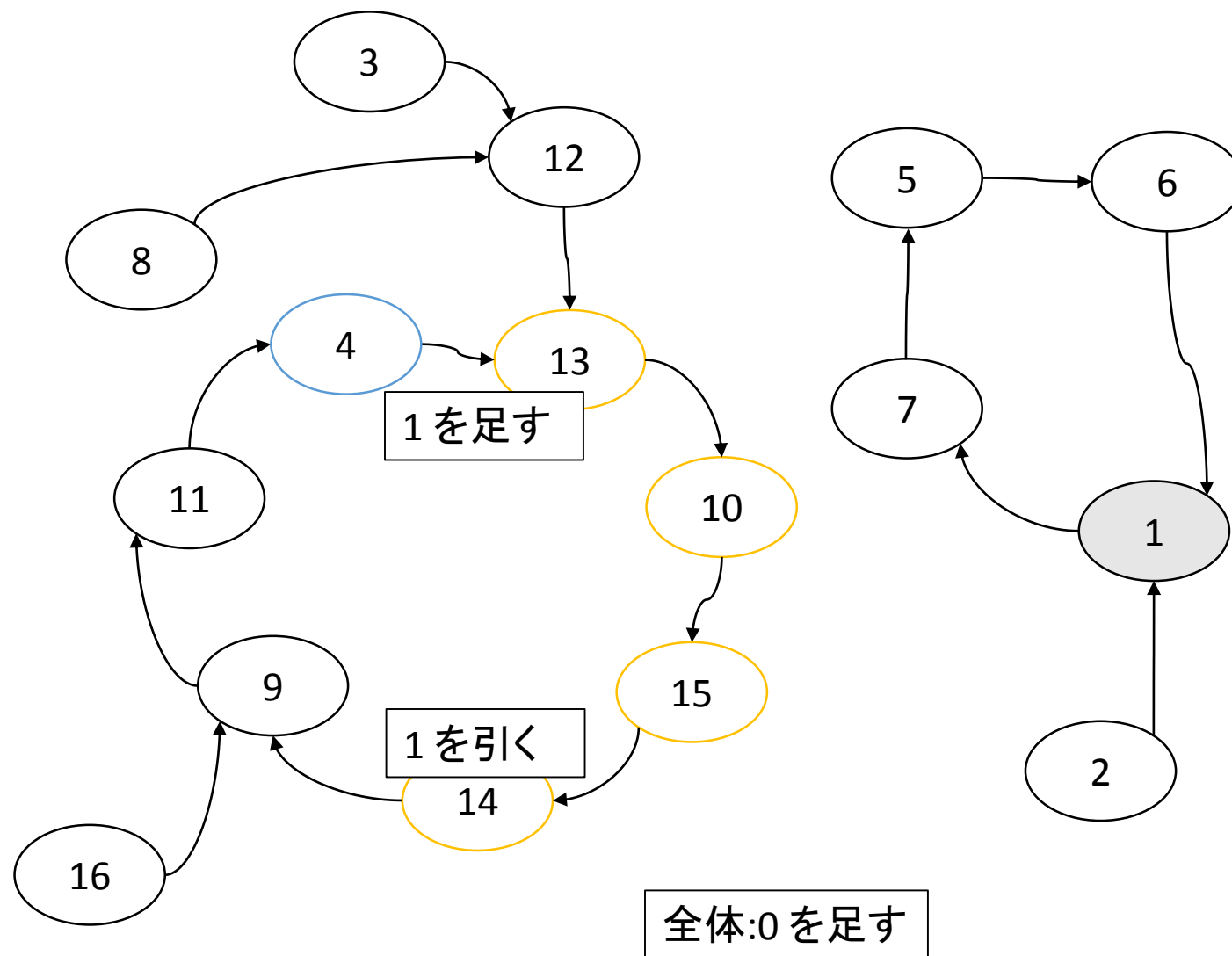
満点解法 – 場合分け(2)

- B=12の場合を考える
- 頂点10,15,14,9に1ずつ足せばよいことがわかった
- これをこのように記録する



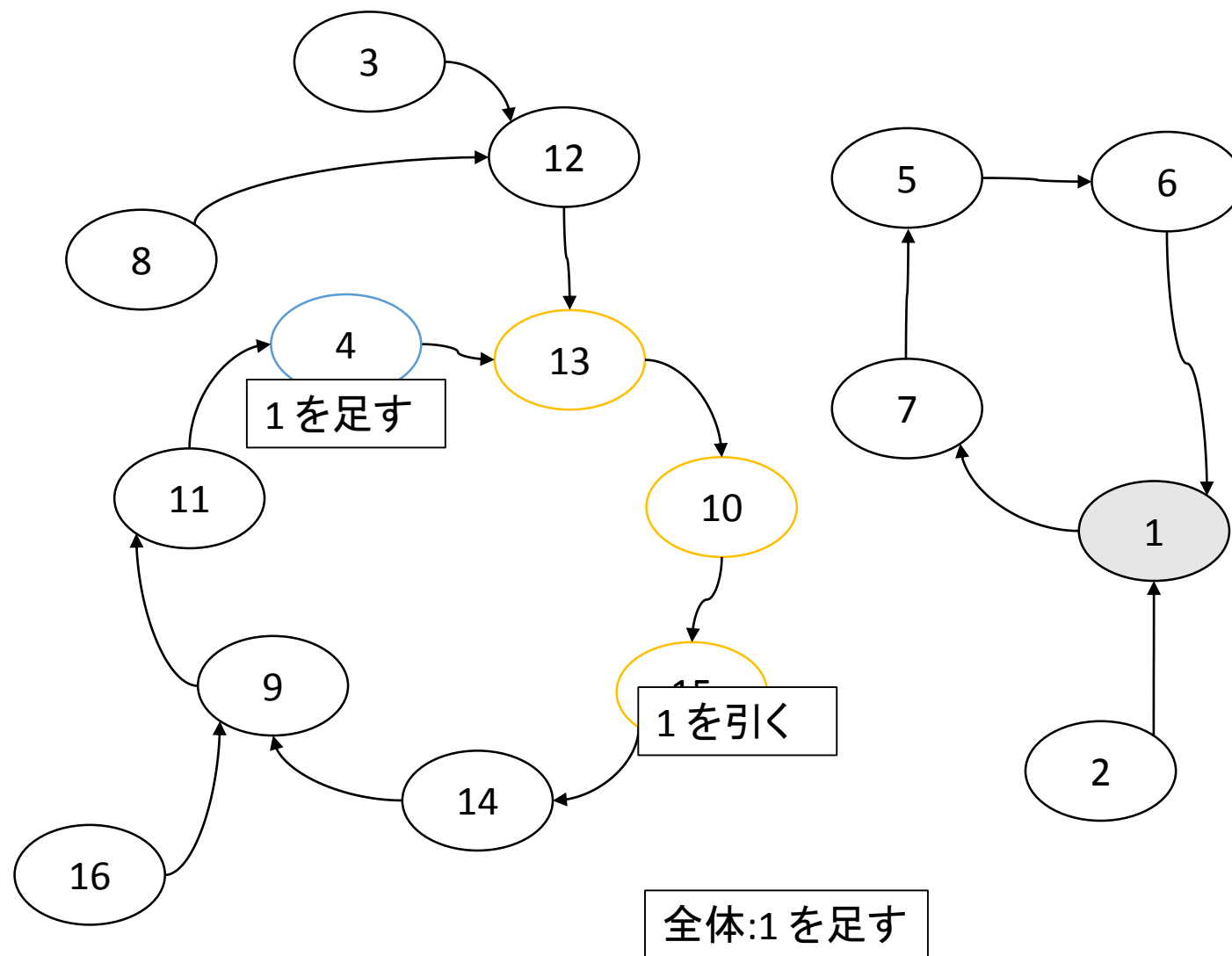
満点解法 – 場合分け(2)

- B=3,8,11の場合



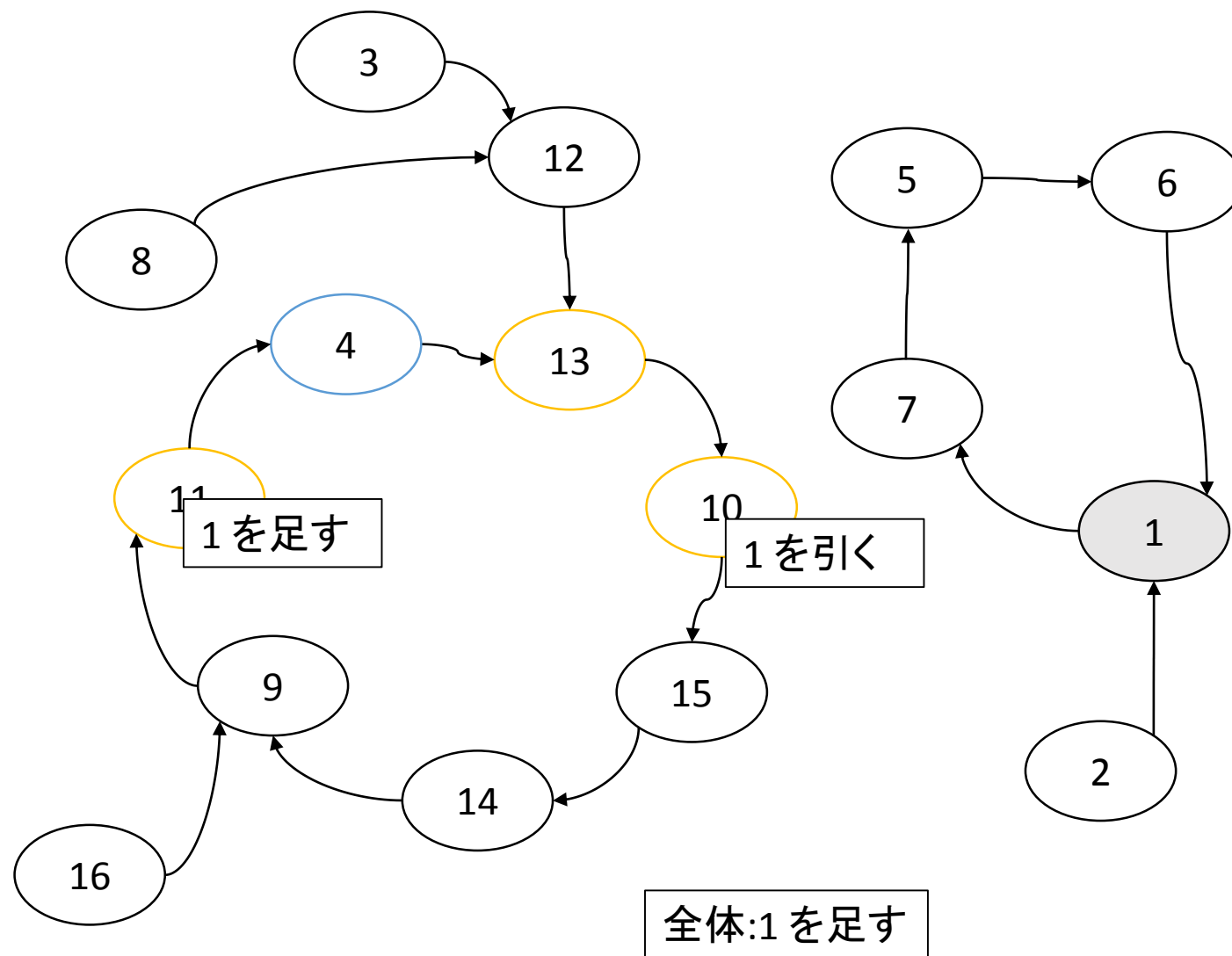
満点解法 – 場合分け(2)

- B=9の場合



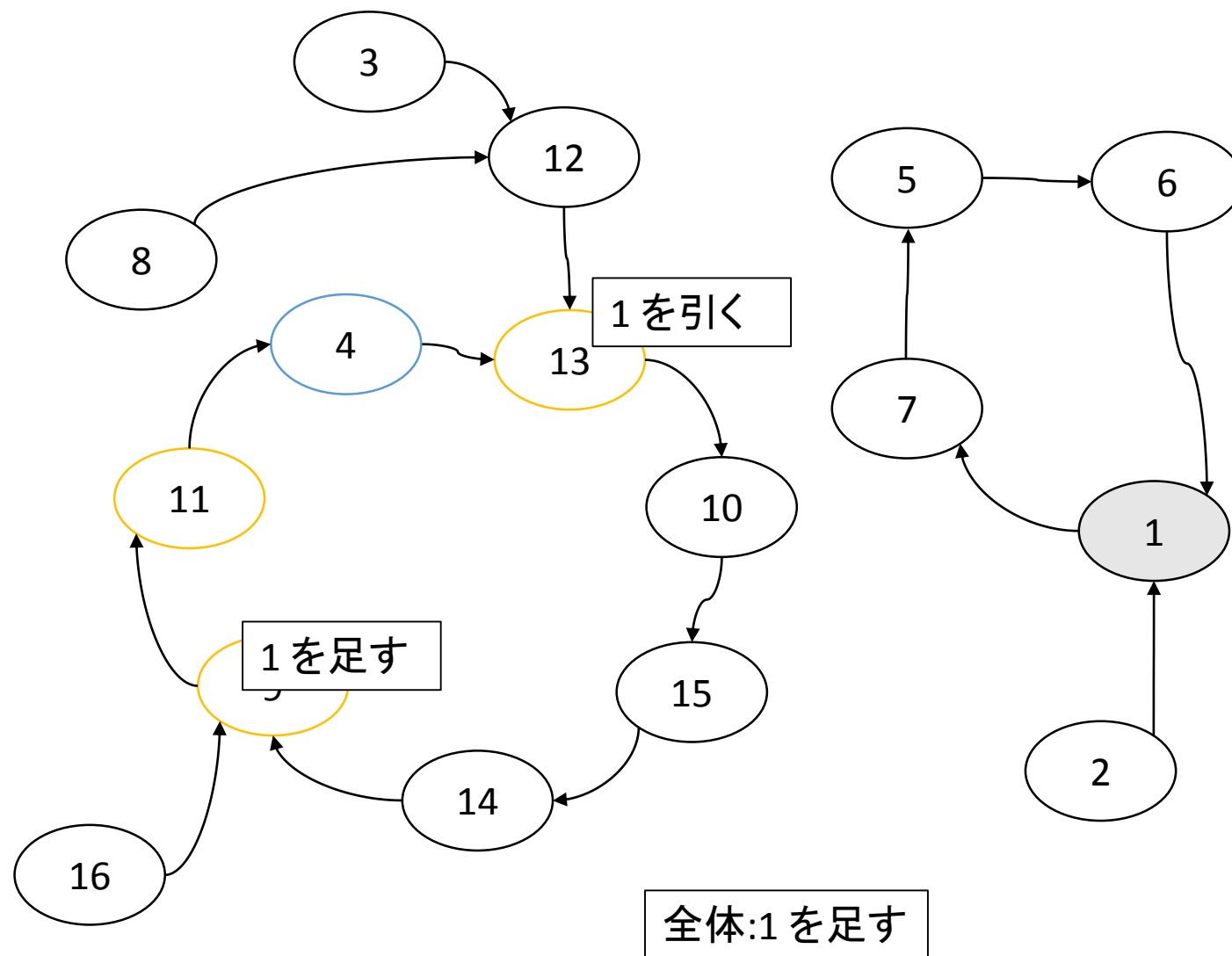
満点解法 – 場合分け(2)

- B=14,16の場合



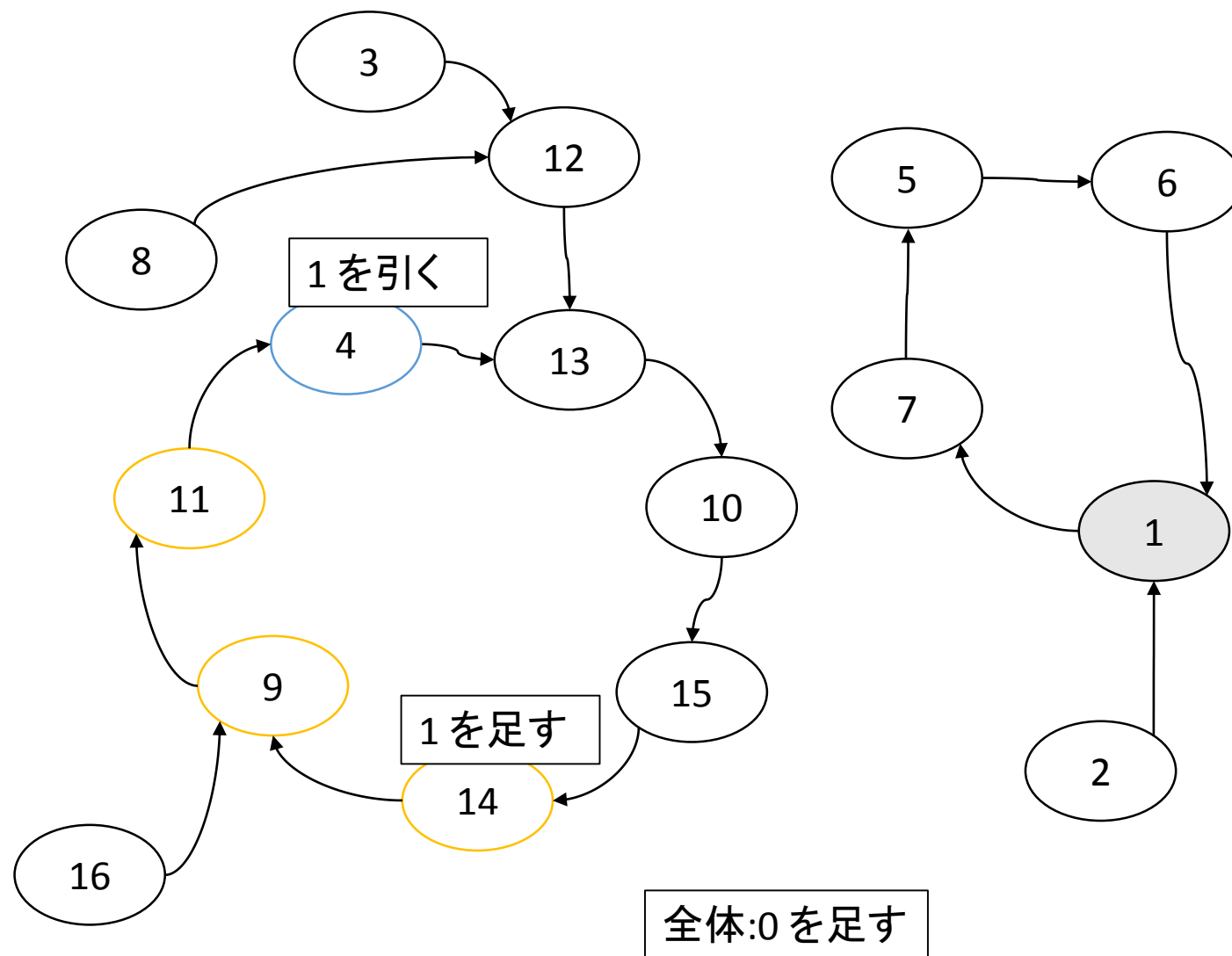
満点解法 – 場合分け(2)

- B=15の場合



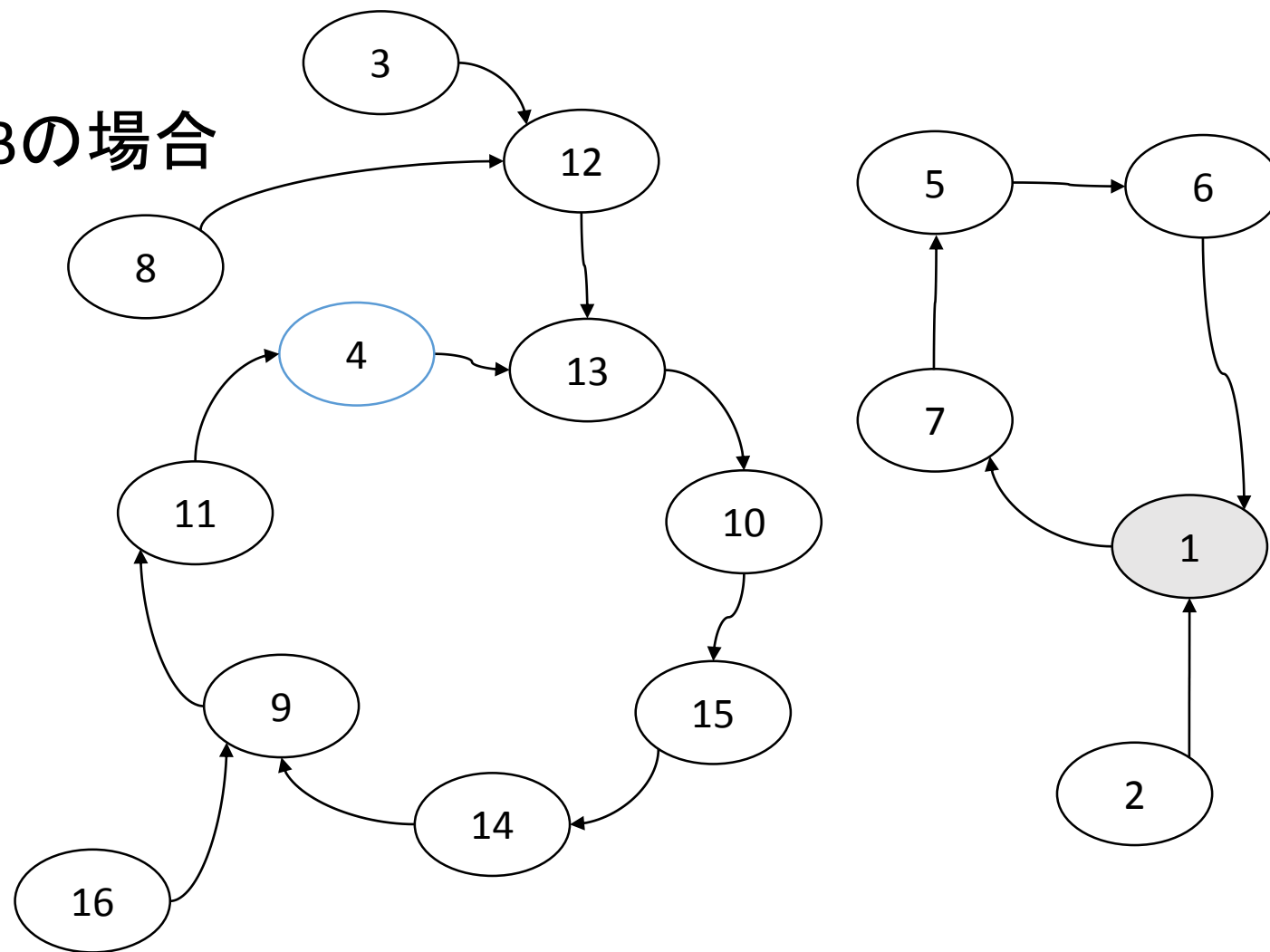
満点解法 – 場合分け(2)

- B=10の場合



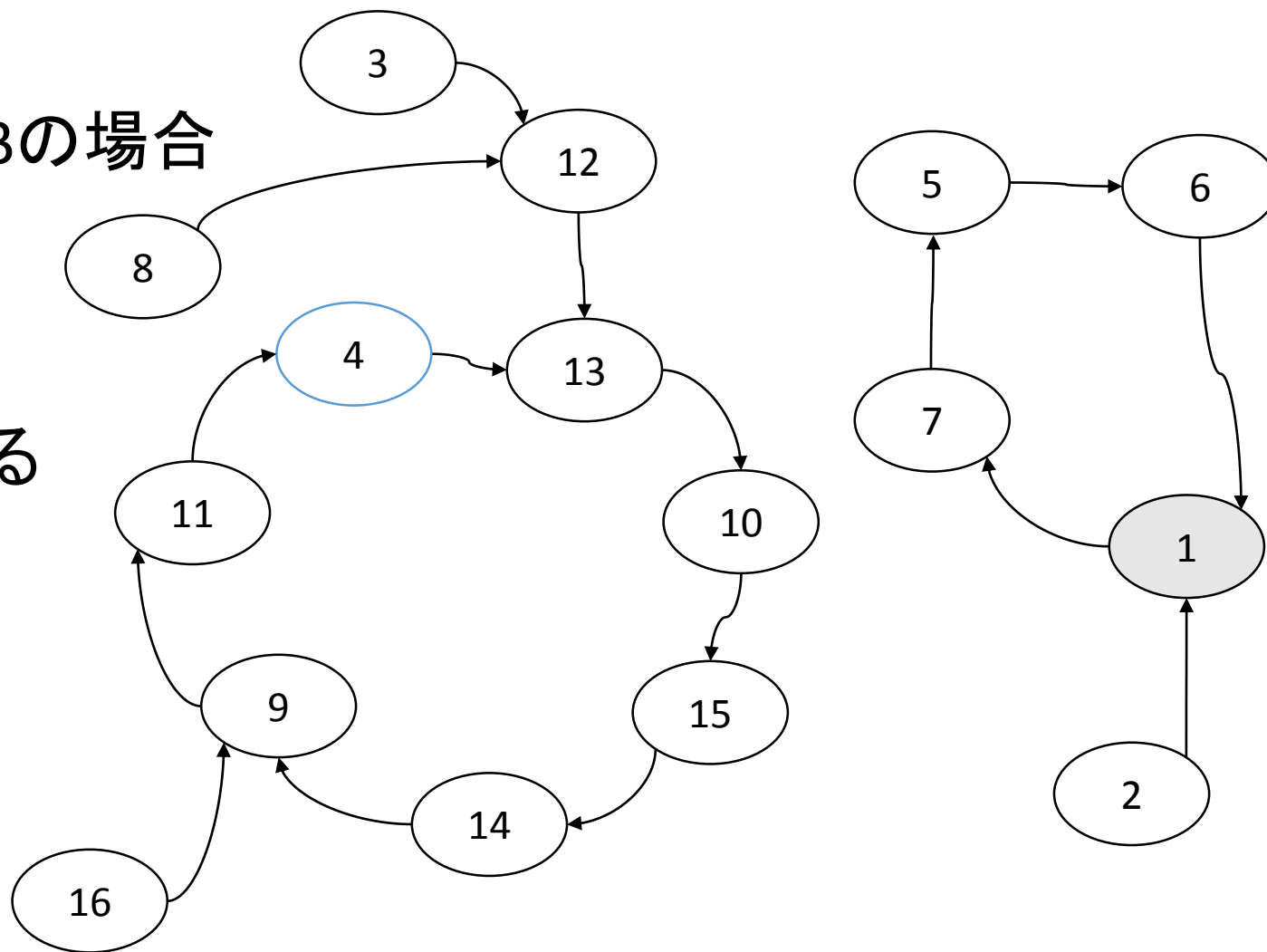
満点解法 – 場合分け(2)

- 連結成分内の各頂点がBの場合
を全て処理する



満点解法 – 場合分け(2)

- 連結成分内の各頂点がBの場合を全て処理する
- →頂点13から頂点4に向かって部分和をとると本来のカウントが得られる
- (ループ全体に対するカウントを忘れずに)

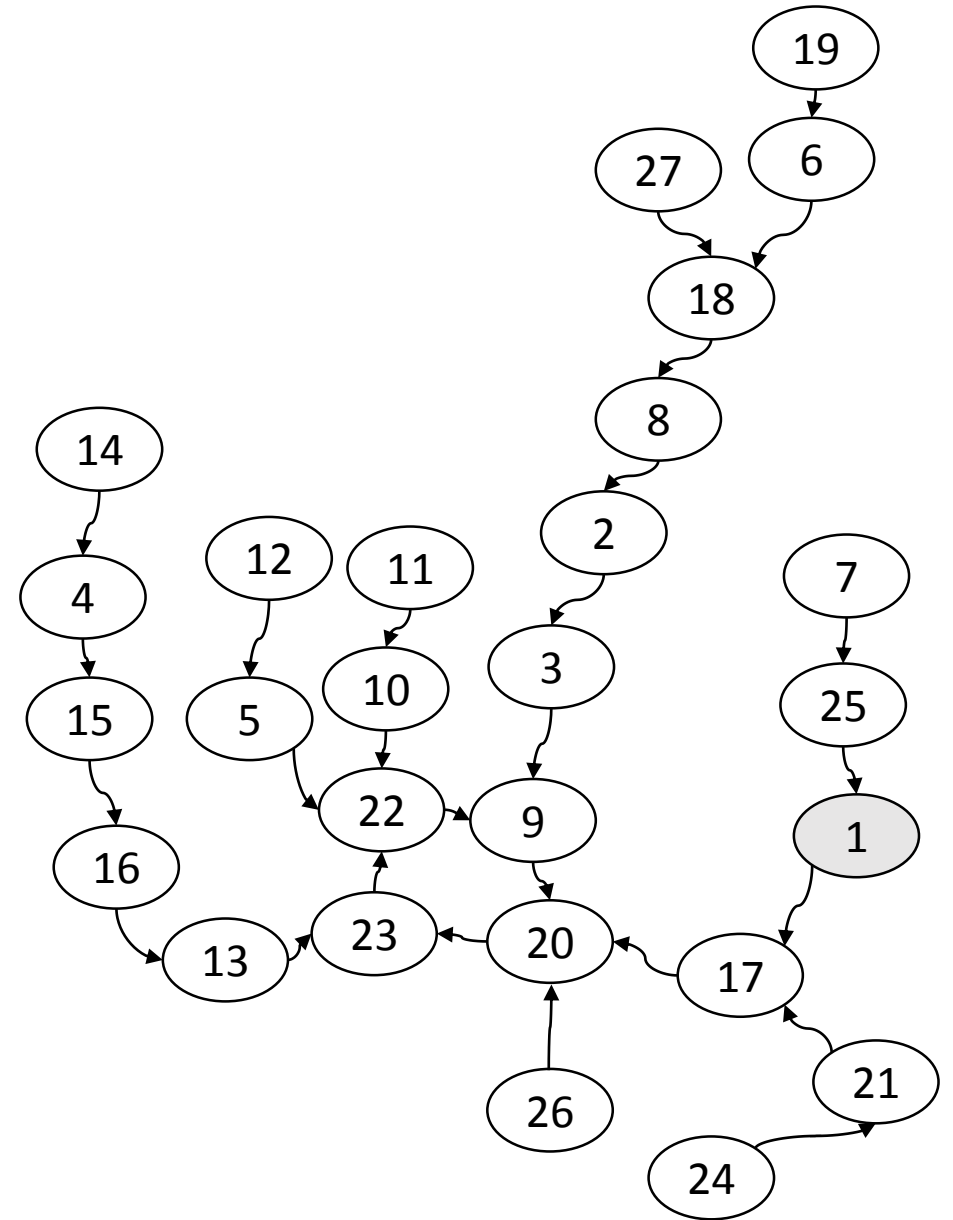


満点解法

- 以下、Bが頂点1と同じ連結成分に属する場合を考える

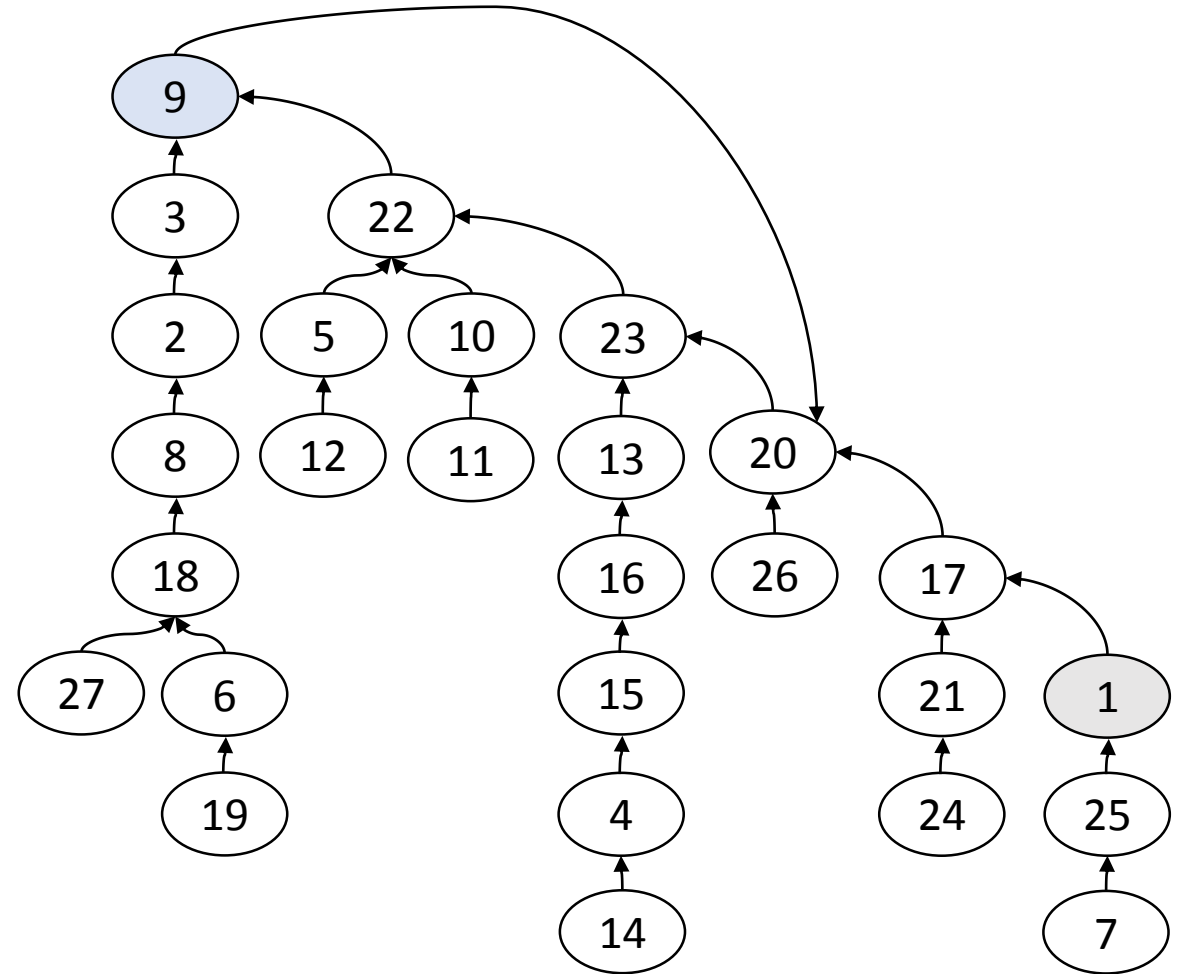
満点解法 – 場合分け(3)

- ここで使うグラフ($N=27$, $K=35$)



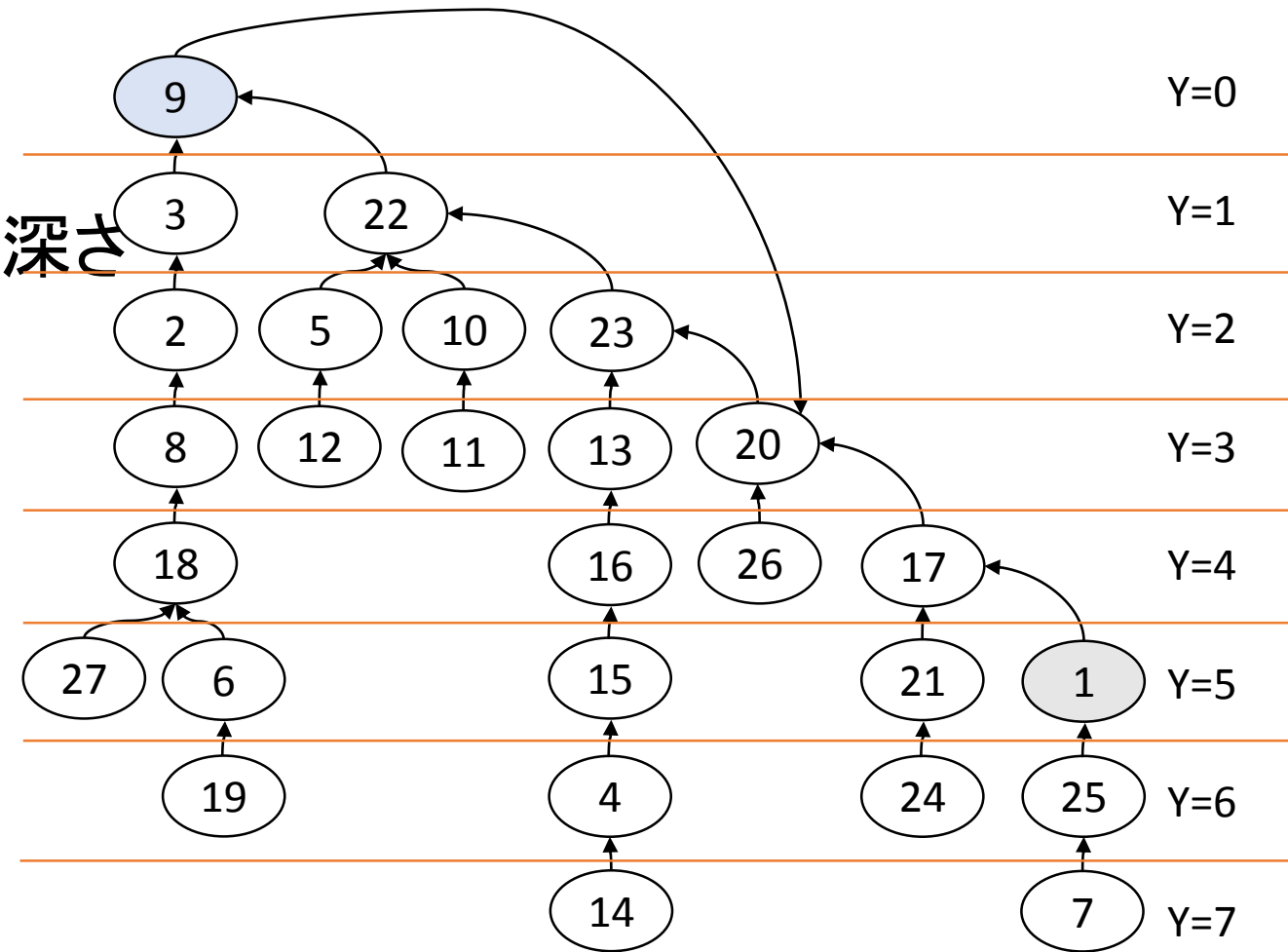
満点解法 – 場合分け(3)

- 根を決める
- 根 = 頂点1から辿ってはじめて
ループに入る直前の頂点



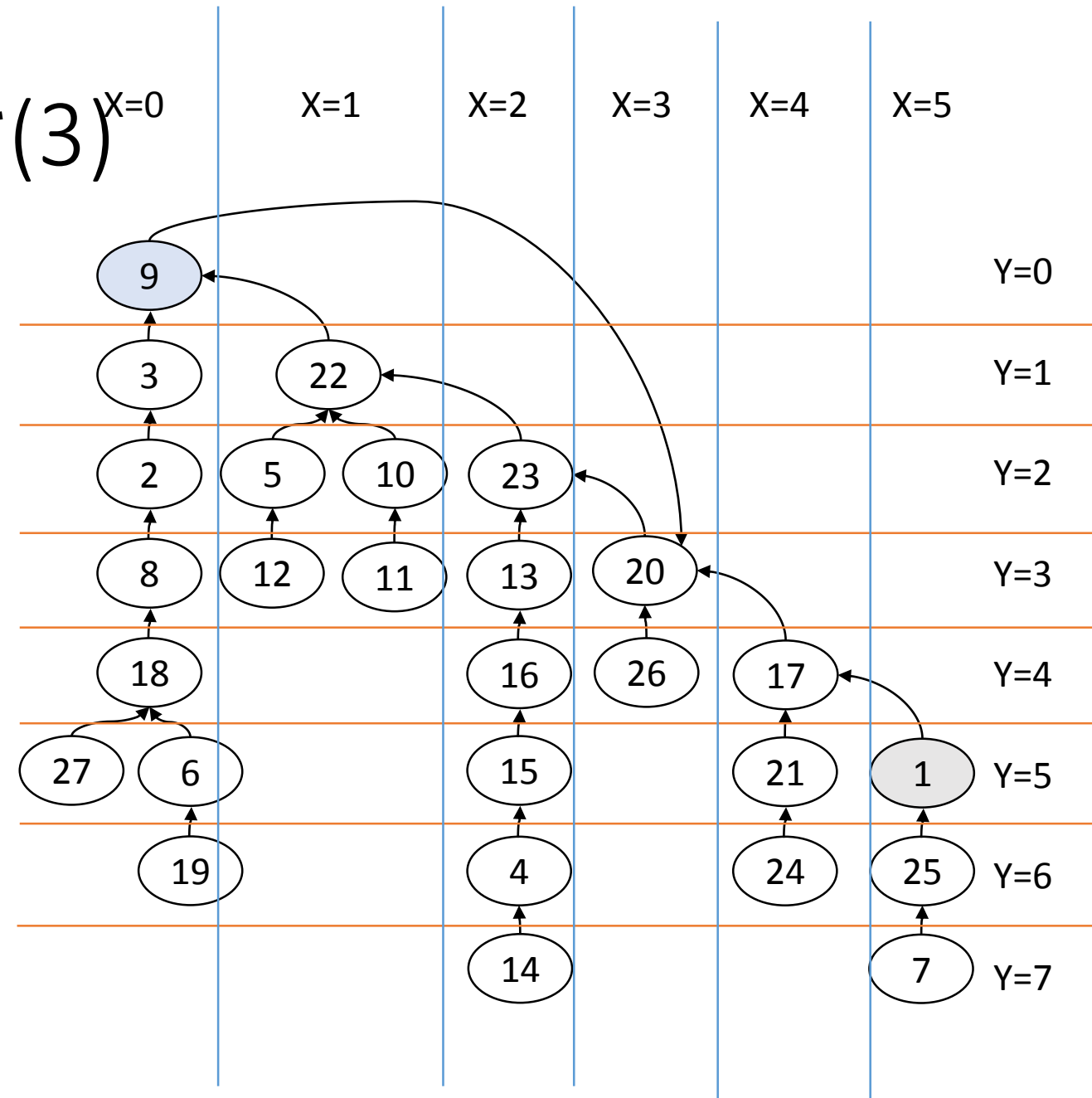
満点解法 – 場合分け(3)

- Y座標を定める
- Y座標 : これを木として見た時の深さ



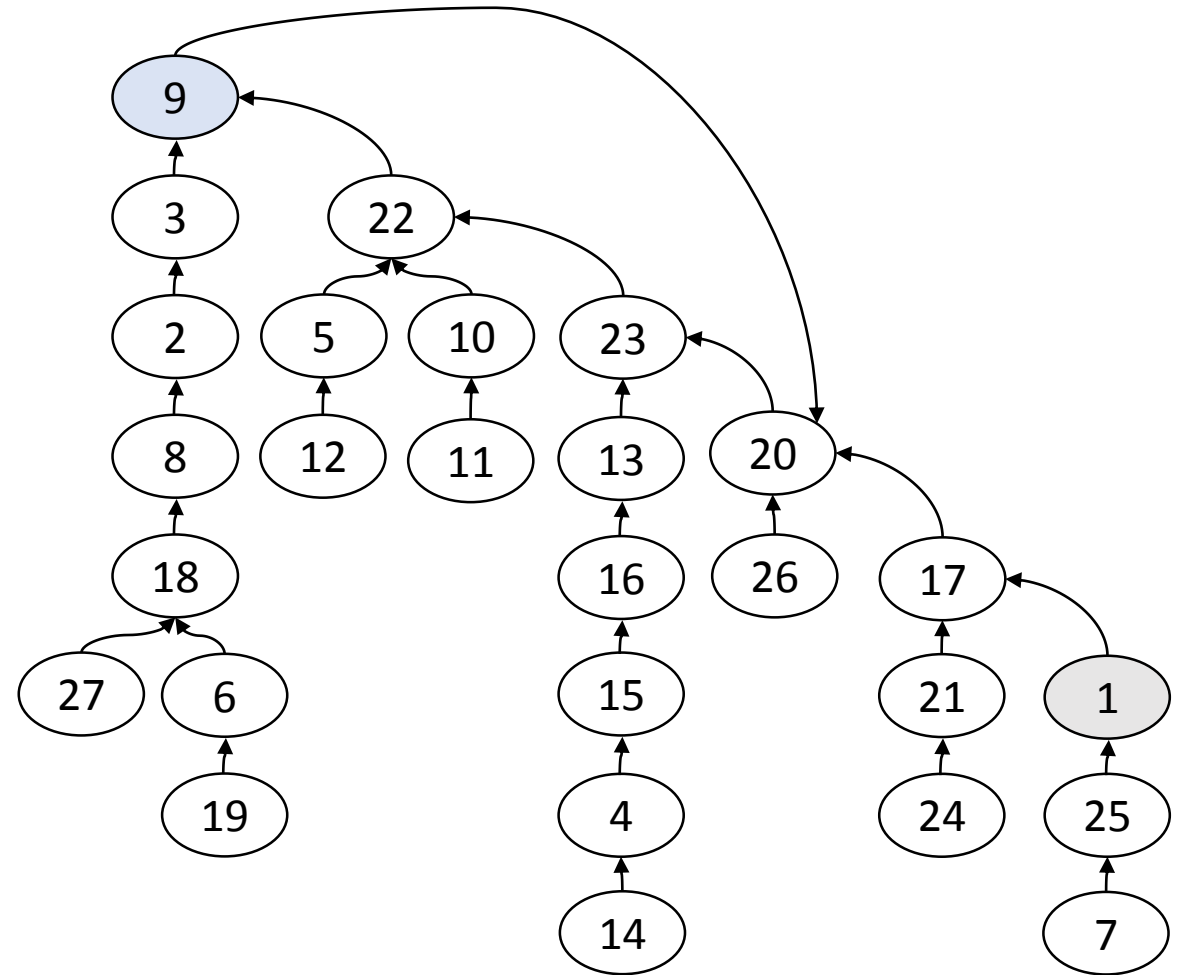
満点解法 – 場合分け(3)^{X=0}

- X座標を定める
- X座標：
頂点1とのLCAのY座標



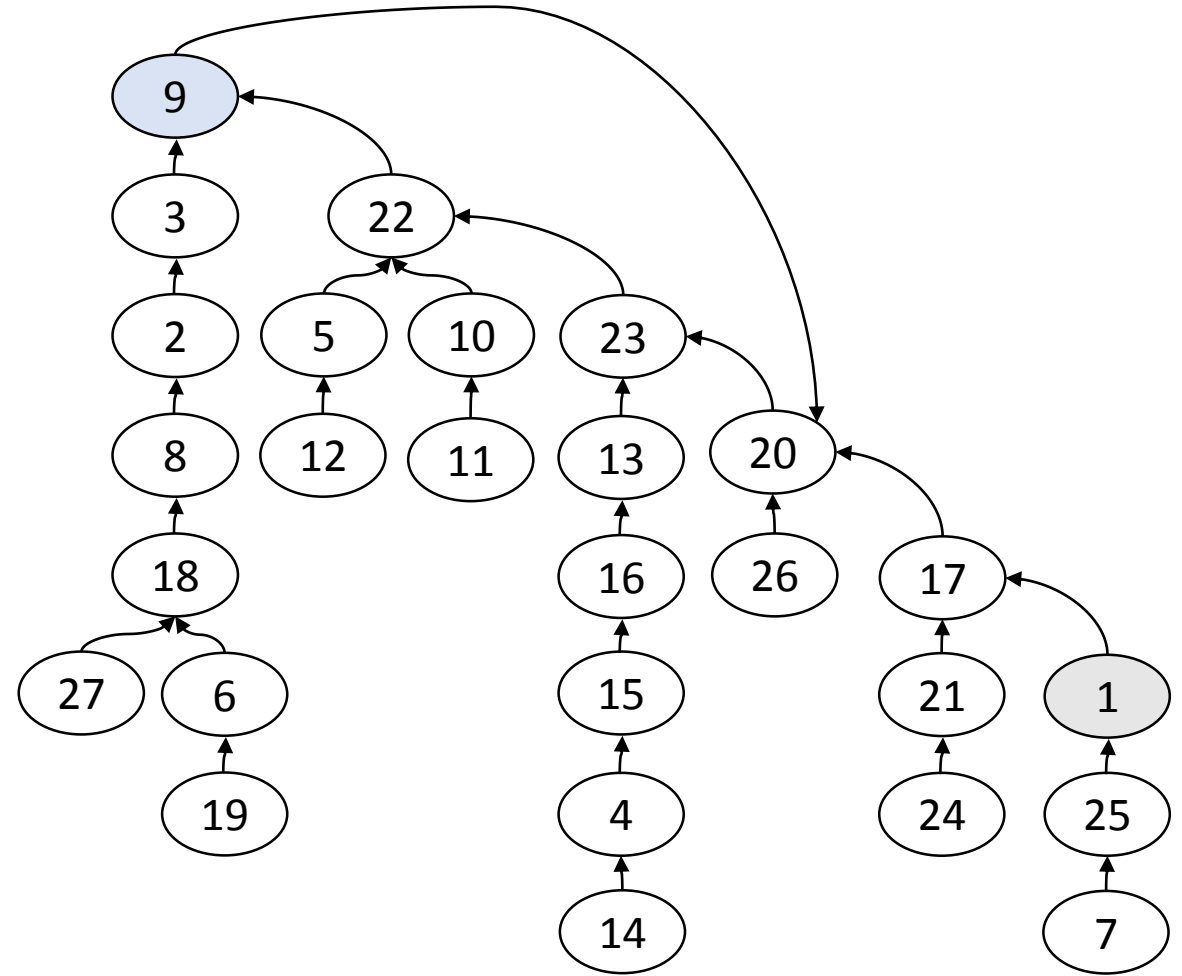
満点解法 – 場合分け(3)

- Dを次の値とする:
(BのY座標) – (AのY座標)
- Lを次の値とする:
(初期状態のループ長)
- この例ではL=4
(DはA,Bの選び方に依存する値)



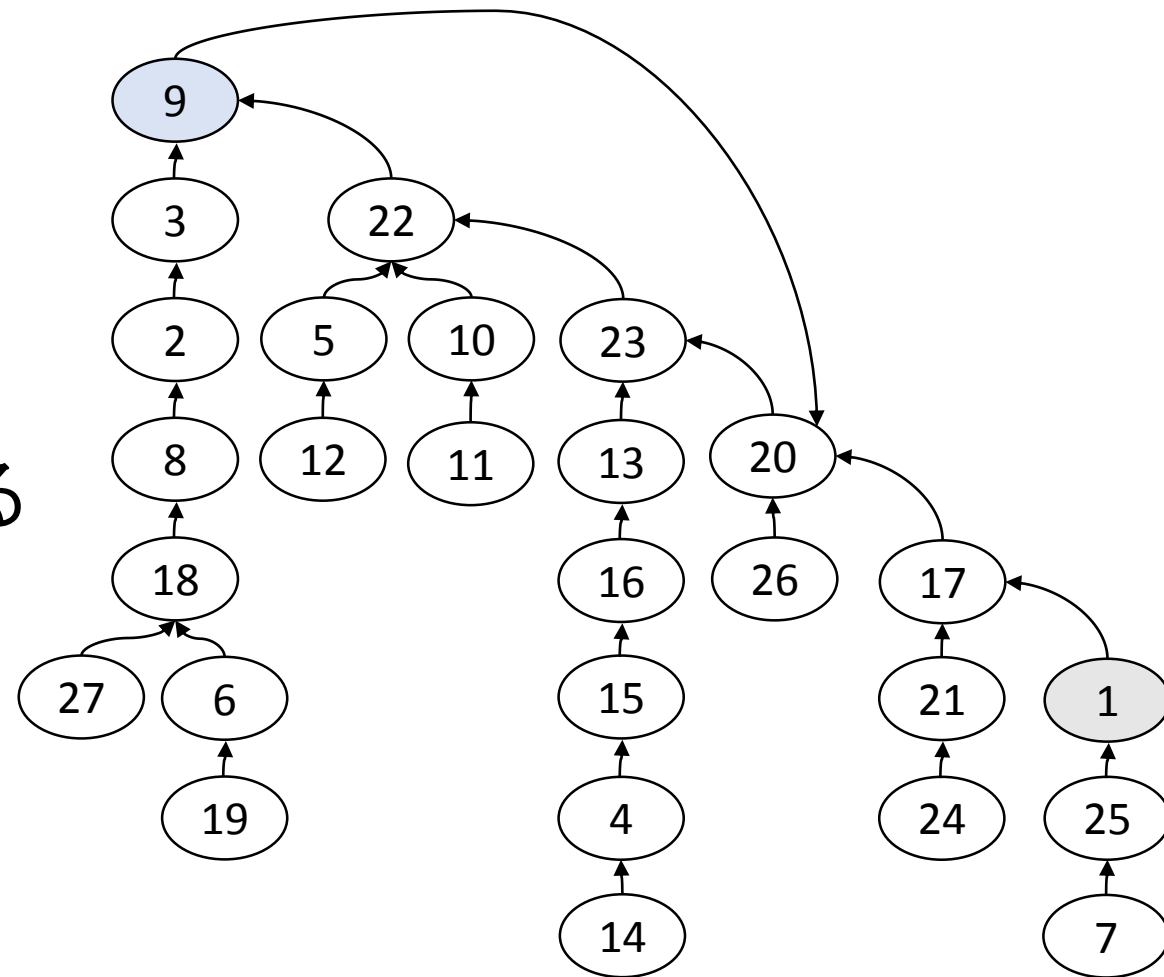
満点解法 – 場合分け(3)

- AがBの祖先でない場合を
まず考える



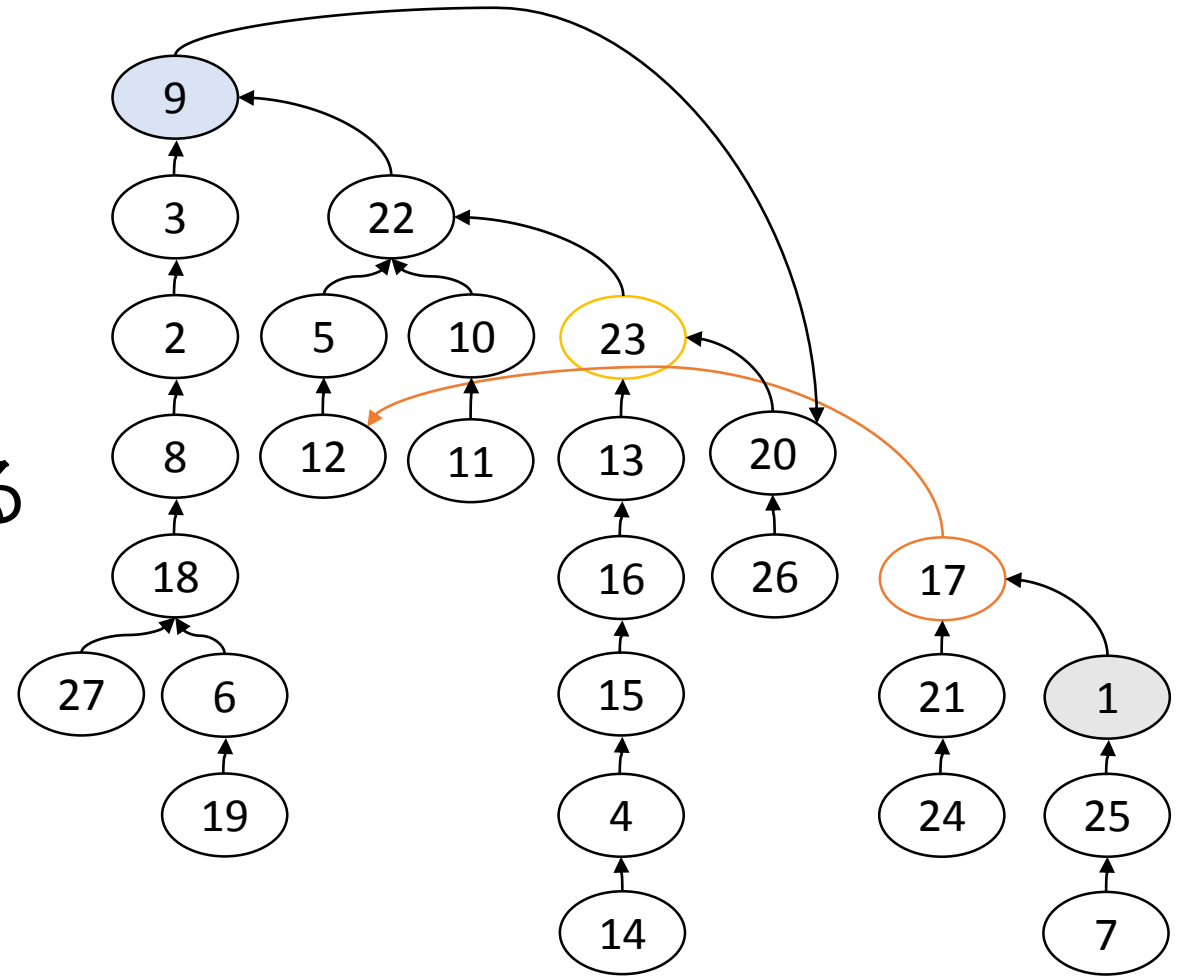
満点解法－場合分け(3)

- AがBの祖先でない場合を
まず考える
- 特に、Aがサイクルに
含まれない場合
(この例ではA=1,17)をまず考える



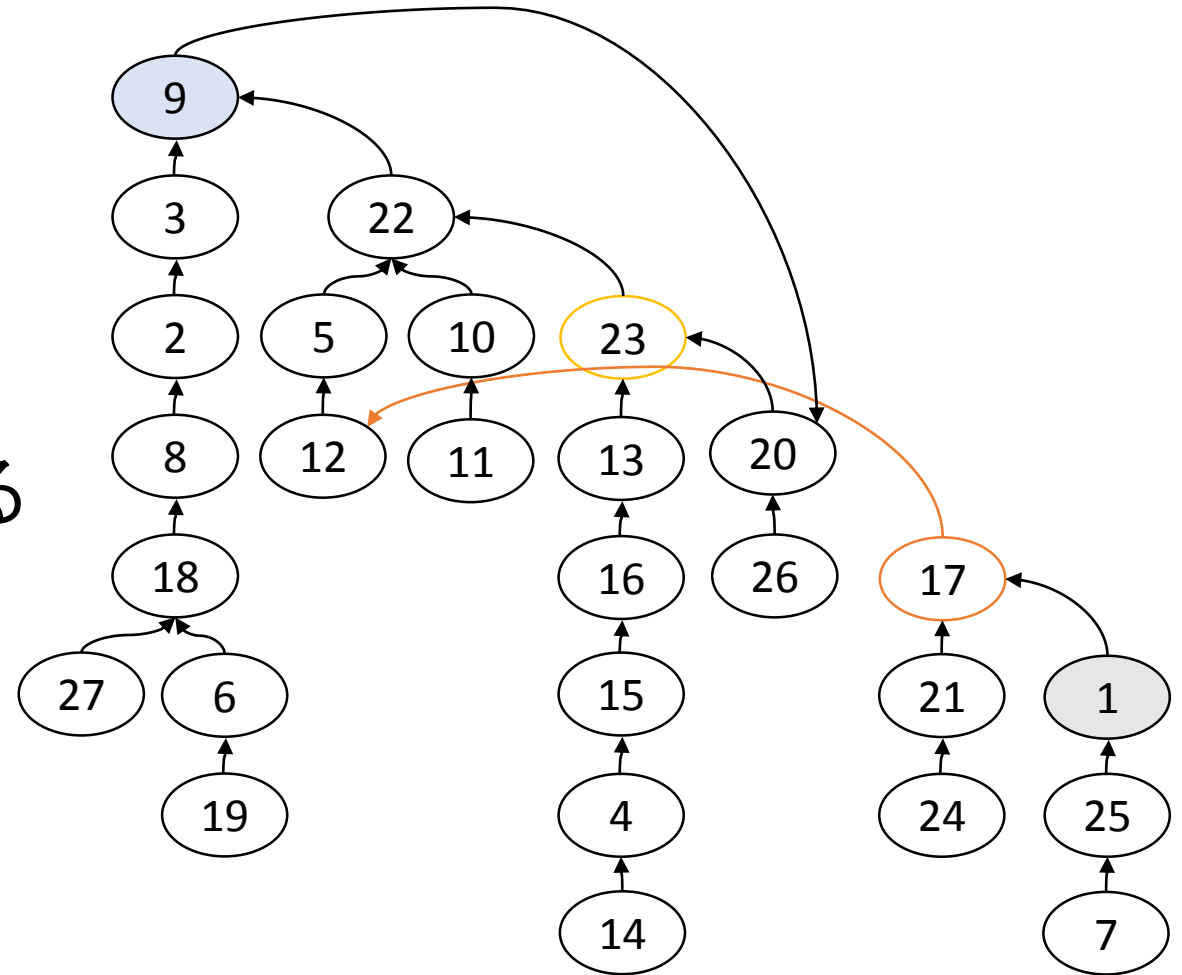
満点解法－場合分け(3A)

- AがBの祖先でない場合を
まず考える
- 特に、Aがサイクルに
含まれない場合
(この例ではA=1,17)をまず考える
- このときはサイクル長は一定



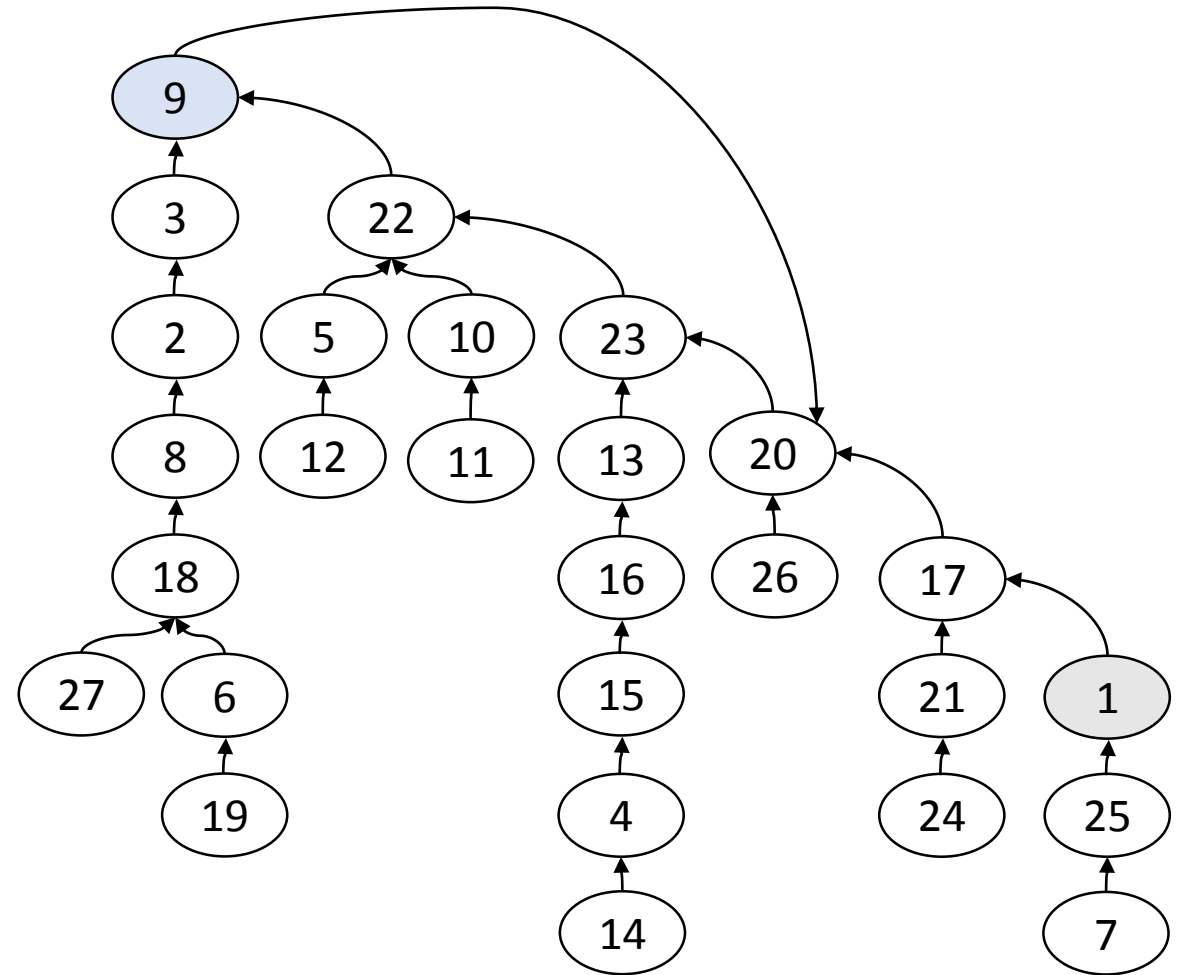
満点解法 – 場合分け(3A)

- AがBの祖先でない場合を
まず考える
- 特に、Aがサイクルに
含まれない場合
(この例ではA=1,17)をまず考える
- このときはサイクル長は一定
- 場合分け(2)と同様に処理可能



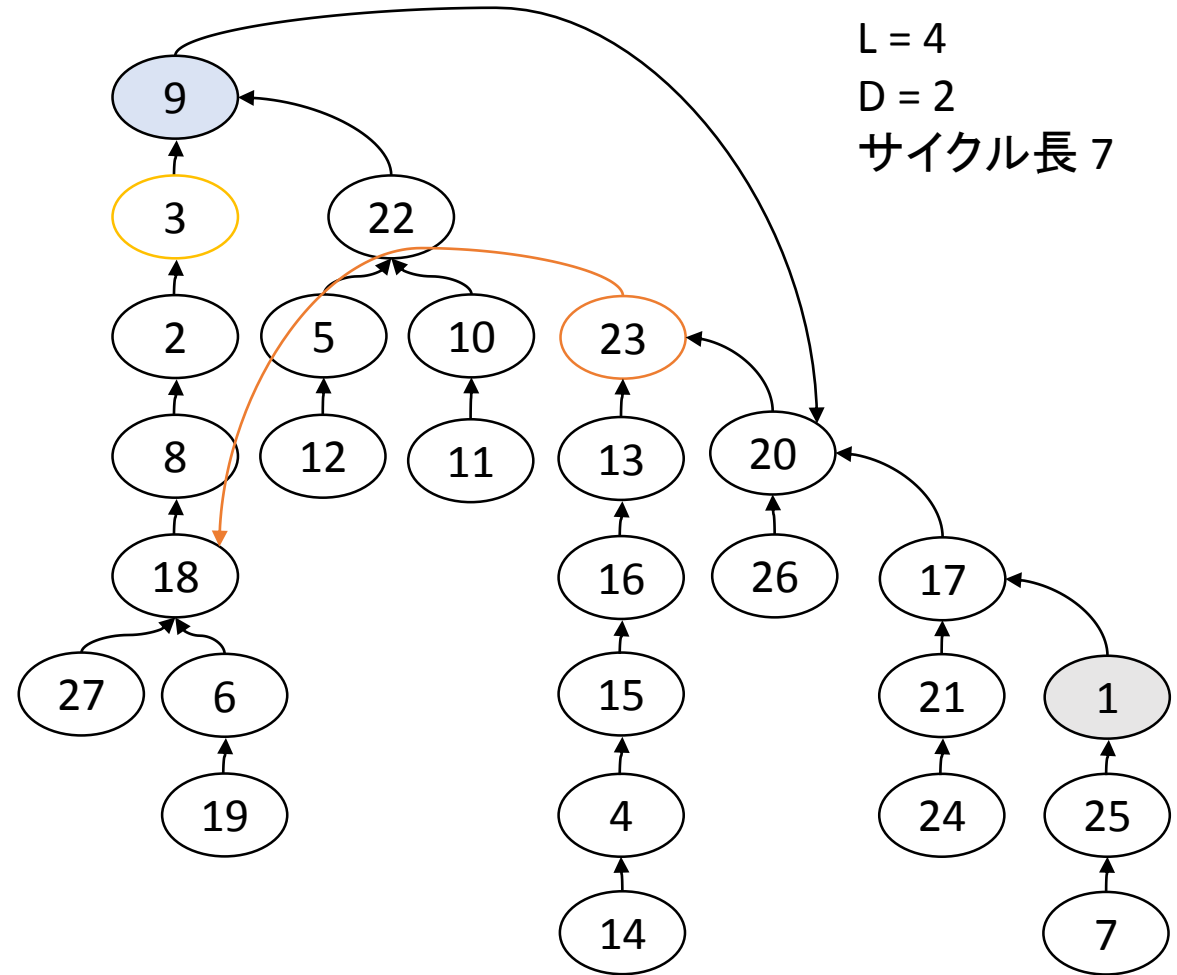
満点解法 – 場合分け(3)

- AがBの祖先でない場合を
まず考える
- 次:Aがサイクルに含まれる場合
(この例ではA=20,23,22,9)



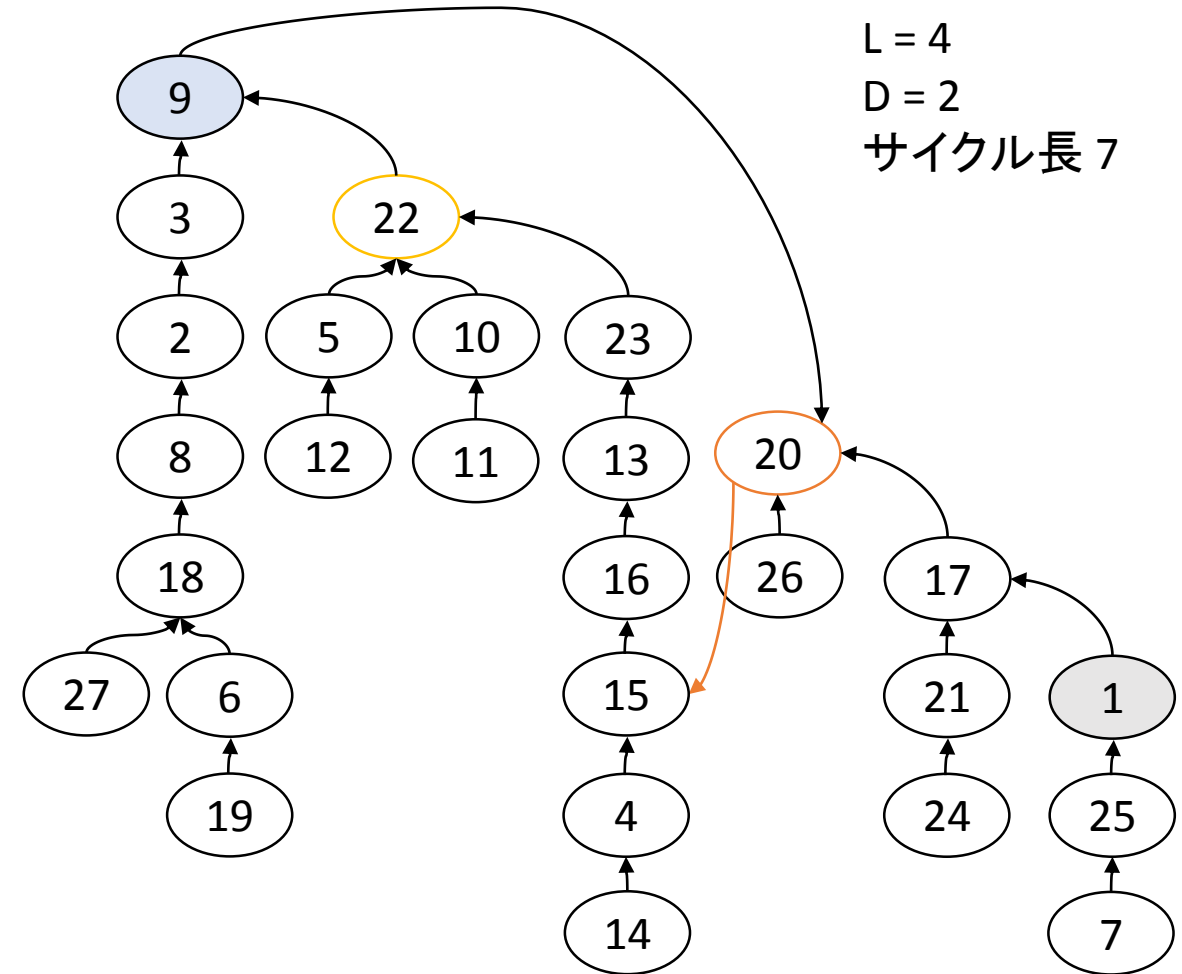
満点解法－場合分け(3B)

- AがBの祖先でない場合を
まず考える
- 次:Aがサイクルに含まれる場合
(この例では $A=20, 23, 22, 9$)
- このとき、できるサイクル長は
 $L+D+1$ である。



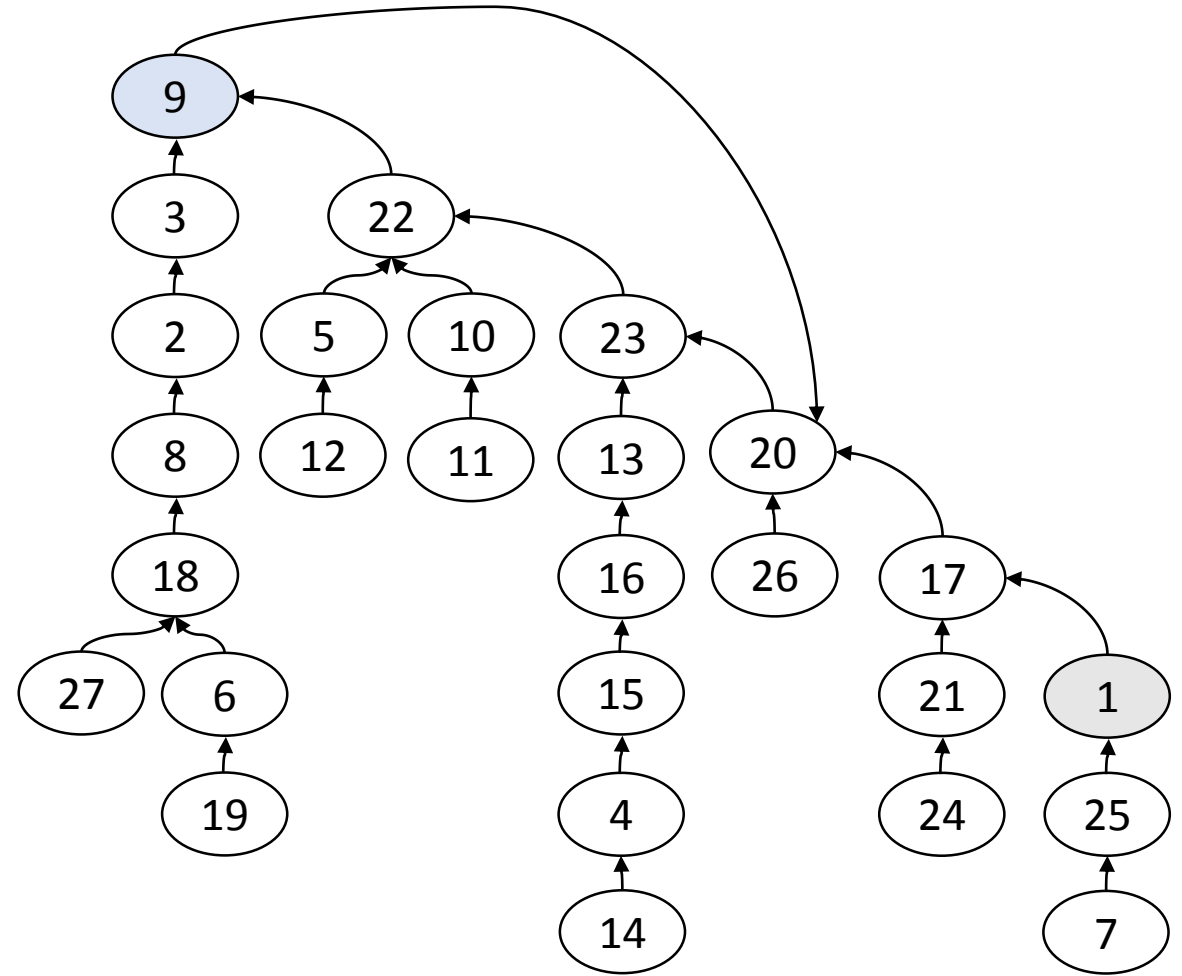
満点解法 – 場合分け(3B)

- AがBの祖先でない場合を
まず考える
- 次:Aがサイクルに含まれる場合
(この例ではA=20,23,22,9)
- このとき、できるサイクル長は
 $L+D+1$ である。
- D一定のとき、到達先の
Y座標は一定...？



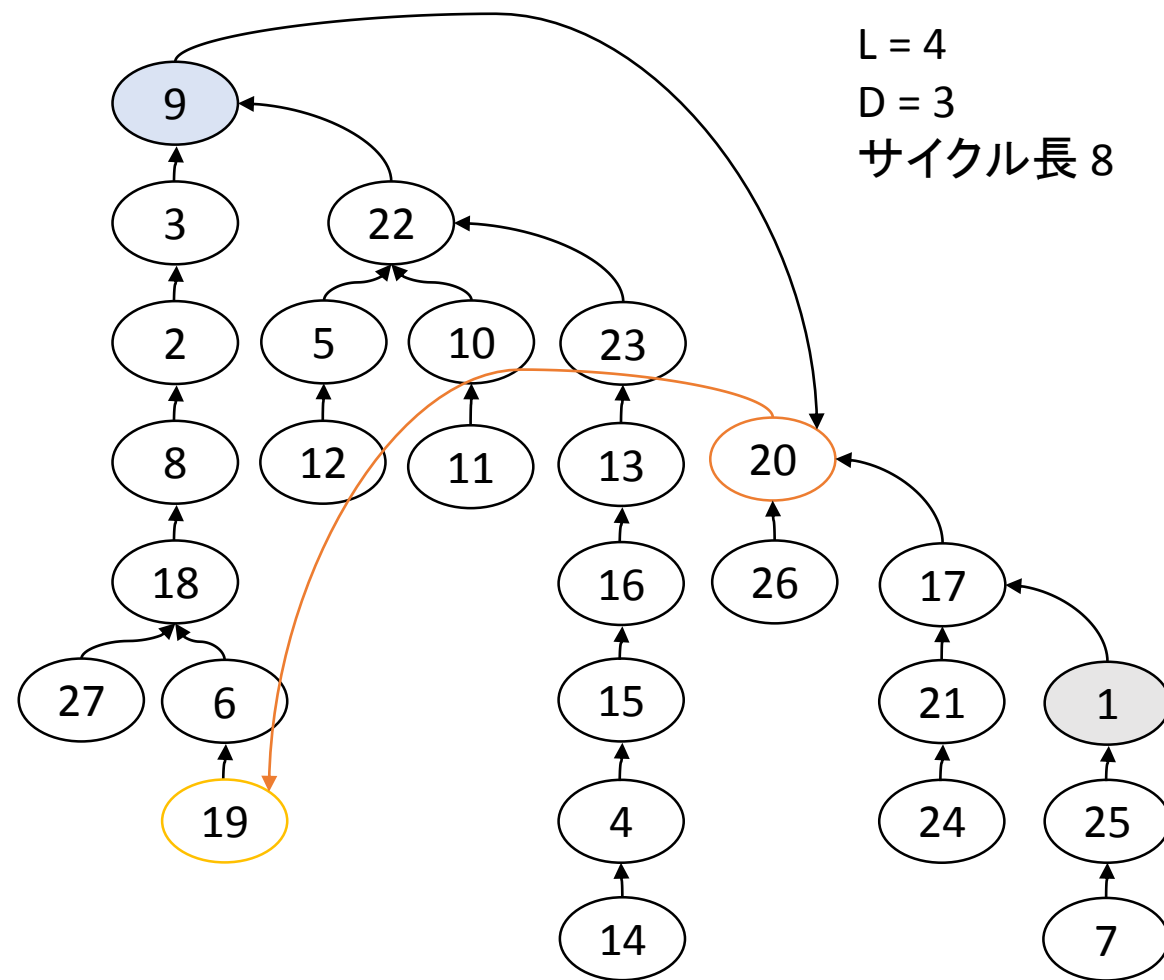
満点解法 – 場合分け(3B)

- D一定のとき、到達先のY座標は一定ではない



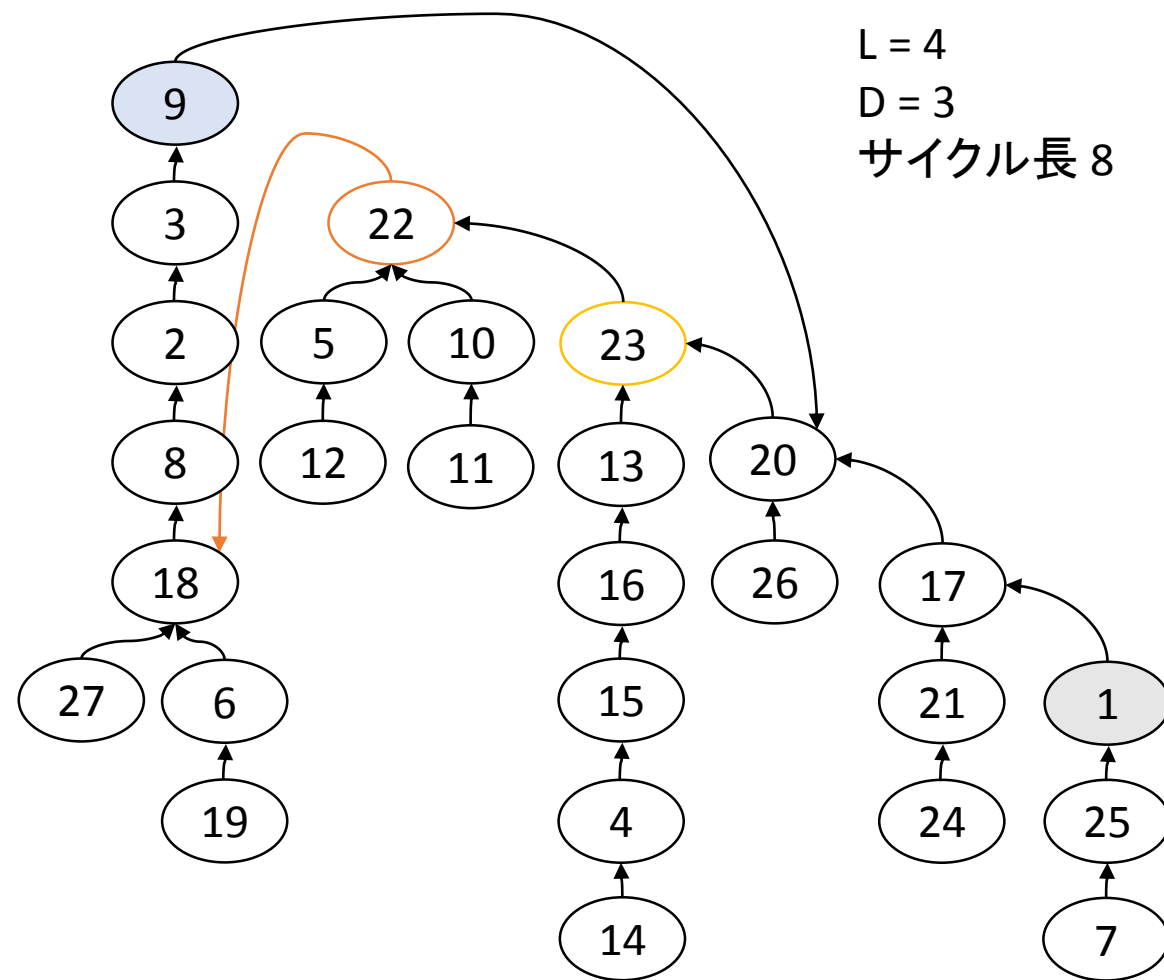
満点解法－場合分け(3B)

- D一定のとき、到達先のY座標は一定ではない
- このときは $Y=6$



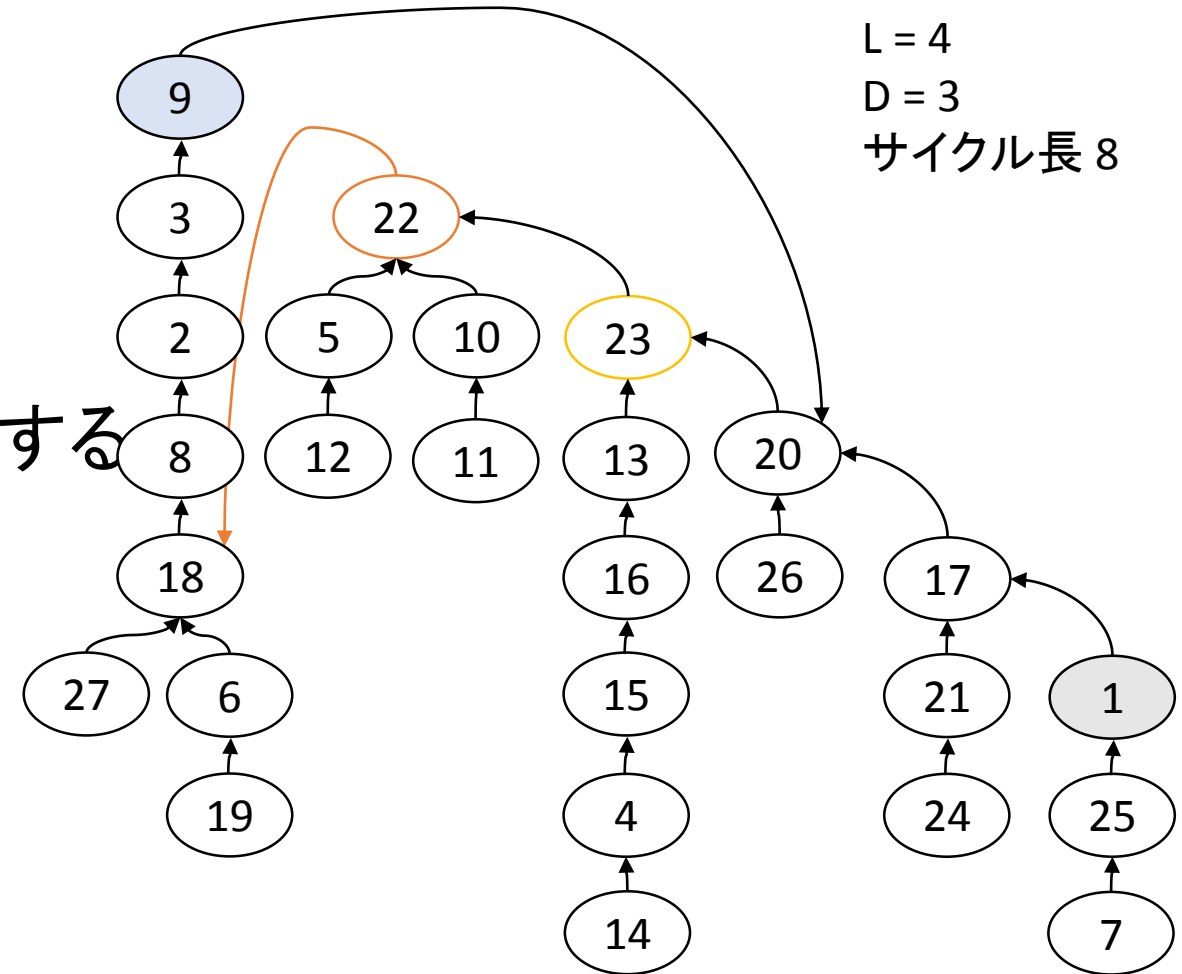
満点解法－場合分け(3B)

- D一定のとき、到達先のY座標は一定ではない
- このときは $Y=2$ となる



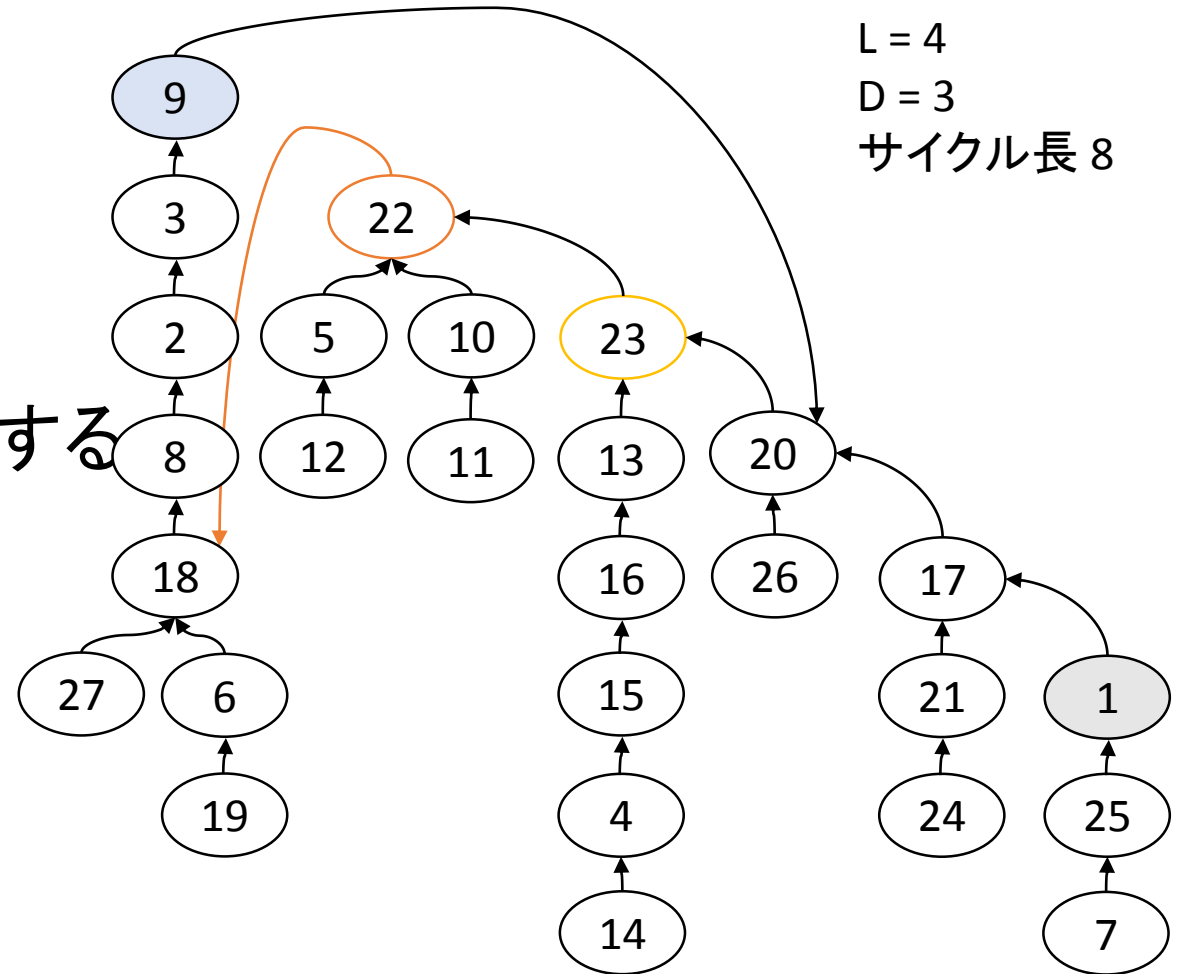
満点解法 – 場合分け(3B)

- D一定のとき、到達先のY座標は一定ではない
- このときは $Y=2$ となる
- このように一点だけ例外が存在する



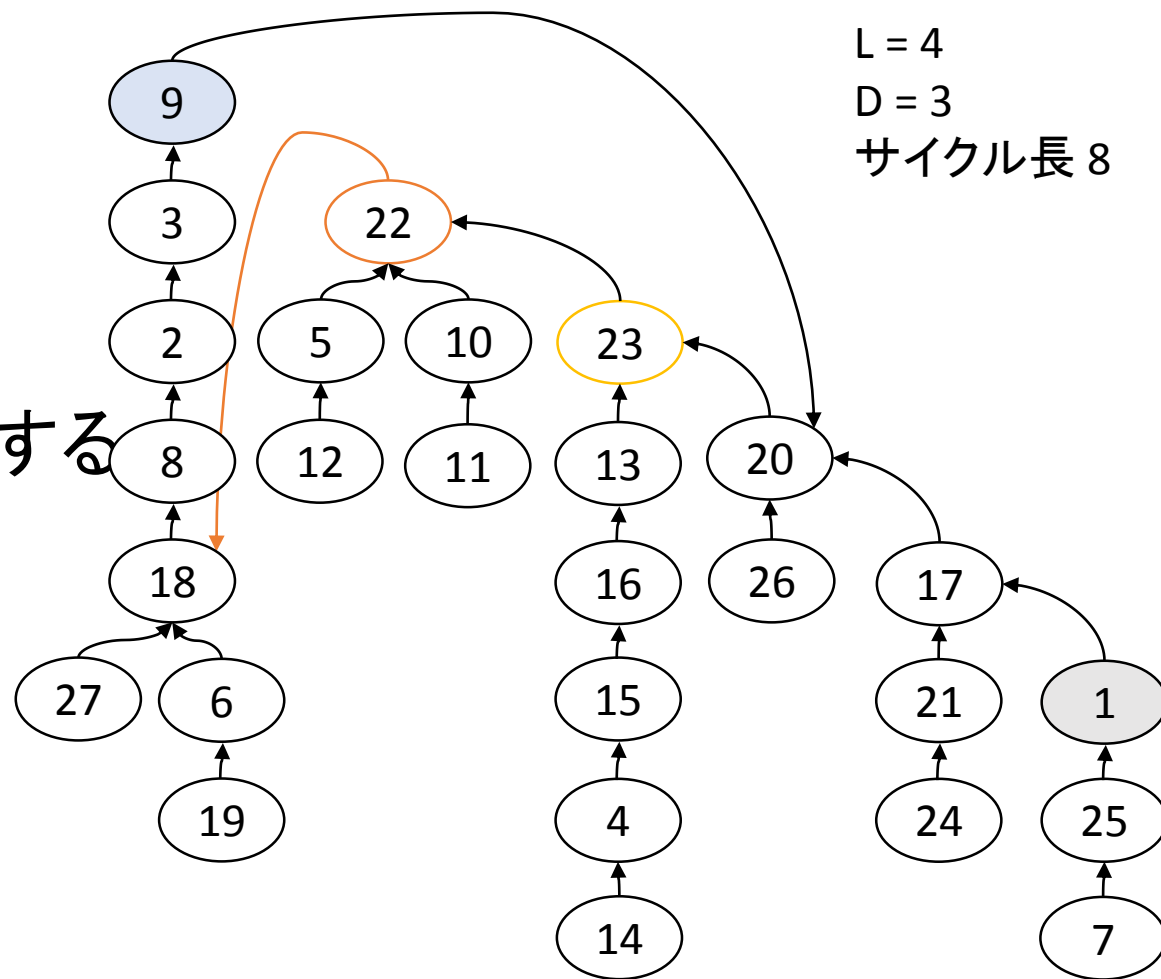
満点解法 – 場合分け(3B)

- D一定のとき、到達先のY座標は一定ではない
- このときは $Y=2$ となる
- このように一点だけ例外が存在する
- この例外はループ上に存在



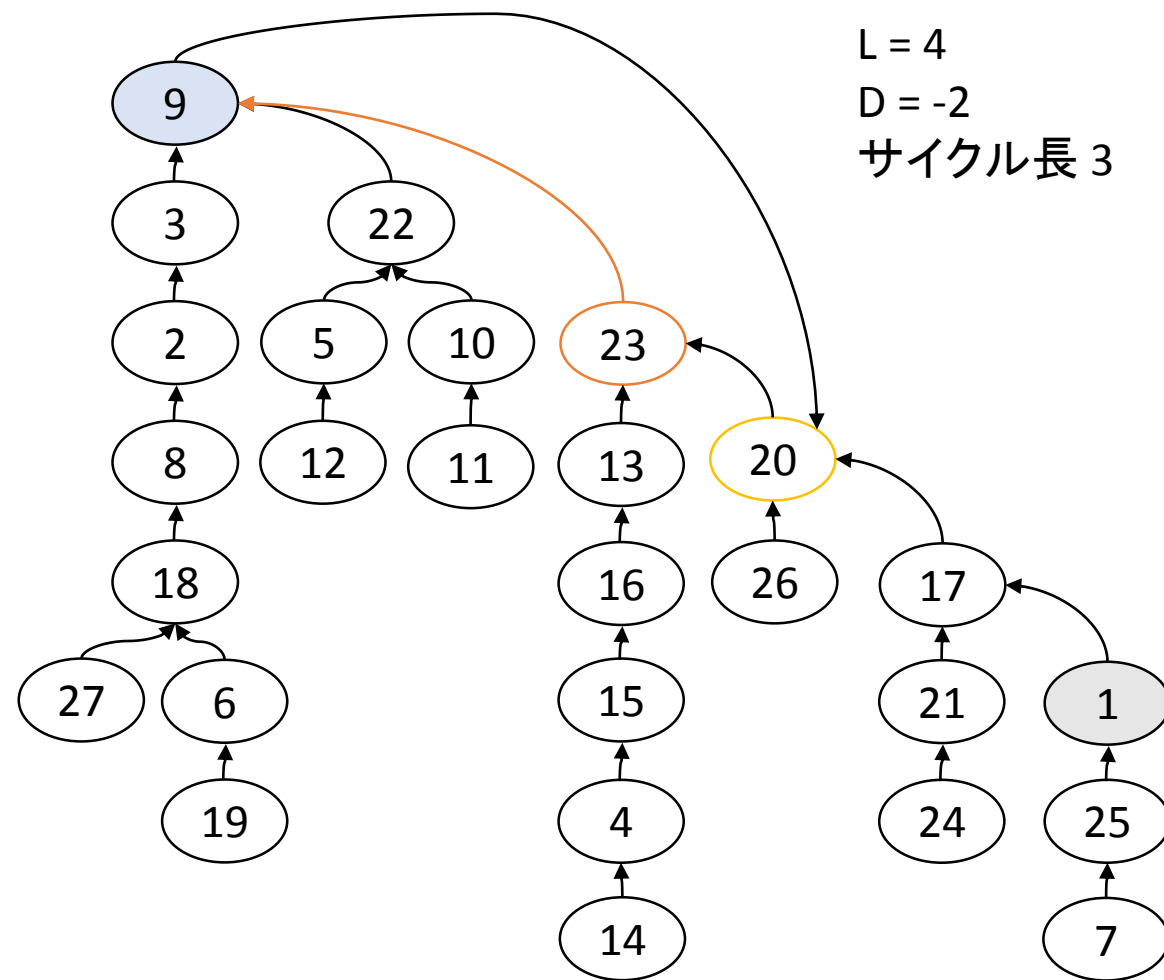
満点解法－場合分け(3B)

- D一定のとき、到達先のY座標は一定ではない
- このときは $Y=2$ となる
- このように一点だけ例外が存在する
- この例外はループ上に存在
- この座標は通常のものY座標から $D+1$ を引いたものとなる



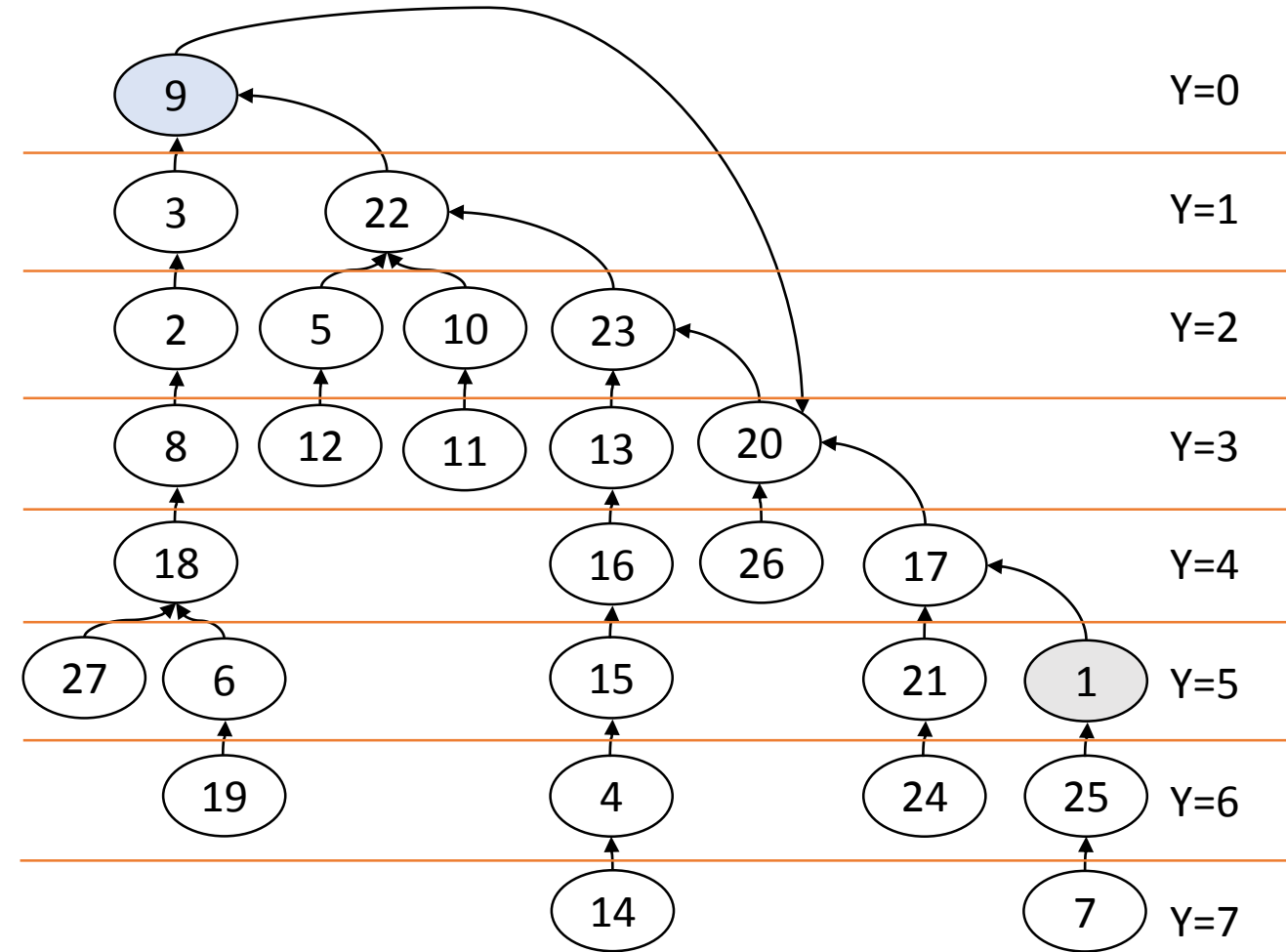
満点解法 – 場合分け(3B)

- Dはマイナスにもなるので注意



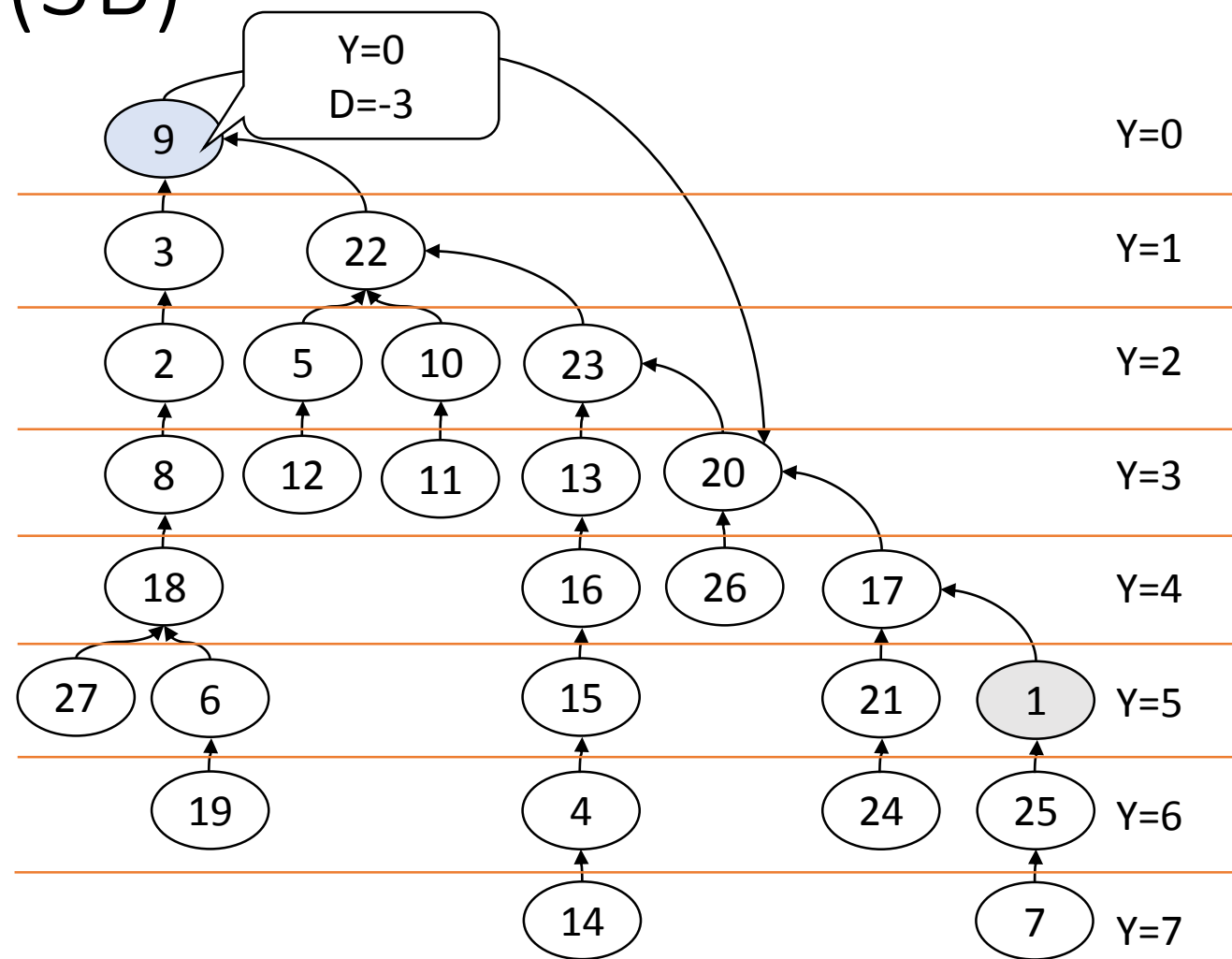
満点解法 – 場合分け(3B)

- 以上をイベントとして管理



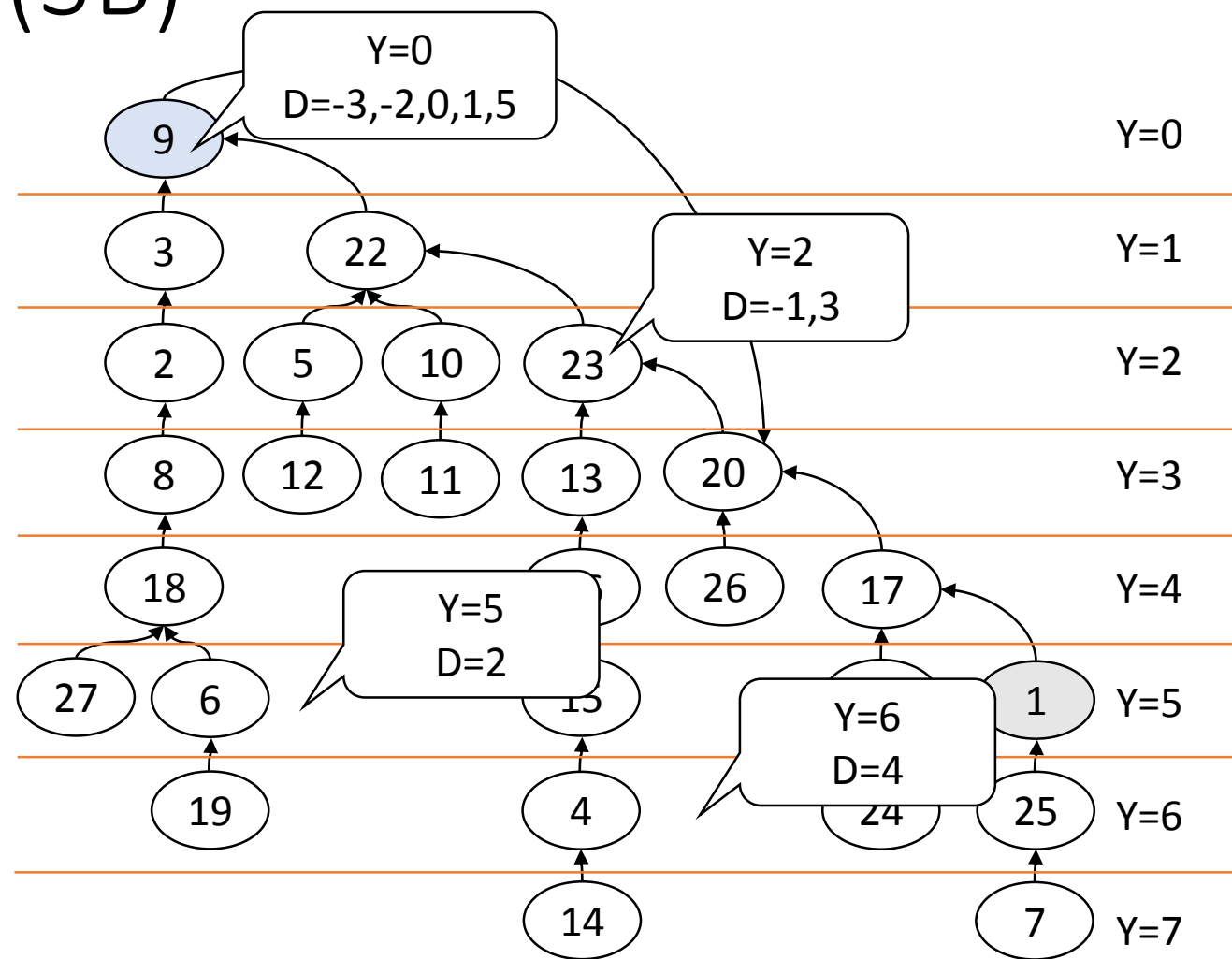
満点解法 – 場合分け(3B)

- 以上をイベントとして管理



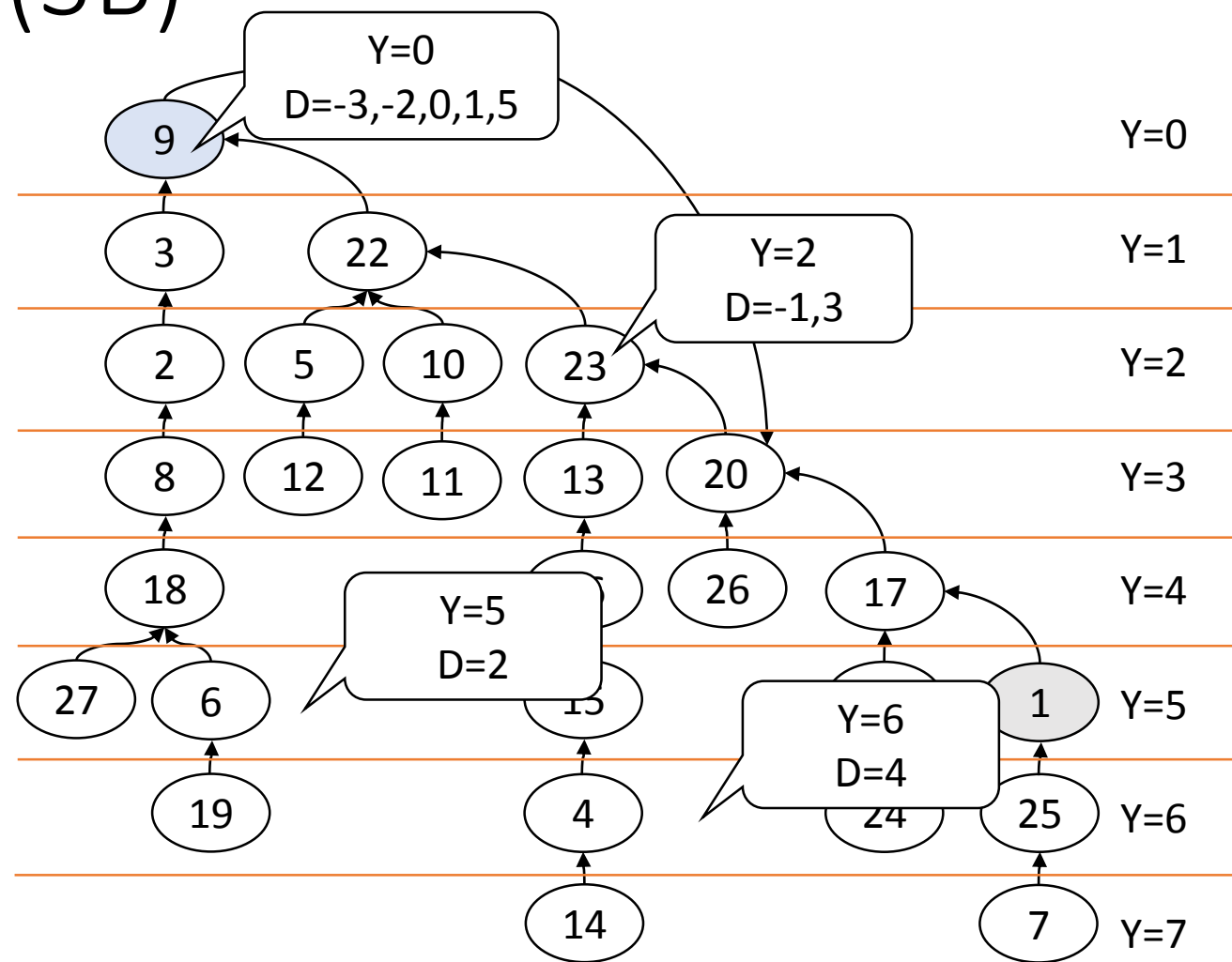
満点解法 – 場合分け(3B)

- 以上をイベントとして管理
- 各Dについて、対応するY座標を計算



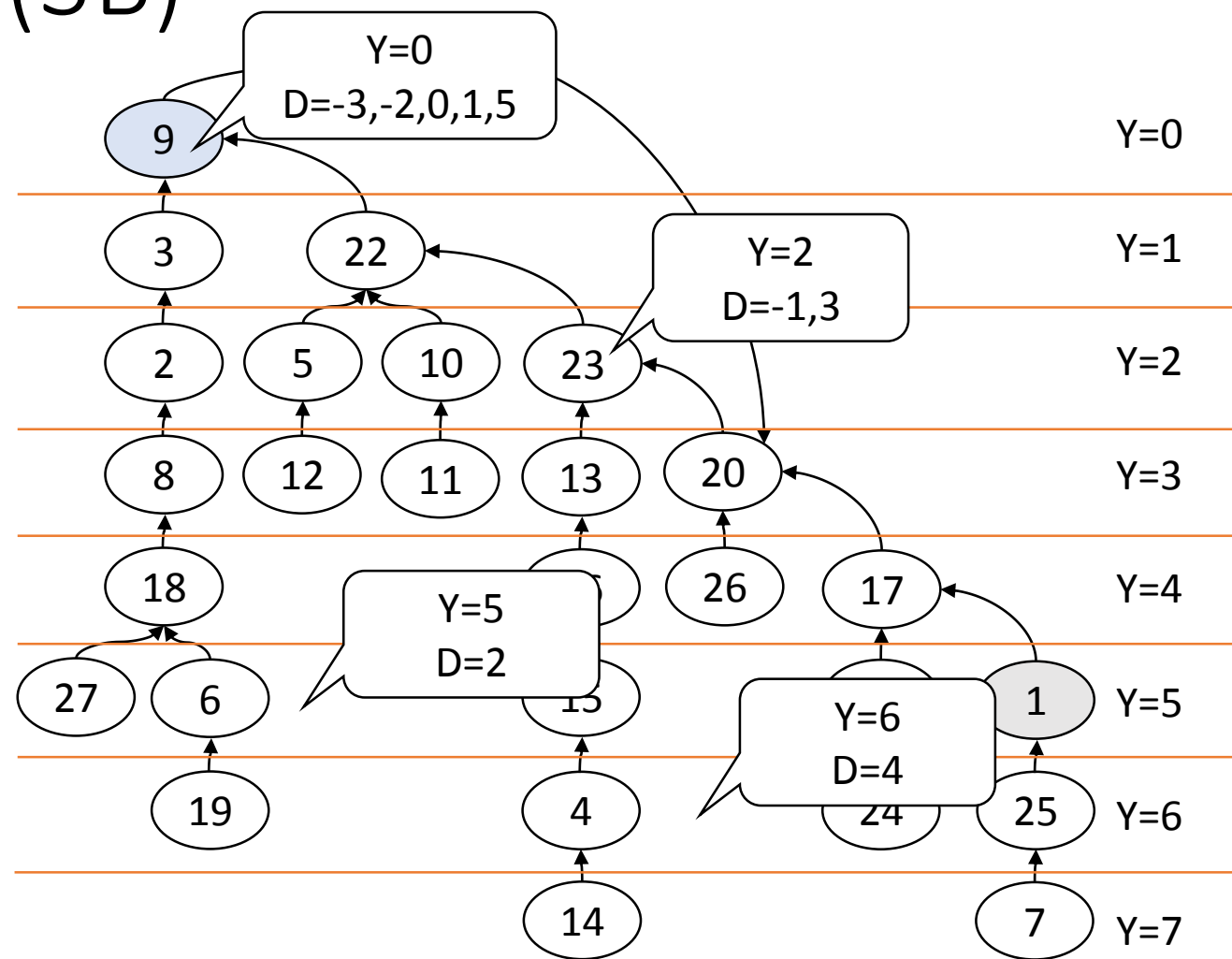
満点解法 – 場合分け(3B)

- 以上をイベントとして管理
- 各Dについて、対応するY座標を計算
- 例外的な到達先のY座標はこの値からD+1を減じたものになる



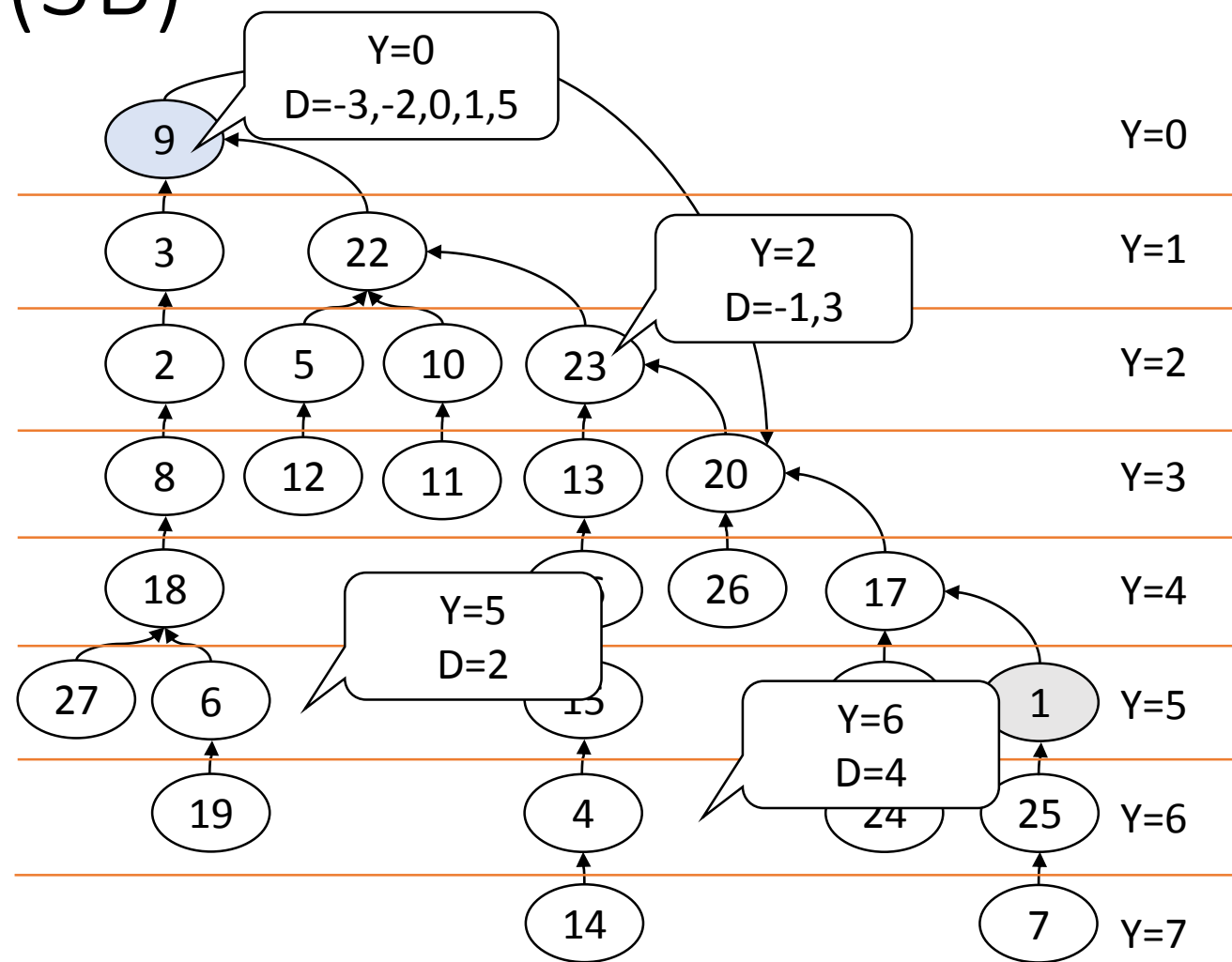
満点解法 – 場合分け(3B)

- 実際の数え上げ:
下から走査する



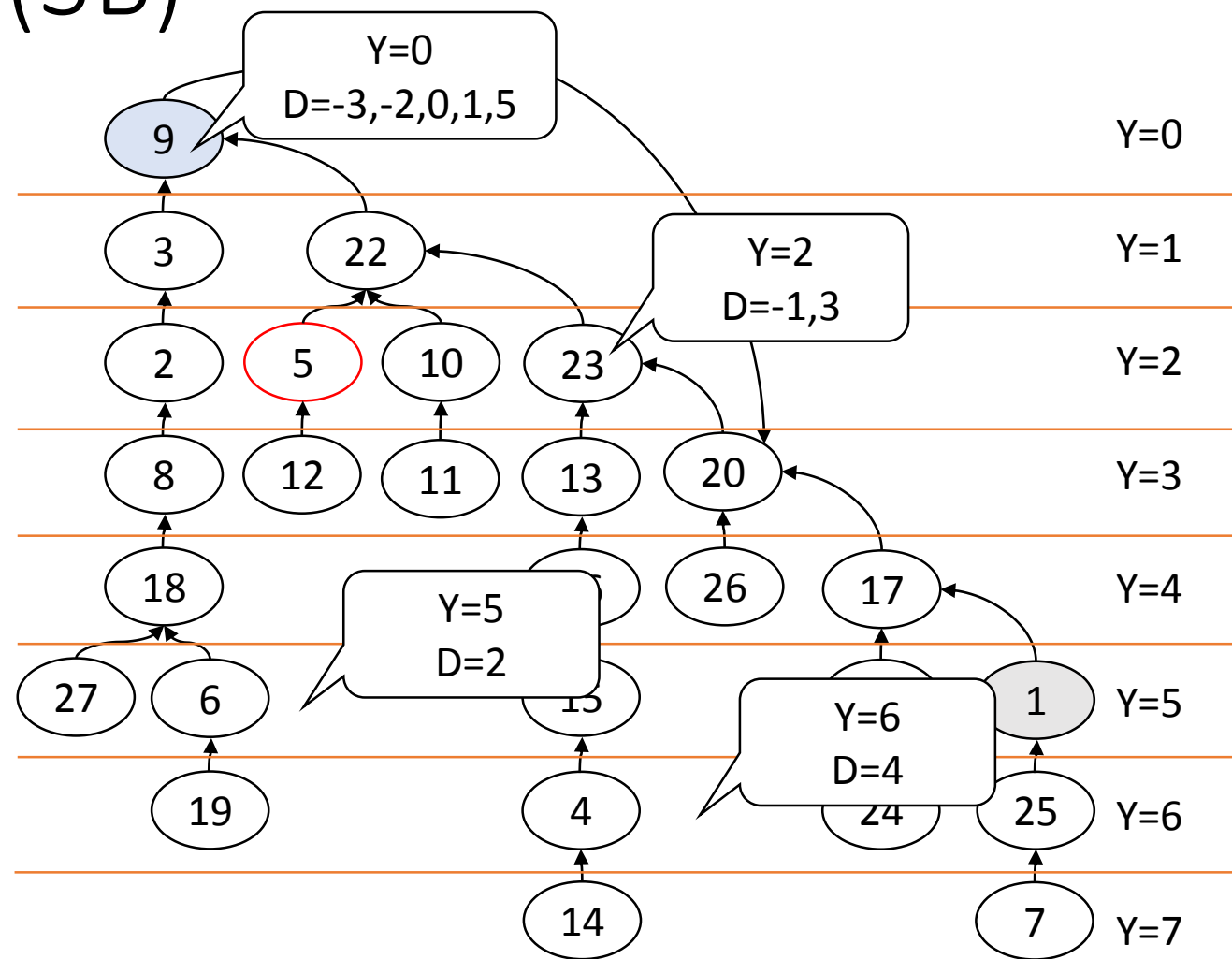
満点解法 – 場合分け(3B)

- 実際の数え上げ:
下から走査する
- このときループで回す頂点は
頂点Aの候補ではなく、
到達先の頂点の候補を
意味する



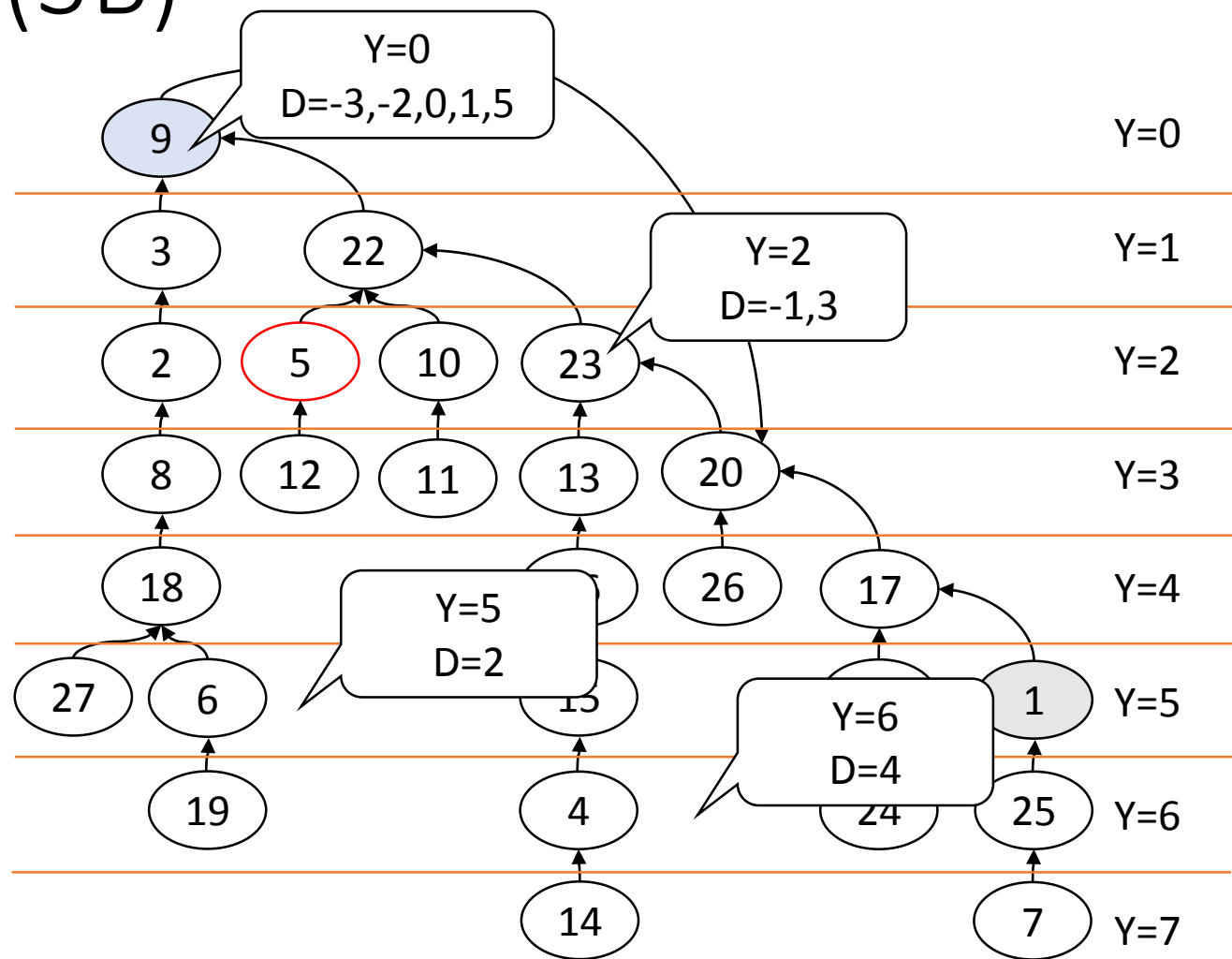
満点解法 – 場合分け(3B)

- 各到達先候補について:



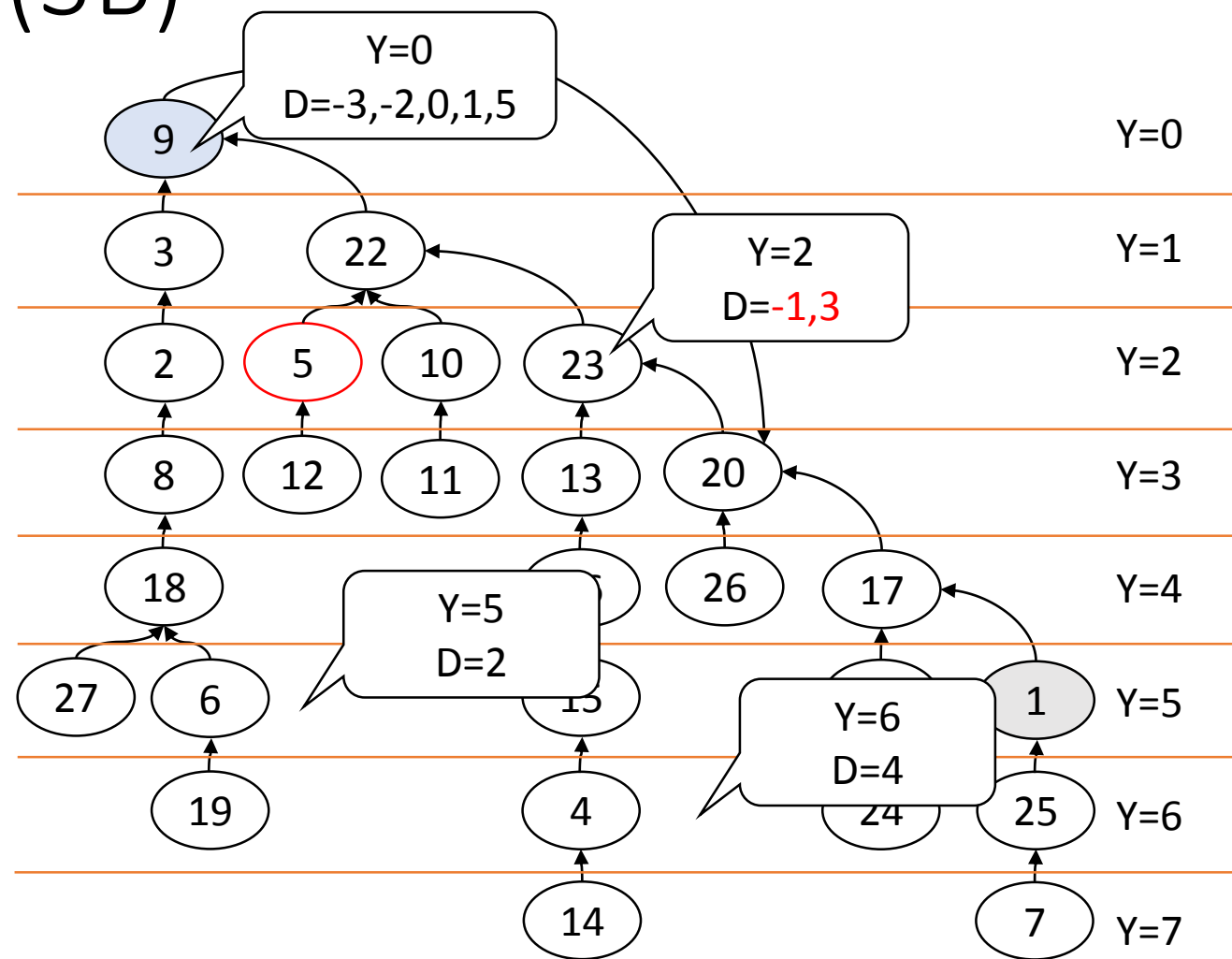
満点解法－場合分け(3B)

- 各到達先候補について:
 - Dの候補を列挙する



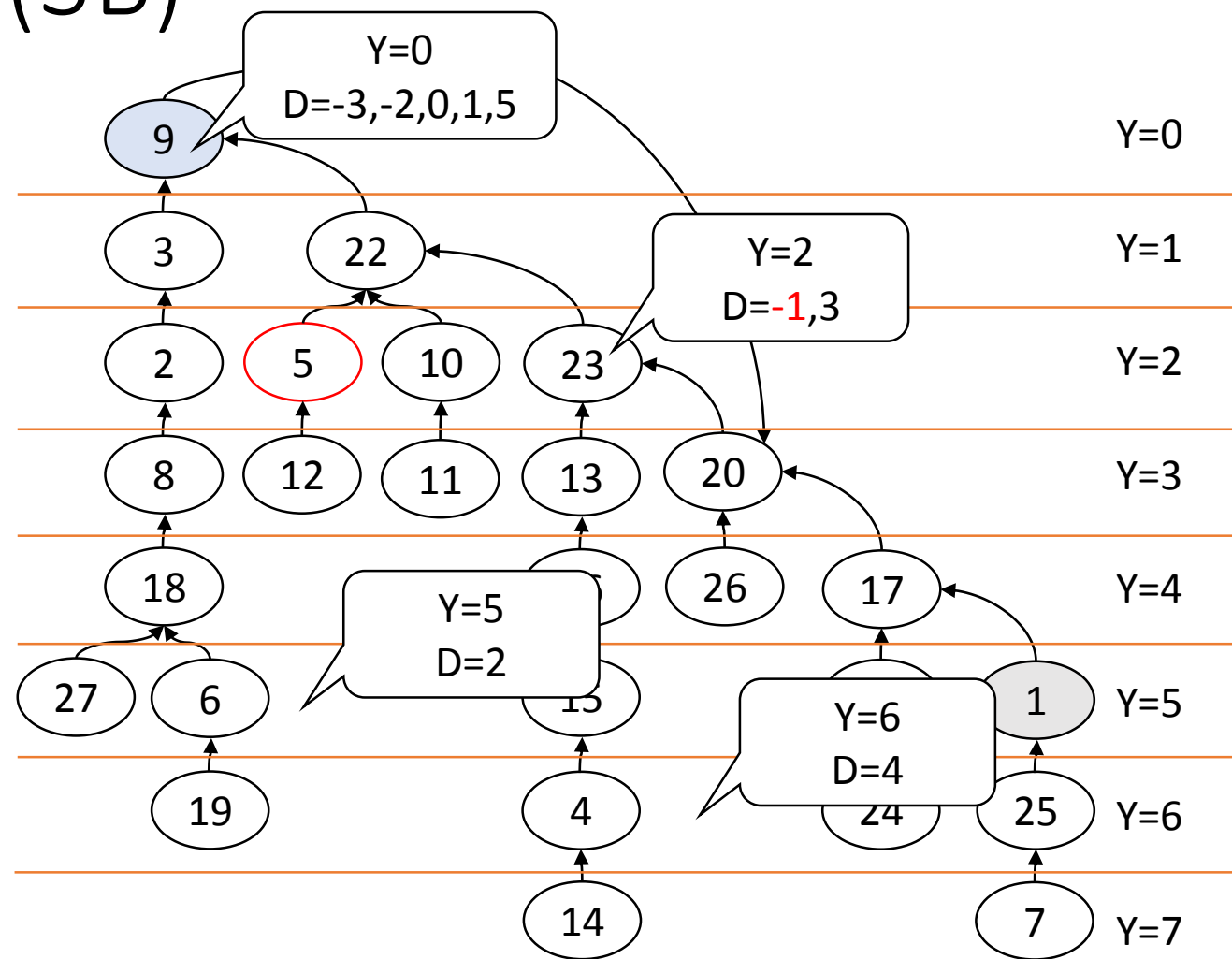
満点解法 – 場合分け(3B)

- 各到達先候補について:
 - Dの候補を列挙する
 - この場合は-1, -3



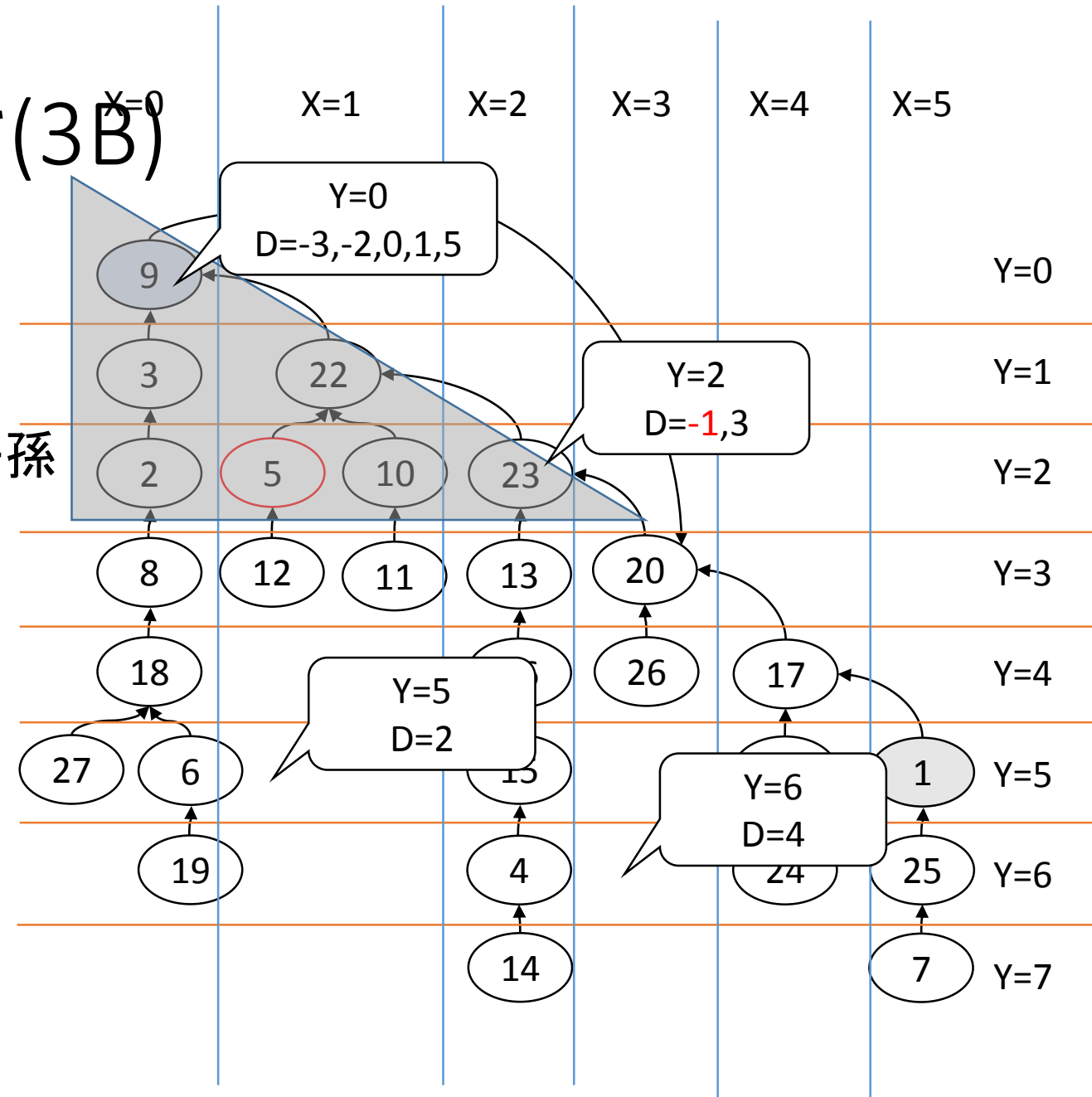
満点解法 – 場合分け(3B)

- 各(到達先候補,D)について:
 - Bの候補を列挙する



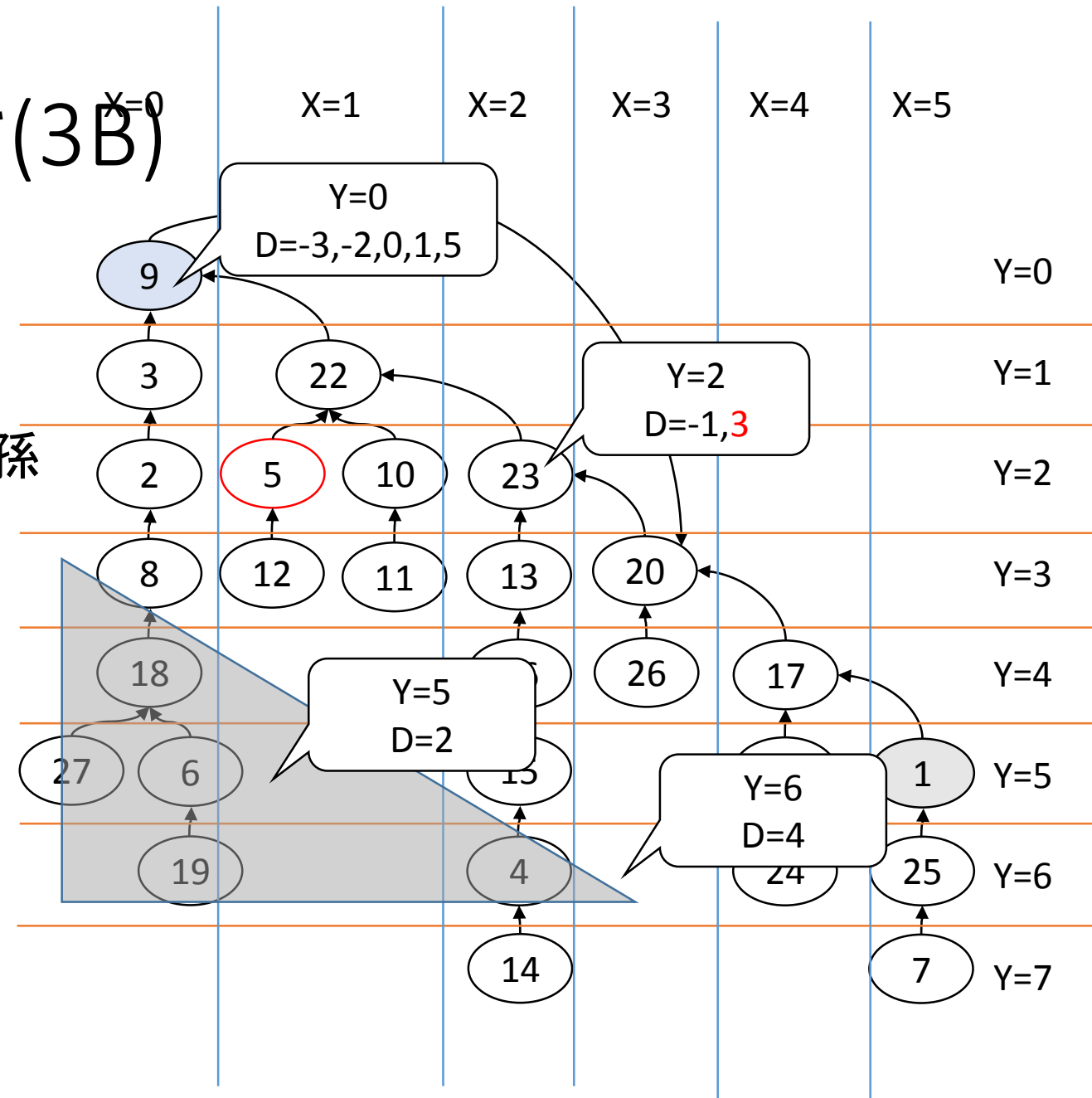
満点解法 – 場合分け(3B)

- 各(到達先候補,D)について:
 - Bの候補を列挙する
 - D=-1のとき、候補はこの範囲の子孫
 - この三角形はDのみに依存



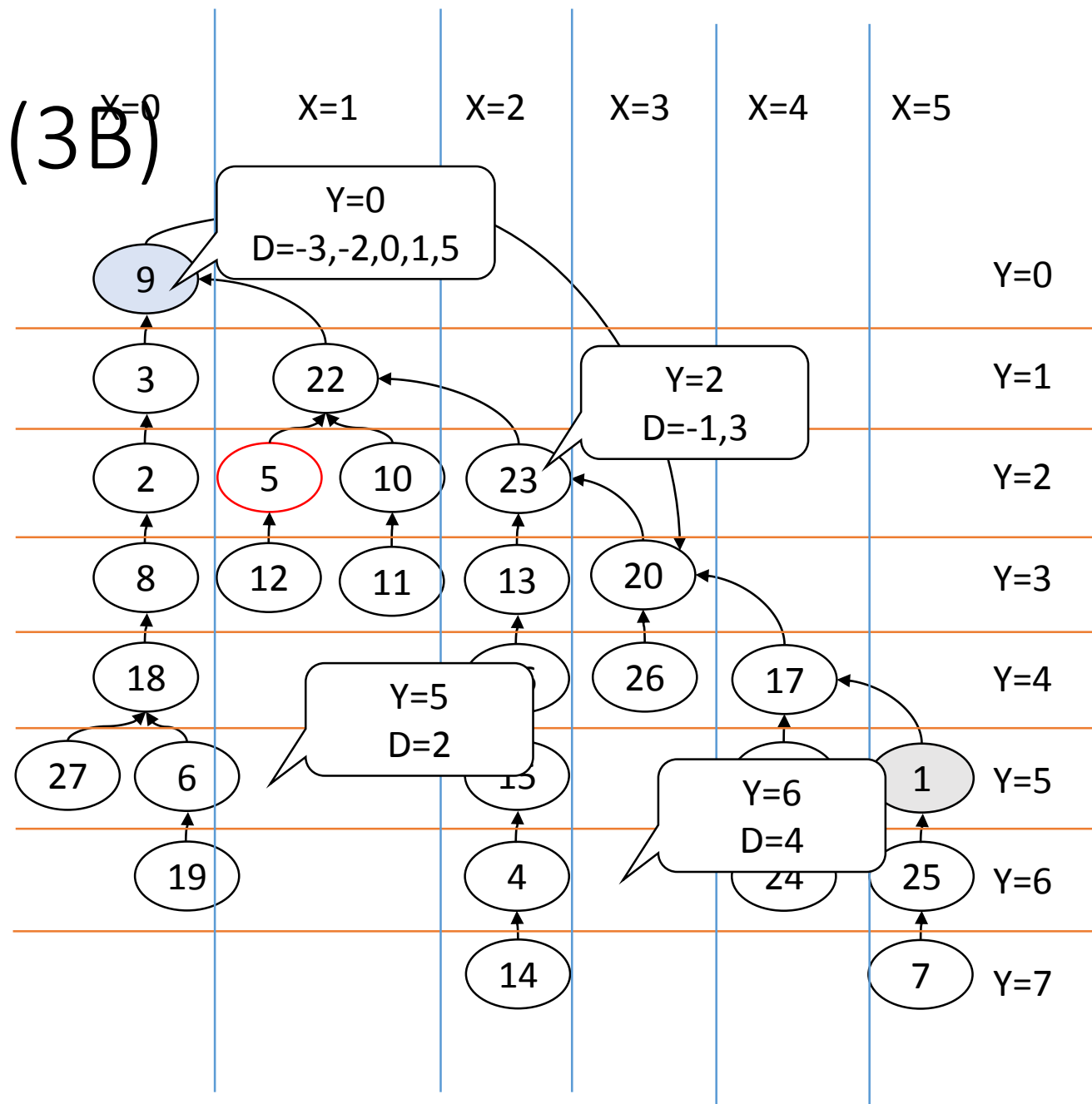
満点解法 – 場合分け(3B)

- 各(到達先候補,D)について:
 - Bの候補を列挙する
 - D=3のとき、候補はこの範囲の子孫
 - この三角形はDのみに依存



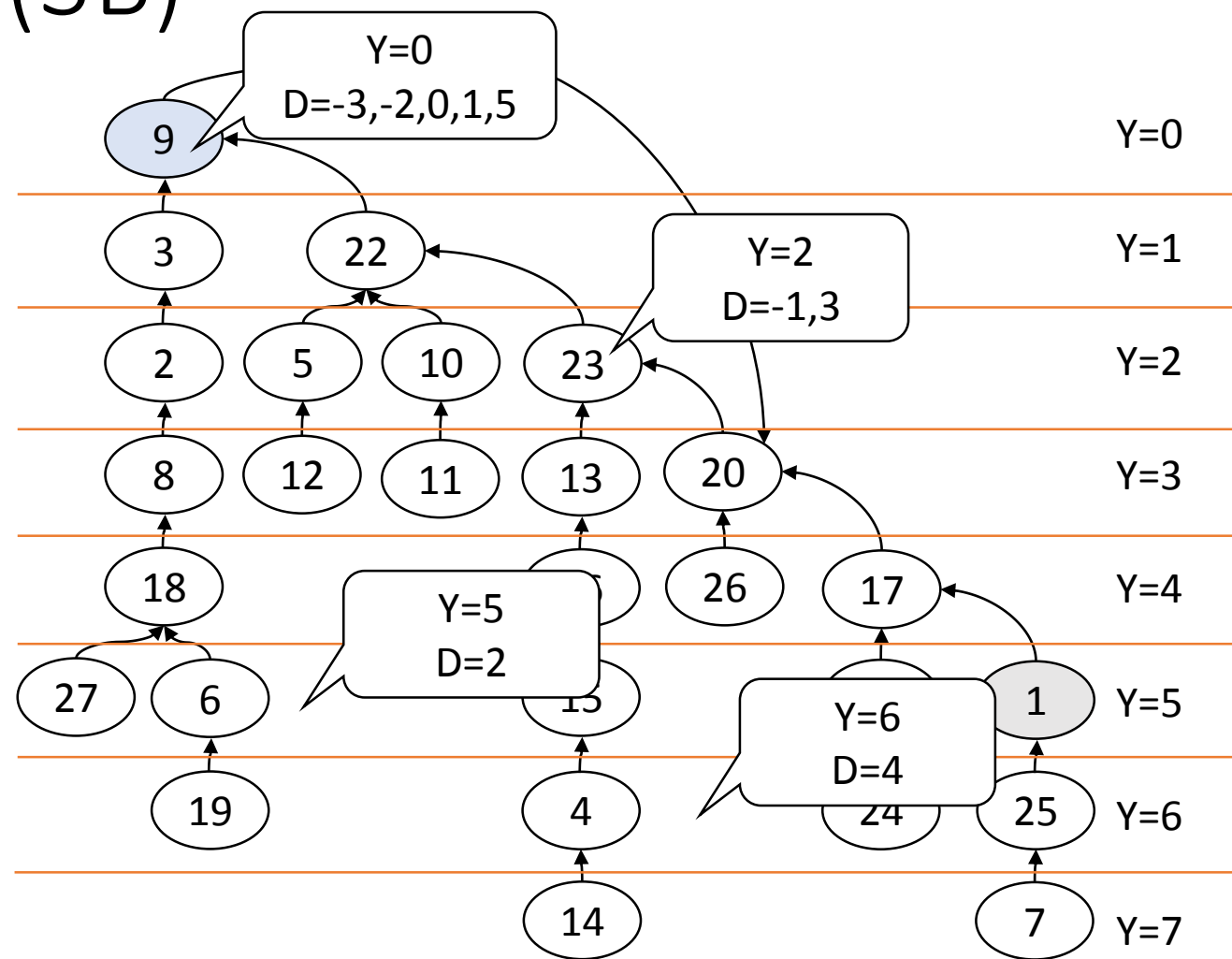
満点解法 – 場合分け(3B)

- 各(到達先候補,D)について:
 - Bの候補を列挙する
 - Bに対してAは一意に定まる
 - Bの候補の個数を数えるだけでよい



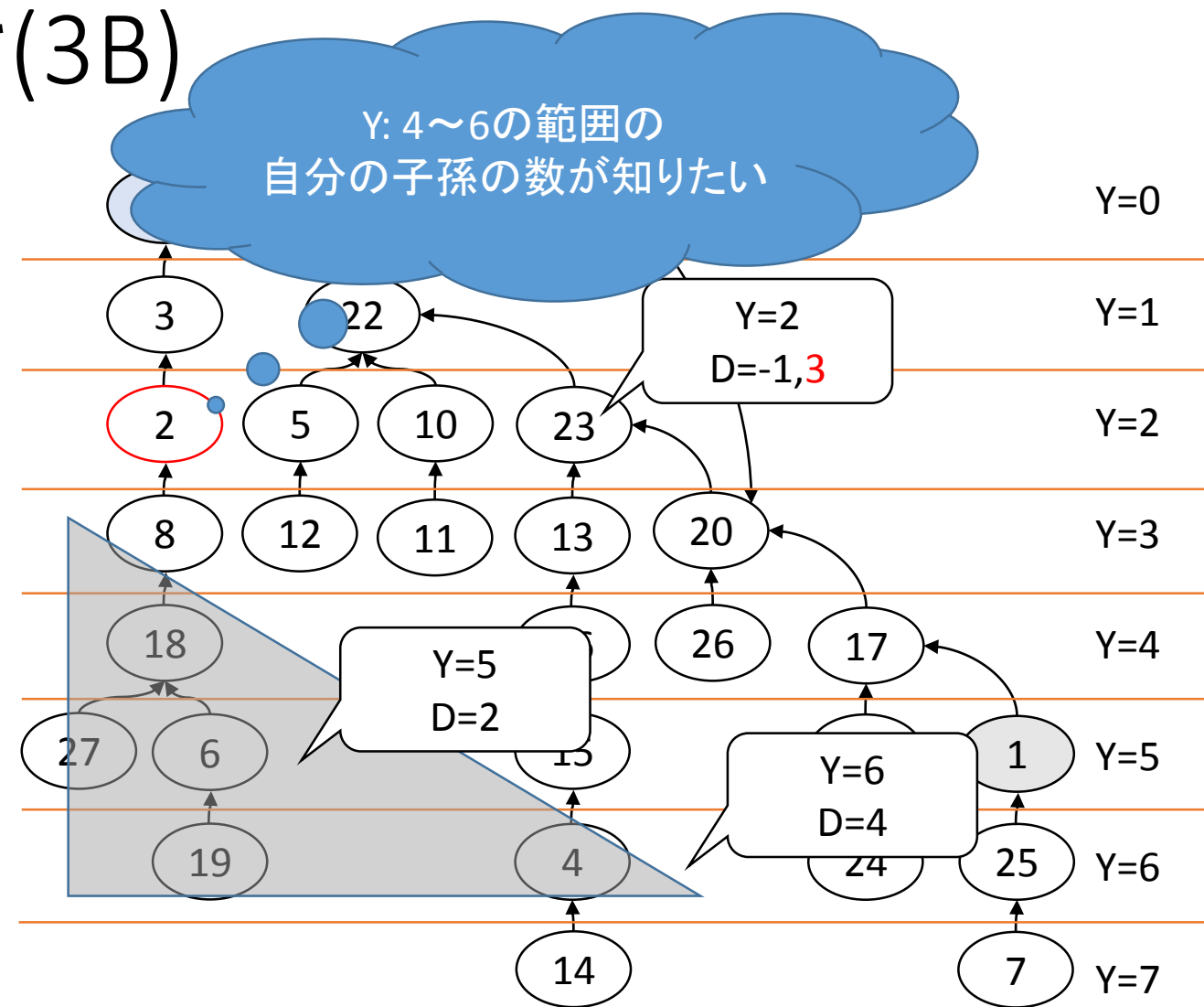
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい



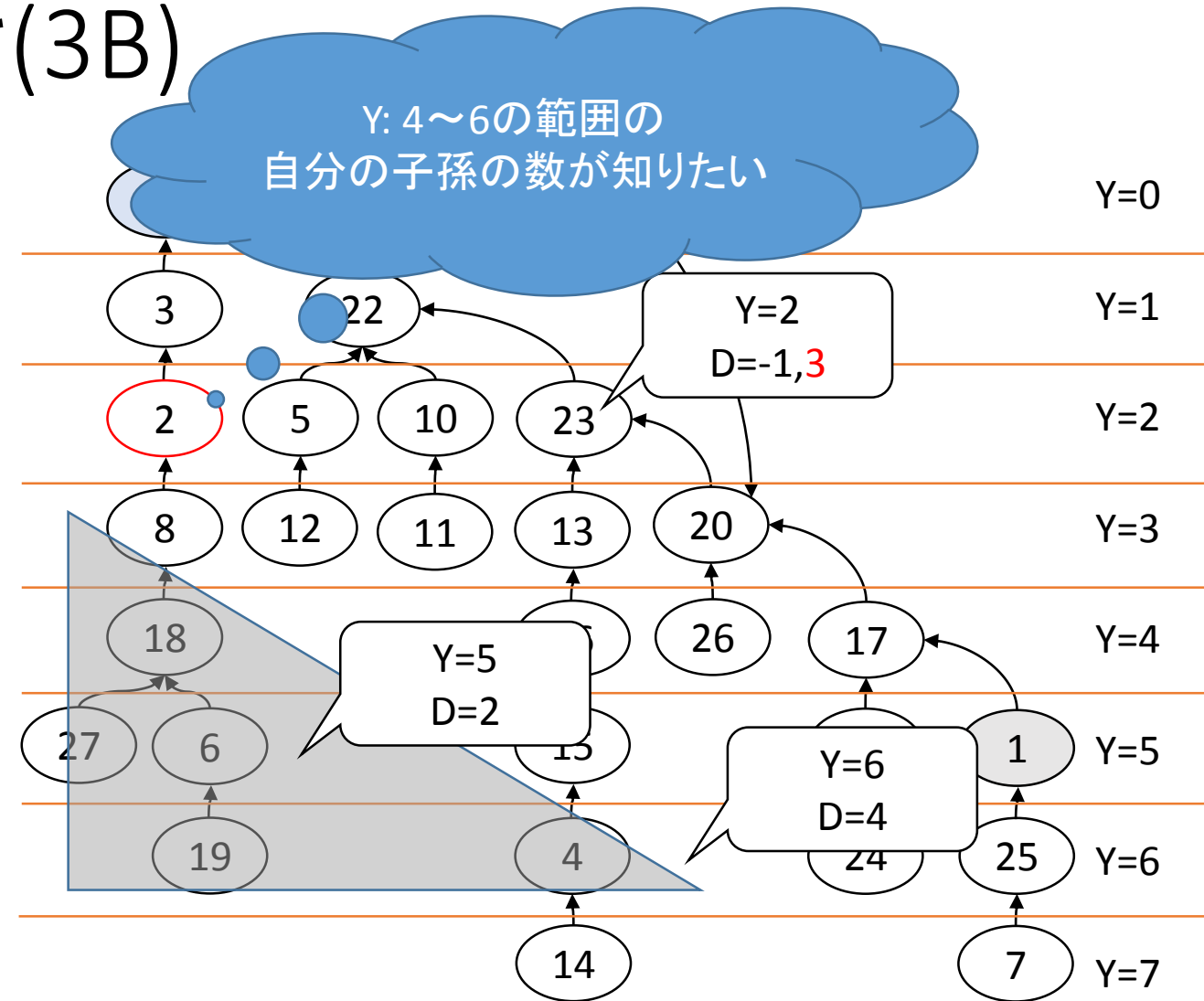
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい



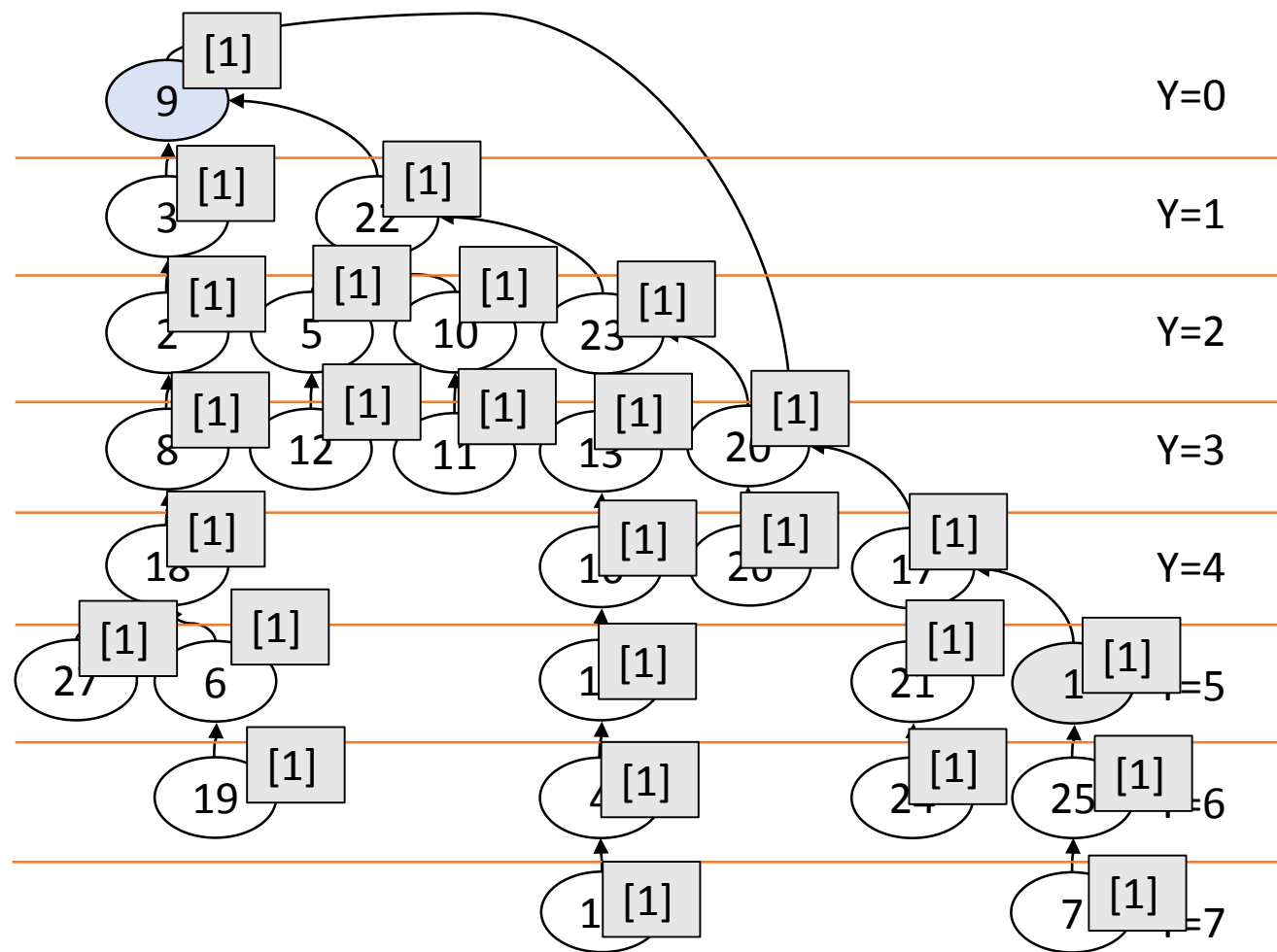
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
 - この場合は4個



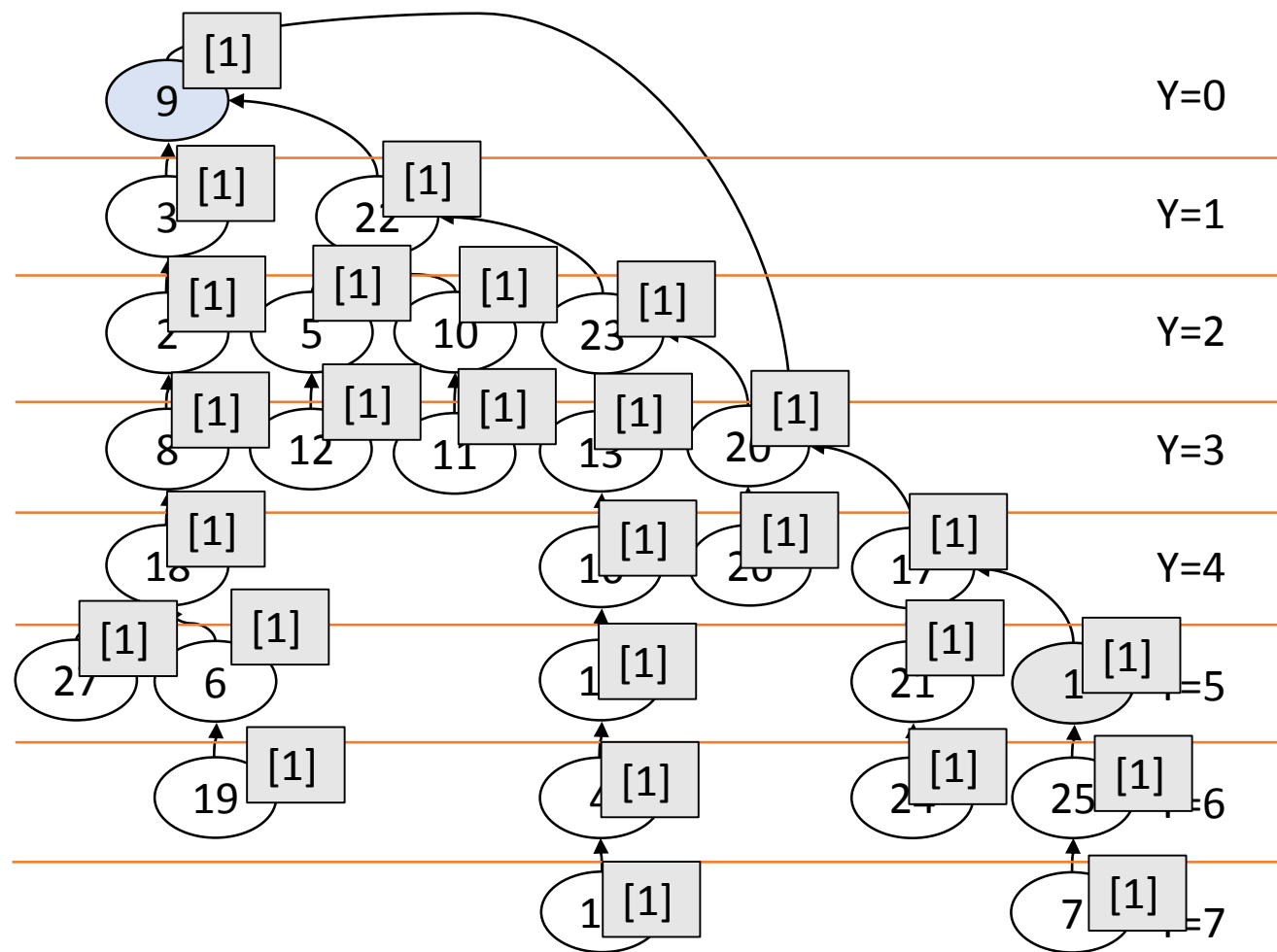
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化



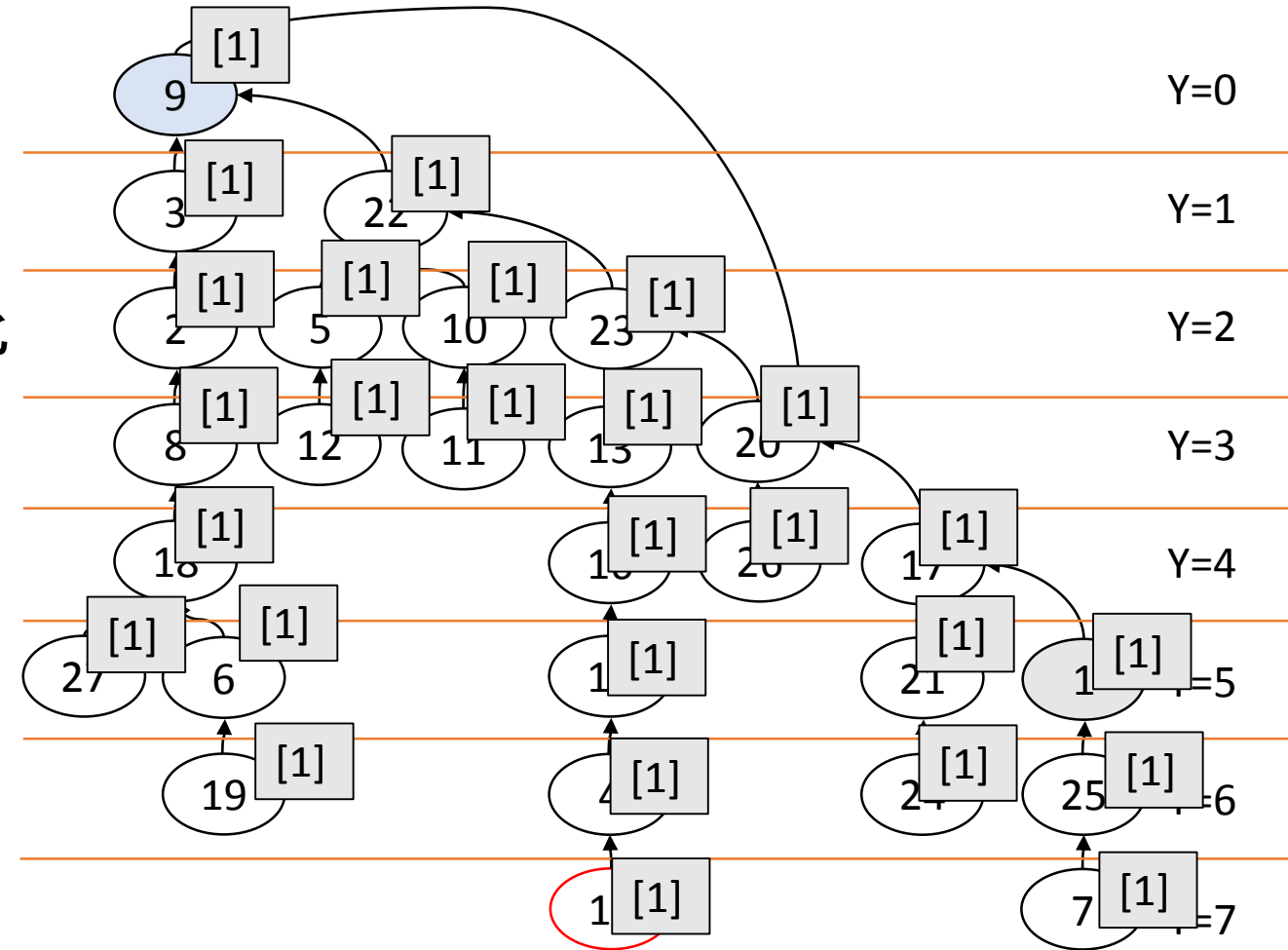
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



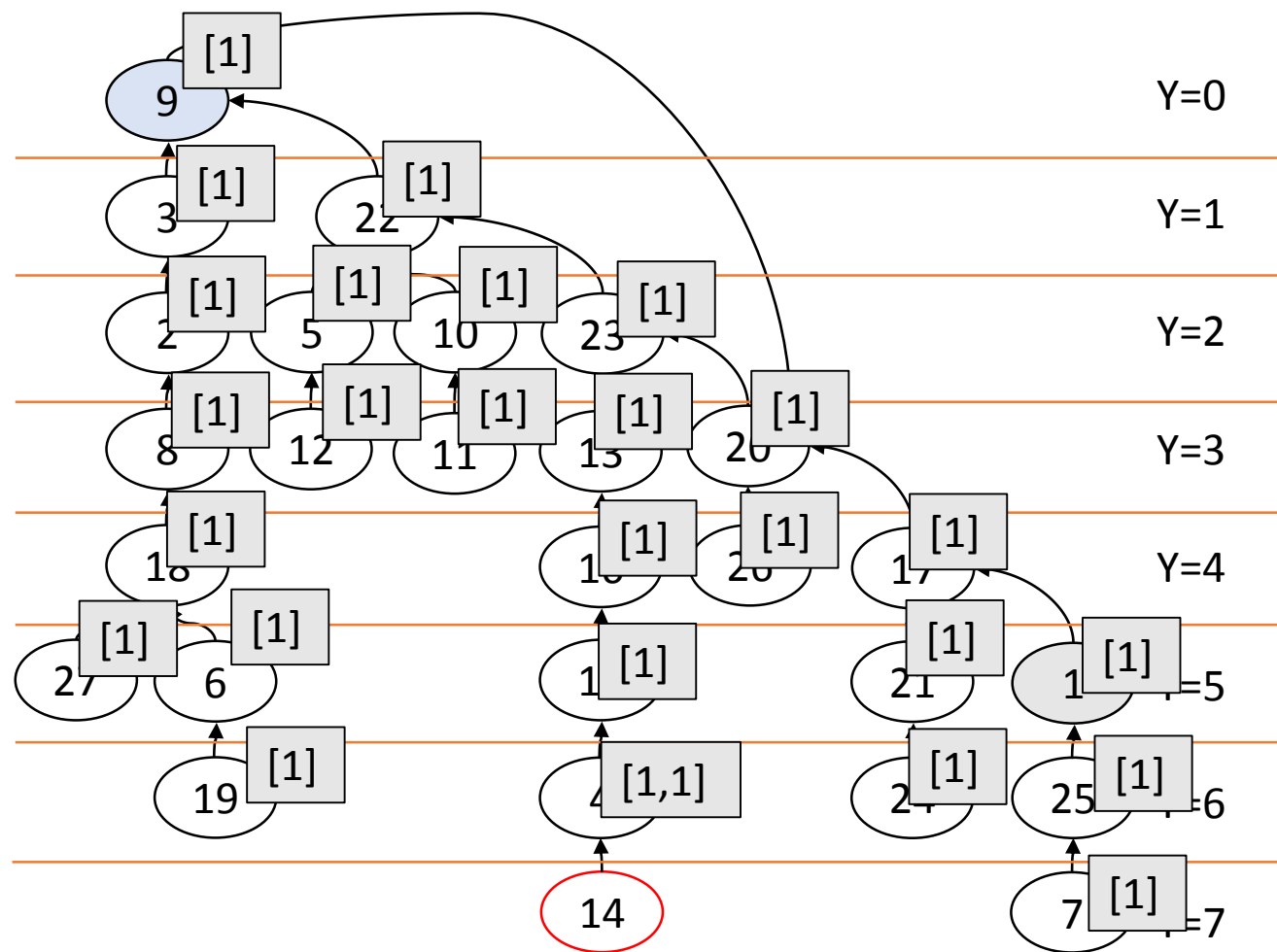
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査
 - 処理中の頂点のFenwick Treeは子孫の個数を正しく表している



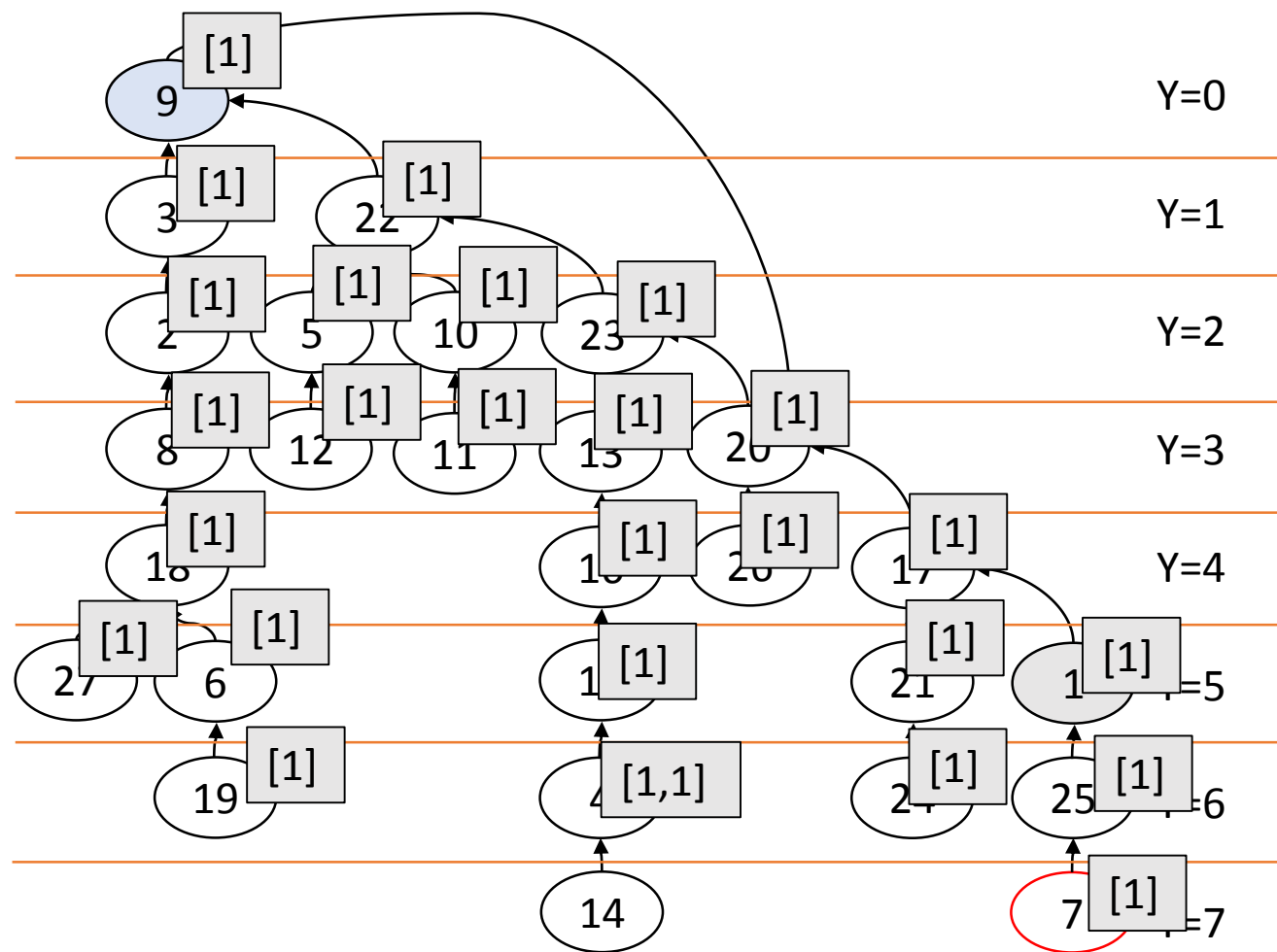
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査
 - 終わったら上の木とマージ



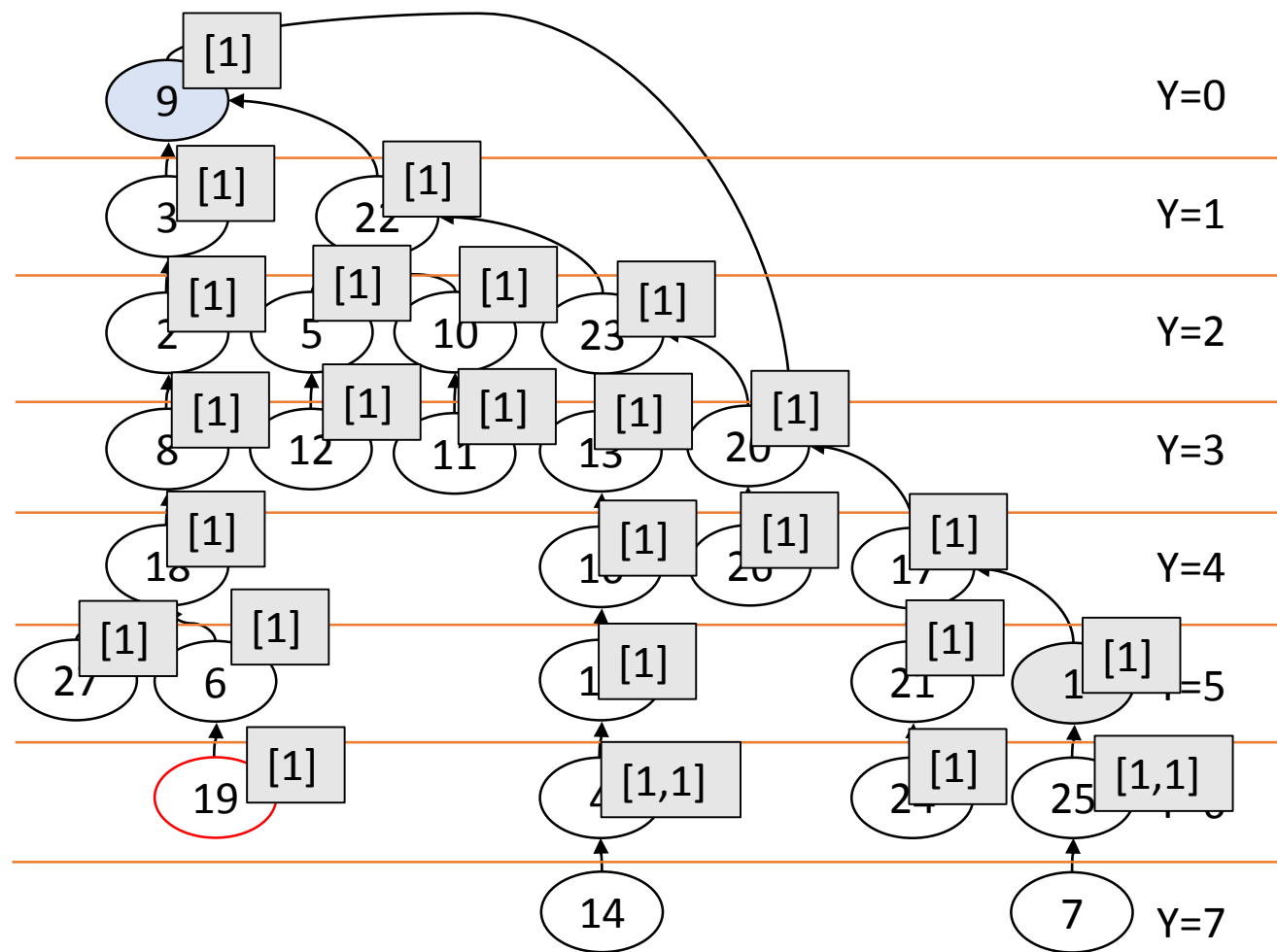
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



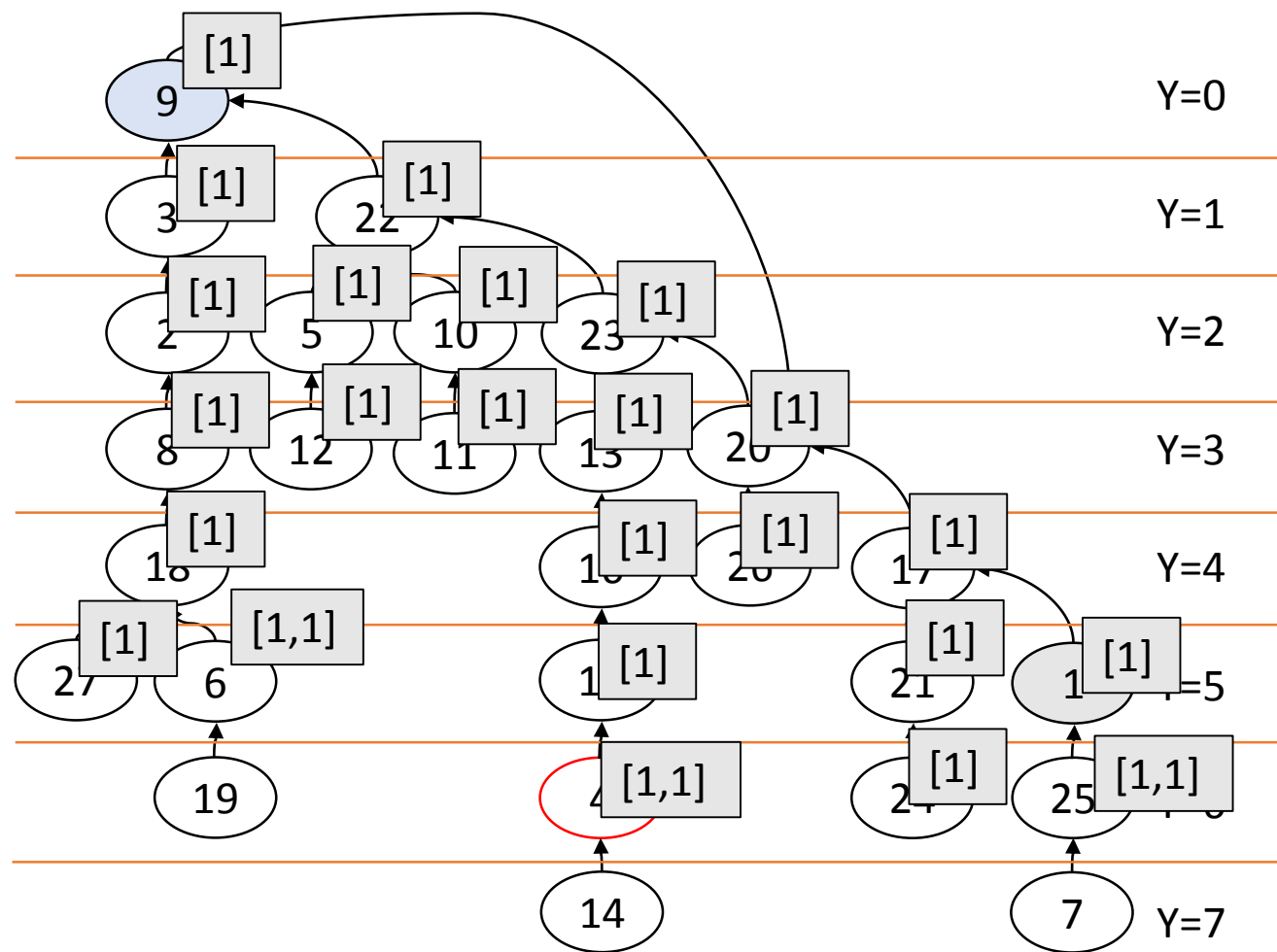
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



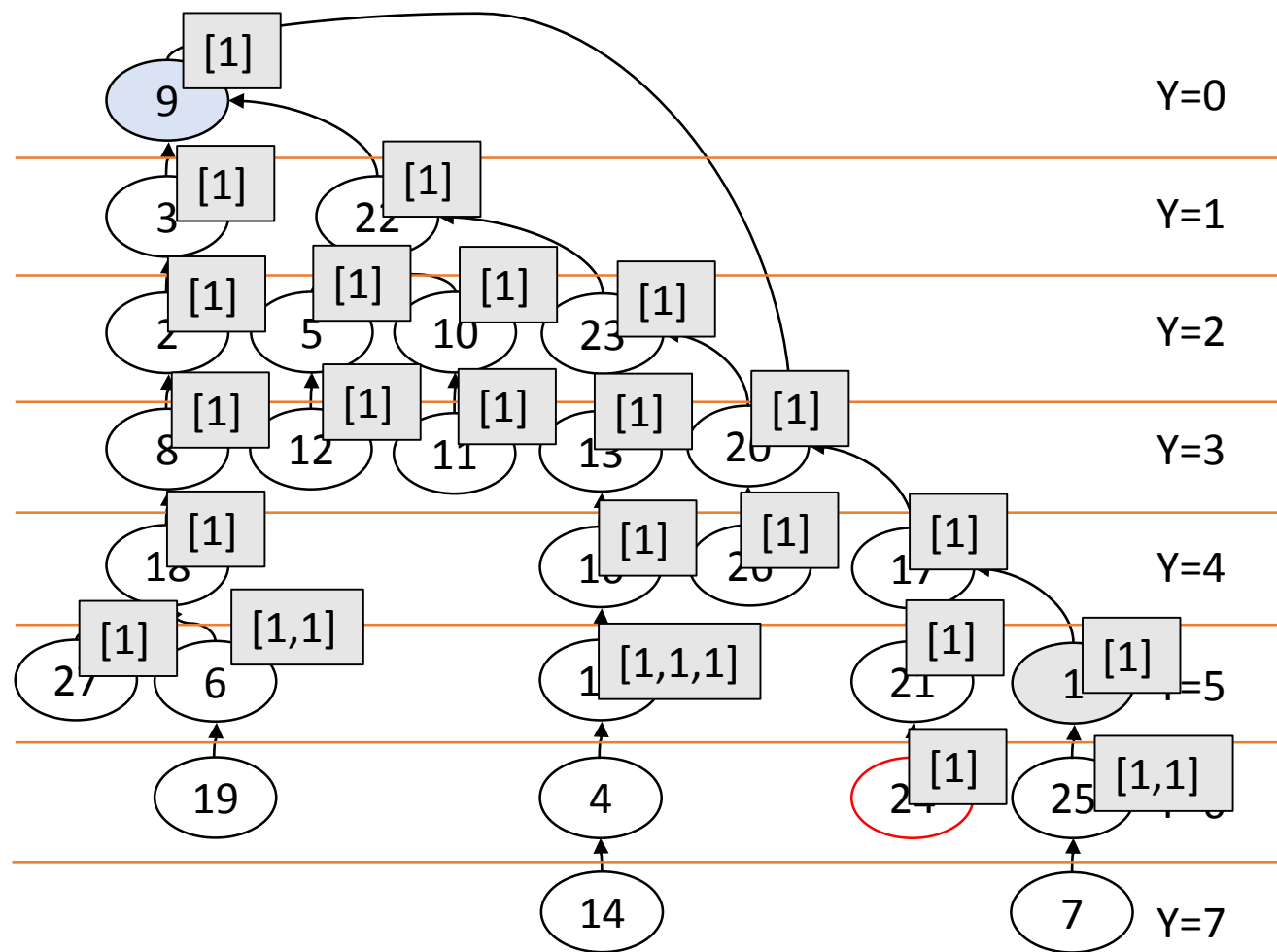
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



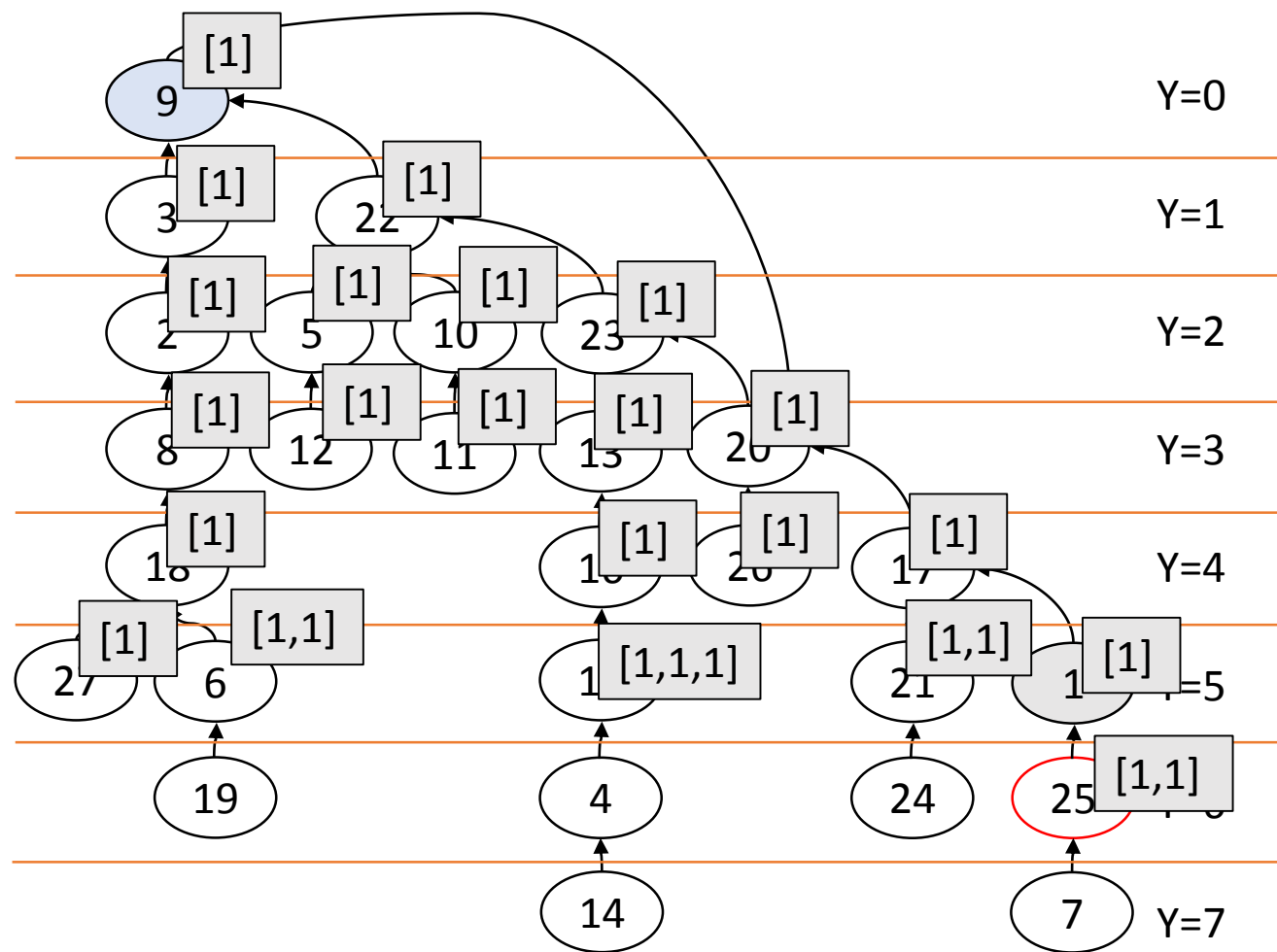
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



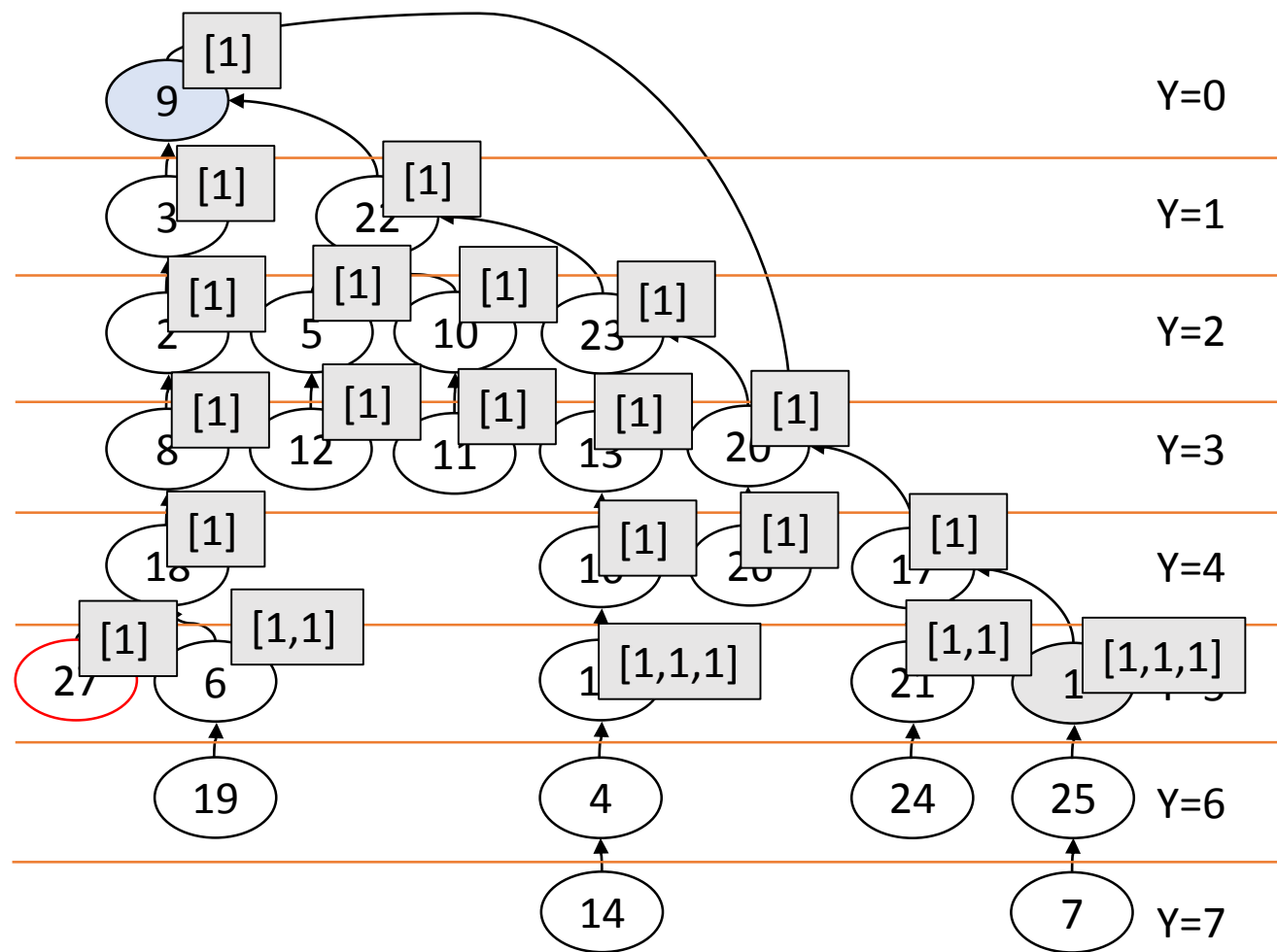
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



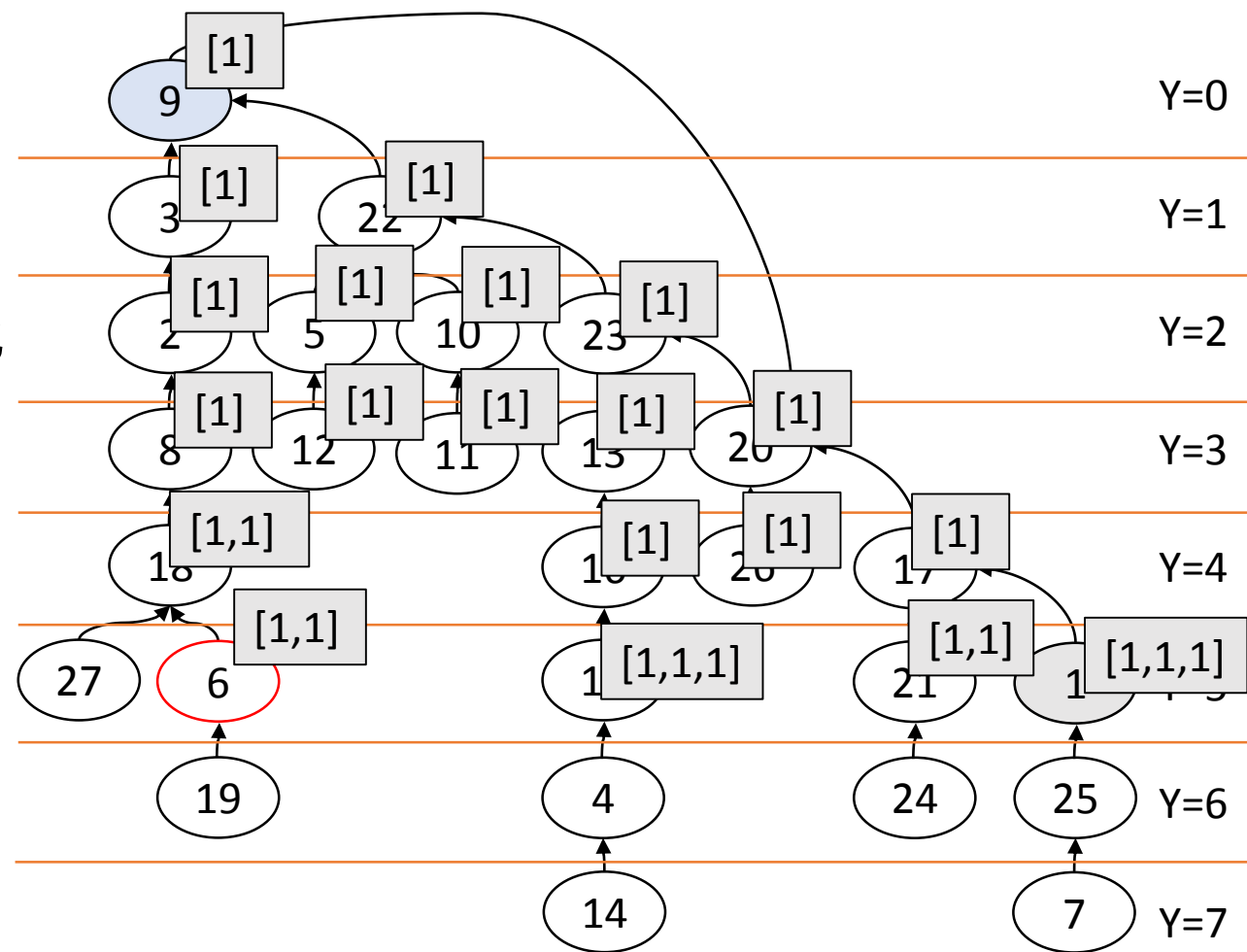
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



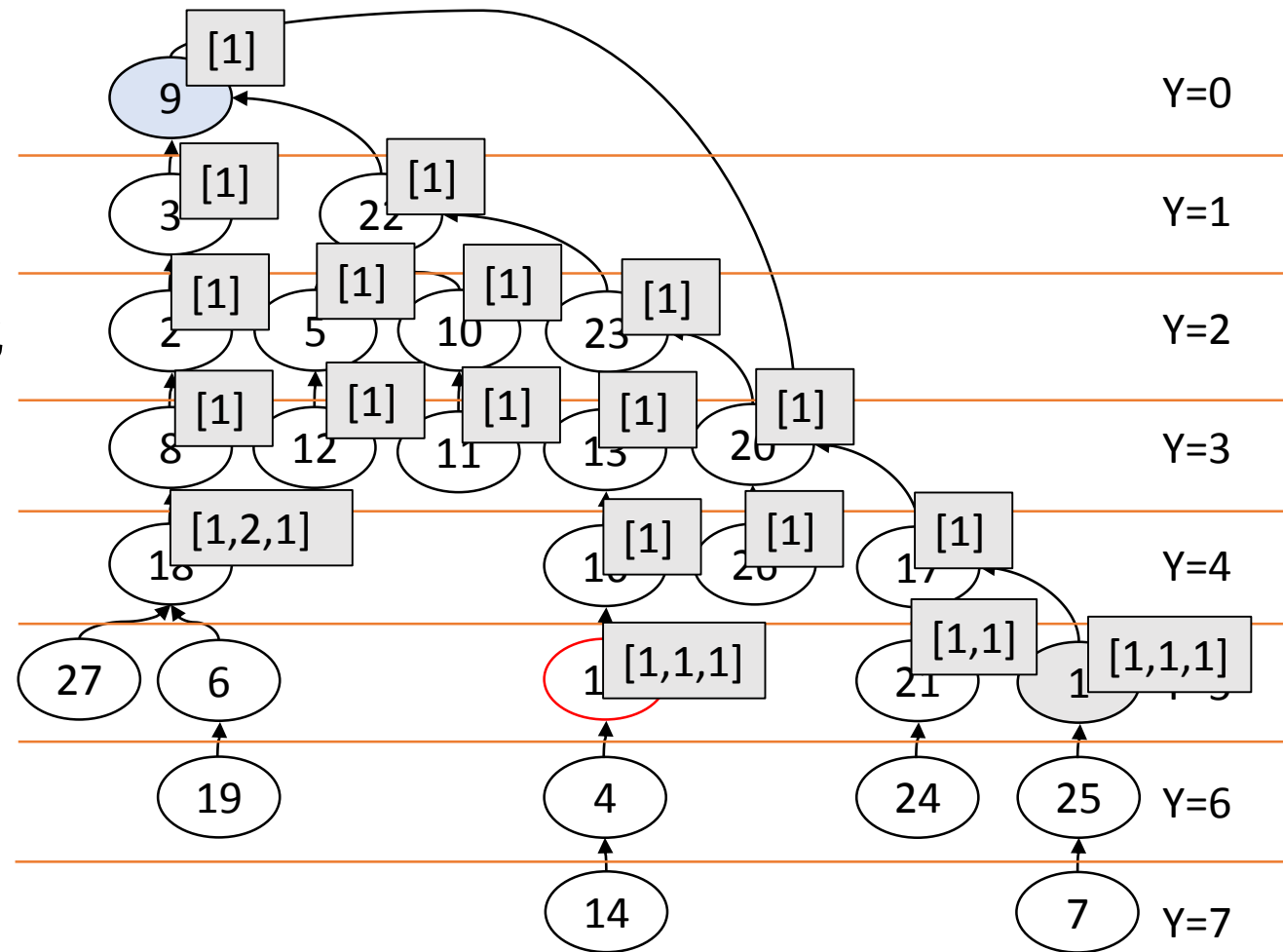
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



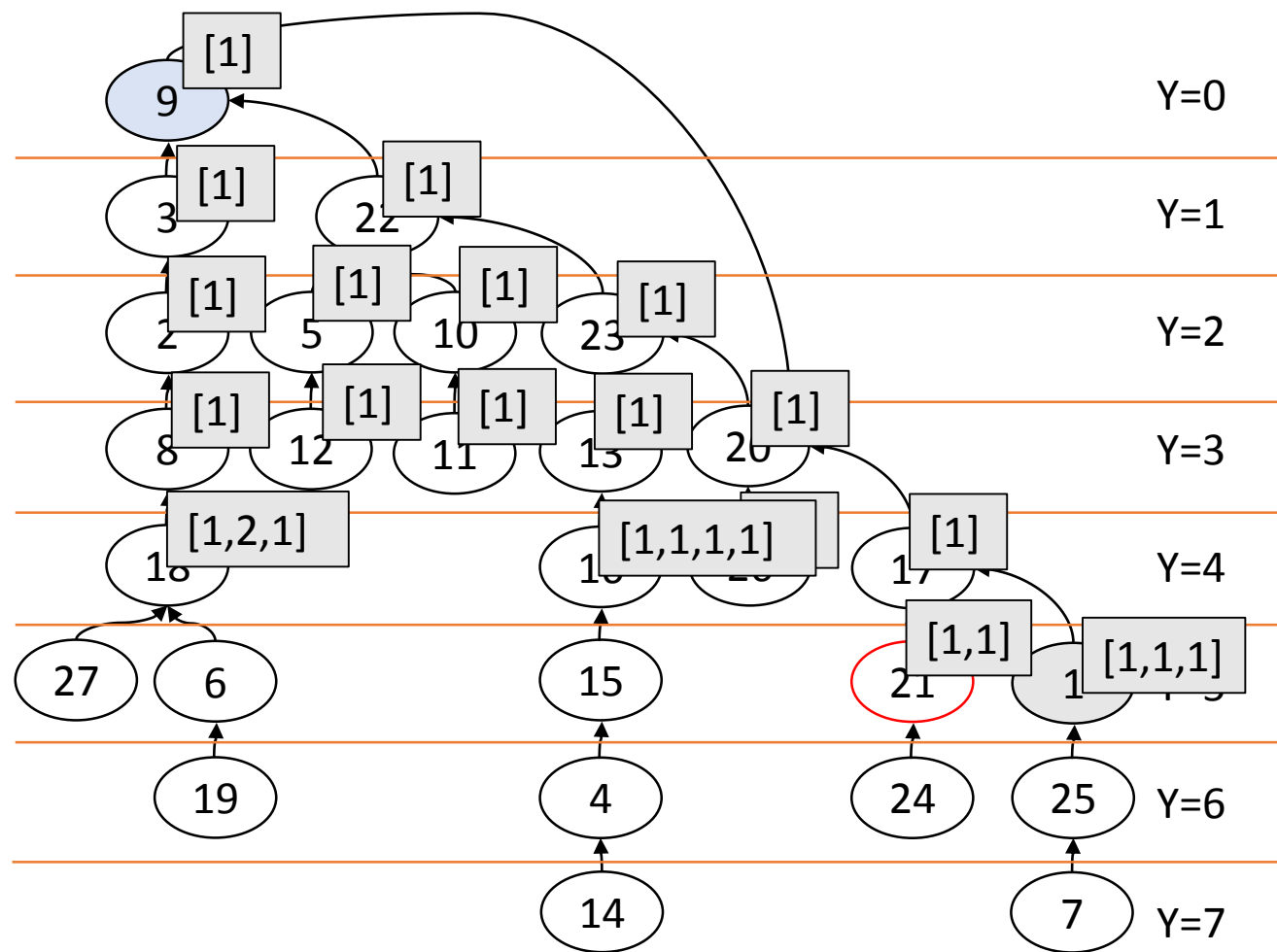
満点解法－場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



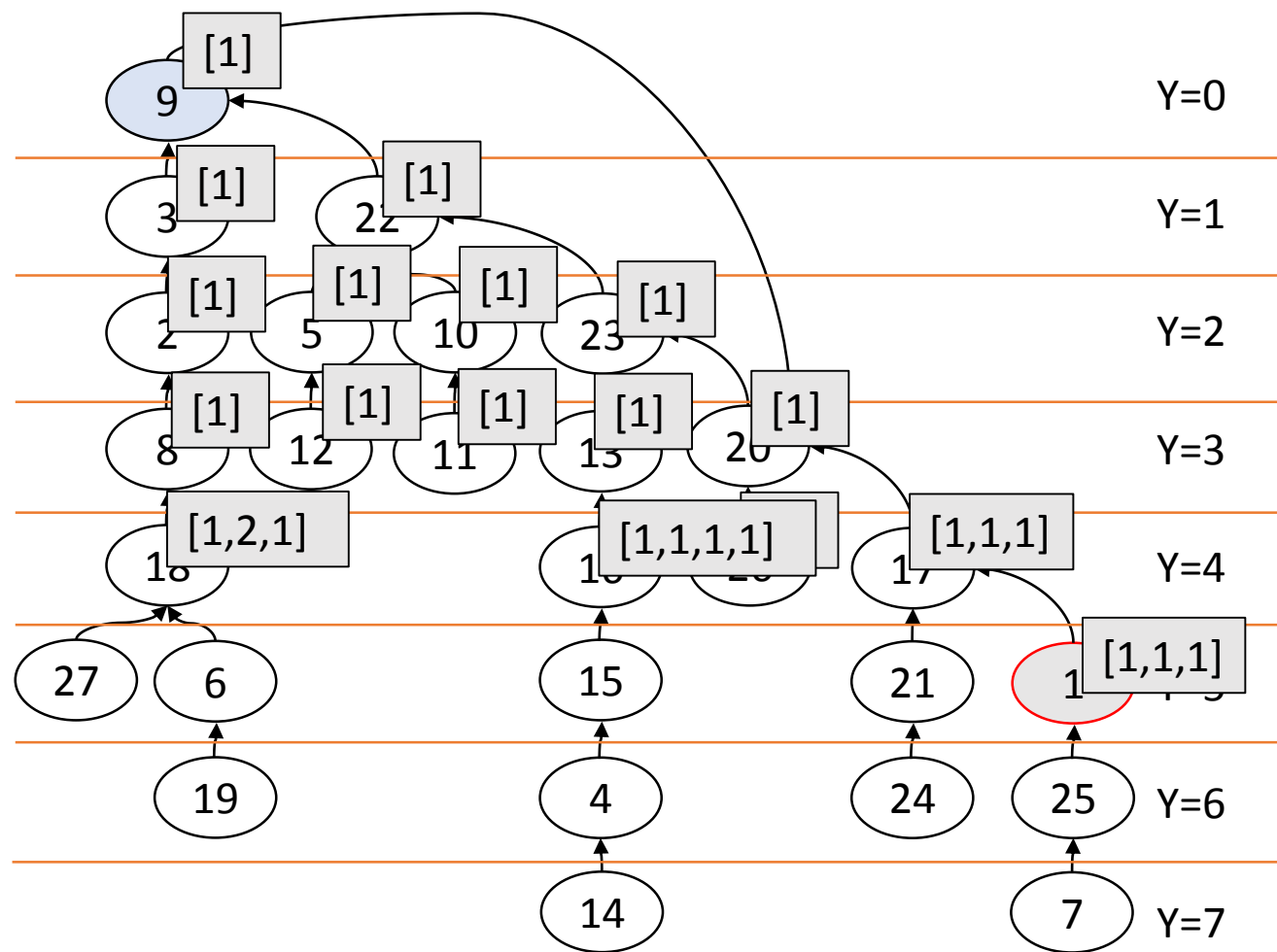
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



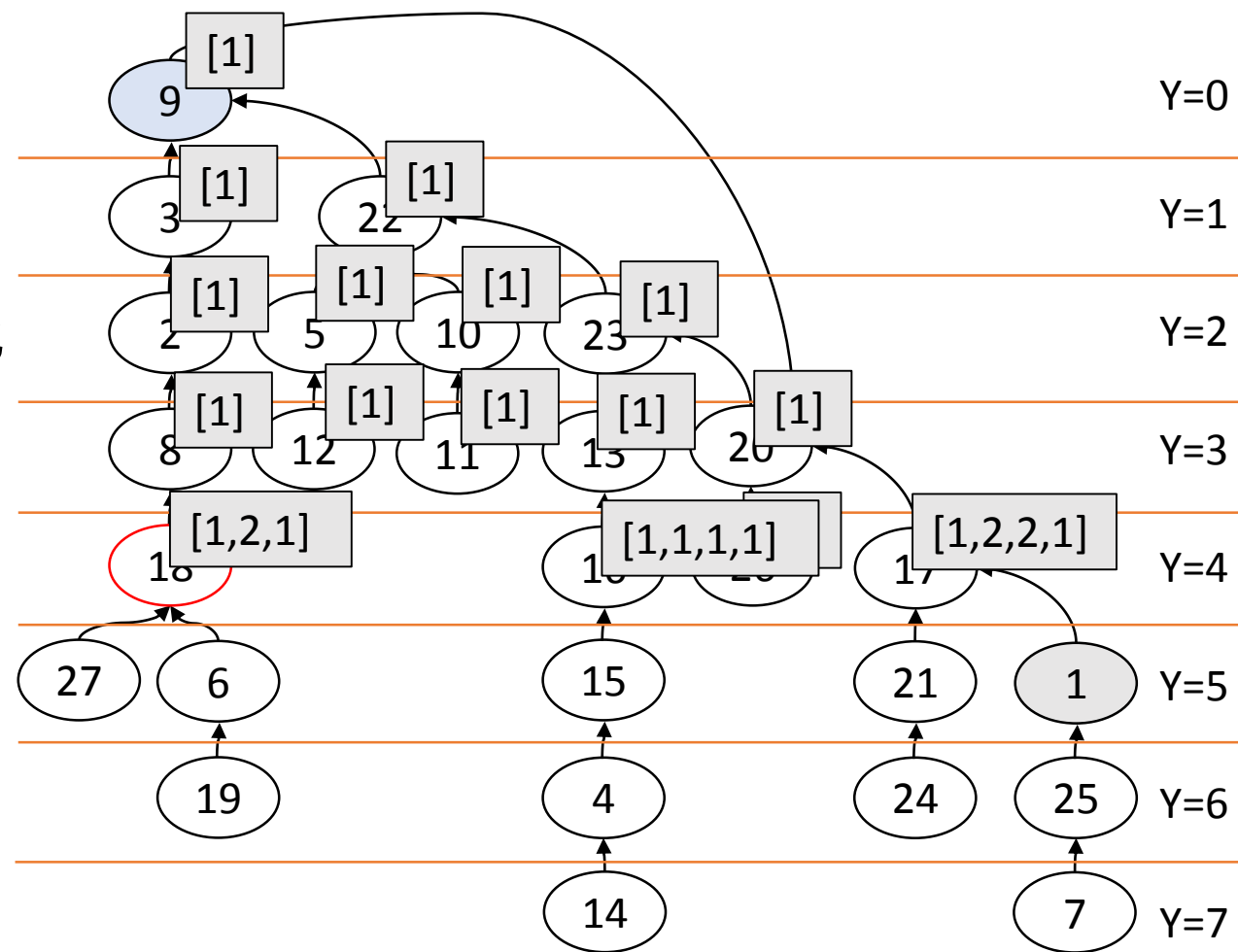
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



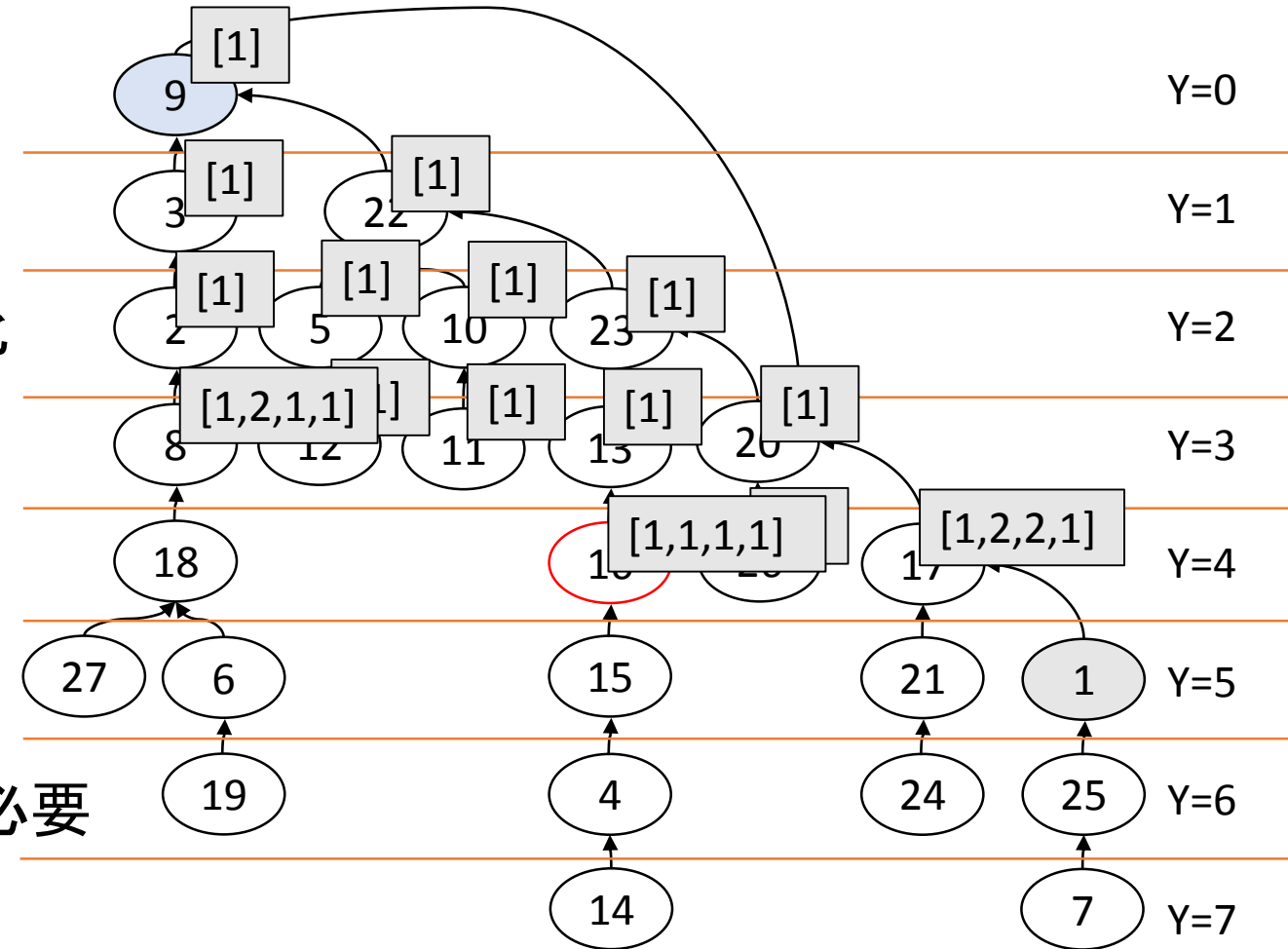
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



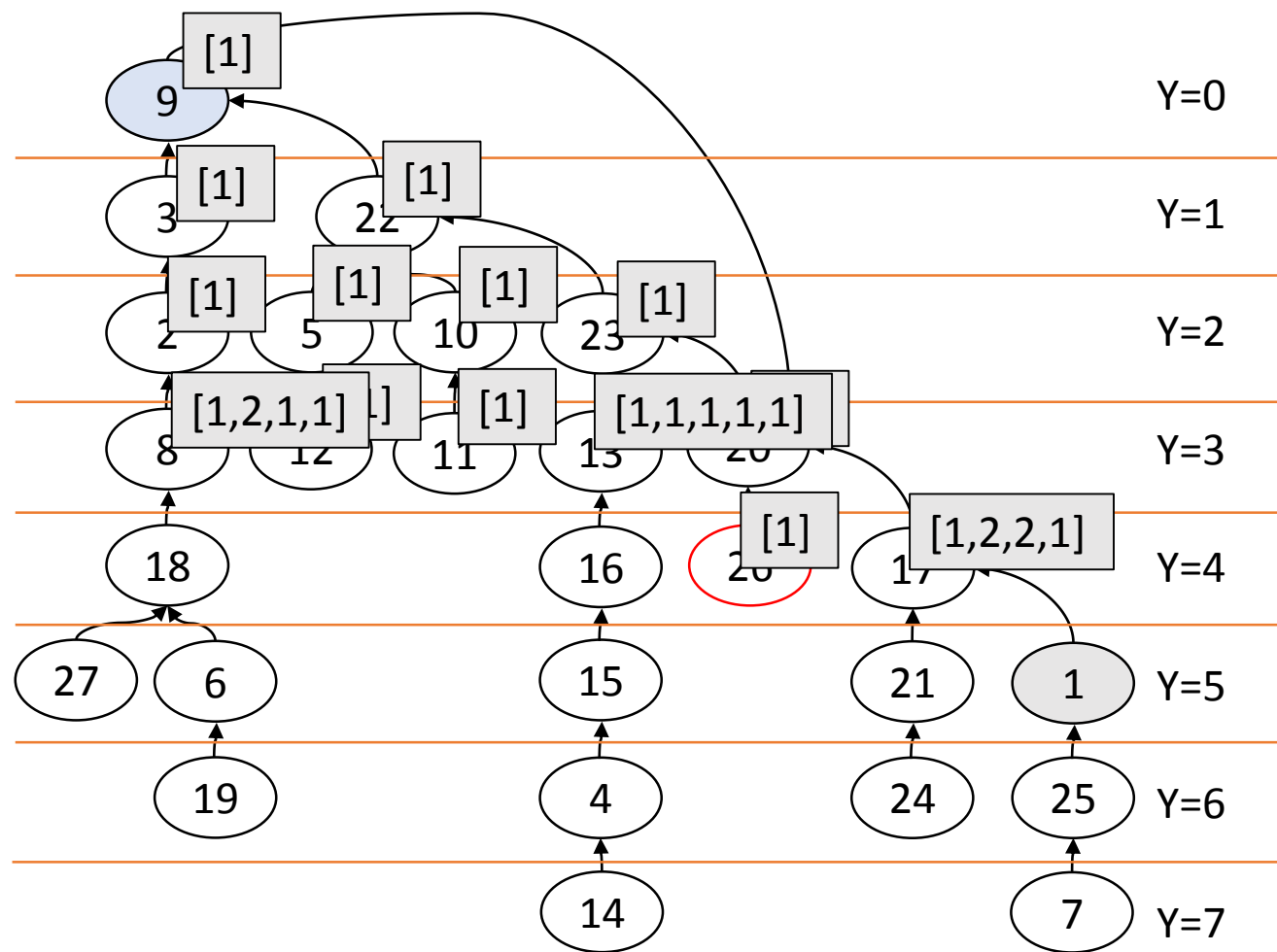
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査
 - Fenwick Treeの数字は、左ほど深い(Yが大きい)頂点の個数を表すように保持する
 - これはマージの効率化のために必要



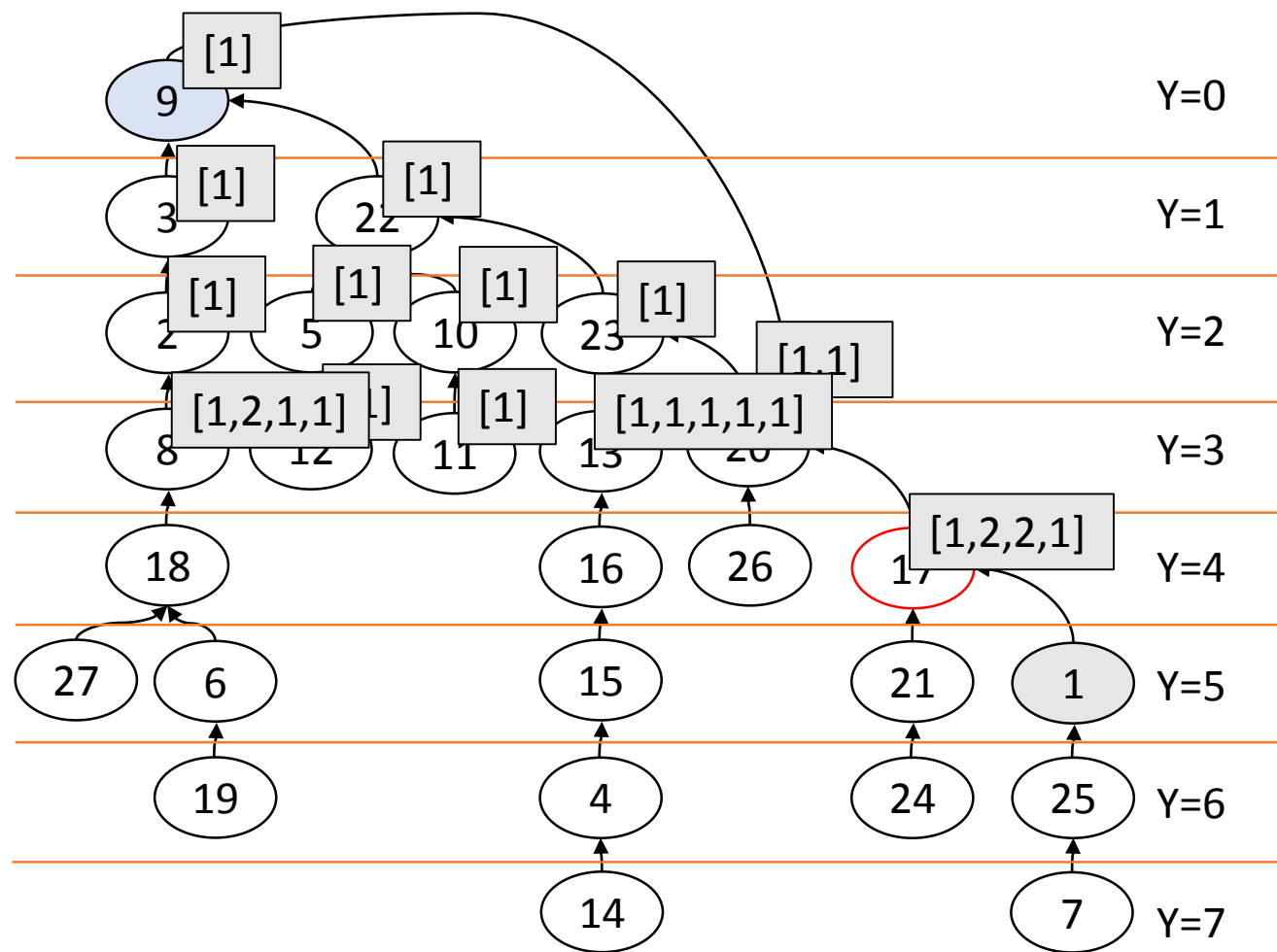
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



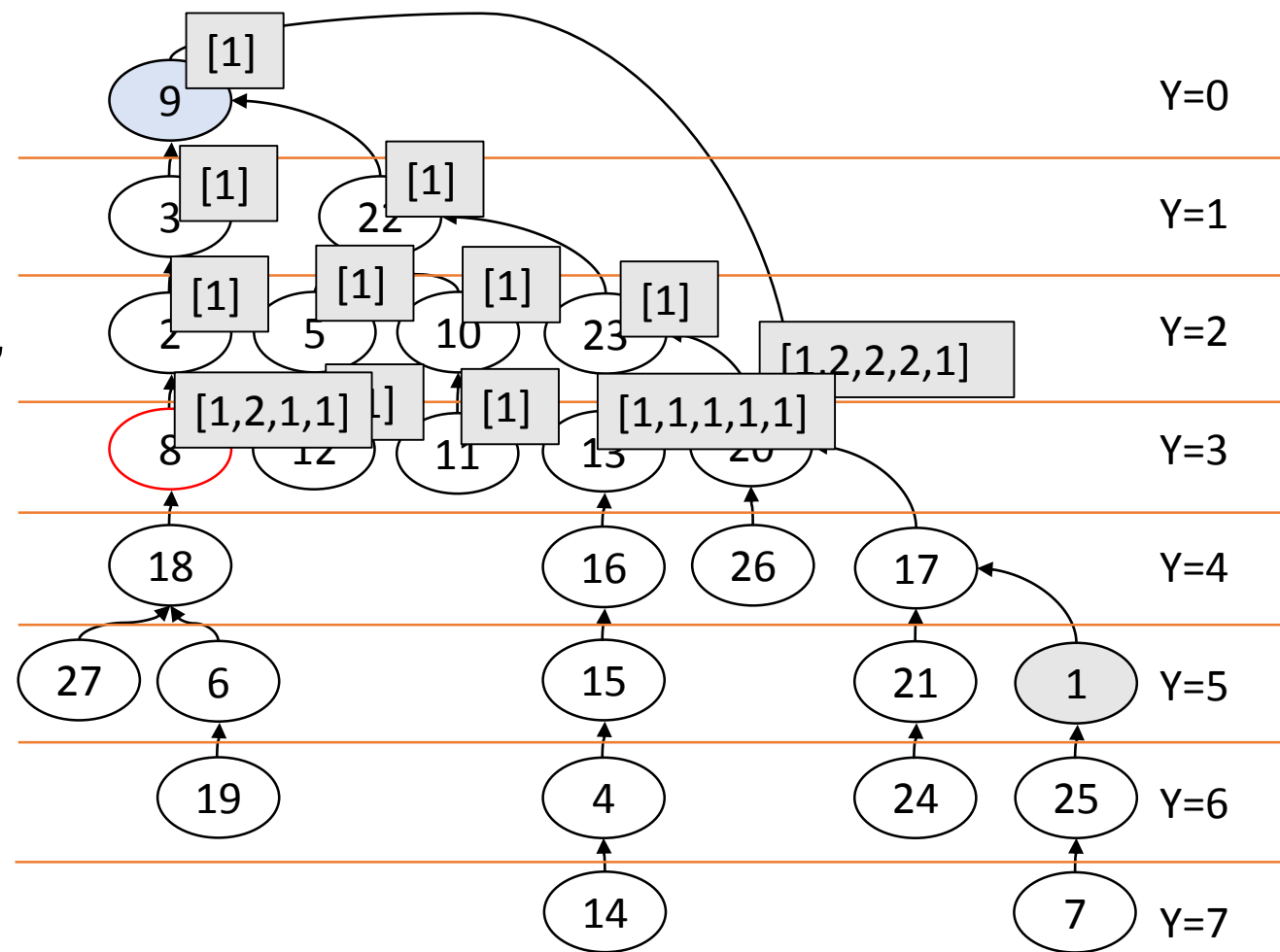
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



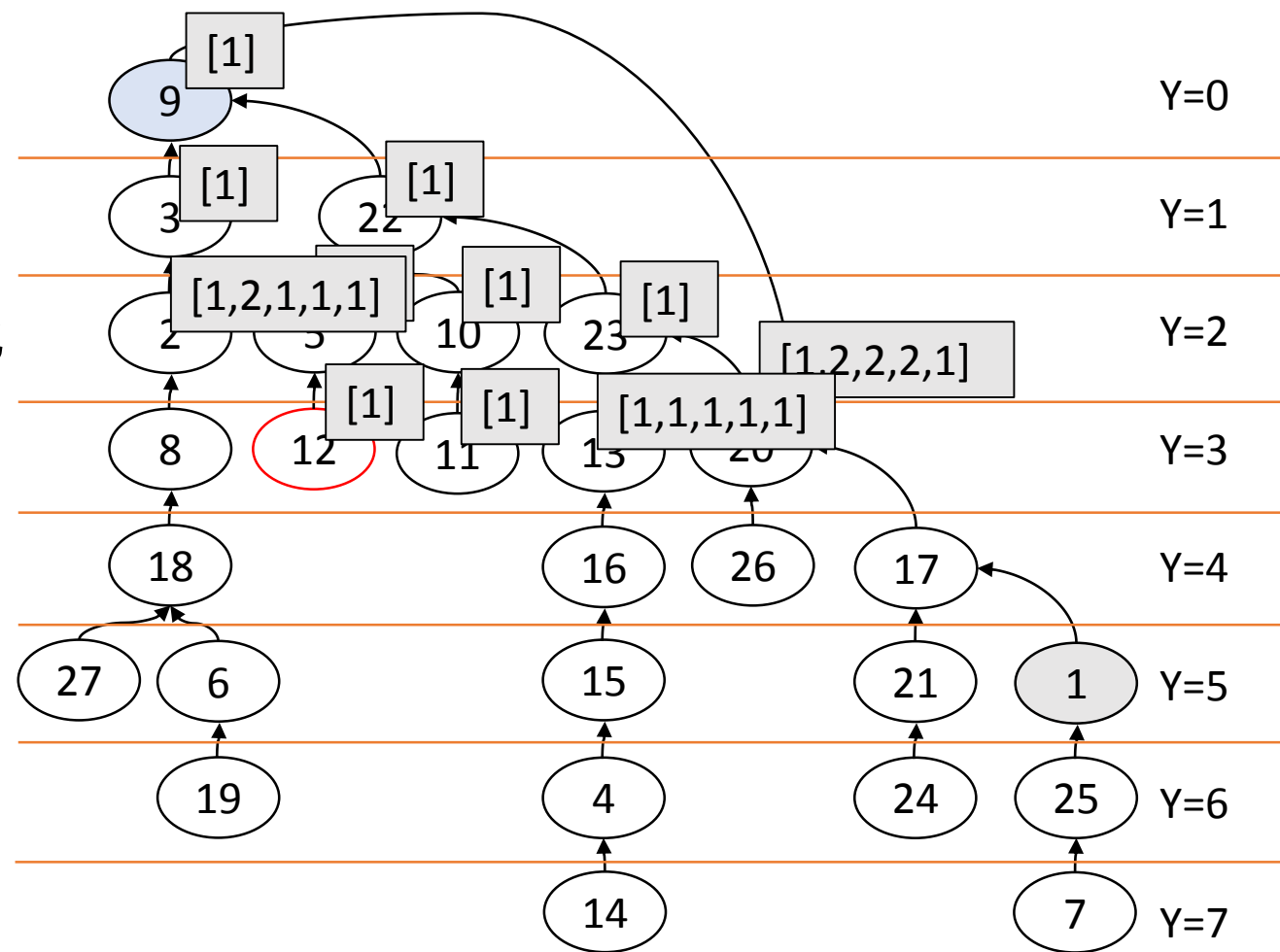
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



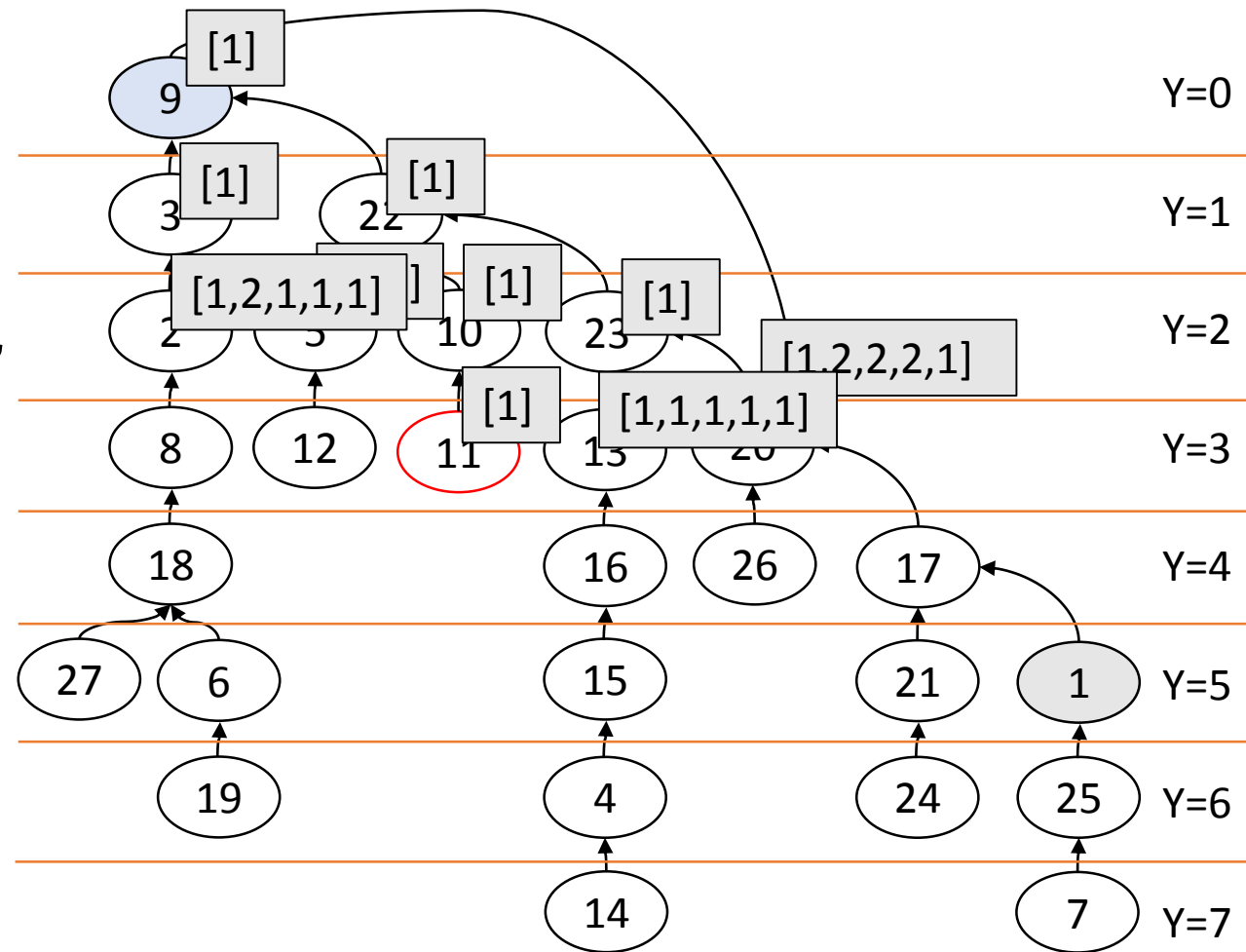
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



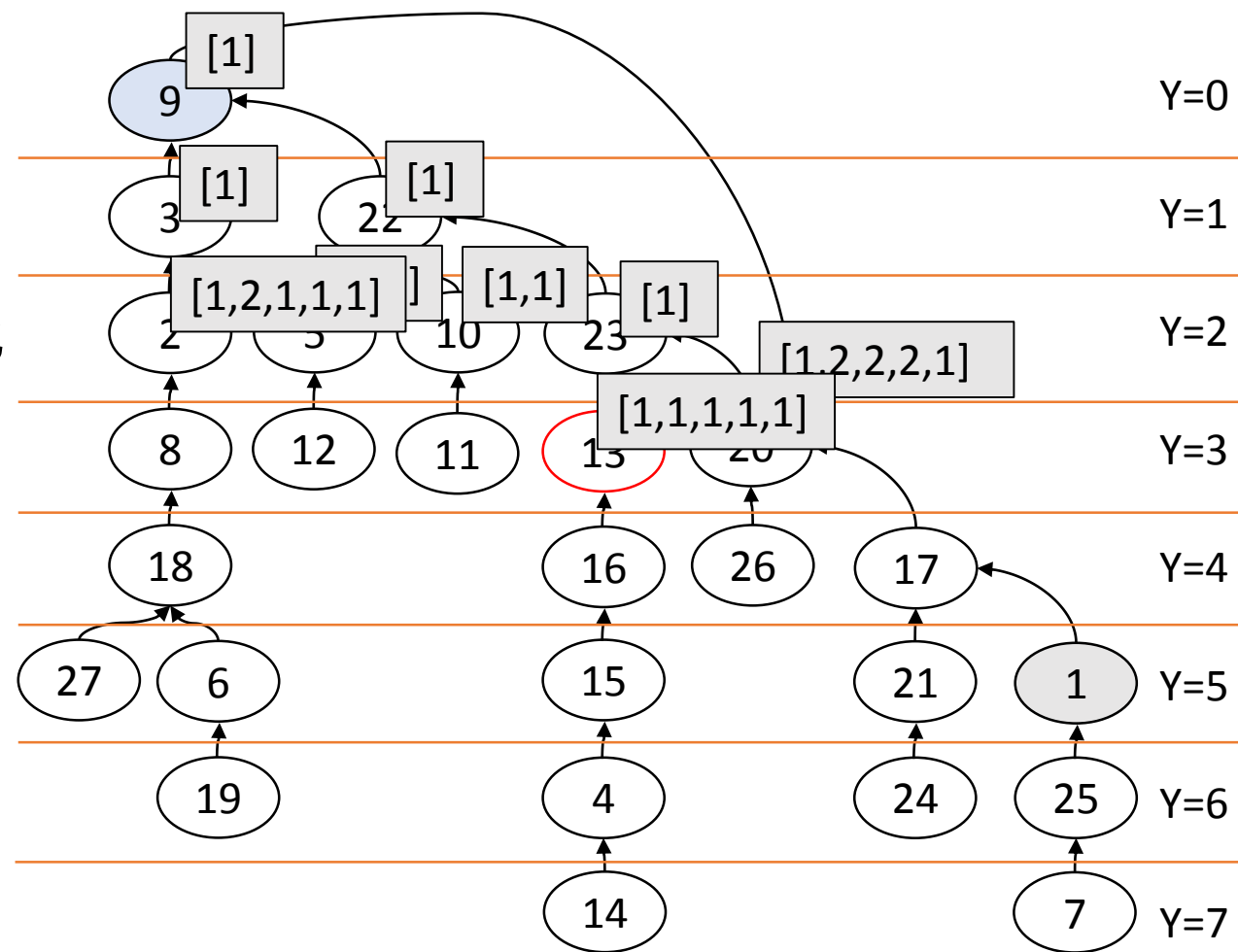
満点解法－場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



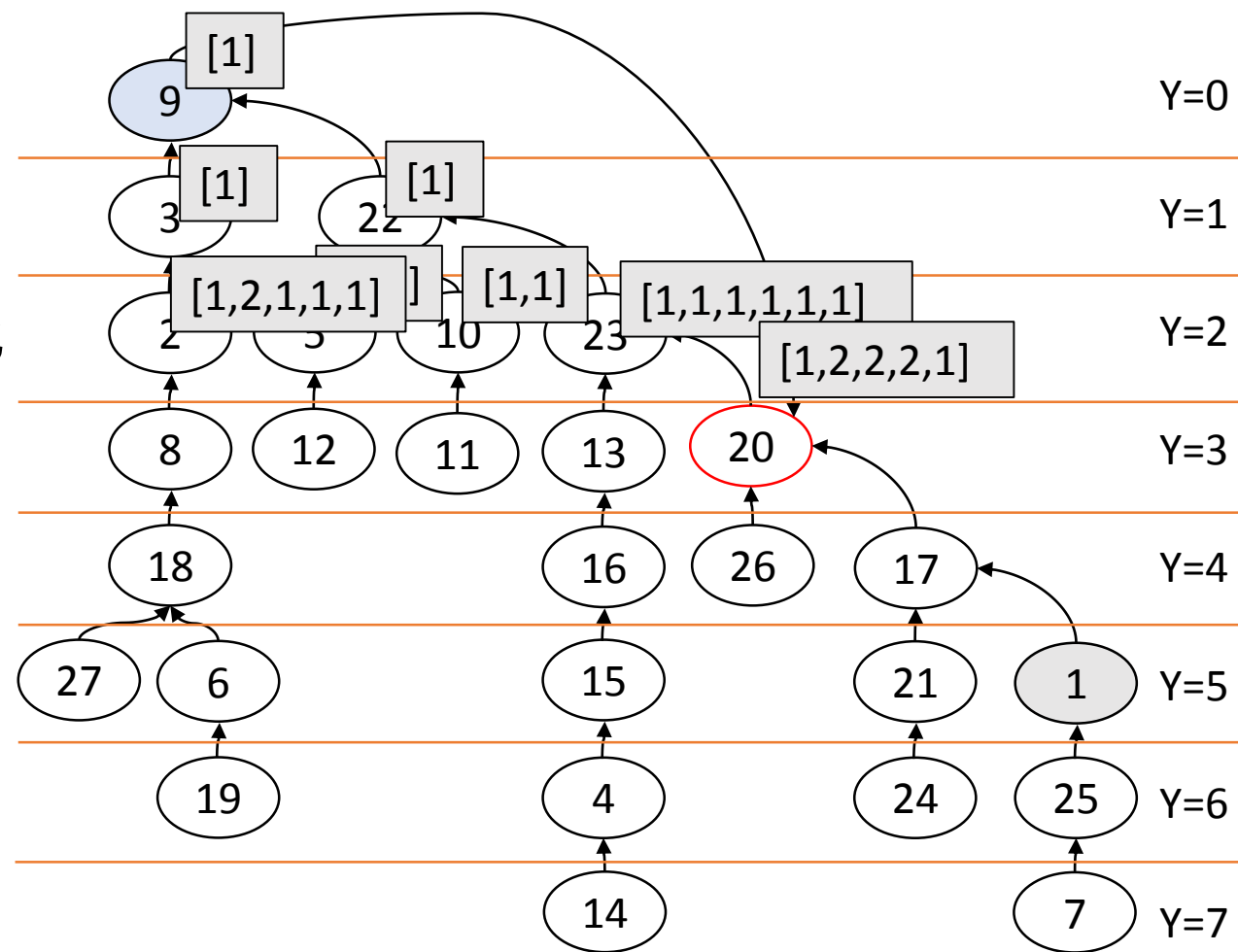
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



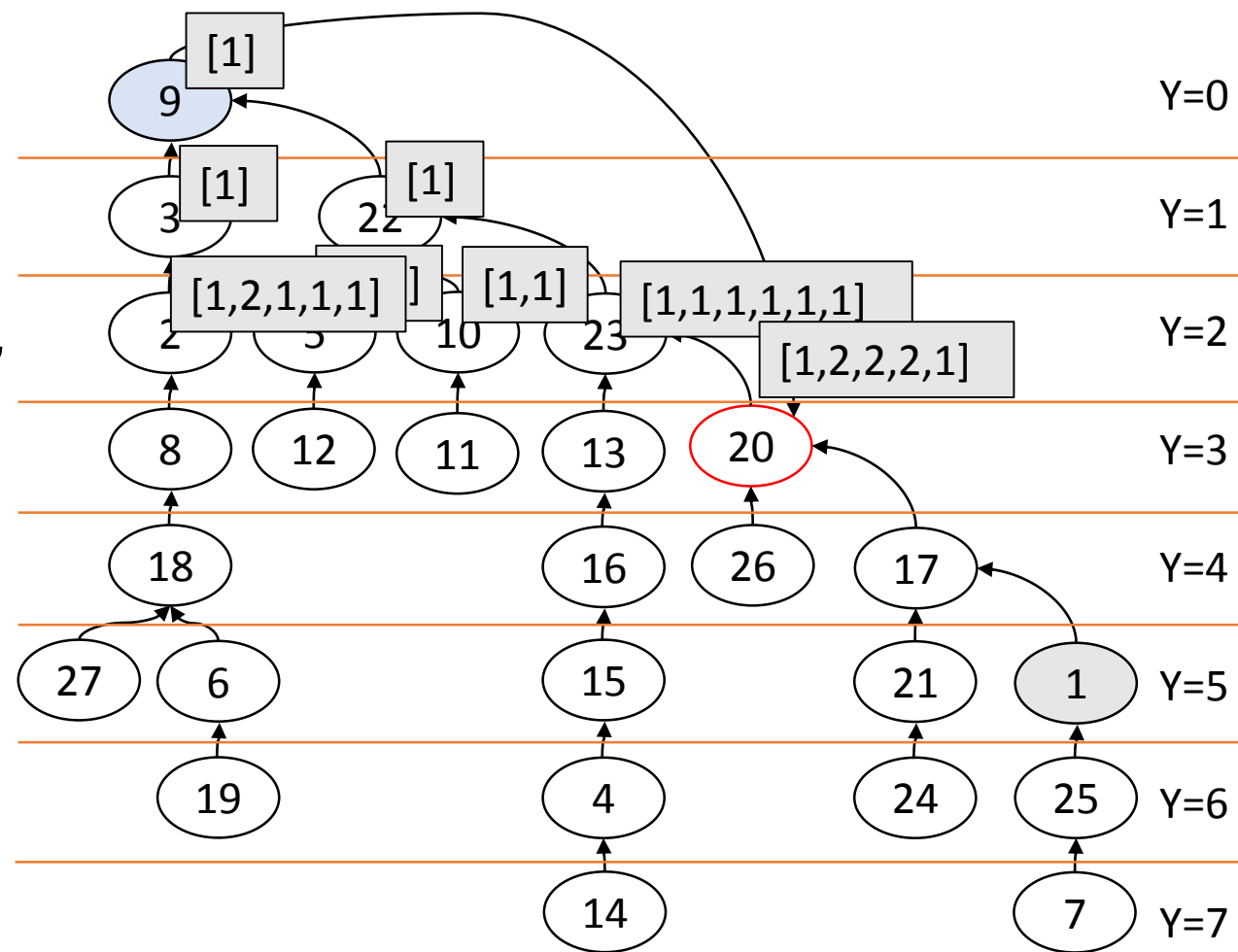
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



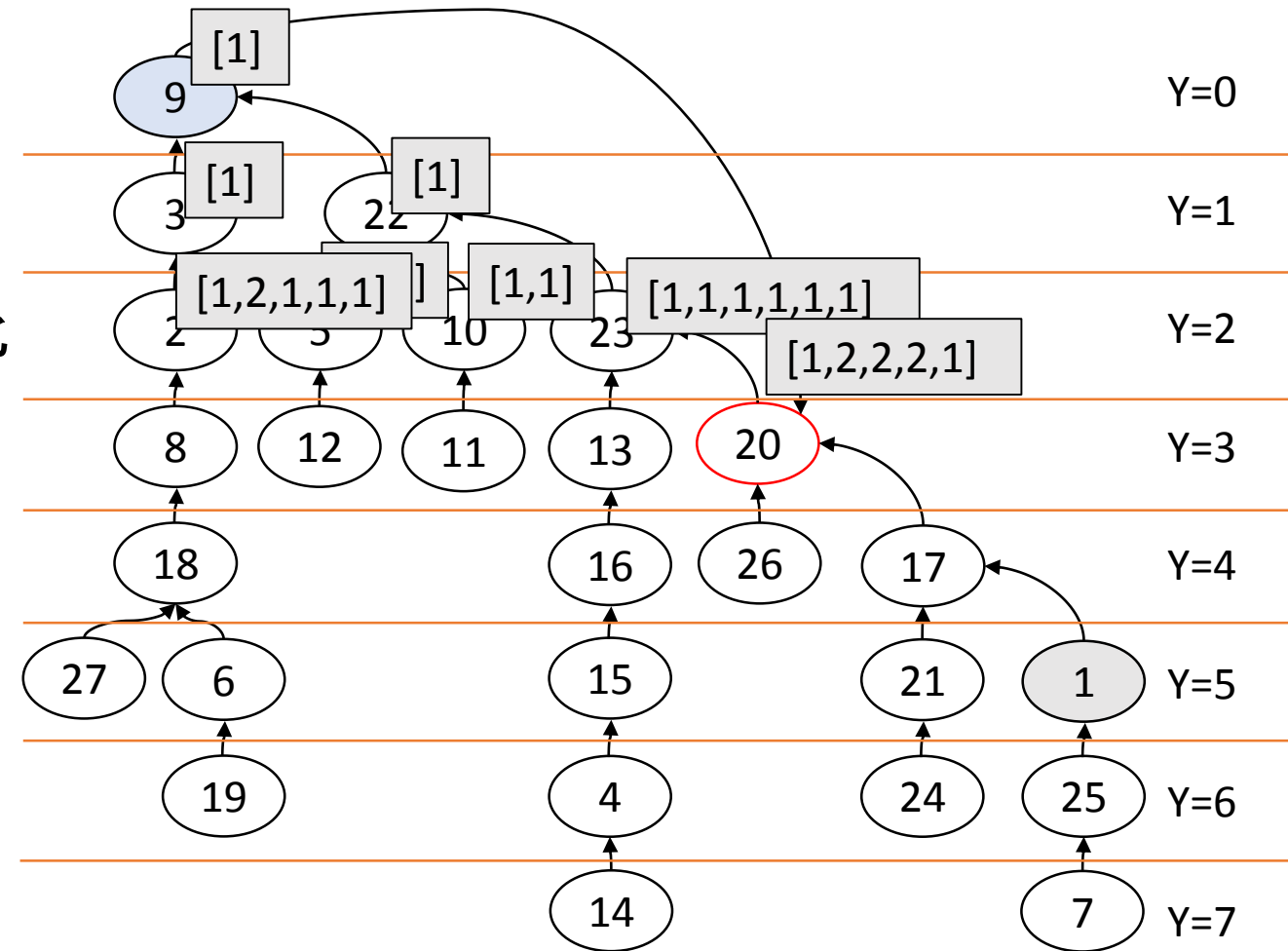
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査
 - この頂点の処理には注意



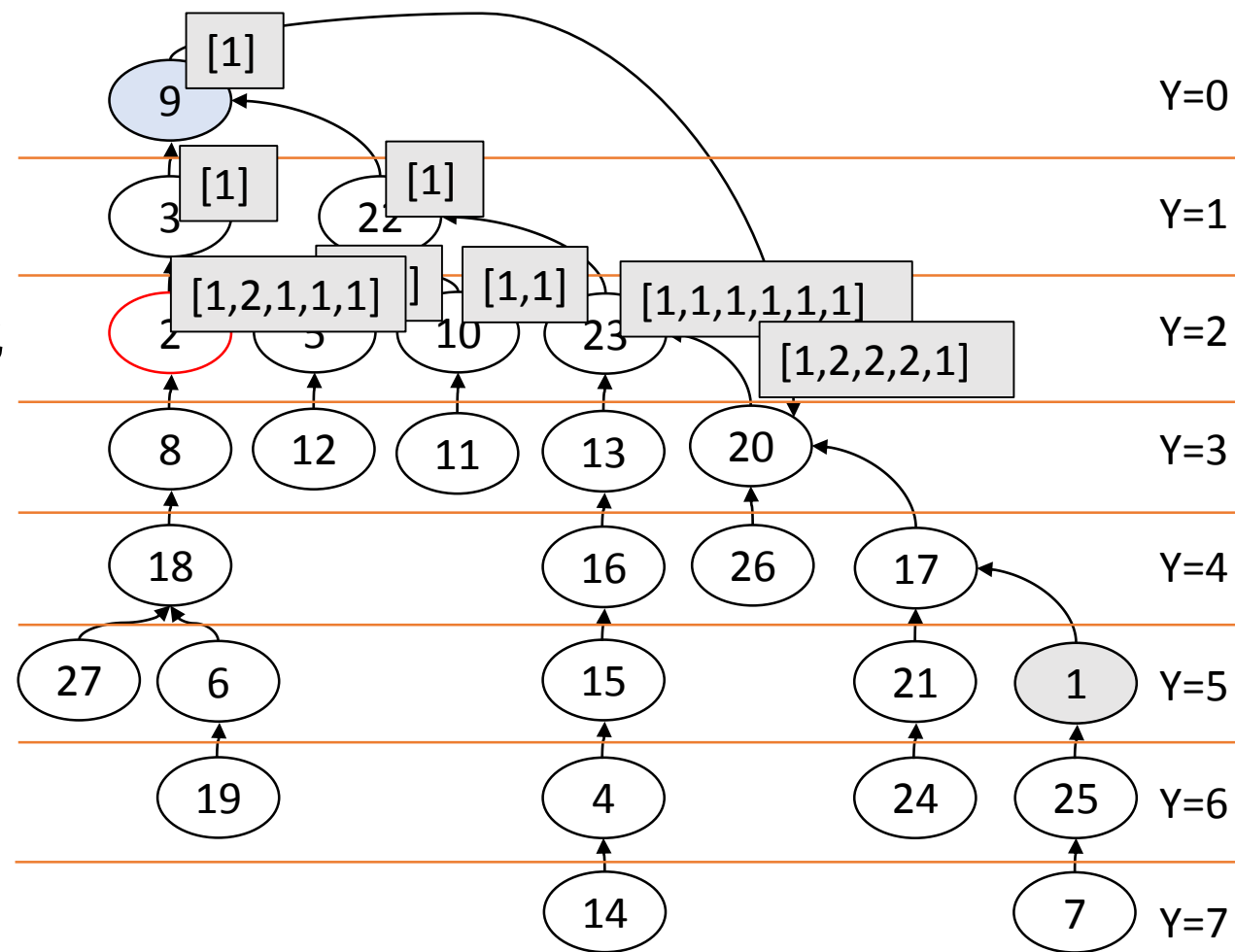
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査
 - この頂点の処理には注意
(3B)の場合、この個数は親に
遺伝させないようにするとよい



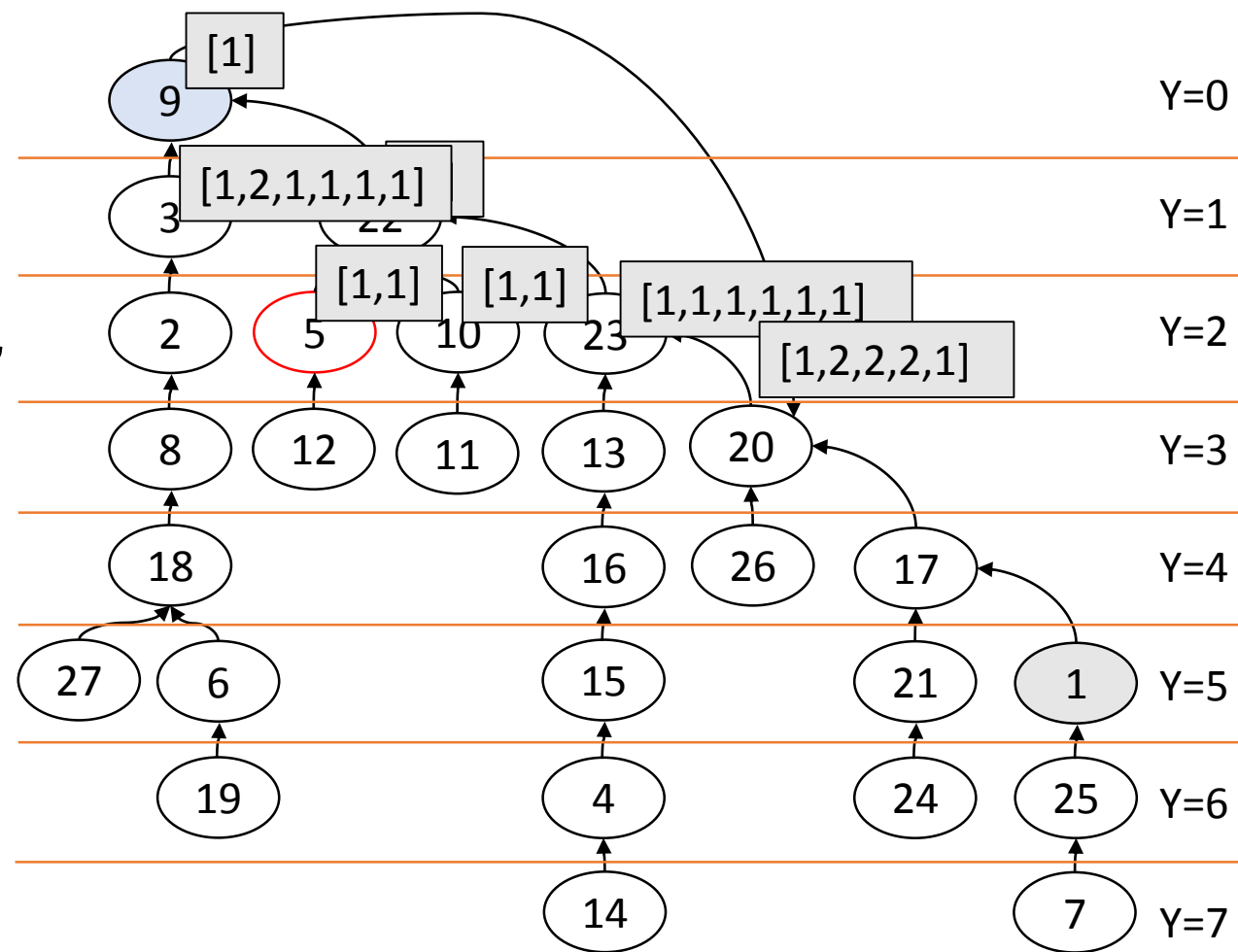
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



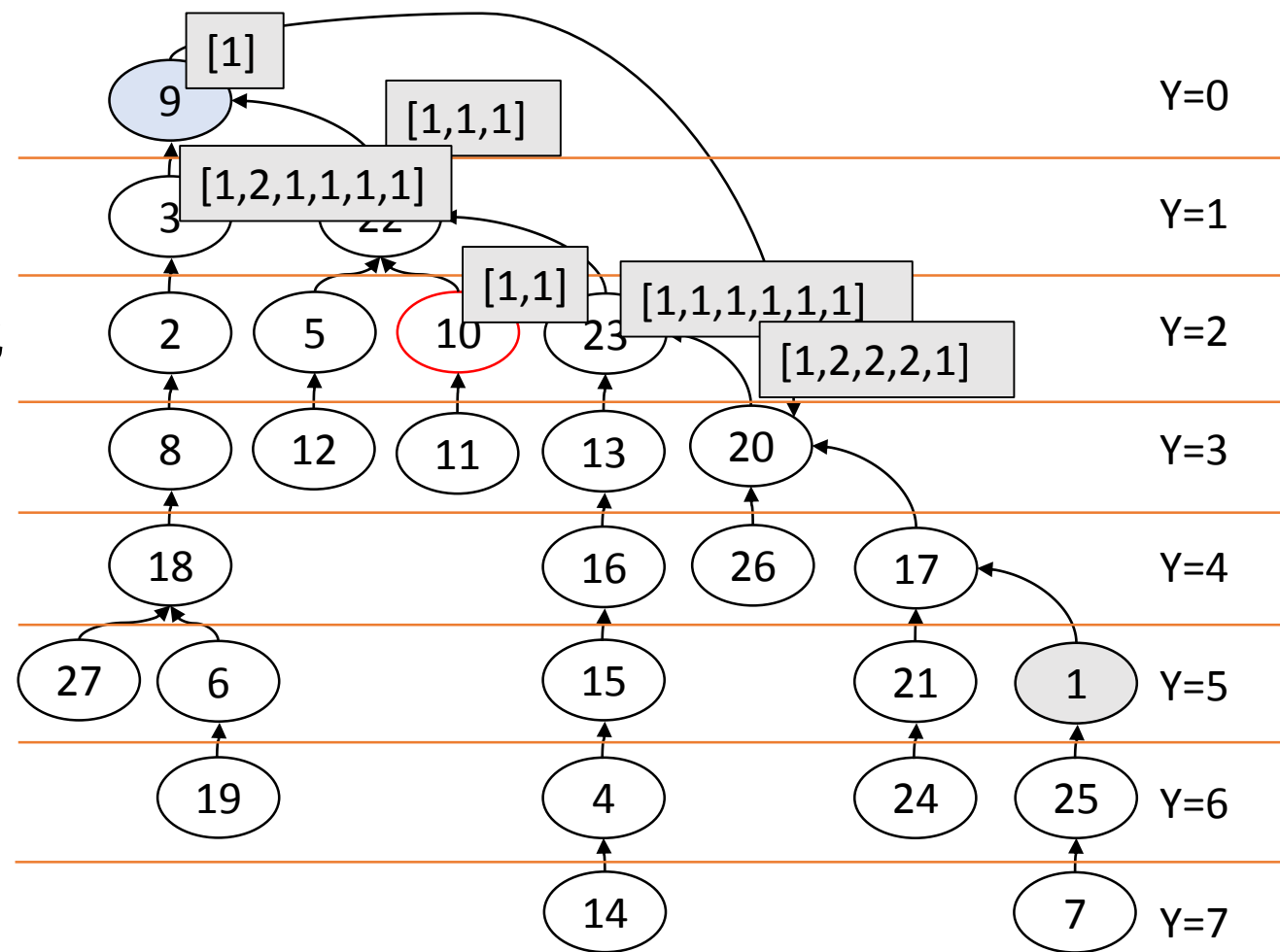
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



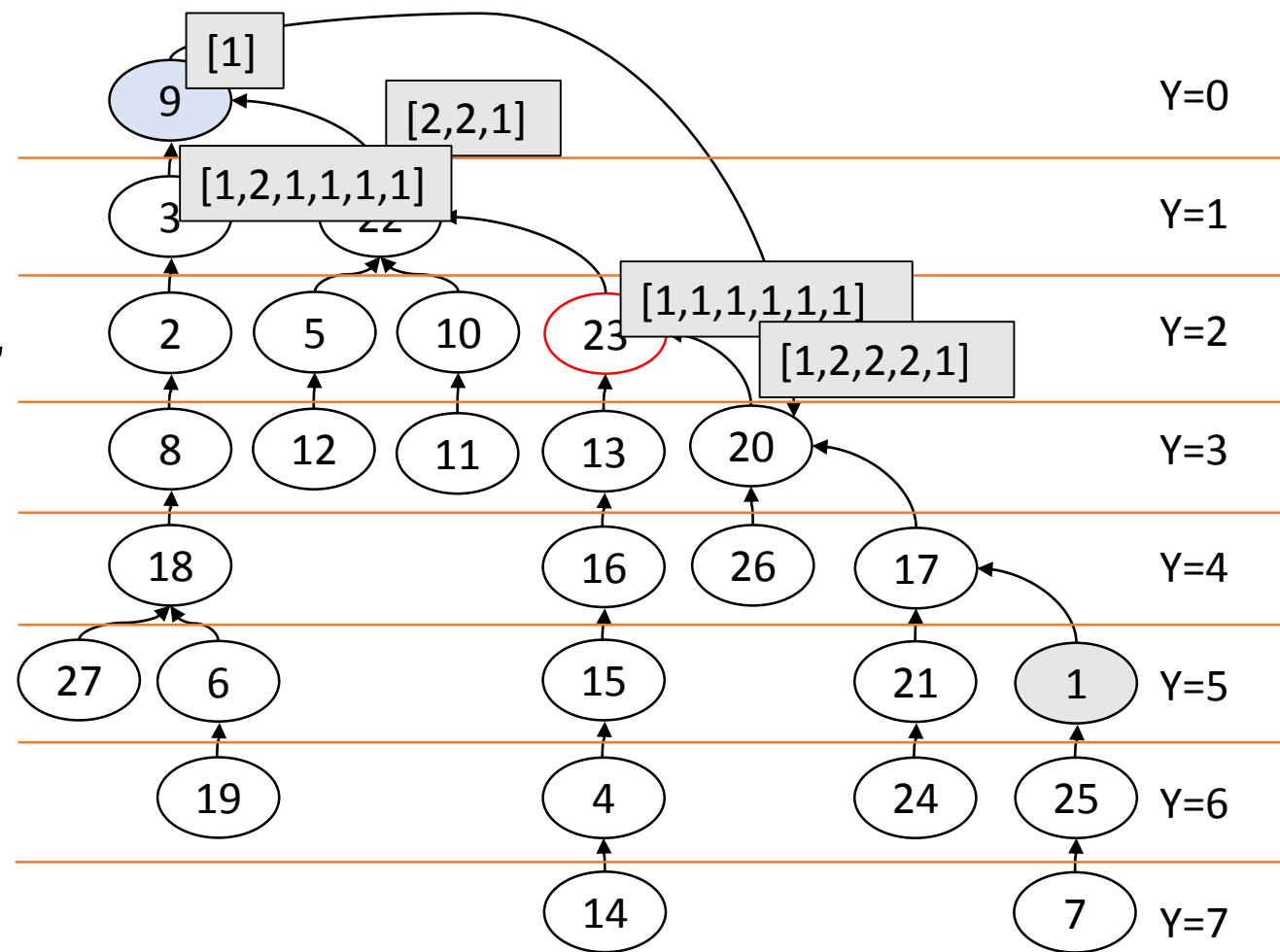
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



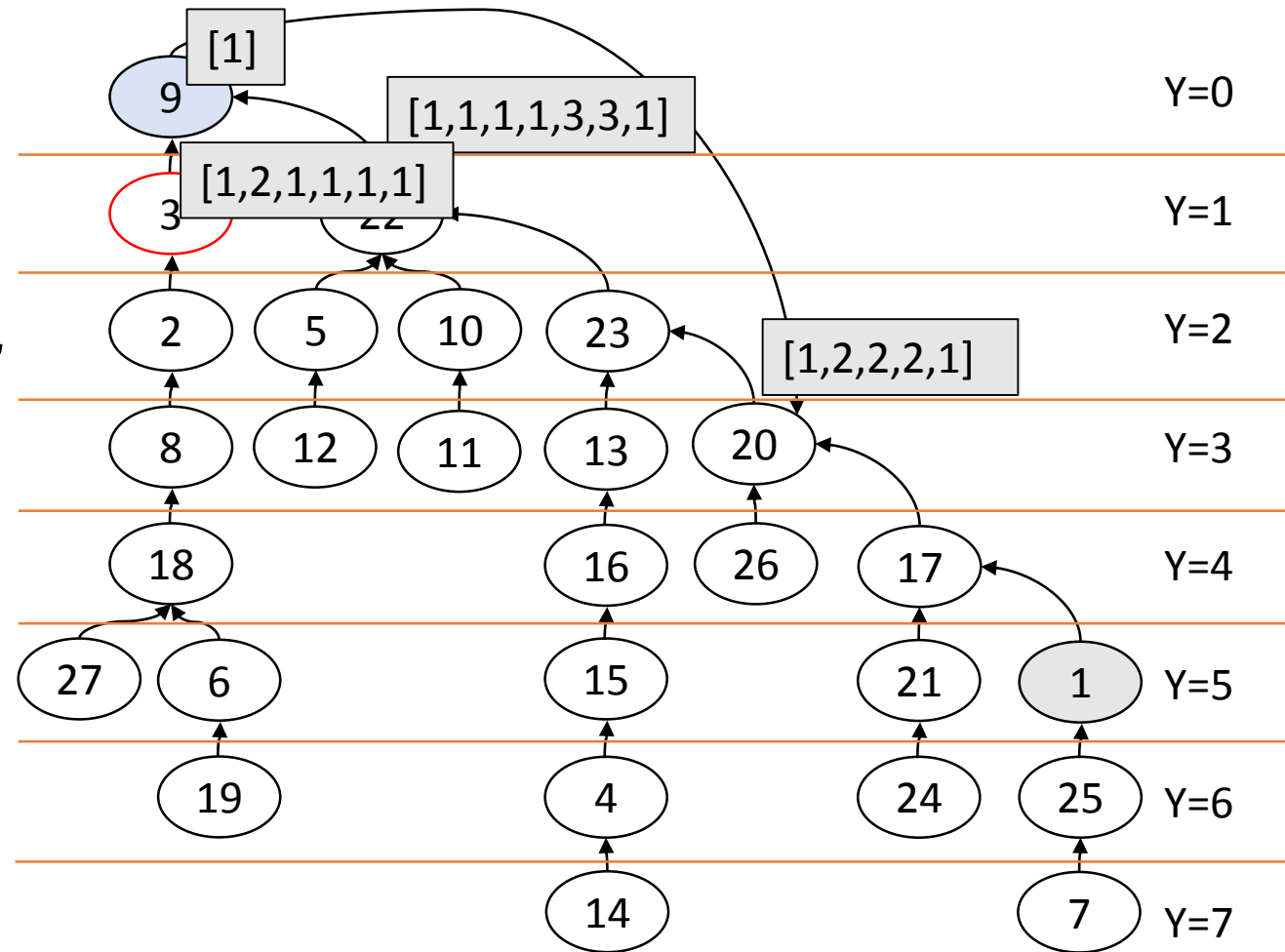
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



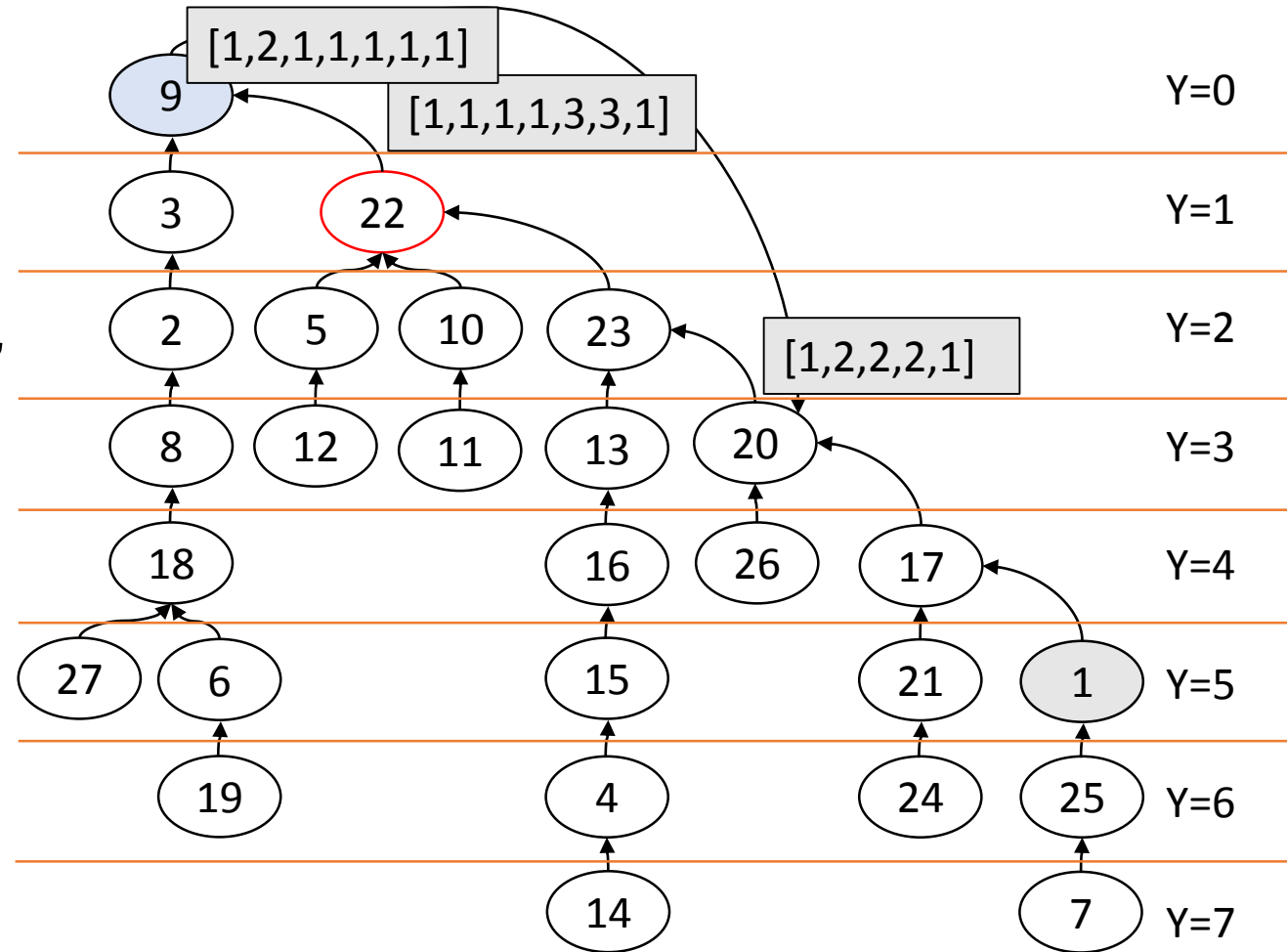
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



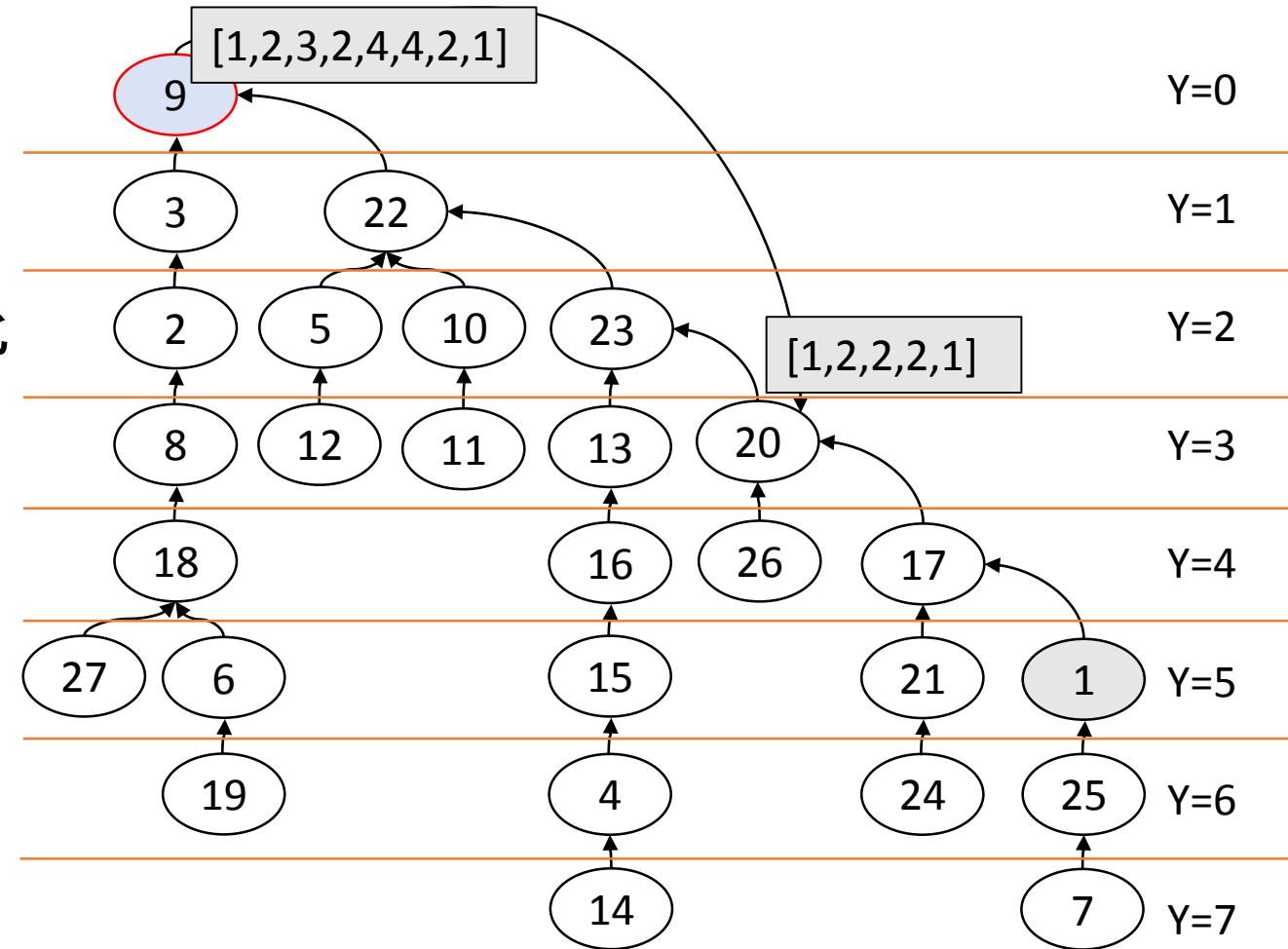
満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



満点解法 – 場合分け(3B)

- Bの候補を高速に数えたい
- 各頂点でFenwick Treeを保持
 - はじめは[1]を表す木として初期化
- イベントを処理しながら下から走査



満点解法 – 場合分け(3B)

- Fenwick Treeのマージ

満点解法 – 場合分け(3B)

- Fenwick Treeのマージ
- 親が $[1,2,3]$ 、子が $[2,5,6]$ というデータを保持していたとする

満点解法 – 場合分け(3B)

- Fenwick Treeのマージ
- 親が $[1,2,3]$ 、子が $[2,5,6]$ というデータを保持していたとする
- 子の末尾に0を付加する $[2,5,6,0]$

満点解法 – 場合分け(3B)

- Fenwick Treeのマージ
- 親が $[1,2,3]$ 、子が $[2,5,6]$ というデータを保持していたとする
- 子の末尾に0を付加する $[2,5,6,0]$
- 親の $[1,2,3]$ と $[2,5,6,0]$ を右揃えで加算し、新しい列を作る

満点解法 – 場合分け(3B)

- Fenwick Treeのマージ
- 親が $[1,2,3]$ 、子が $[2,5,6]$ というデータを保持していたとする
- 子の末尾に0を付加する $[2,5,6,0]$
- 親の $[1,2,3]$ と $[2,5,6,0]$ を右揃えで加算し、新しい列を作る
- $\rightarrow [2,6,8,3]$

満点解法 – 場合分け(3B)

- Fenwick Treeのマージ
- 親が $[1,2,3]$ 、子が $[2,5,6]$ というデータを保持していたとする
- 子の末尾に0を付加する $[2,5,6,0]$
- 親の $[1,2,3]$ と $[2,5,6,0]$ を右揃えで加算し、新しい列を作る
- $\rightarrow [2,6,8,3]$
- このとき、小さい方のデータを大きい方のデータに加えるようにする

満点解法 – 場合分け(3B)

- Fenwick Treeのマージ
- 親が $[1,2,3]$ 、子が $[2,5,6]$ というデータを保持していたとする
- 子の末尾に0を付加する $[2,5,6,0]$
- 親の $[1,2,3]$ と $[2,5,6,0]$ を右揃えで加算し、新しい列を作る
- $\rightarrow [2,6,8,3]$
- このとき、小さい方のデータを大きい方のデータに加えるようにする
- $\rightarrow$ データ構造をマージする一般的なテク。全体で $O(n \log n)$
 - 対数は2乗にはならない

満点解法 – 場合分け(3B)

- イベント処理の計算量

満点解法 – 場合分け(3B)

- イベント処理の計算量
- イベント：到達先候補ごとに処理していた

満点解法 – 場合分け(3B)

- イベント処理の計算量
- イベント：到達先候補ごとに処理していた
- 到達先候補ごとのイベント数が均等ならば、イベント処理回数は $O(n)$

満点解法 – 場合分け(3B)

- イベント処理の計算量
- イベント：到達先候補ごとに処理していた
- 到達先候補ごとのイベント数が均等ならば、イベント処理回数は $O(n)$
- 1つのイベントは $O(\log n)$ で処理できるから、 $O(n \log n)$

満点解法 – 場合分け(3B)

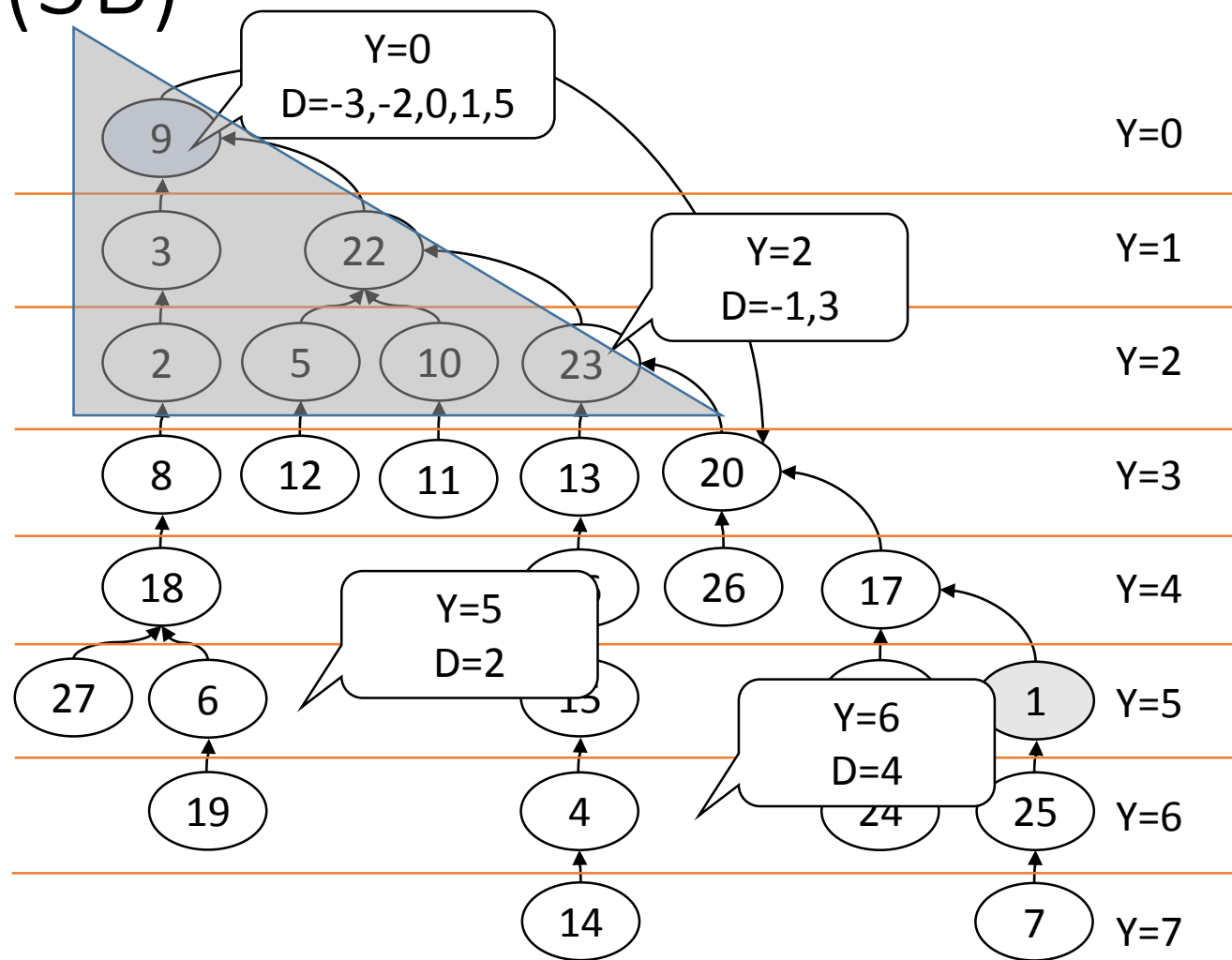
- イベント処理の計算量
- イベント：到達先候補ごとに処理していた
- 到達先候補ごとのイベント数が均等ならば、イベント処理回数は $O(n)$
- 1つのイベントは $O(\log n)$ で処理できるから、 $O(n \log n)$

満点解法 – 場合分け(3B)

- こういうケースがありそう:
 - $Y=2$ にたくさんの頂点がある かつ
 - $Y=2$ にたくさんのイベントがある
- この場合、イベント処理回数は $O(n^2)$ になりかねない

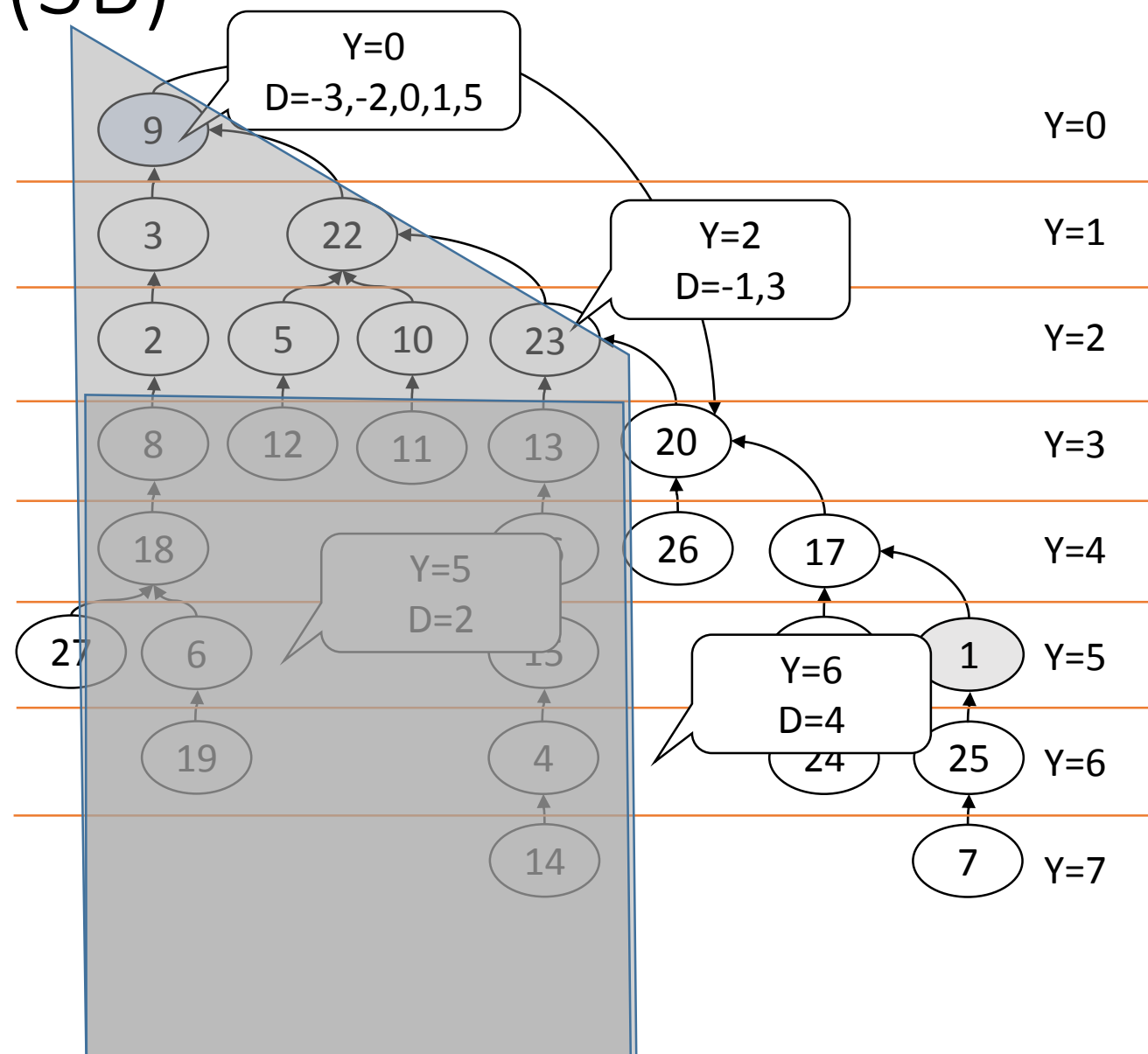
満点解法－場合分け(3B)

- ・イベントの図を思い出そう



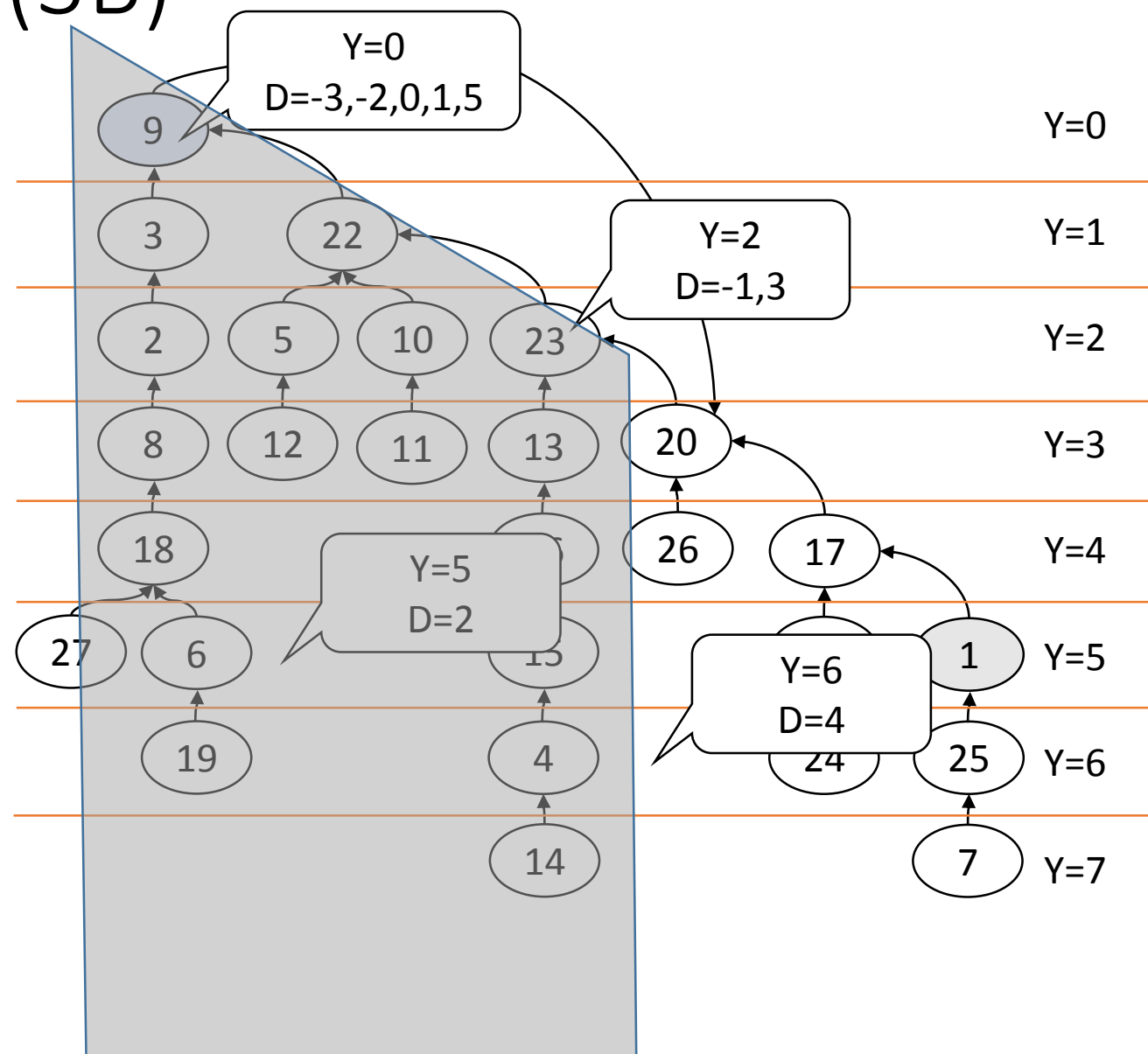
満点解法 – 場合分け(3B)

- 簡単のため、数え上げ処理を2つに分割



満点解法 – 場合分け(3B)

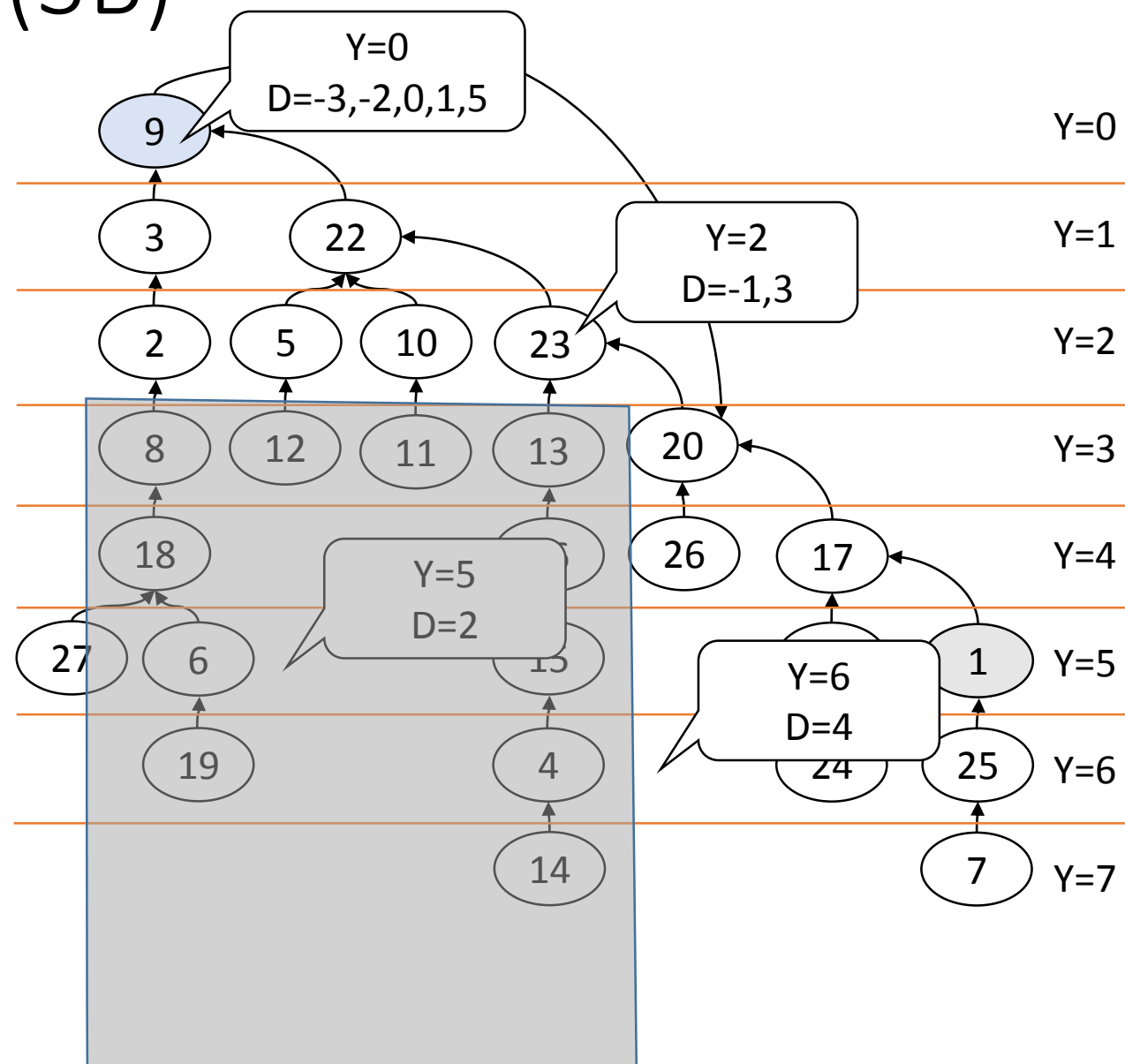
- 簡単のため、数え上げ処理を2つに分割
 - 加算処理



満点解法 – 場合分け(3B)

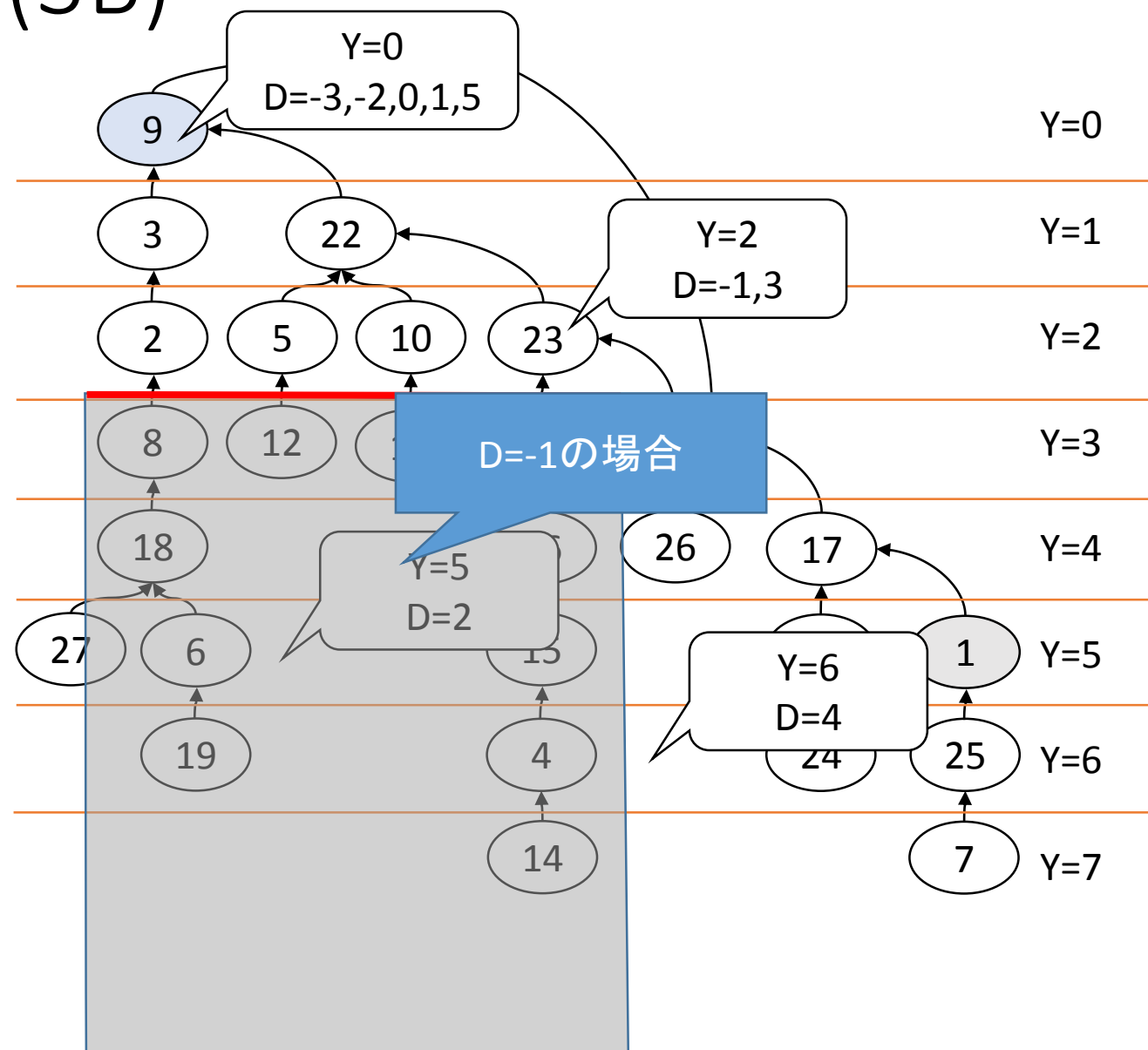
- 簡単のため、数え上げ処理を2つに分割

- 加算処理
- 減算処理



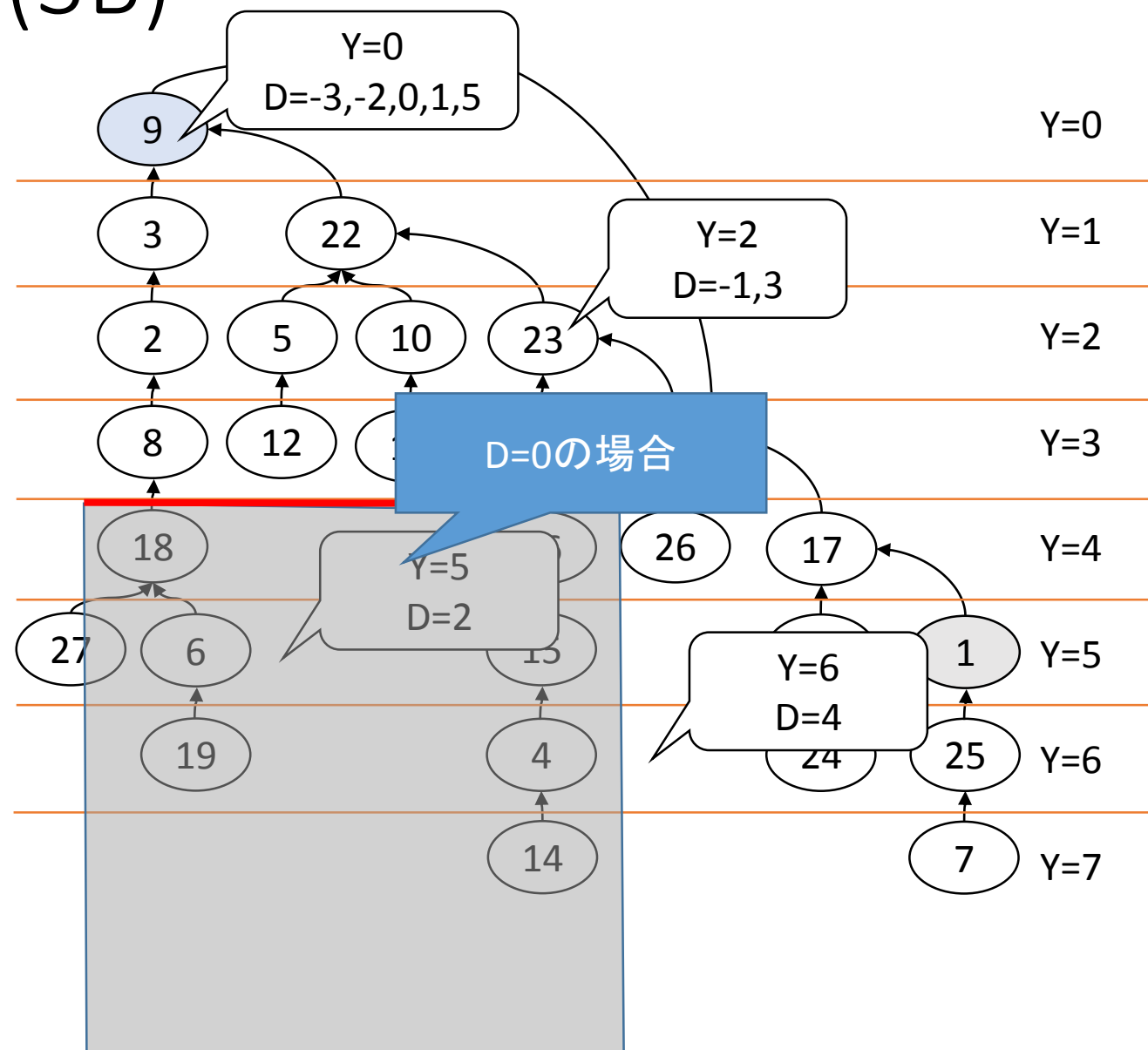
満点解法 – 場合分け(3B)

- 各イベントの数え上げ範囲はDが変化すると上下に動く



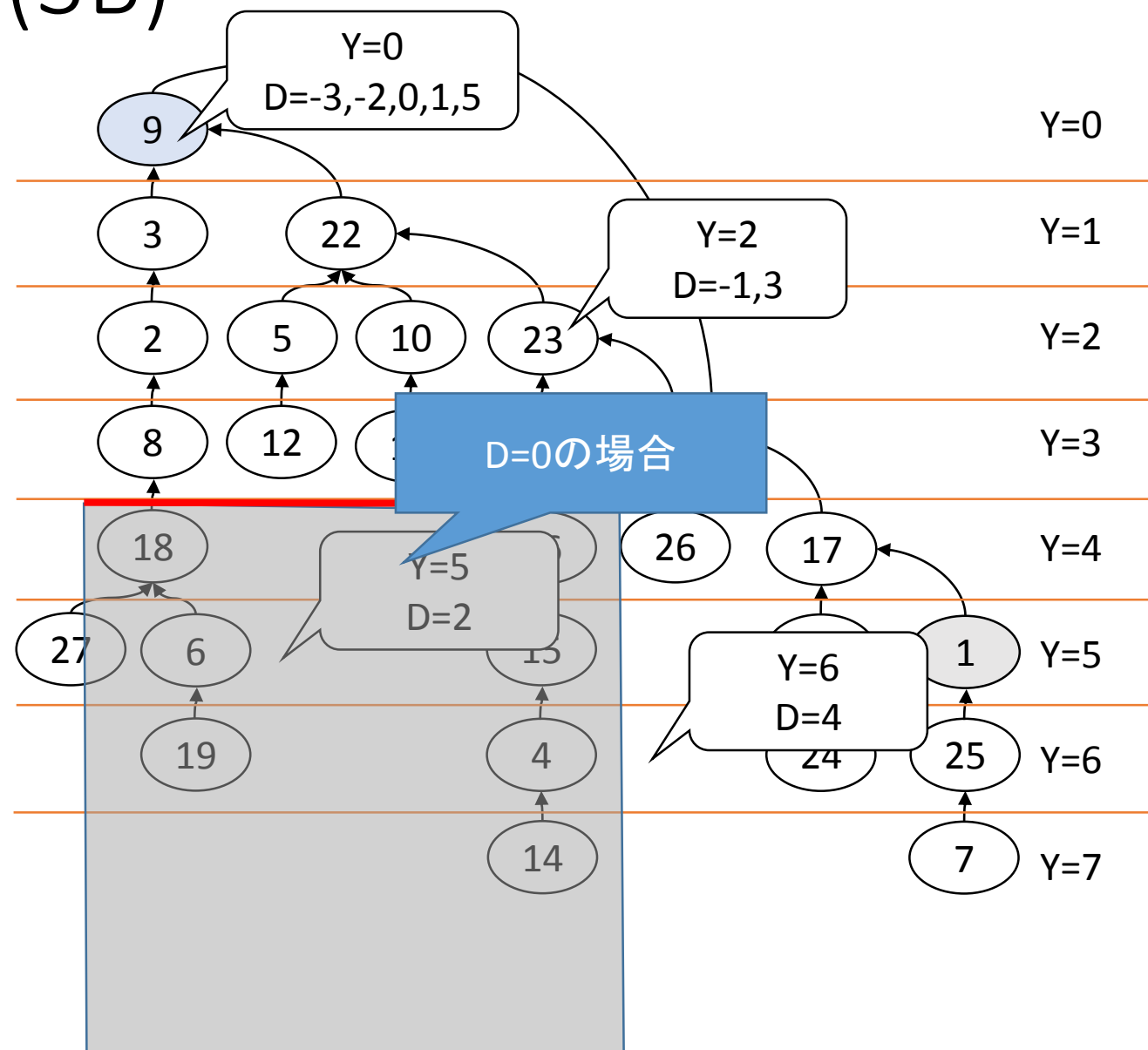
満点解法 – 場合分け(3B)

- 各イベントの数え上げ範囲はDが変化すると上下に動く



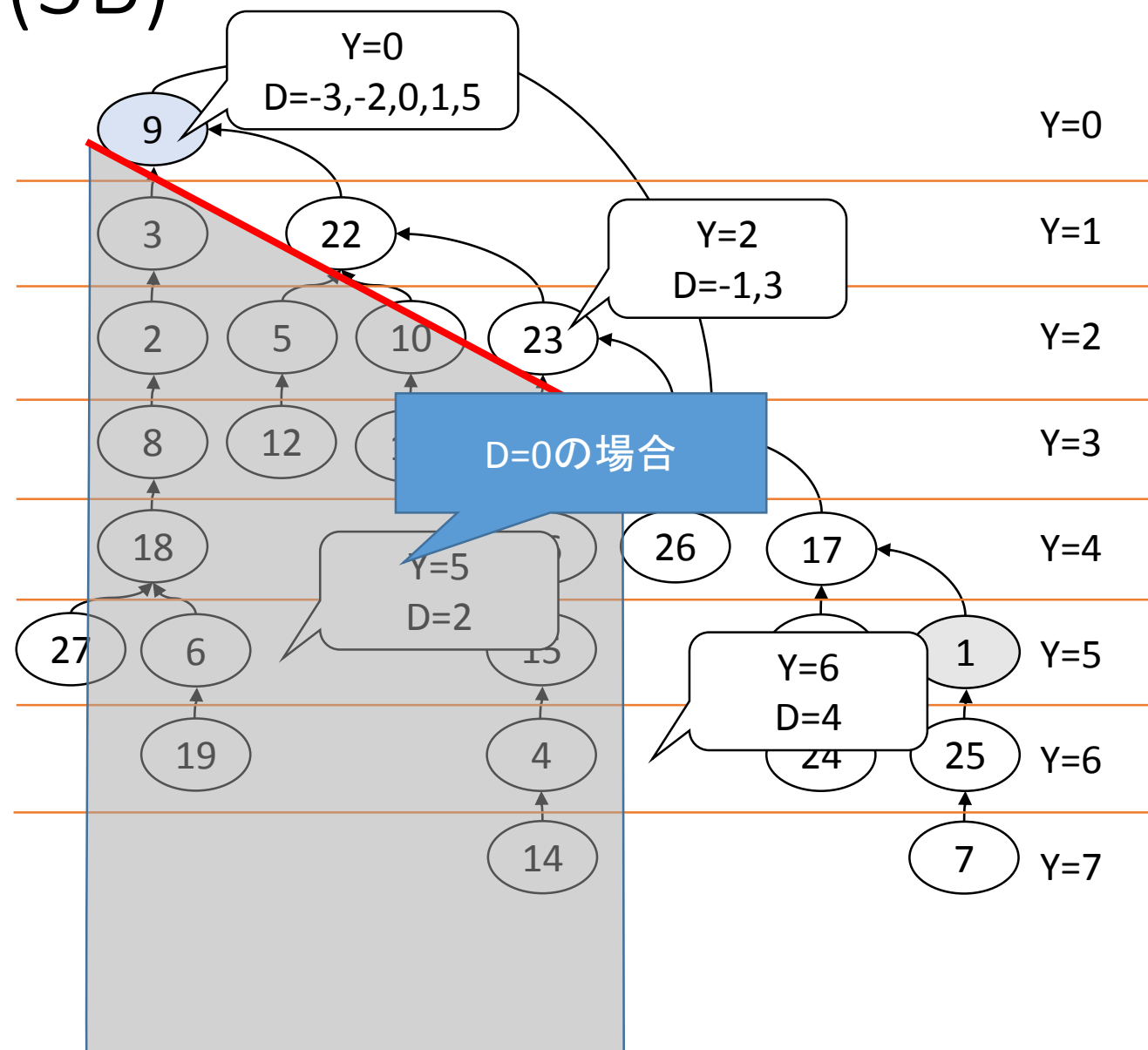
満点解法－場合分け(3B)

- 各イベントの境界部
(赤線で示した)に注目する



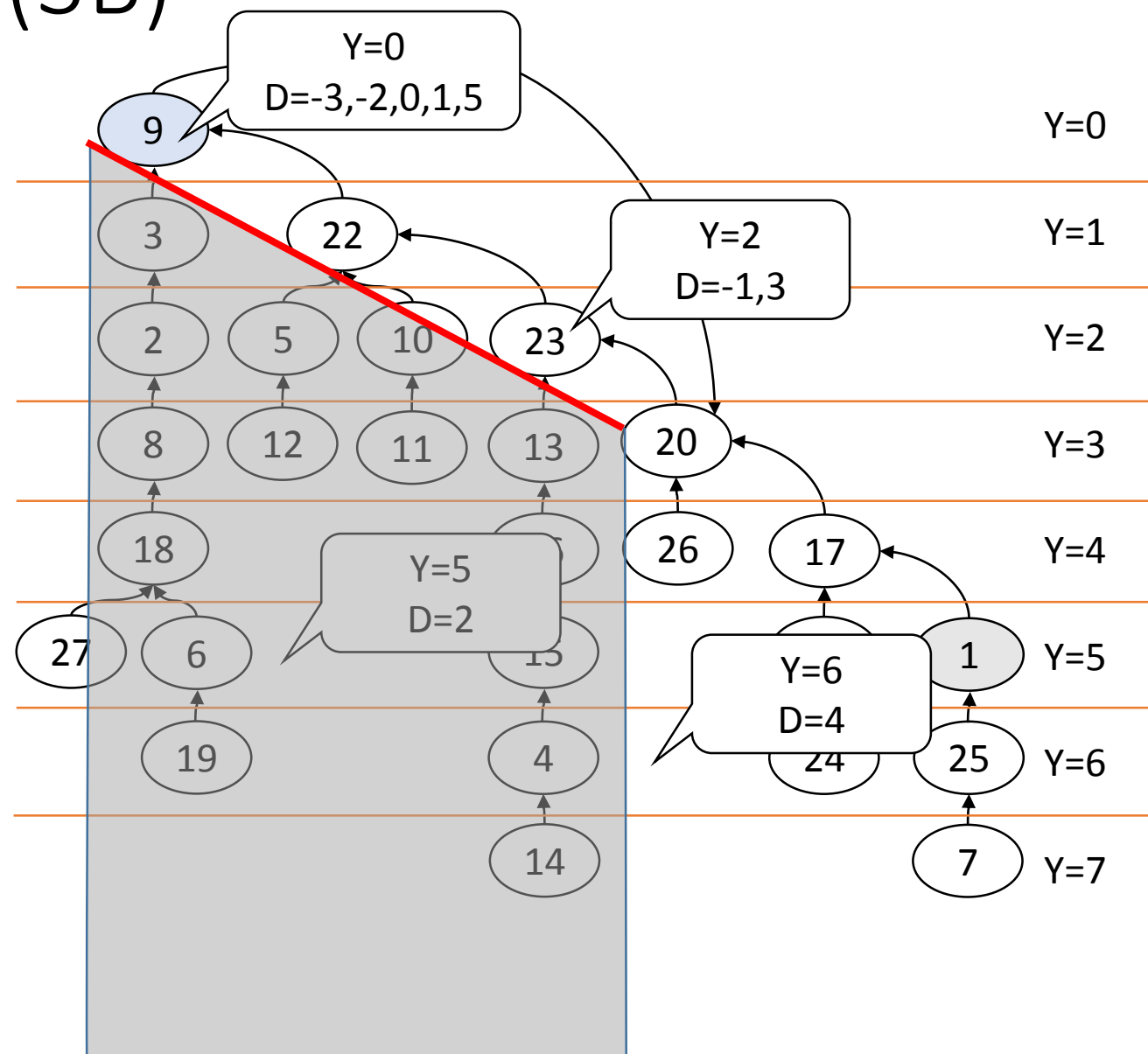
満点解法－場合分け(3B)

- 各イベントの境界部
(赤線で示した)に注目する
 - 加算処理の場合もこんな感じで同様



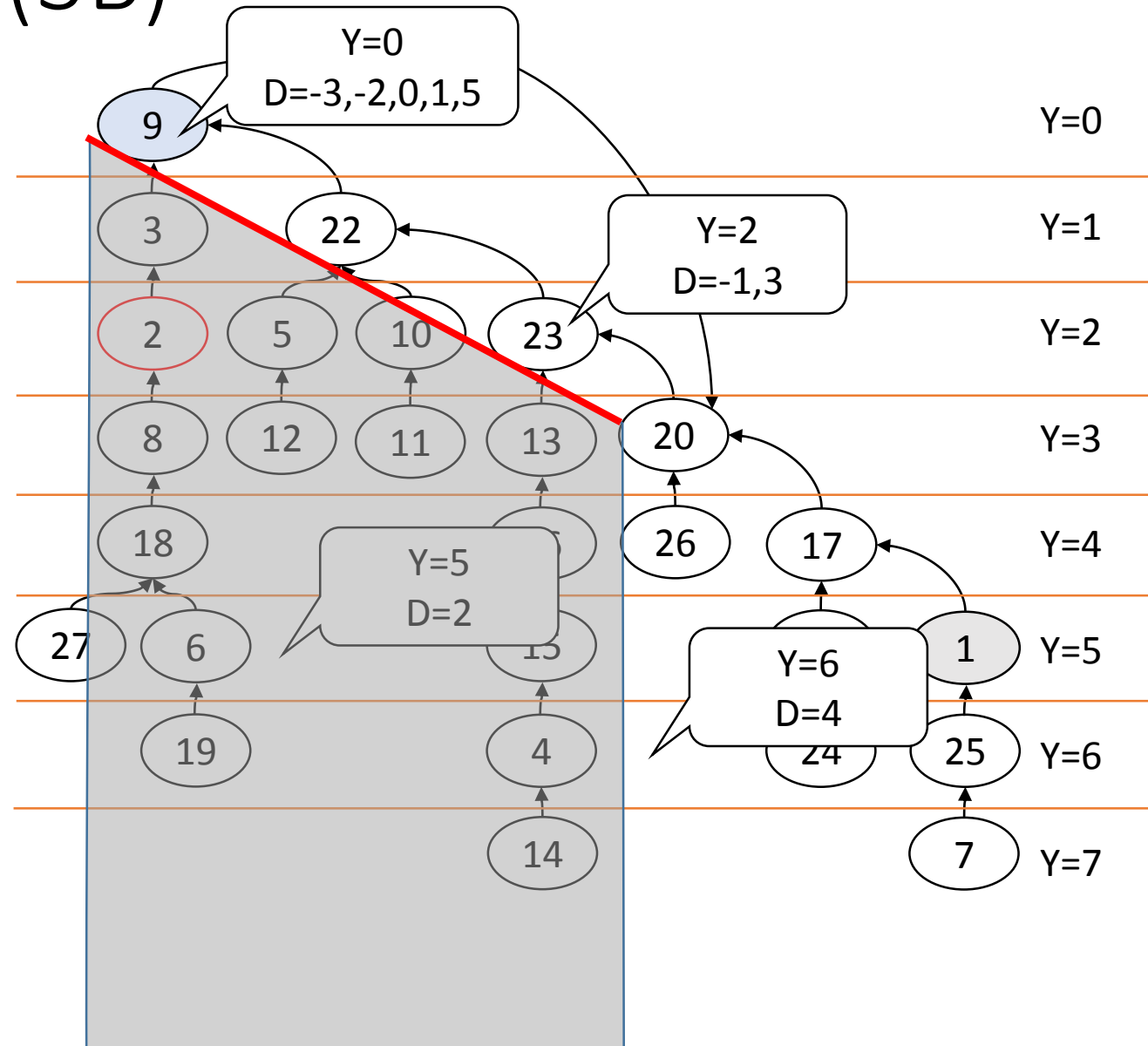
満点解法 – 場合分け(3B)

- 各イベントの境界部
(赤線で示した)に注目する
- 意味があるものだけ処理したい



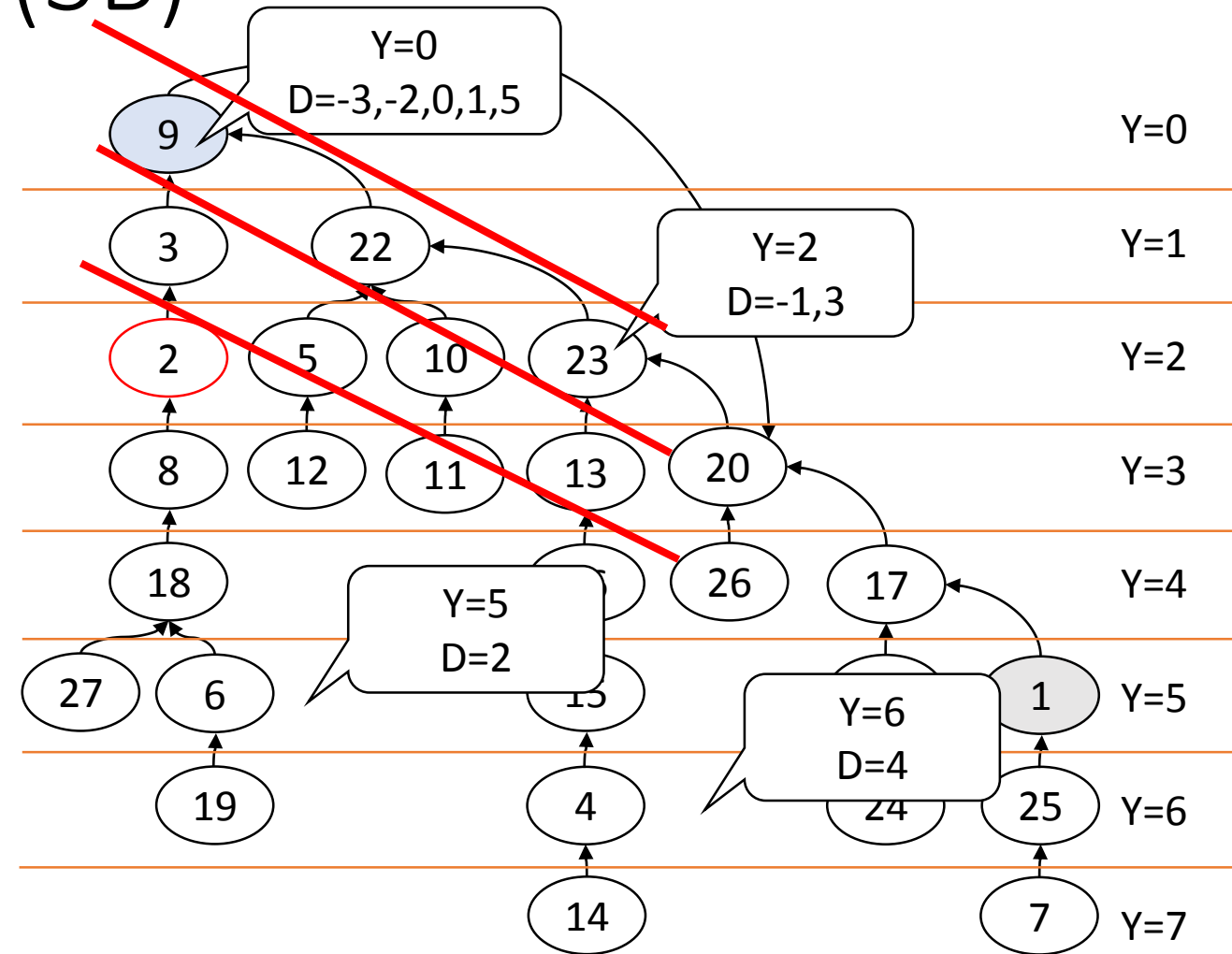
満点解法 – 場合分け(3B)

- 各イベントの境界部
(赤線で示した)に注目する
- 意味があるものだけ処理したい
- 例えば、2を処理中のとき



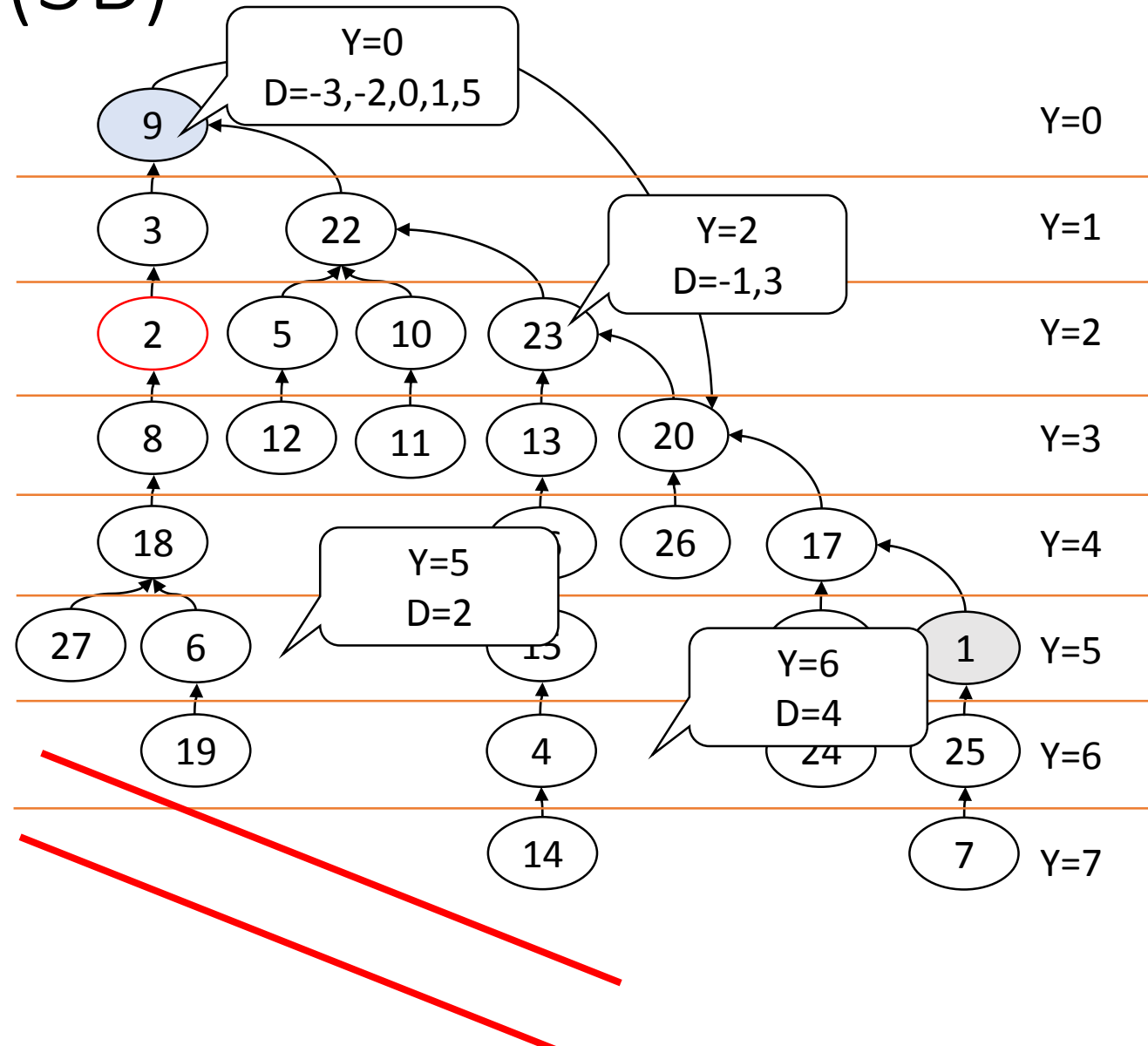
満点解法 – 場合分け(3B)

- 各イベントの境界部
(赤線で示した)に注目する
- 意味があるものだけ処理したい
- 例えば、2を処理中のとき
 - こういった境界をもつイベントは
たくさんあっても一括処理できる
 - (全て、数え上げの結果が一緒)



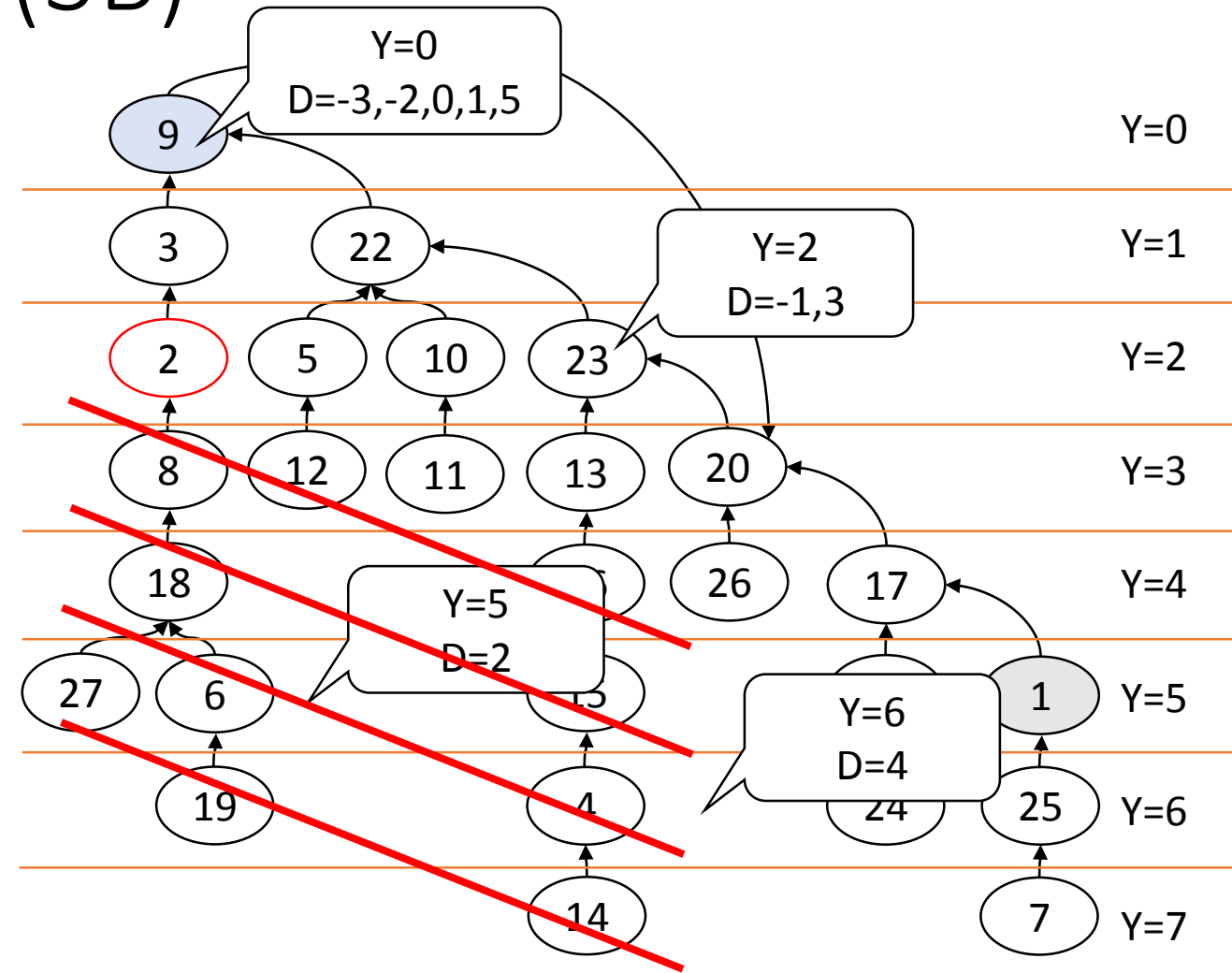
満点解法 – 場合分け(3B)

- 各イベントの境界部
(赤線で示した)に注目する
- 意味があるものだけ処理したい
- 例えば、2を処理中のとき
 - 逆に、こういった境界をもつイベントはたくさんあってもいずれも無視してよい
 - (全て、数え上げの結果は0)



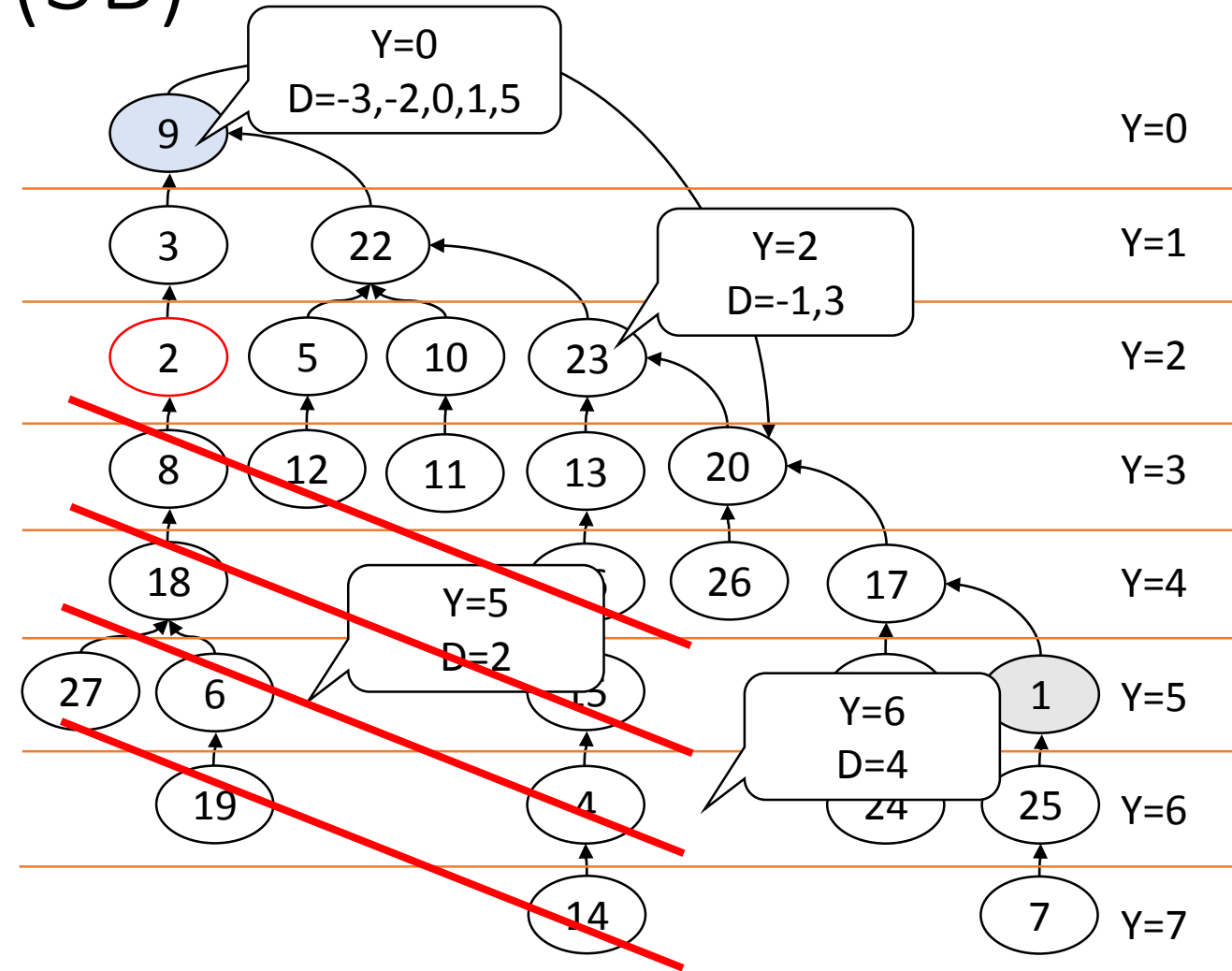
満点解法 – 場合分け(3B)

- 各イベントの境界部
(赤線で示した)に注目する
- 意味があるものだけ処理したい
- 例えば、2を処理中のとき
 - 境界がこの範囲にあるイベント
だけを処理すればよい



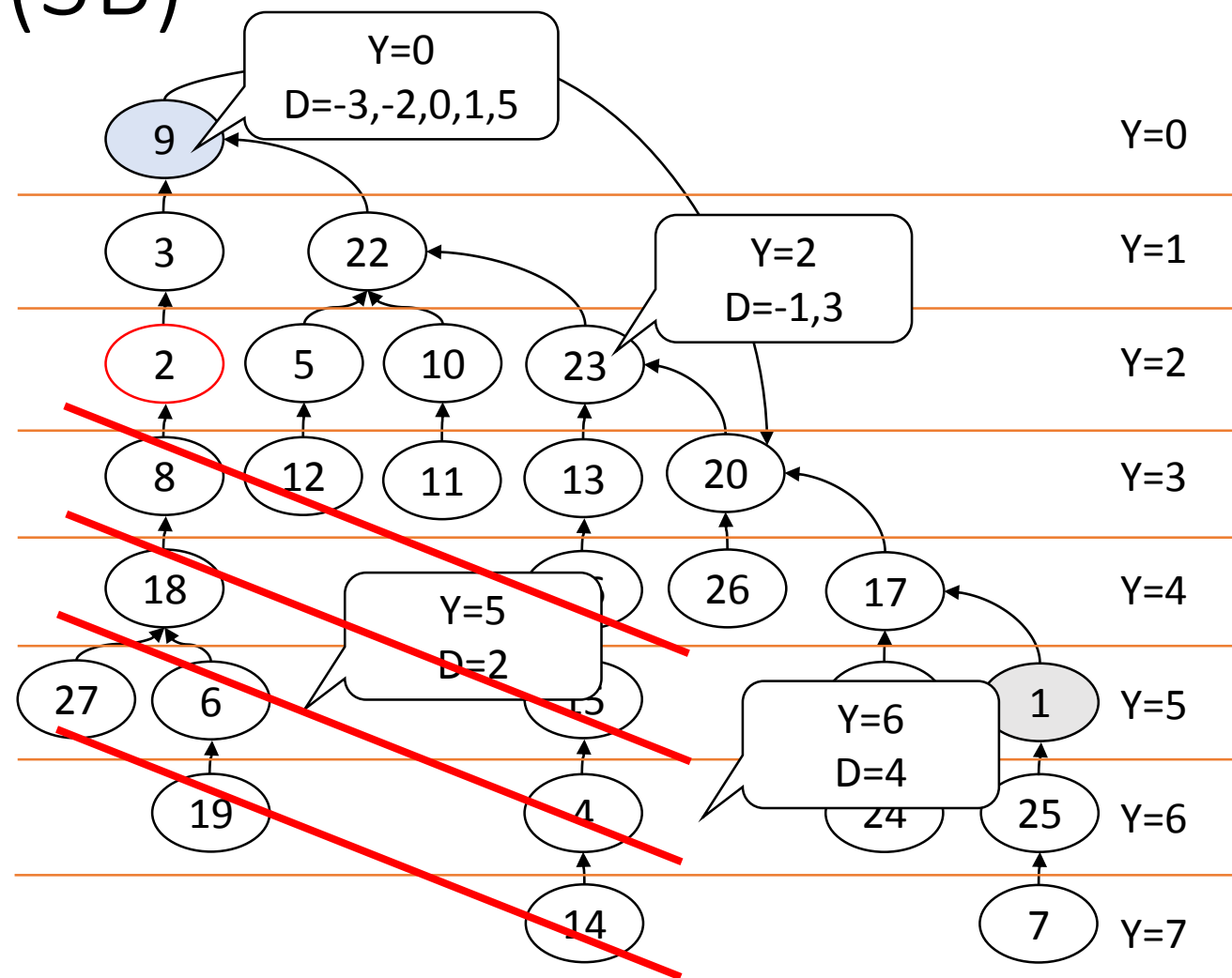
満点解法 – 場合分け(3B)

- 各イベントの境界部
(赤線で示した)に注目する
- 意味があるものだけ処理したい
- 例えば、2を処理中のとき
 - 境界がこの範囲にあるイベント
だけを処理すればよい
 - 実際には、さらにYの値が一致
するイベントのみを対象にする



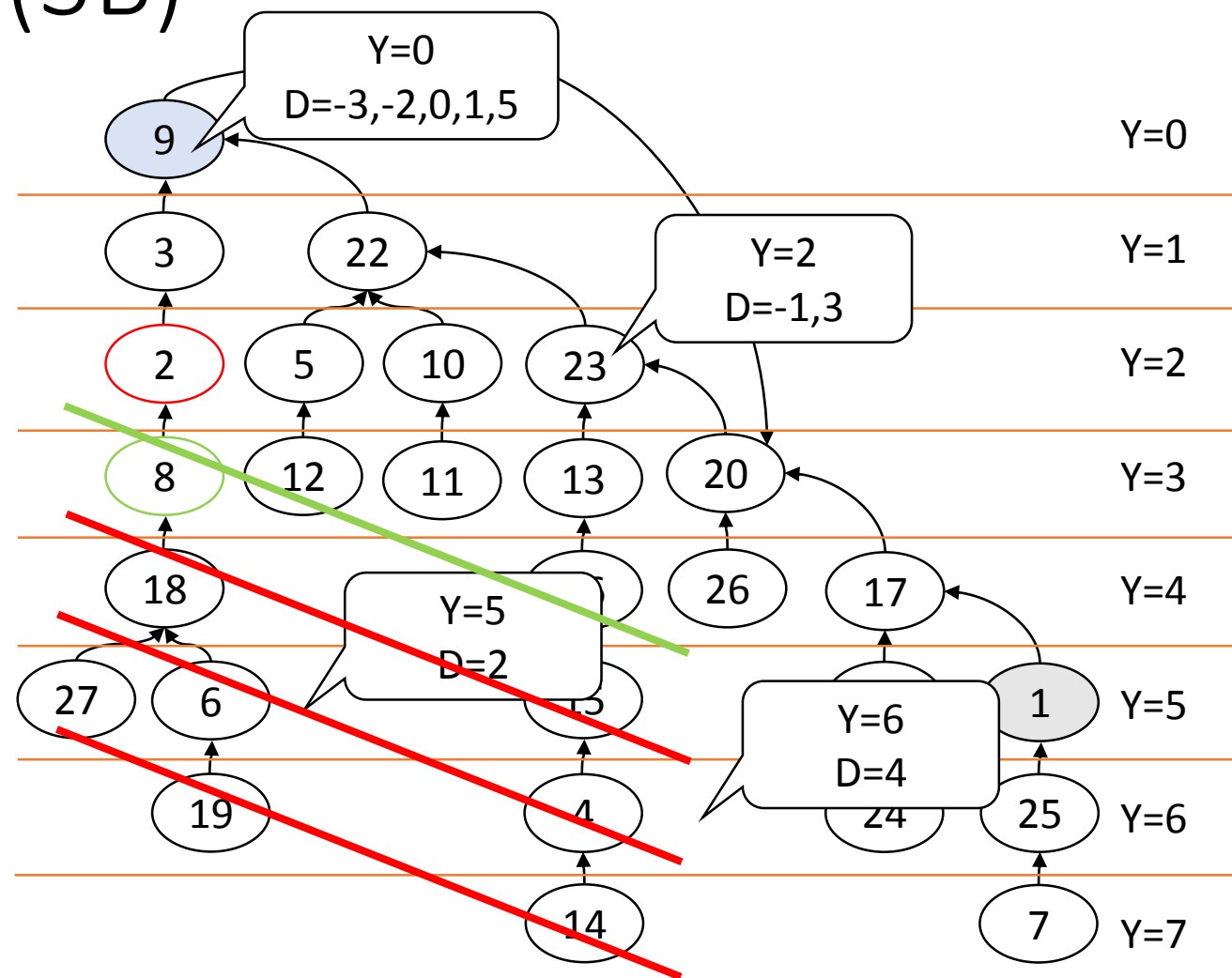
満点解法 – 場合分け(3B)

- 意味のあるイベントに、頂点を対応づける



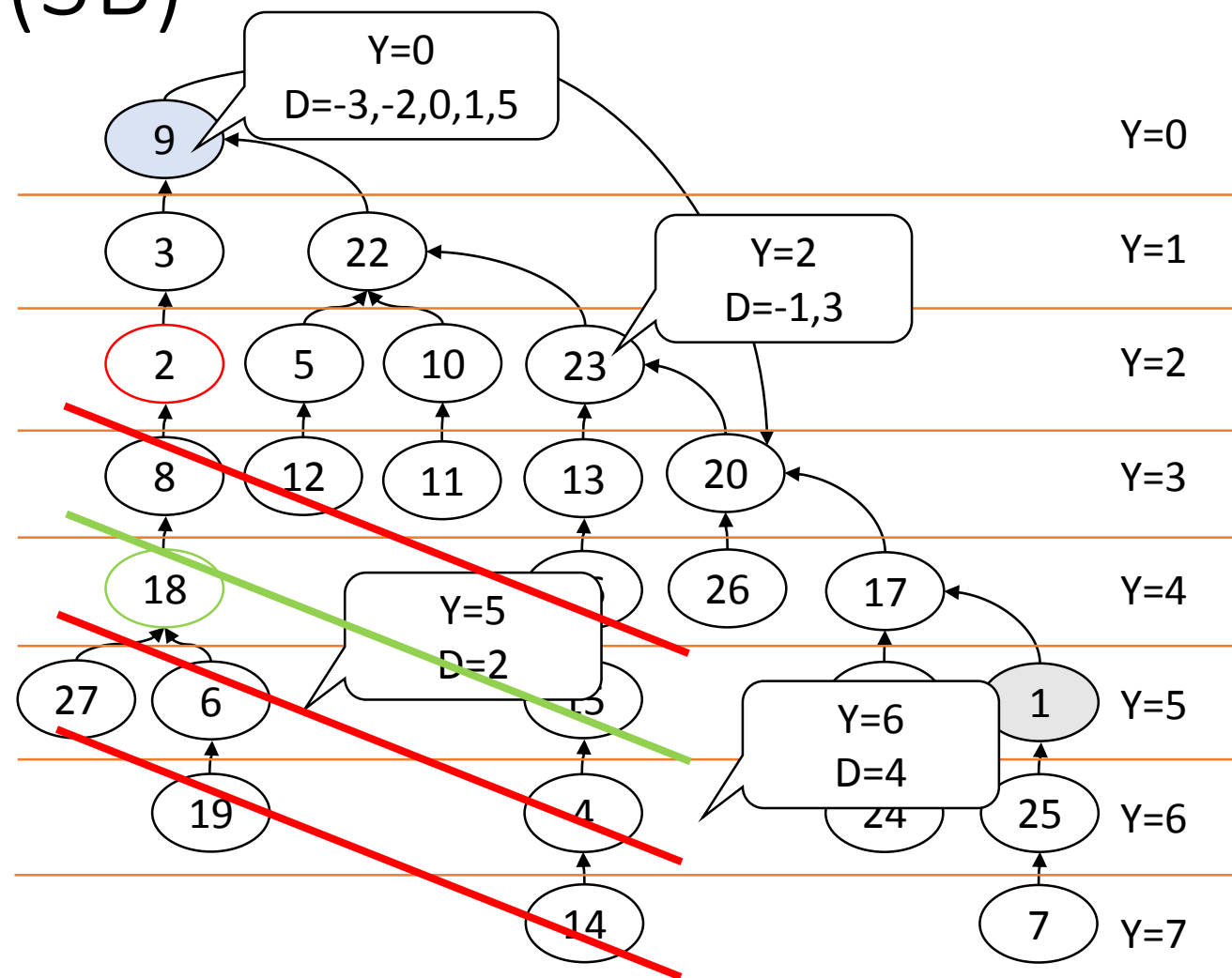
満点解法 – 場合分け(3B)

- 意味のあるイベントに、頂点を対応づける



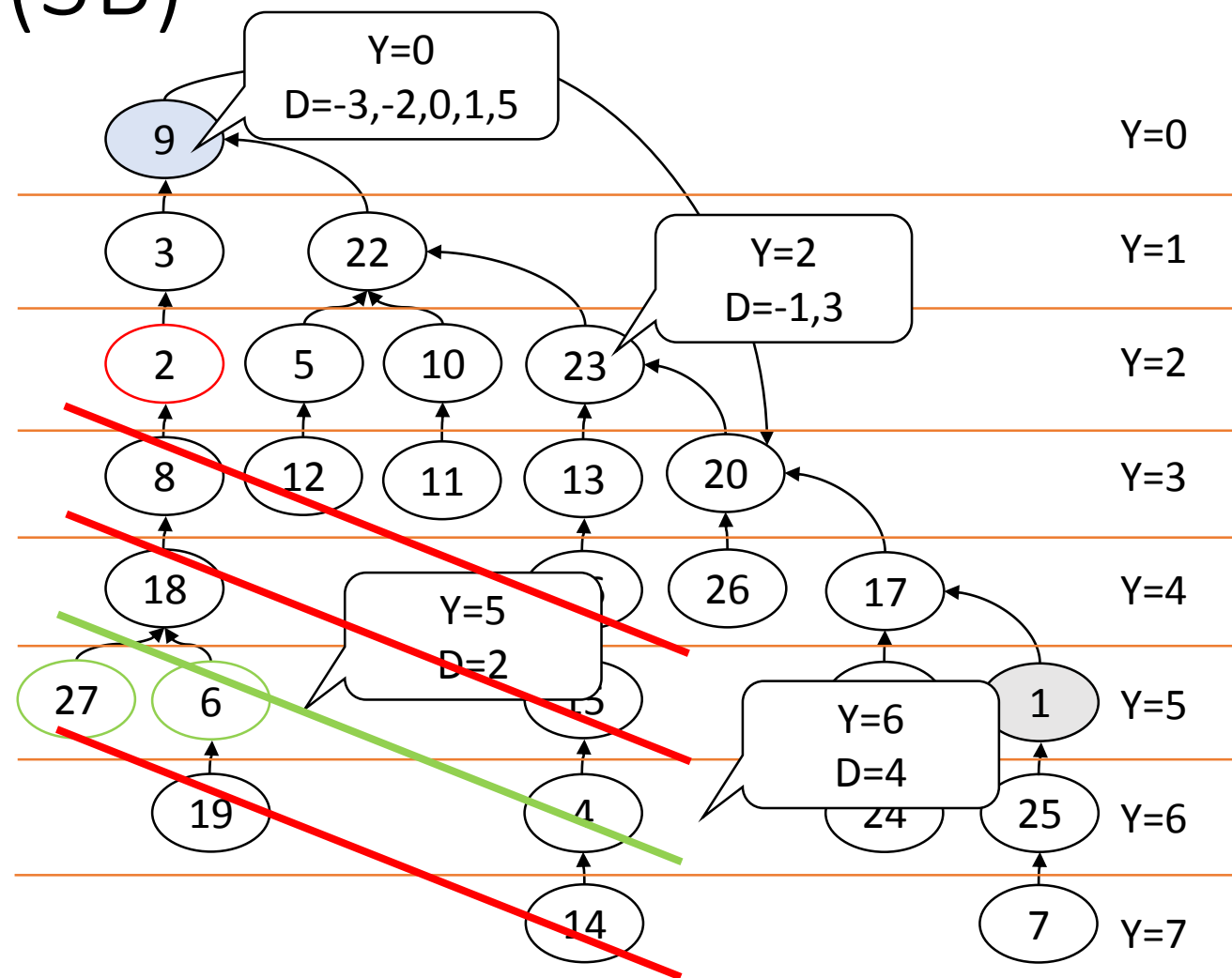
満点解法 – 場合分け(3B)

- 意味のあるイベントに、頂点を対応づける



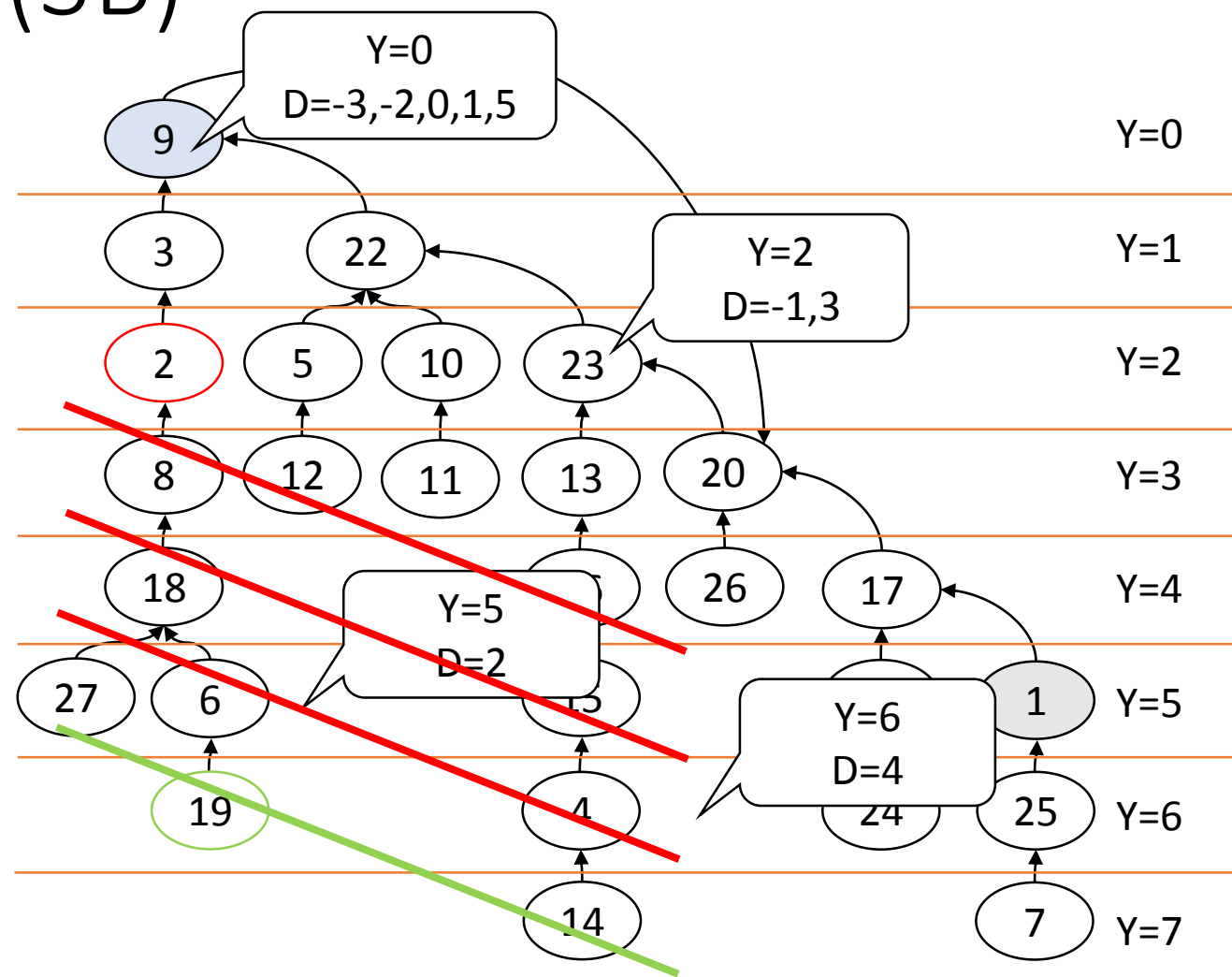
満点解法 – 場合分け(3B)

- 意味のあるイベントに、頂点を対応づける



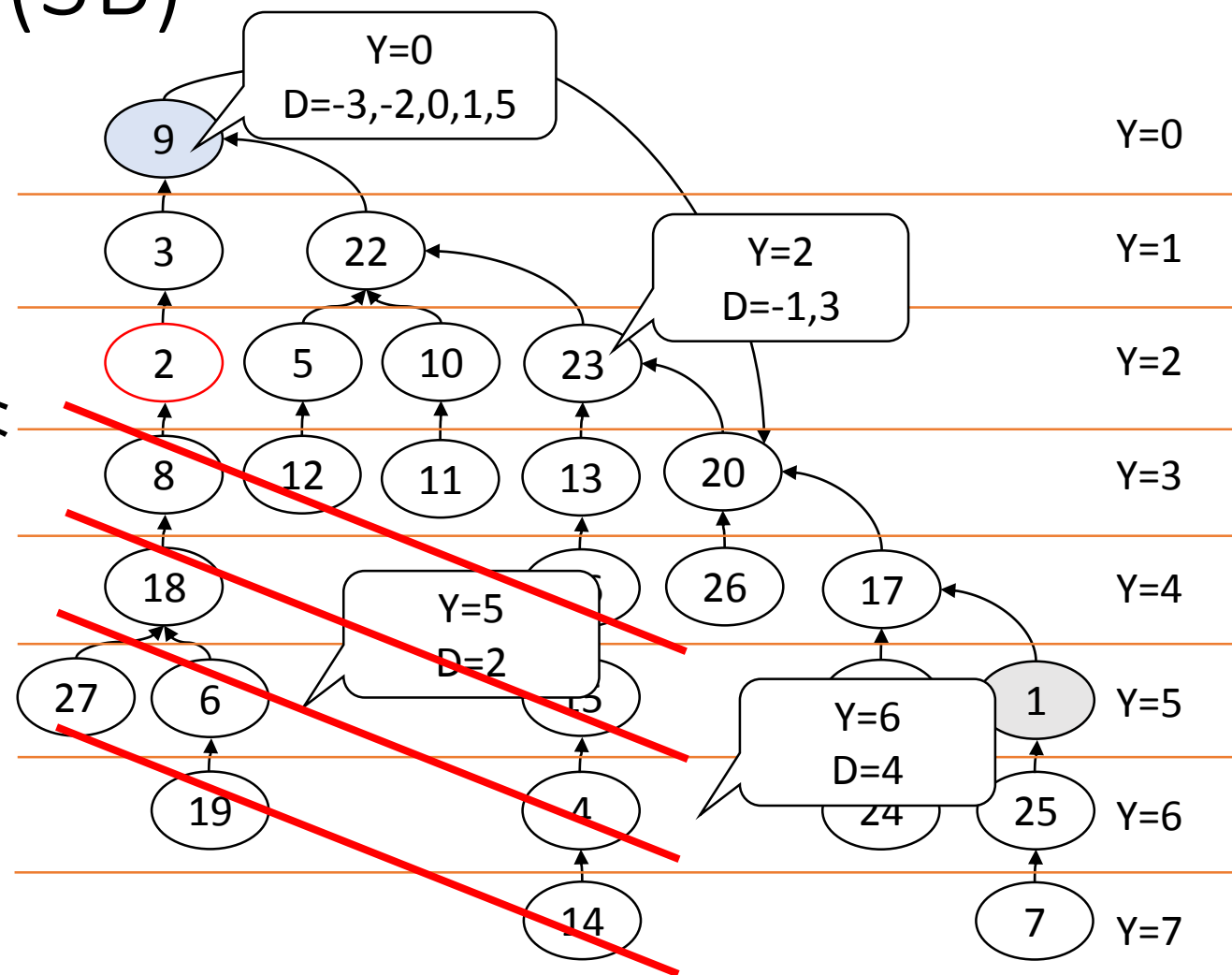
満点解法 – 場合分け(3B)

- 意味のあるイベントに、頂点を対応づける



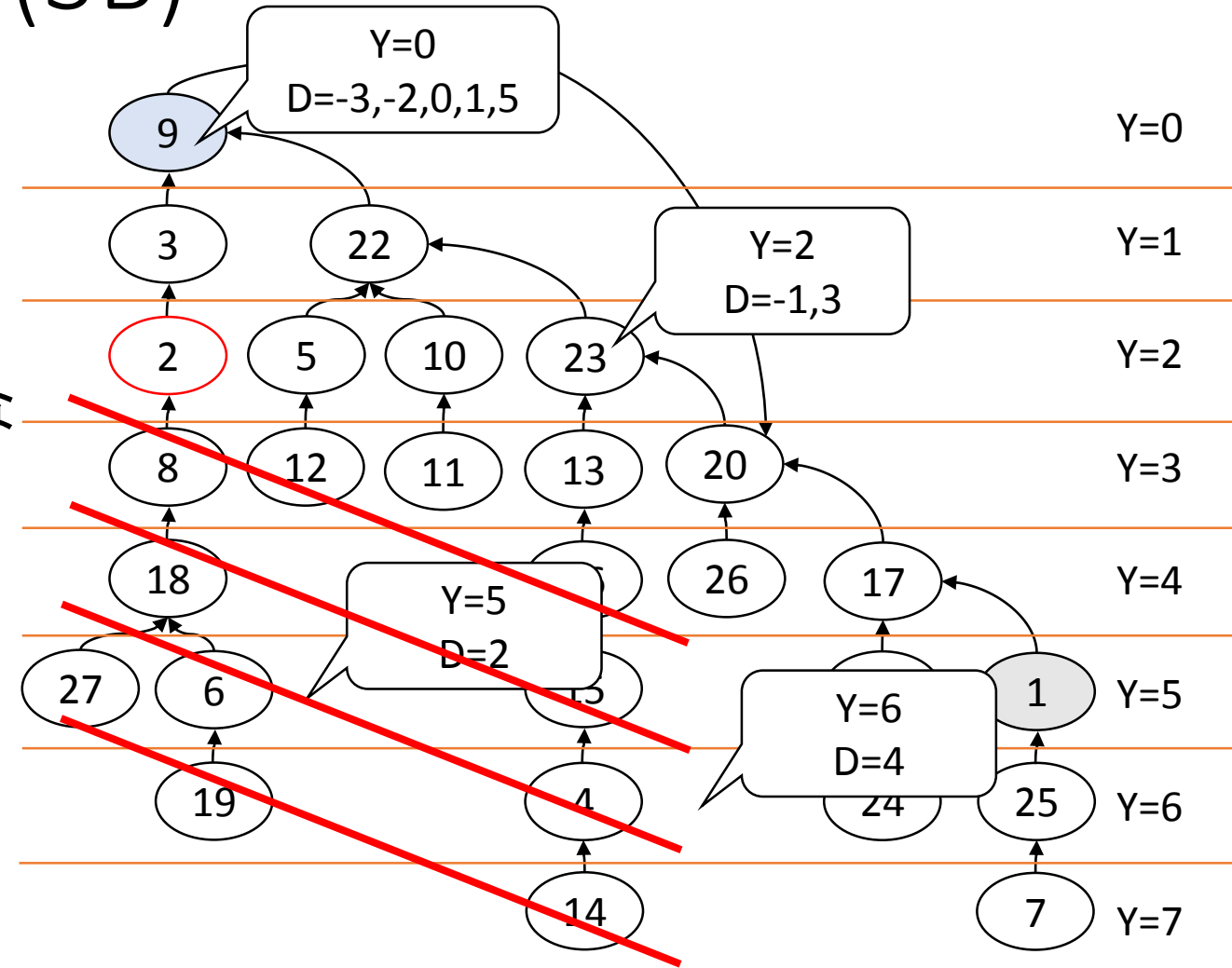
満点解法 – 場合分け(3B)

- 意味のあるイベントに、頂点を対応づける
 - 境界のすぐ下の頂点であって
 - 処理中の頂点の子孫であるようなもの



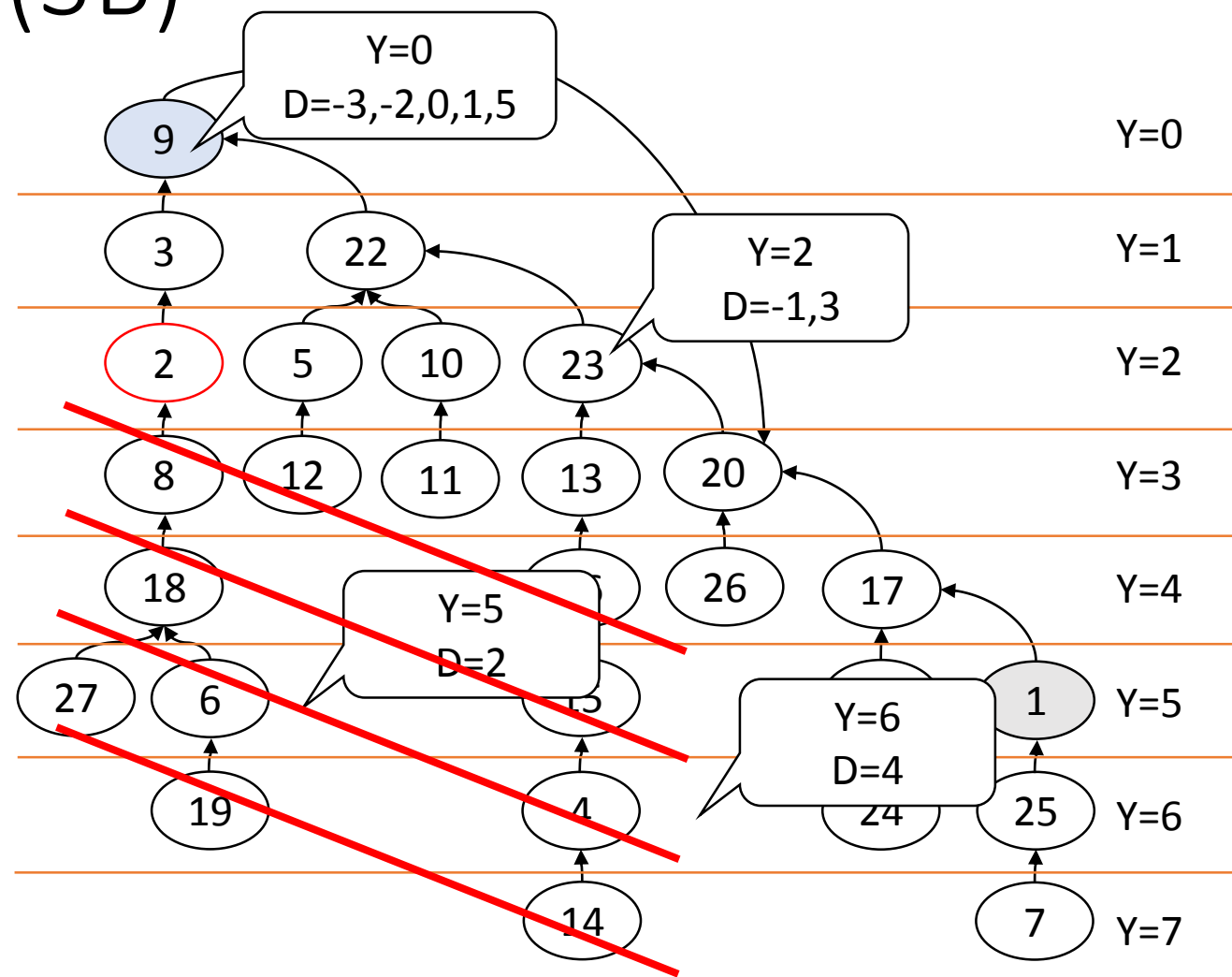
満点解法 – 場合分け(3B)

- 意味のあるイベントに、頂点を対応づける
 - 境界のすぐ下の頂点であって
 - 処理中の頂点の子孫であるようなもの
- 意味のあるイベントには、1つ以上の頂点が対応する



満点解法 – 場合分け(3B)

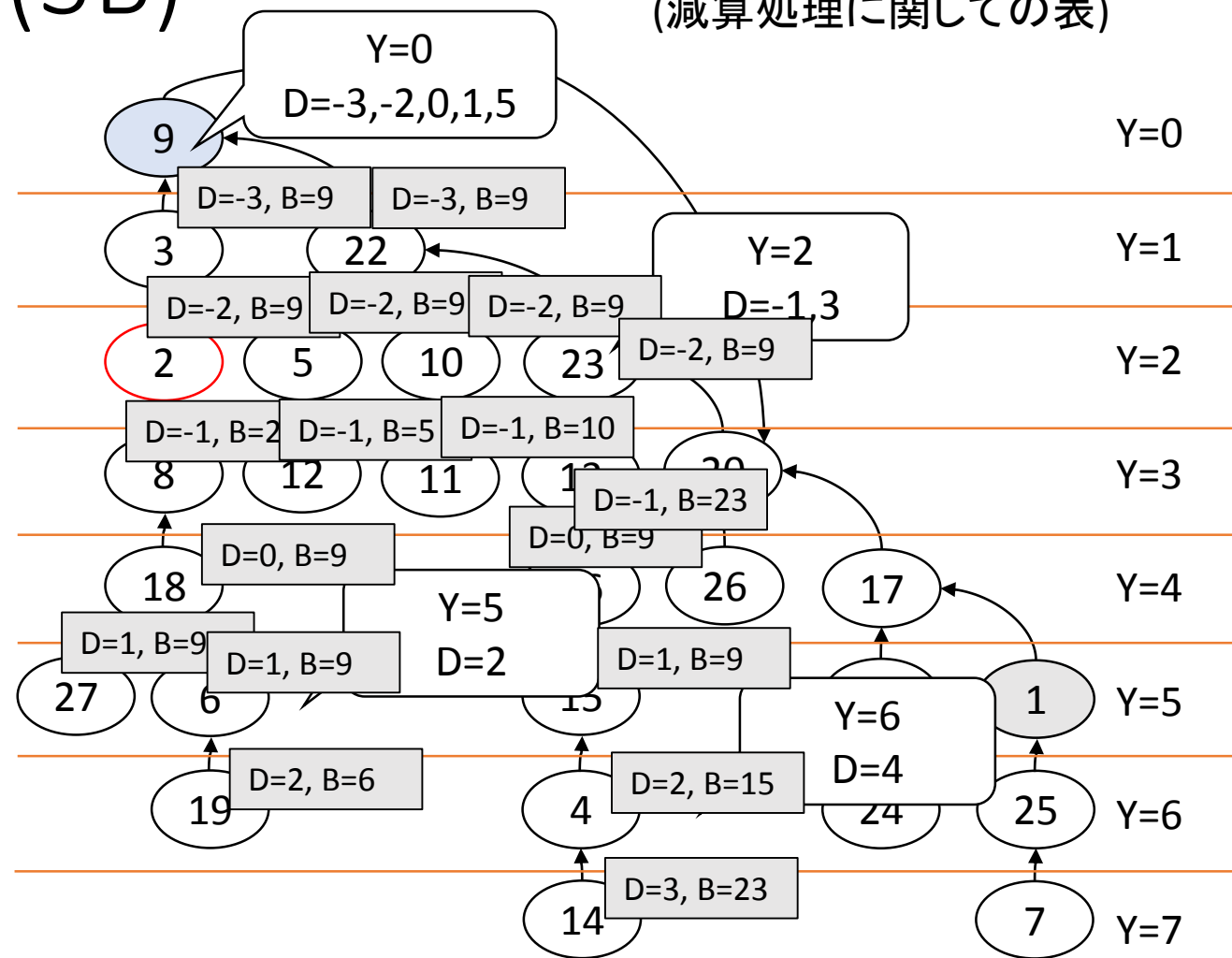
- 同じ頂点に対応するイベント呼び出しが2回行われることはない



満点解法 – 場合分け(3B)

- 同じ頂点に対応するイベント呼び出しが2回行われることはない

各頂点に対応づけられたイベント。
(減算処理に関する表)



満点解法 – 場合分け(3B)

- 同じ頂点に対応するイベント
呼び出しが2回行われる
ことはない
- →意味のあるイベント呼び出し
の回数は $O(n)$ に抑えられた

満点解法 – 場合分け(3B)

- 例外的な場合の処理

満点解法 – 場合分け(3B)

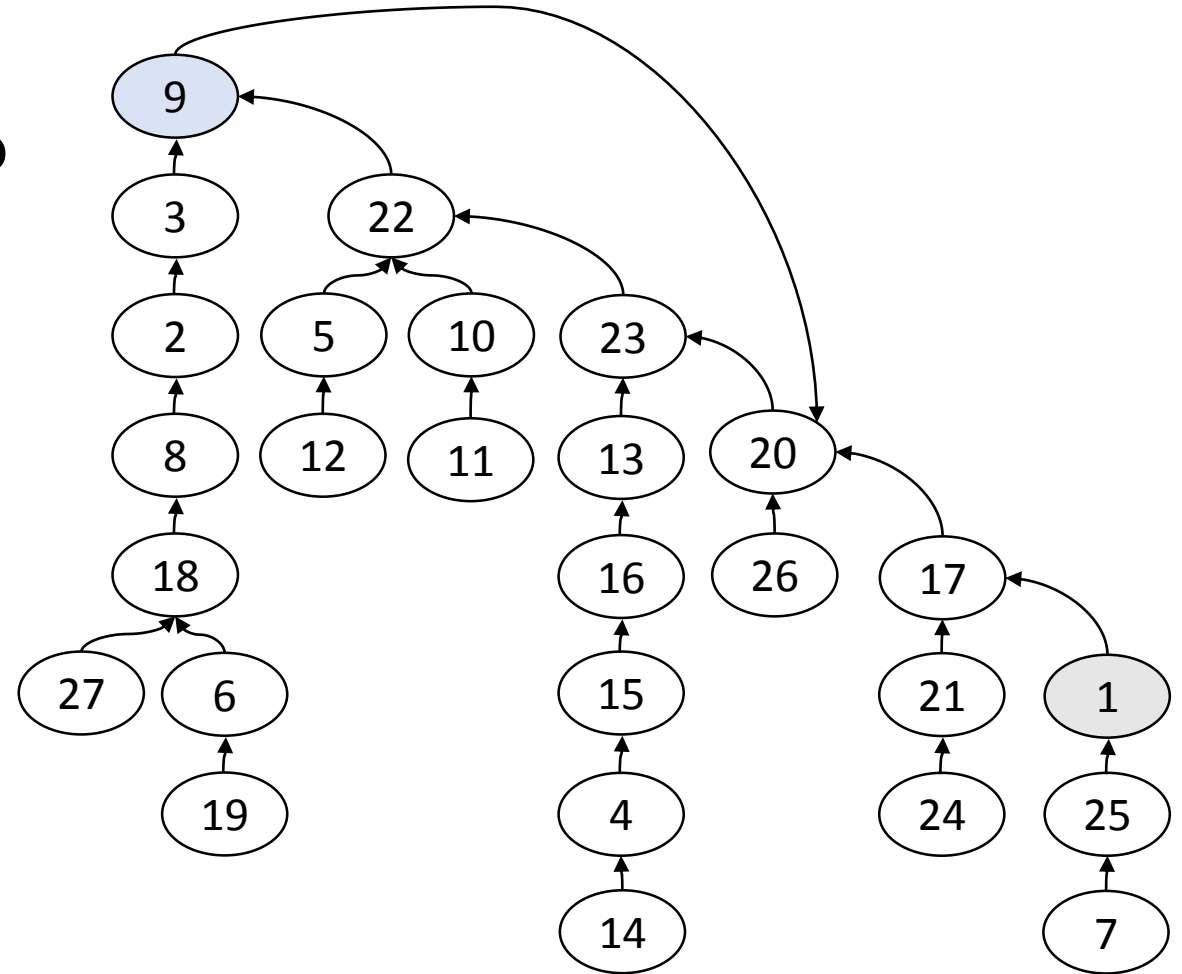
- 例外的な場合の処理
- 各イベント(Y, D)について、 Y 座標が $Y-(D+1)$ となるようなループ上の点
が到達先の候補である。

満点解法 – 場合分け(3B)

- 例外的な場合の処理
- 各イベント(Y, D)について、 Y 座標が $Y-(D+1)$ となるようなループ上の点
が到達先の候補である。
- $Y-(D+1)$ がマイナスになる可能性があることだけ注意すればOK

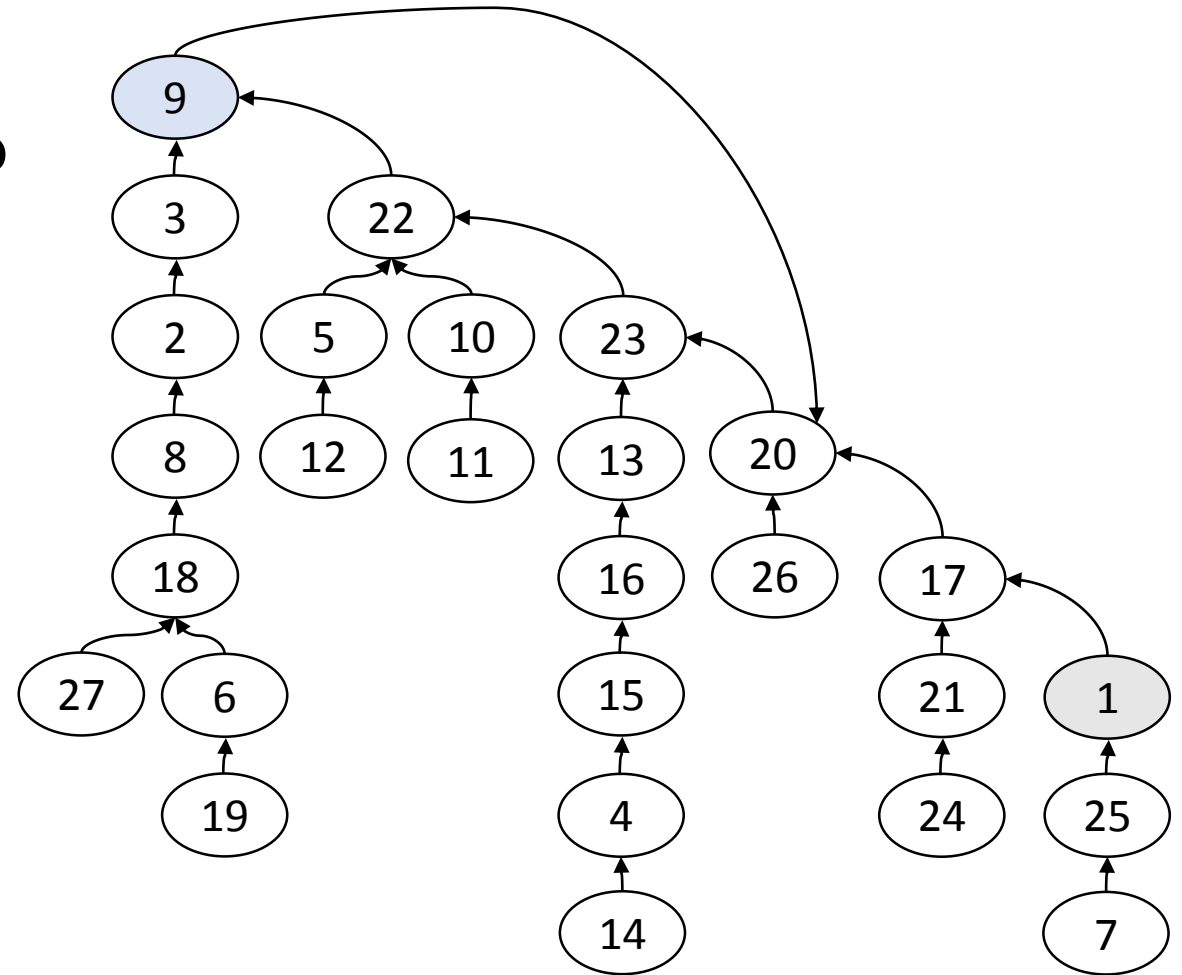
満点解法 – 場合分け(3C)

- AがBの祖先である場合を考える



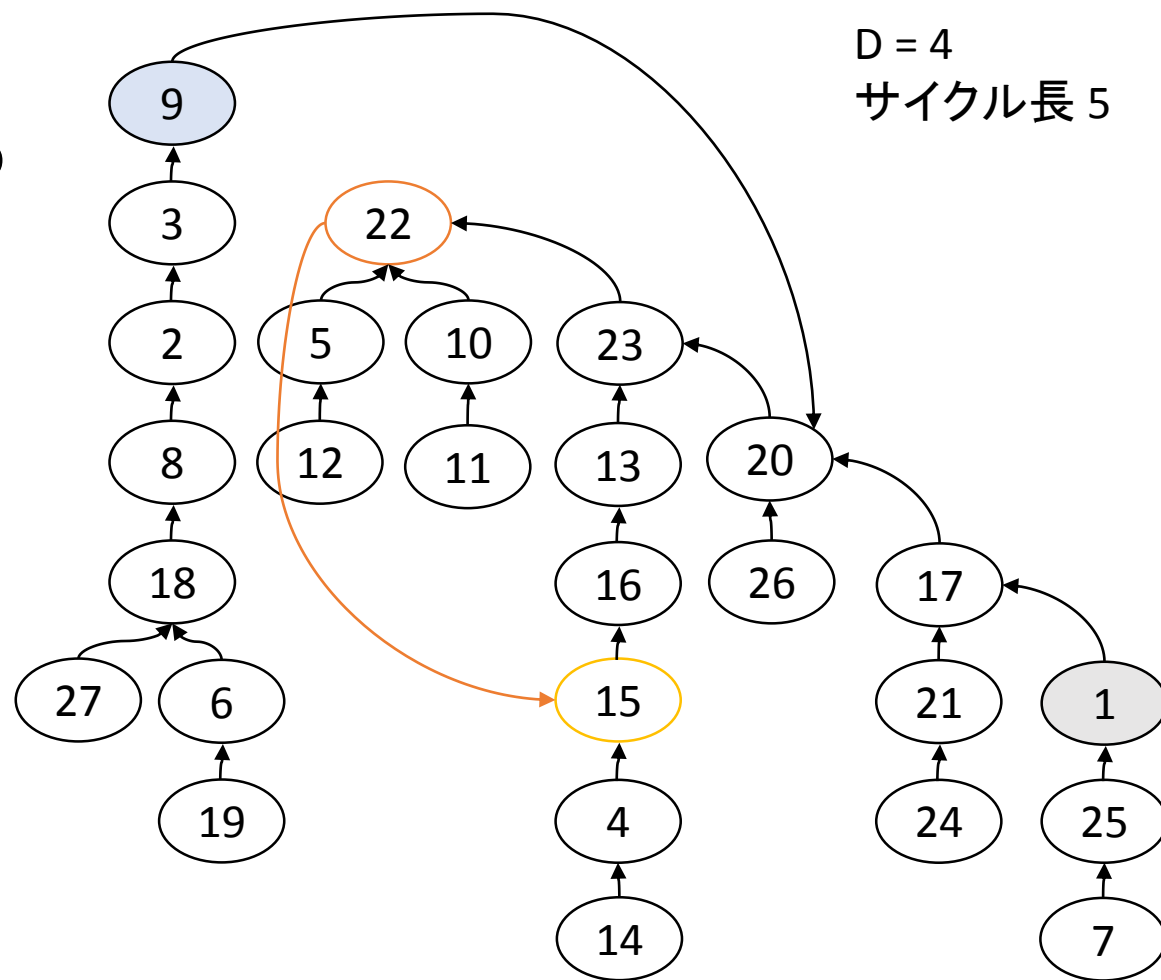
満点解法 – 場合分け(3C)

- AがBの祖先である場合を考える
- このとき、できるサイクル長は $D + 1$ である。



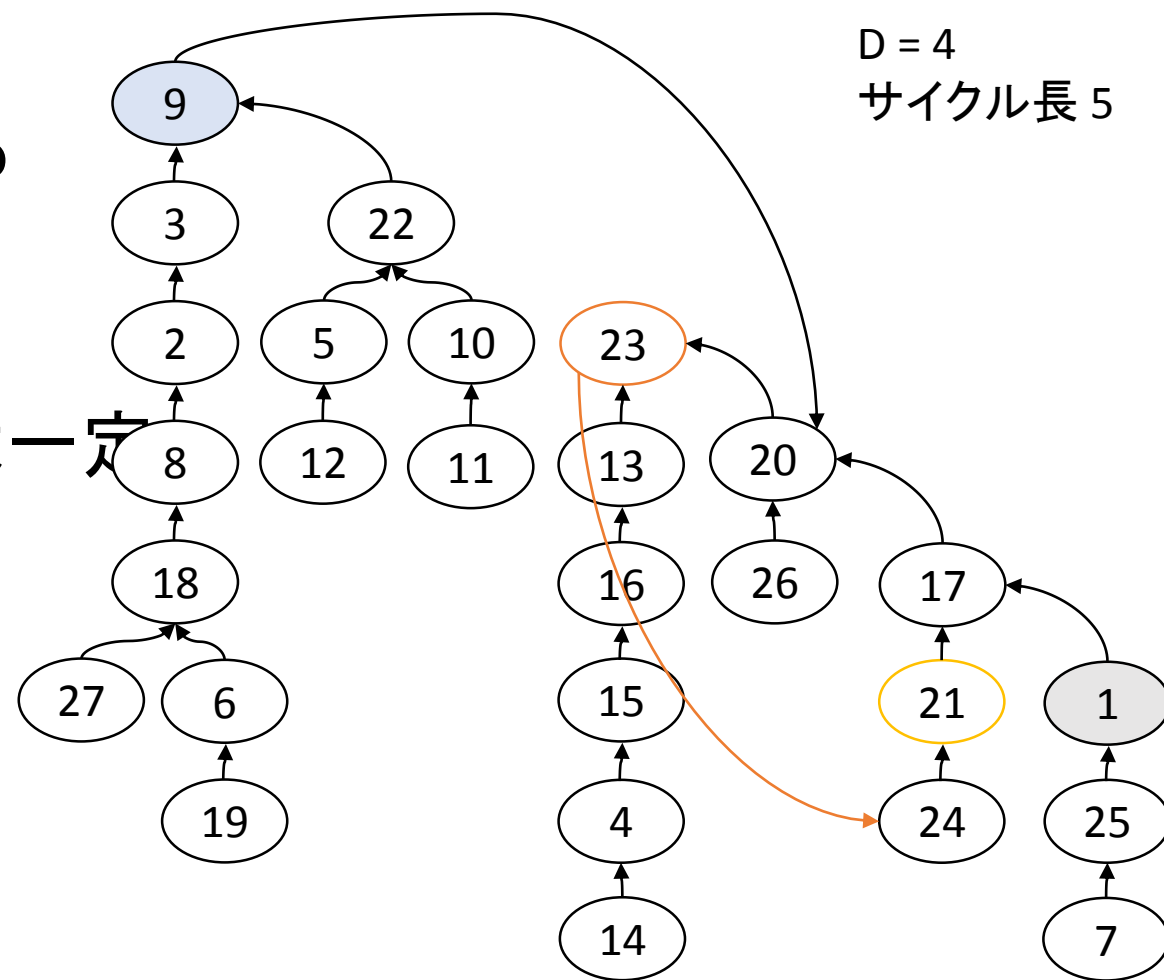
満点解法 – 場合分け(3C)

- AがBの祖先である場合を考える
- このとき、できるサイクル長は $D + 1$ である。



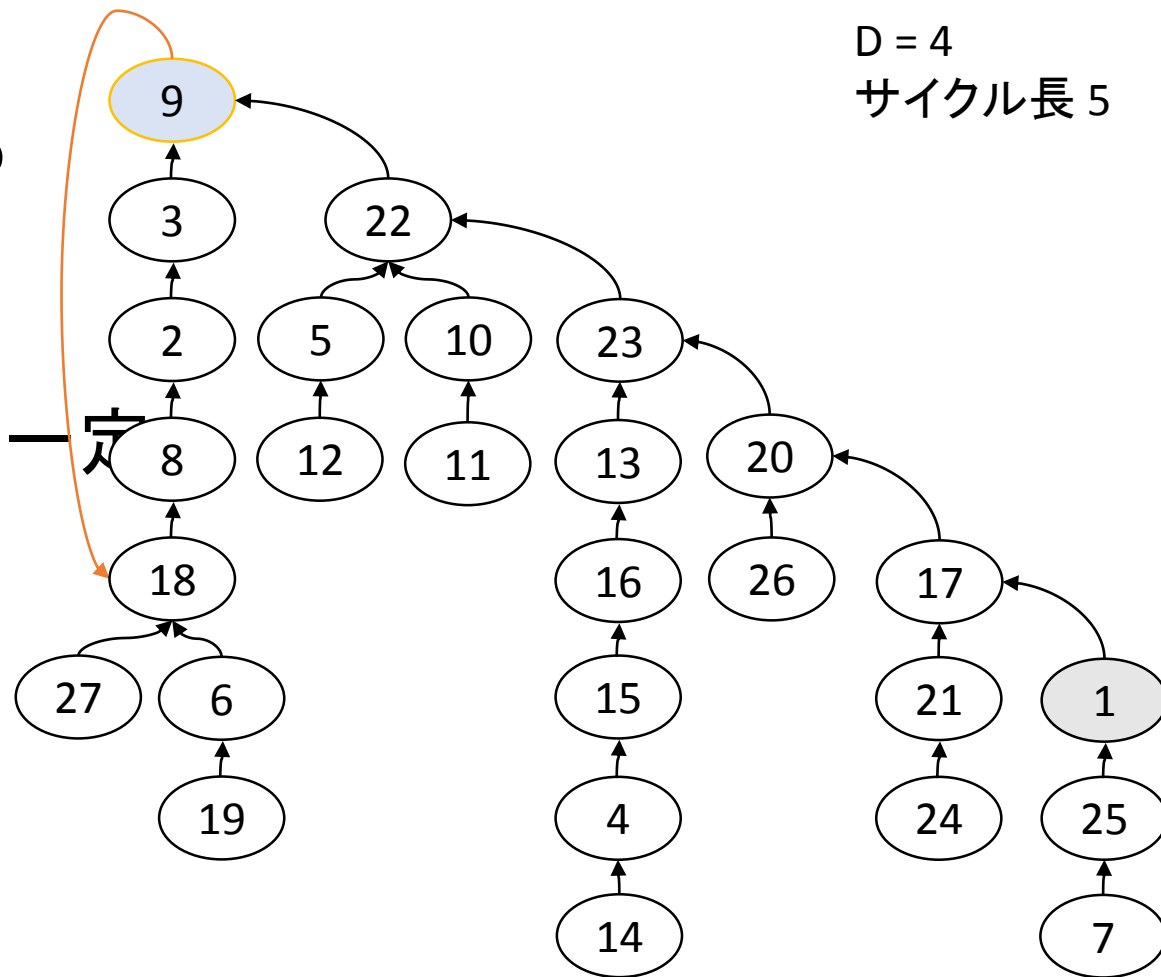
満点解法 – 場合分け(3C)

- AがBの祖先である場合を考える
- このとき、できるサイクル長は $D + 1$ である。
- D一定のとき、到達先のY座標は一定



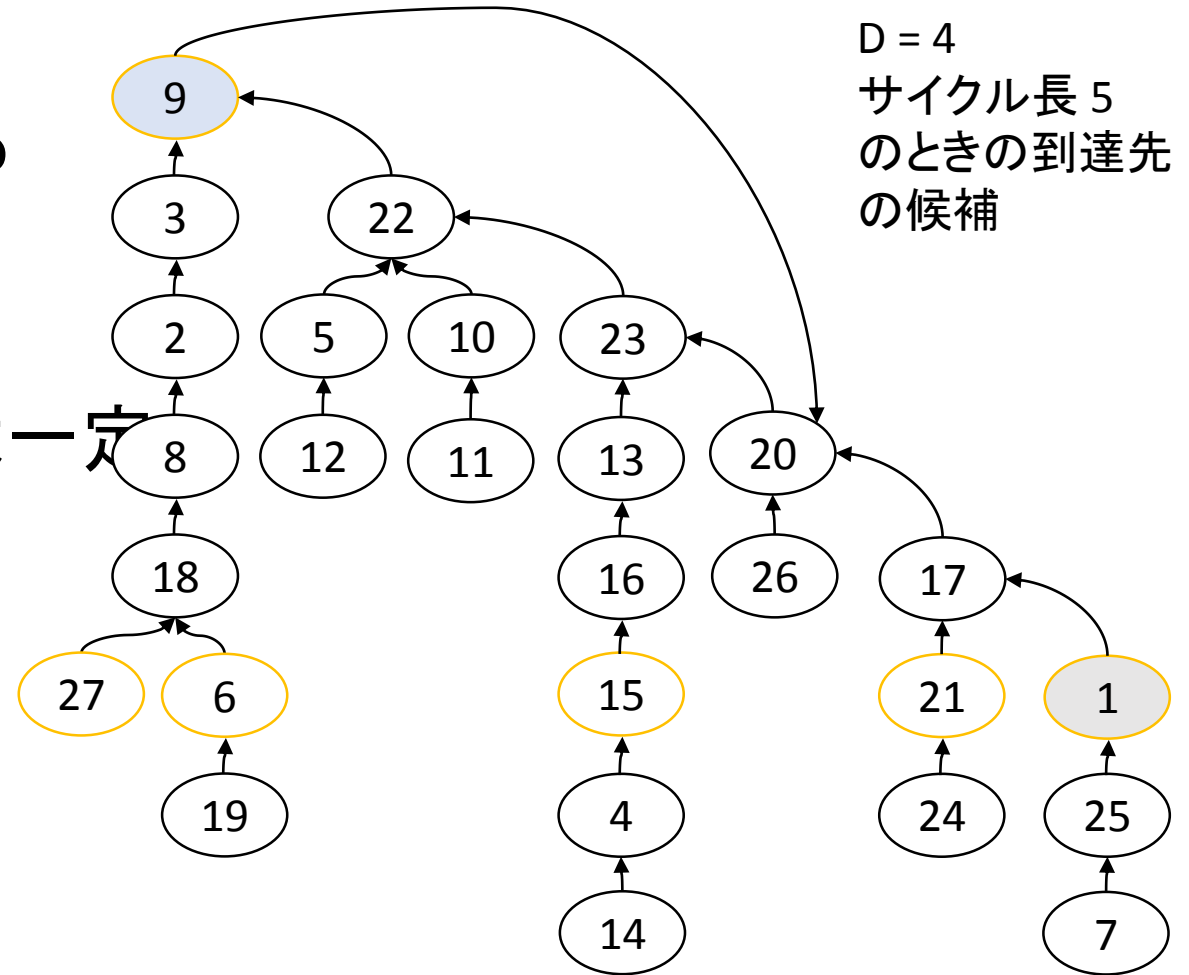
満点解法－場合分け(3C)

- AがBの祖先である場合を考える
- このとき、できるサイクル長は $D + 1$ である。
- D 一定のとき、到達先のY座標は一定である。
- というわけではないが、 $D+1$ を法として一定である。



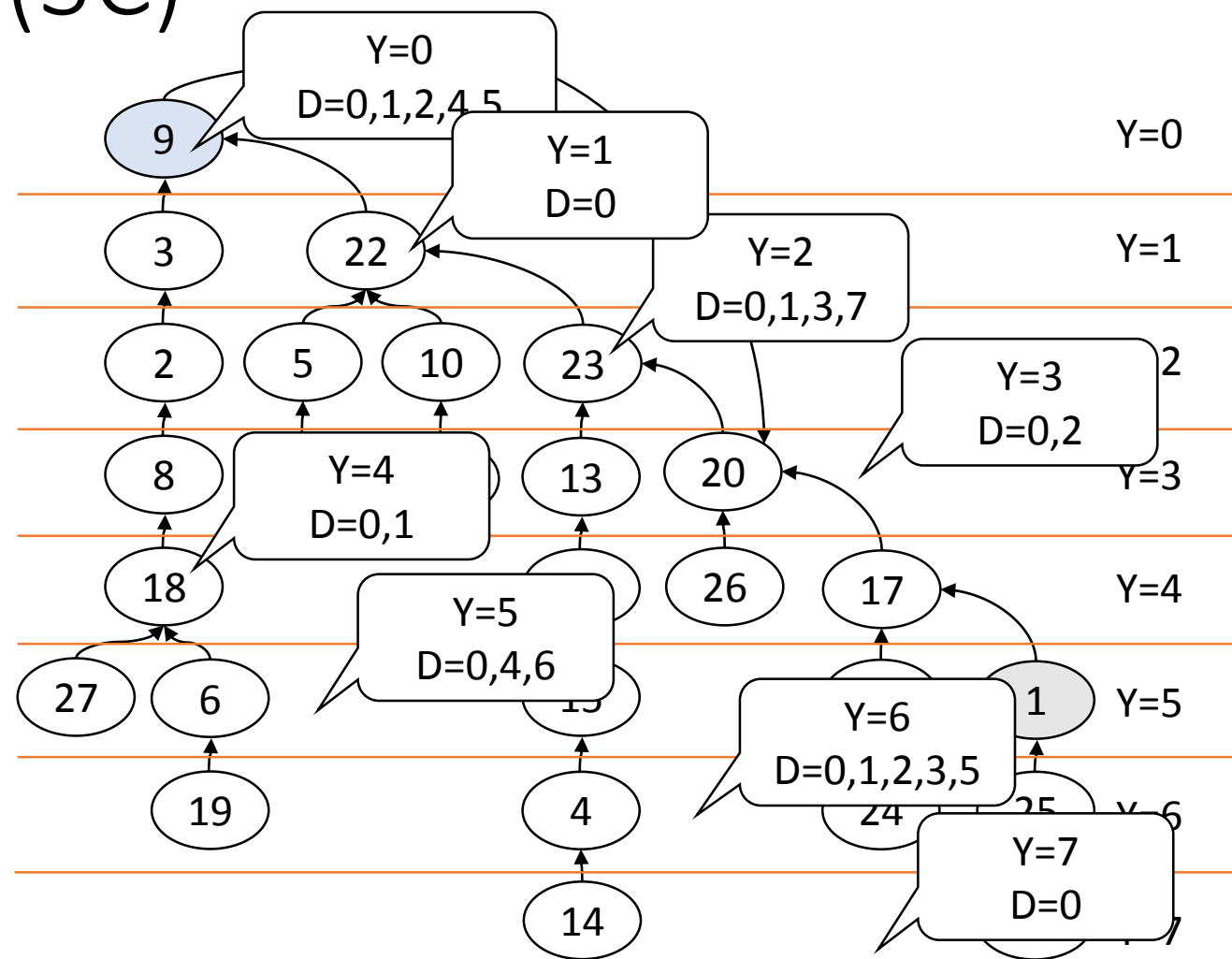
満点解法 – 場合分け(3C)

- AがBの祖先である場合を考える
- このとき、できるサイクル長は $D + 1$ である。
- D 一定のとき、到達先のY座標は一定
- というわけではないが、 $D+1$ を法として一定である。



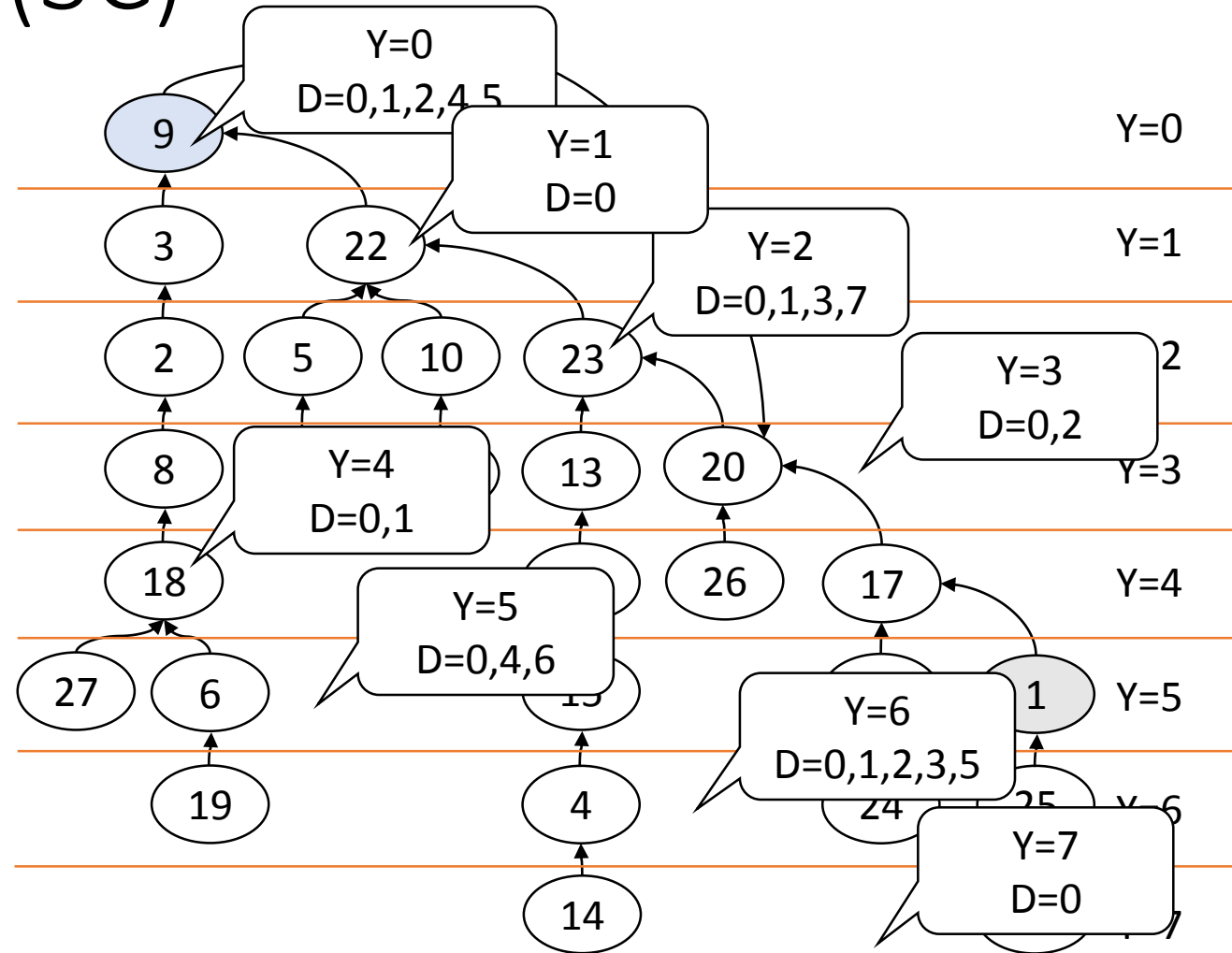
満点解法 – 場合分け(3C)

- 以上をイベントとして管理



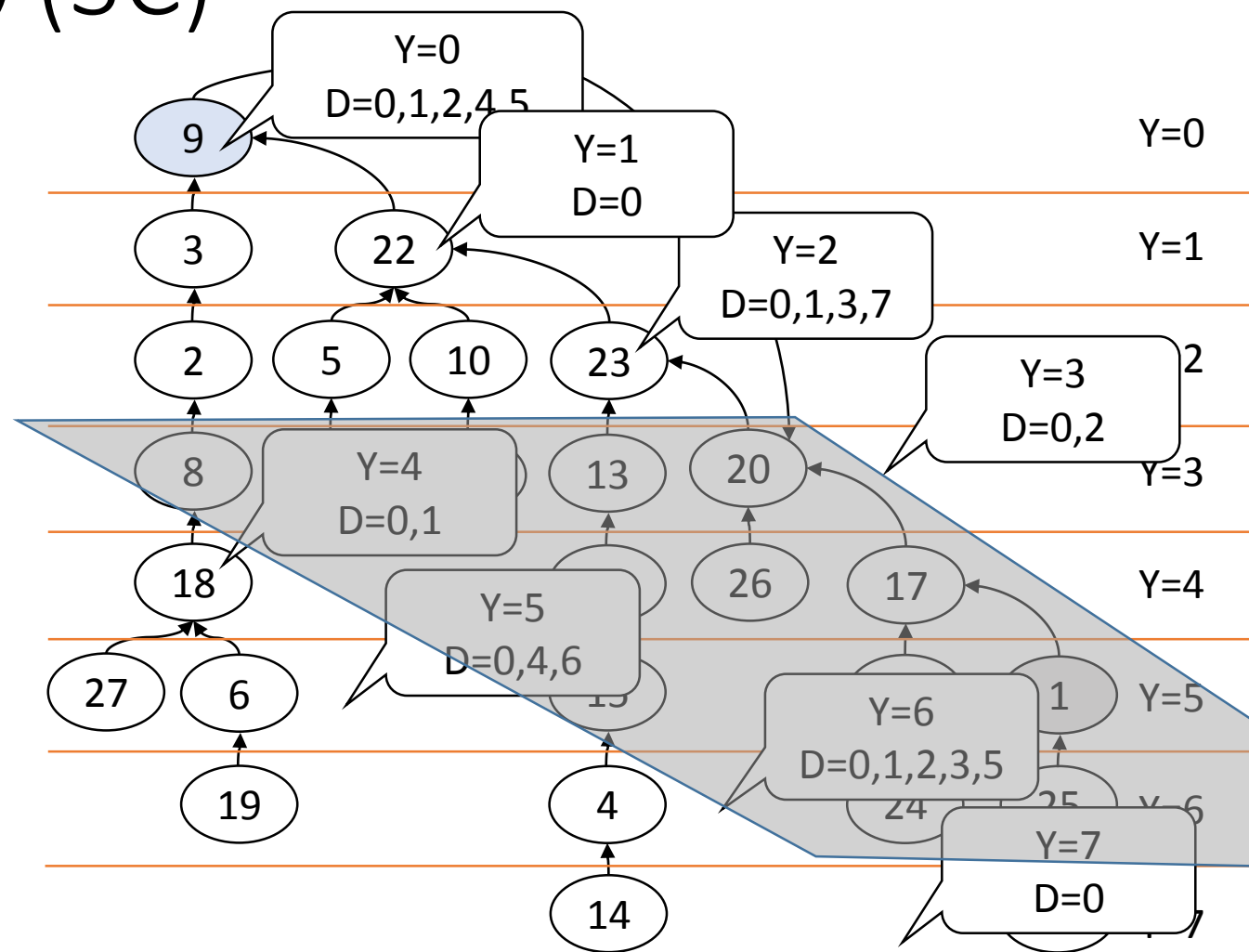
満点解法 – 場合分け(3C)

- 以上をイベントとして管理
- イベントの個数は逆数の和だから、 $O(n \log n)$



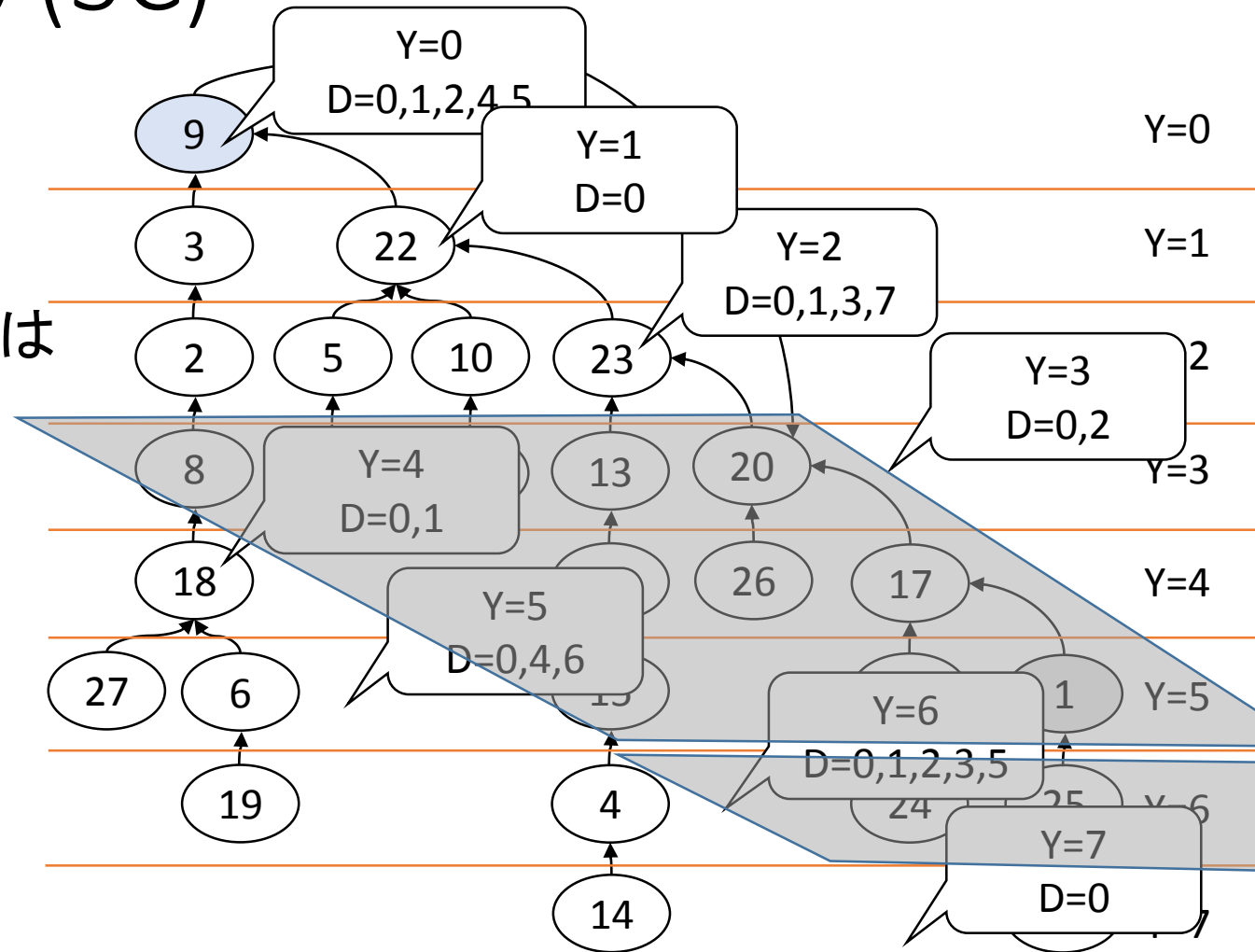
満点解法 – 場合分け(3C)

- D=3のイベントに引がかかる
Bの範囲は図のようになる



満点解法 – 場合分け(3C)

- D=3のイベントに引がかかる
Bの範囲は図のようになる
 - そのうち、Y=2, Y=6のものの範囲は
それぞれ図のようになる



満点解法 – 場合分け(3C)

- あとは(3B)の場合と同様に、木を下から走査する

満点解法 – 場合分け(3C)

- あとは(3B)の場合と同様に、木を下から走査する
 - 木を走査しながら、頂点ごとに子孫の個数を記録したFenwick木を構築
 - 各頂点について、意味のあるイベントを列挙し、それらのイベントについて数え上げを行う
 - 各頂点について、意味のないイベントは一括して数え上げを行う

満点解法 – 場合分け(3C)

- これによって、同様の計算量解析が使える
- 到達先が尾根($Y=X$)にないときの処理の計算量は合わせて $O(n \log n)$ になることが示せる

満点解法 – 場合分け(3C)

- これによって、同様の計算量解析が使える
- 到達先が尾根($Y=X$)にないときの処理の計算量は合わせて $O(n \log n)$ になることが示せる
- 到達先が尾根にあるときの処理 → 一回あたり $O(\log n)$ の処理を合計 $O(n \log n)$ 回行うので、 $O(n \log^2 n)$ までは抑えられる

満点解法

- 以上により時間計算量 $O(n \log^2 n)$ を達成できた

PROJECT OF MIGRATION 解説

城下 慎也

基本事項

- 概要
- この課題について
- アプローチの例
- アプローチを考えるにあたって

概要

- 平面の惑星に民族を割り当てる。
- 一部の民族同士は親交が深く、両者の居住地域を結ぶ線路が敷かれる。
- 線路同士の交差回数をできる限り少なくしたい。
- 可能な限り交差回数が少なくなるものを提出せよ。
- **Output Only の課題である。**

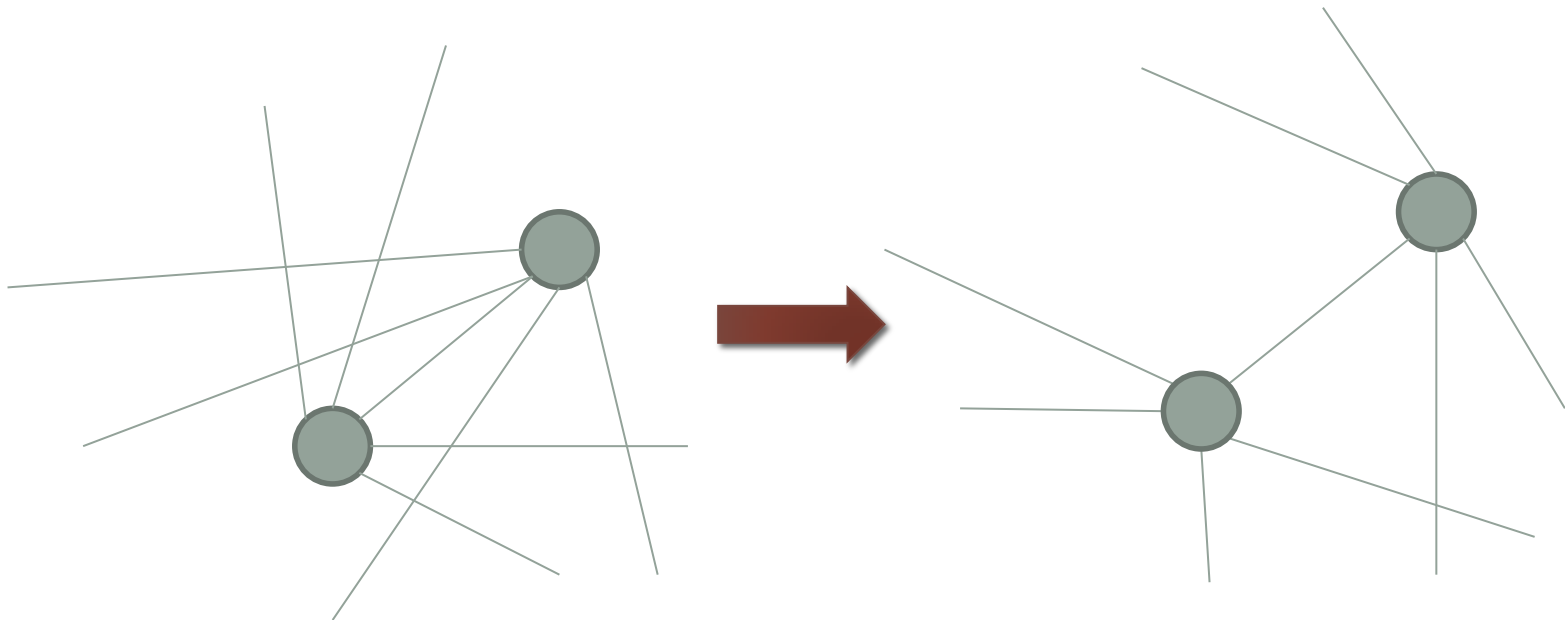
この課題について

- この課題は、出力のみ (Output-Only) の課題です。
- 最適解を効率的に解くことが難しい課題である場合が多く、様々なアプローチをかけて良い解を出す問題となります。
- 最適解を求めるには、逐次改善、ヒューリスティック、近似アルゴリズムなどの手法が考えられます。ただしどのアルゴリズムが良いのかは課題ごとに多種多様となります。
- 入力ごとにソースコードが異なったり(手動で解いても良い)、解を求めるのに費やした時間が異なったりしても構いません。

アプローチの例

局所改善を行う方式

- 例えば、2つの居住地域を取り出し、割り当てを交換することで交差回数が減るかどうかを見ます。
- 山登り法や焼きなまし法で適用できる方法になります。



アプローチを考えるにあたって

- アルゴリズムにもよりますが、試行回数の多さも重要です。
- 先ほどの例でも、知りたいのは交差回数の増減なので、入れ替えを行わない辺同士の交差については見る必要がありません。
- 個々の試行が速くなれば、コンテスト中に行える試行回数を増やすことができ、結果良い解が得られるようになります。

その他事項

- 入力 2 について
- 入力 2 用解法の例
- Visualizer について

入力 2 について

- 入力 2 は、木構造となります。
- 木構造は、平面グラフなので交差がなくなるような埋め込みが可能です。
- すべての平面グラフについて、この問題で交差がなくなるように埋め込めるとは限りませんが(頂点の座標に指定があるため)、今回の木構造については可能です。
- 例えば外周部から凸包の縁をなぞっていく割り当て方で実現可能です。

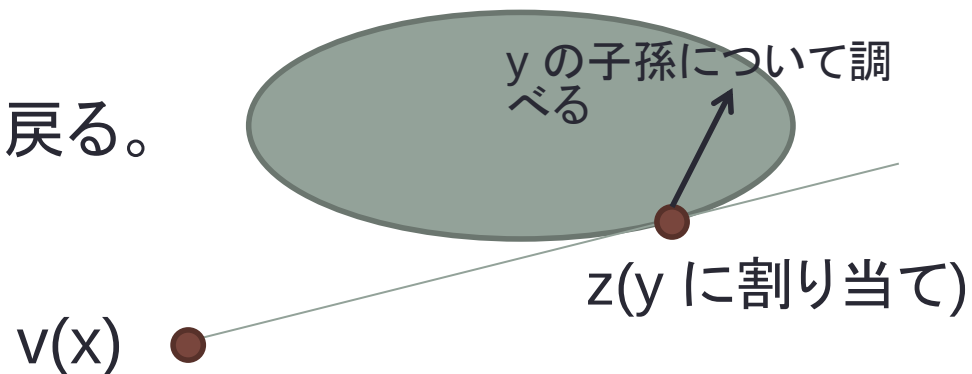
入力 2 用解法の例

- 木構造を深さ優先探索の順序にする。
- 現在ノード v にいて、 v への割り当てが居住地域 x であると決まっている場合に、未探索の v の子ノード y について、

1. どの民族も割り当てられていない居住地域の集合の中で、外周(凸包)を取り、 x を通る、凸包の右側の接線上にある点 z を y の割り当て先にする(x は常に凸包の外側にあるので、 z を一意に定義することができる)。

2. y の子孫について、このアルゴリズムを再帰的に実行することで割り当て先を決定する。

3. y を探索済みノードとして戻る。



Visualizer について

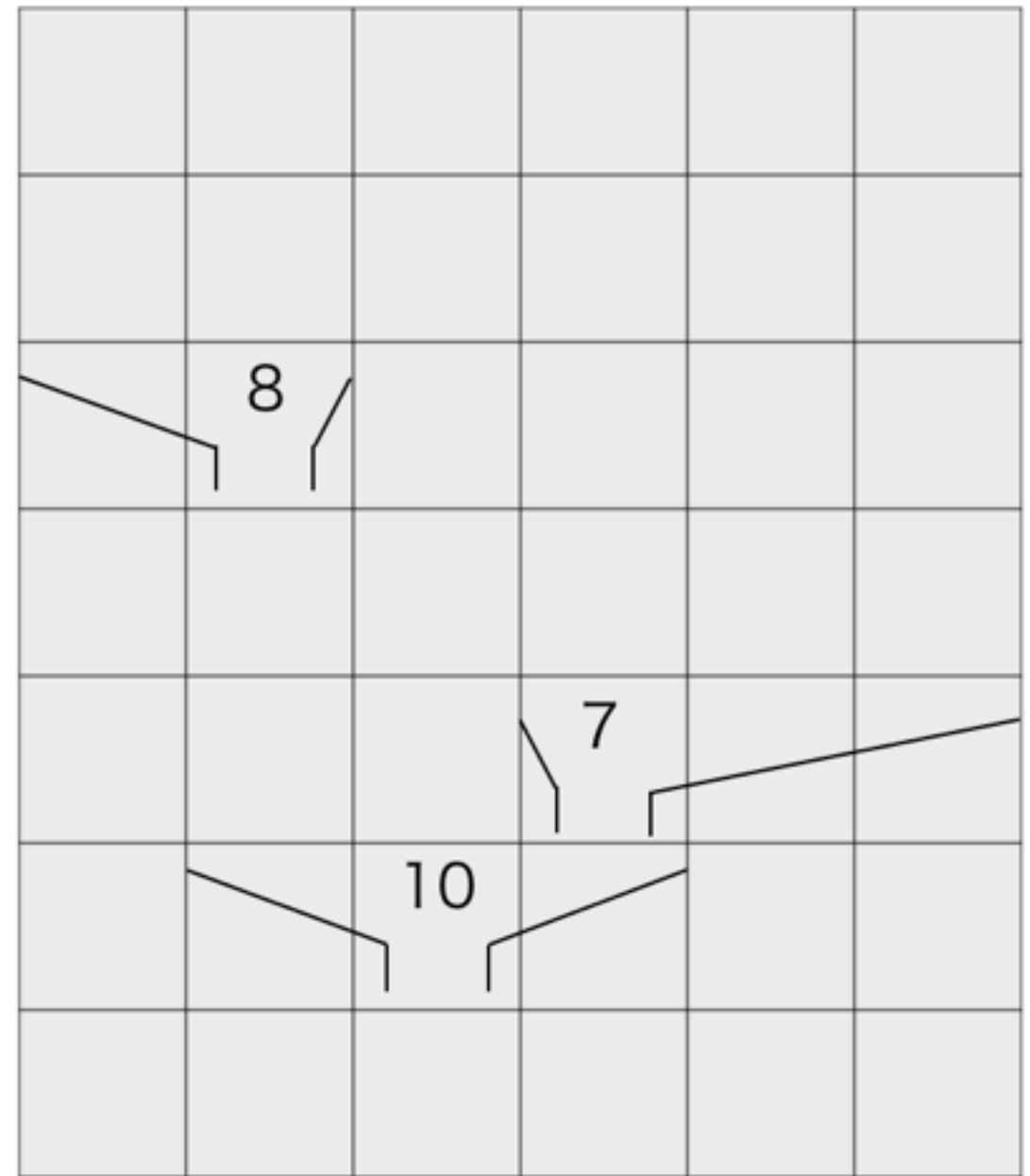
- 入出力を可視化するためのプログラムおよび可視化したものを指します。
- 例としては、入力と出力を受け取り、Canvas(html) コードを提出するプログラムを実装します。
- 可視化することで、出力をチェックでき、改善すべき指針を立てたり、手動改善のお供になったりします。
- Visualizer の実装は必須ではないので、コンテストの残り時間と配分に応じて実装するかどうかを決めてください。

Pinball 解説

平野湧一郎 (nai)

問題概要

- ・ アリスがピンボールをする
- ・ ボールが降ってくるところを1ヶ所にしたい



考えること

- ・ 総当たりはちょっと無理（一応小課題1は解ける）
- ・ DP?

部分点解(1)

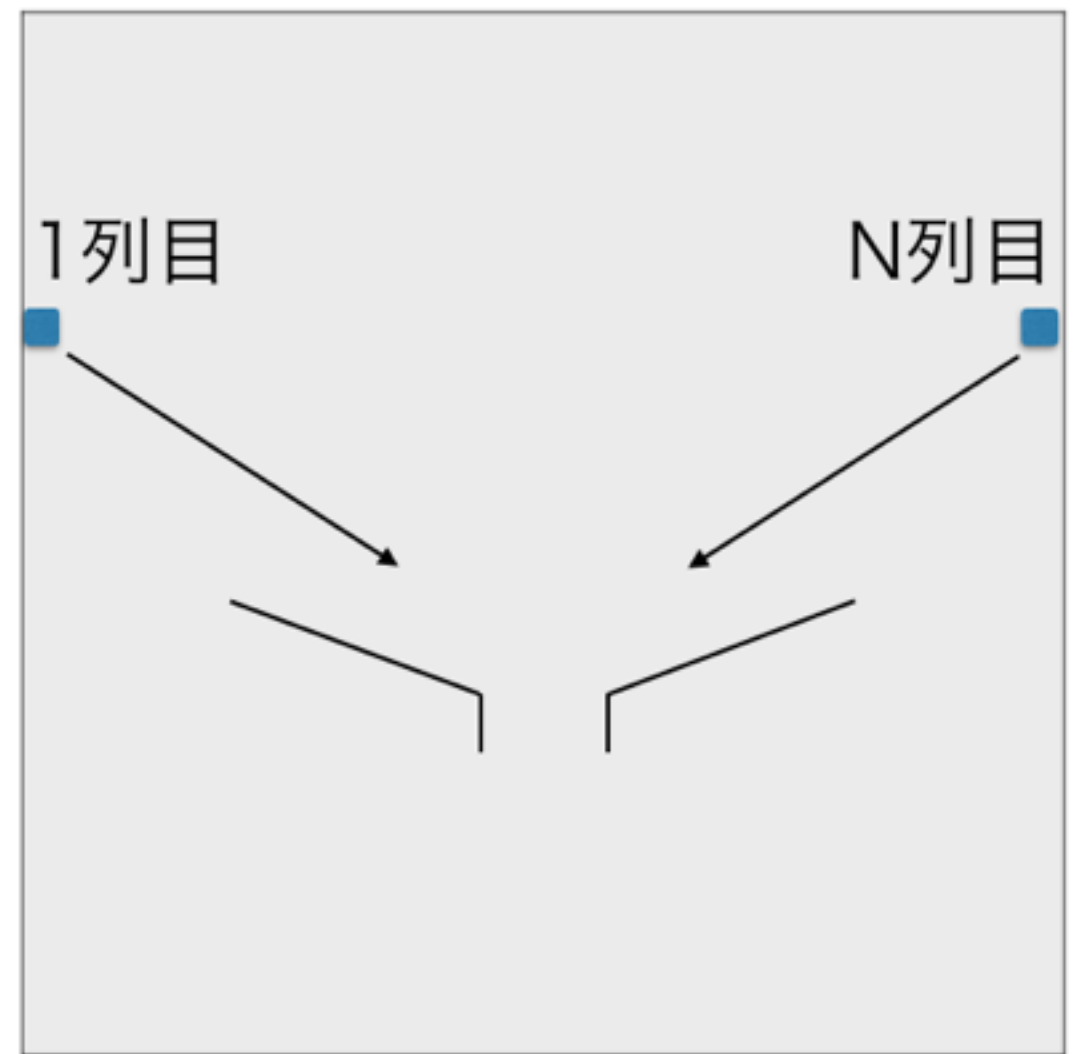
- ・ まず座標圧縮
- ・ $dp[i][s][t] :=$ 「 i 行目で, 列 s から t までに落ちたボールを1ヶ所に集めるためのコストのmin」
- ・ これを下の行から順に求める
- ・ これで $O(M^3)$, 29点

重要な考察

- ・ $A_i :=$ 「列 i に現れたボールが最終的に落ちる列」
- ・ A_i は広義単調増加
 - ・ $i < j \Rightarrow A_i \leq A_j$
- ・ よって、どのボールも同じ列に落ちるための条件は,
 $A_1 == A_N$
- ・ 1列目と N 列目だけ考えればよくなった！

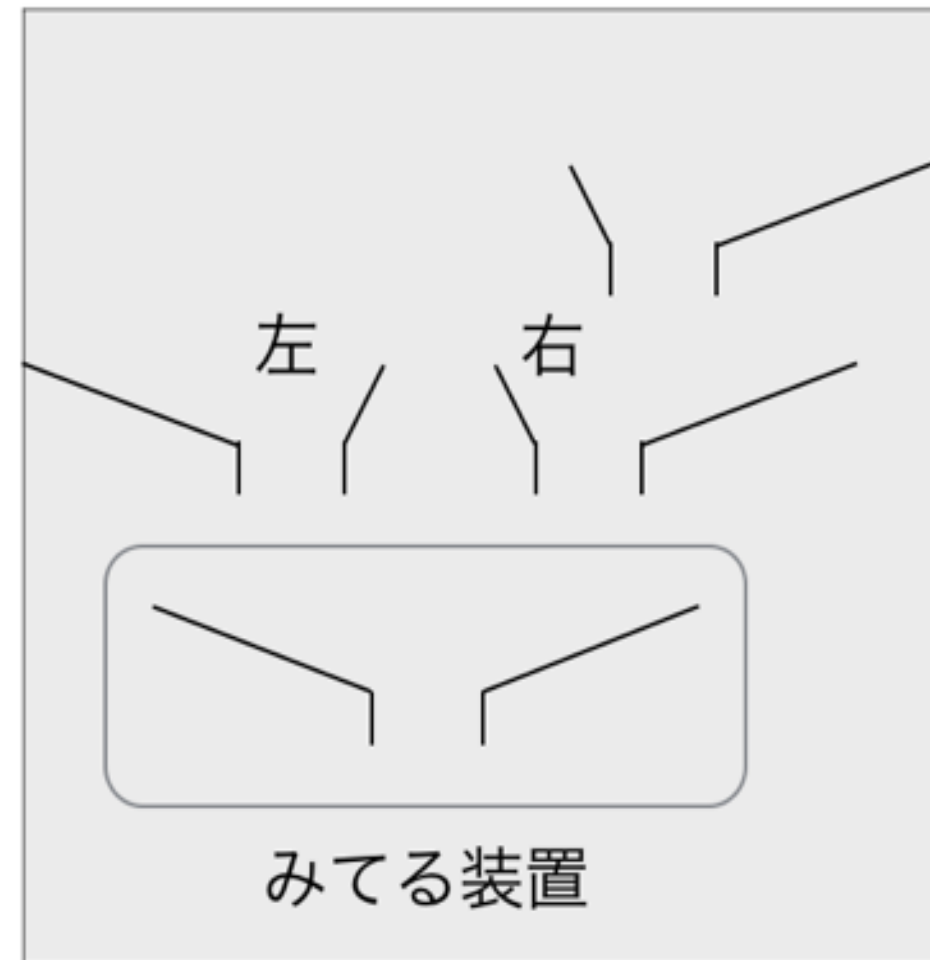
重要な考察(2)

- ・ ある装置Xを使って全てのボールを1箇所に集めるには,
- ・ 1列目のボールがXに落ちてくる(左条件)
- ・ N列目のボールがXに落ちてくる(右条件)
- ・ ことが必要かつ十分！



部分点解(2)

- ・ 装置Xについて,
 - ・ 1列目のボールをXに集めるためのコストのmin (L_x とする)
 - ・ N列目のボールをXに集めるためのコストのmin (R_x とする)
- ・ を上から順に求める. (コストにはX自身のコスト D_x も含む)



部分点解(2)

- ・ $L_x + R_x - D_x$ が、 X を用いて全てのボールを1ヶ所に集めるためのコストのmin
- ・ この最小値が答え
- ・ これで $O(M^2)$, 51点

さいごのステップ

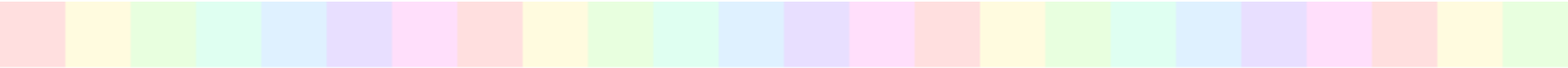
- ・ 問題点：遷移に時間がかかる（自分より上の装置を全て見ないといけないため）
- ・ 「1列目のボールを*i*列目に集めるコストのmin」が爆速でわかればよい（*N*列目も同様）
- ・ これは座標圧縮してから**RMQ** (Range Minimum Query) でできる

想定解 (100点)

- ・ まず座標圧縮
- ・ 上の装置から順に, $L_x =$ 「装置Xを用いて, 1列目のボールを C_x 列目に集めるためのコストのmin」 をRMQを用いて更新してゆく (R_x : N列目も同様)
- ・ $L_x + R_x - D_x$ のminが答え

おしまい

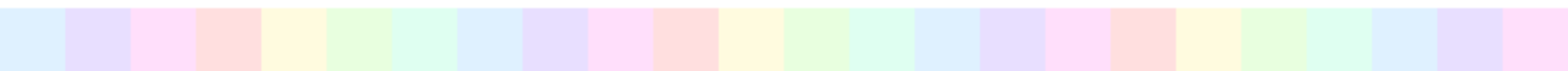
- ・ おつかれさまでした



秘密 (Secret)

JOI Open Contest 2014 Day 2

解説： 保坂 和宏



問題概要

- **Init**: 数列 $A_0, A_1, \dots, A_{N-1}$ が与えられる
- **Query**: $A_L \star A_{L+1} \star \dots \star A_R$ の値は？
- $N \leq 1000$, **Query** は 10000 回以下
- $\star$ は結合的, 関数 **Secret** を呼んで聞く
 - **Init** では 8000 回まで
 - 各 **Query** では 1 回まで

単純な解法 (1) (6 点)

- 各 **Query** に対して **Secret** を $R - L$ 回呼んで計算する

$$(((A_L \star A_{L+1}) \star A_{L+2}) \star \cdots) \star A_R$$

- 最悪で $N - 1$ 回

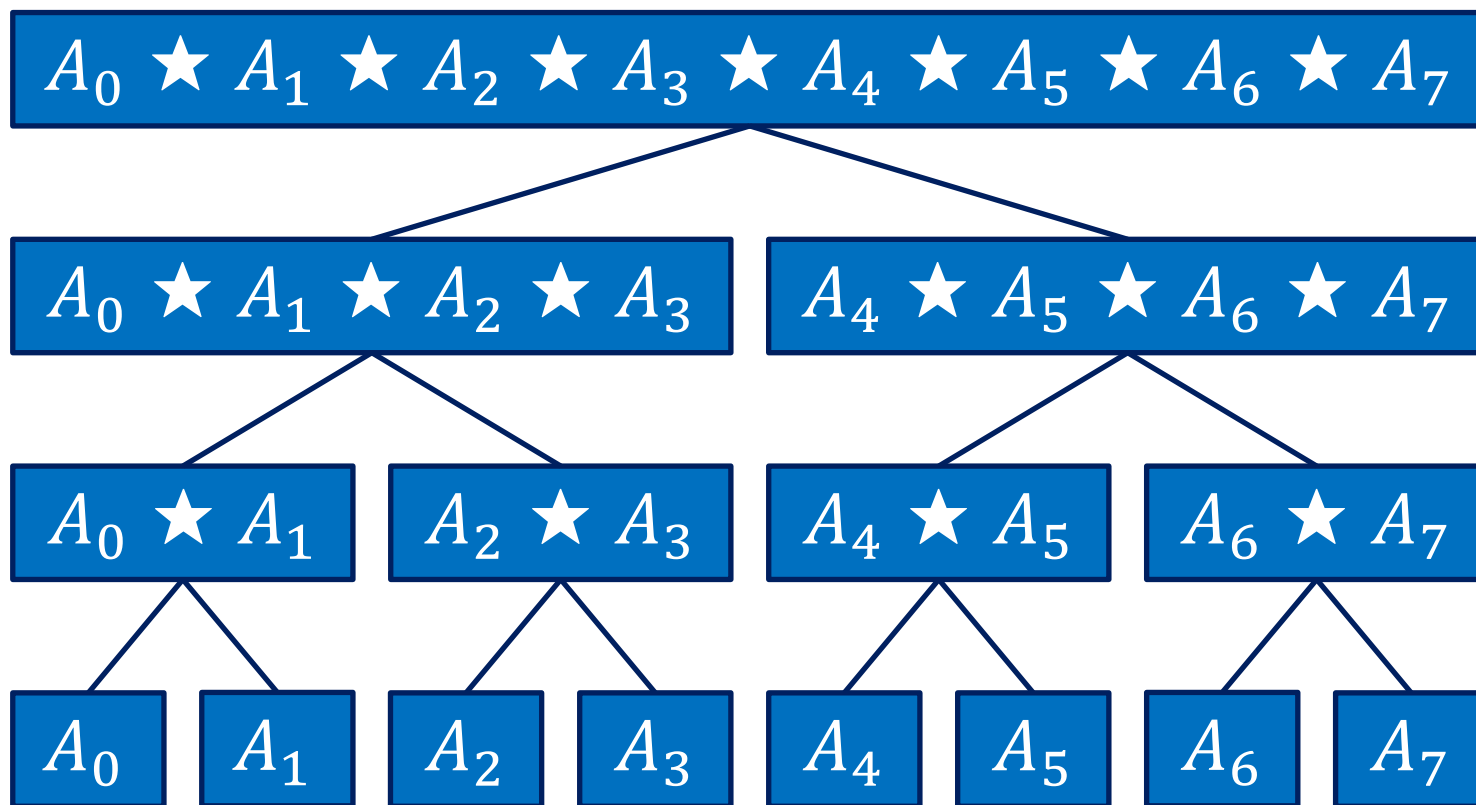
単純な解法 (2) (6 点)

- **Init** ですべての区間 $[L, R]$ に対して答えを求めてしまう
 - $[L, R]$ の答えは $[L, R - 1]$ に対する答えから **Secret** 1 回の呼び出しで求まる
- **Init** で $N(N - 1)/2$ 回
- 各 **Query** で 0 回

改善のためには？

- うまく前計算する
 - **Init** でいくつかの区間に対して答えを求めておく
 - **Query** では、与えられた区間を、答えを求めておいた区間いくつかに分割する

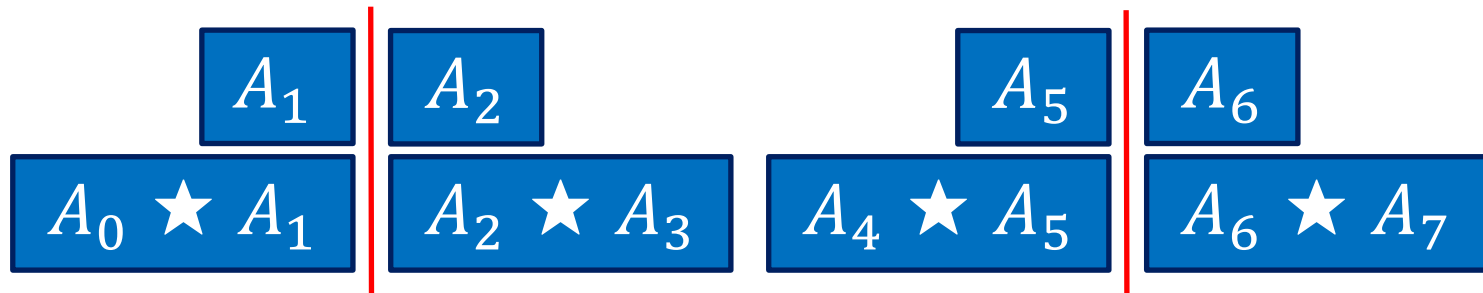
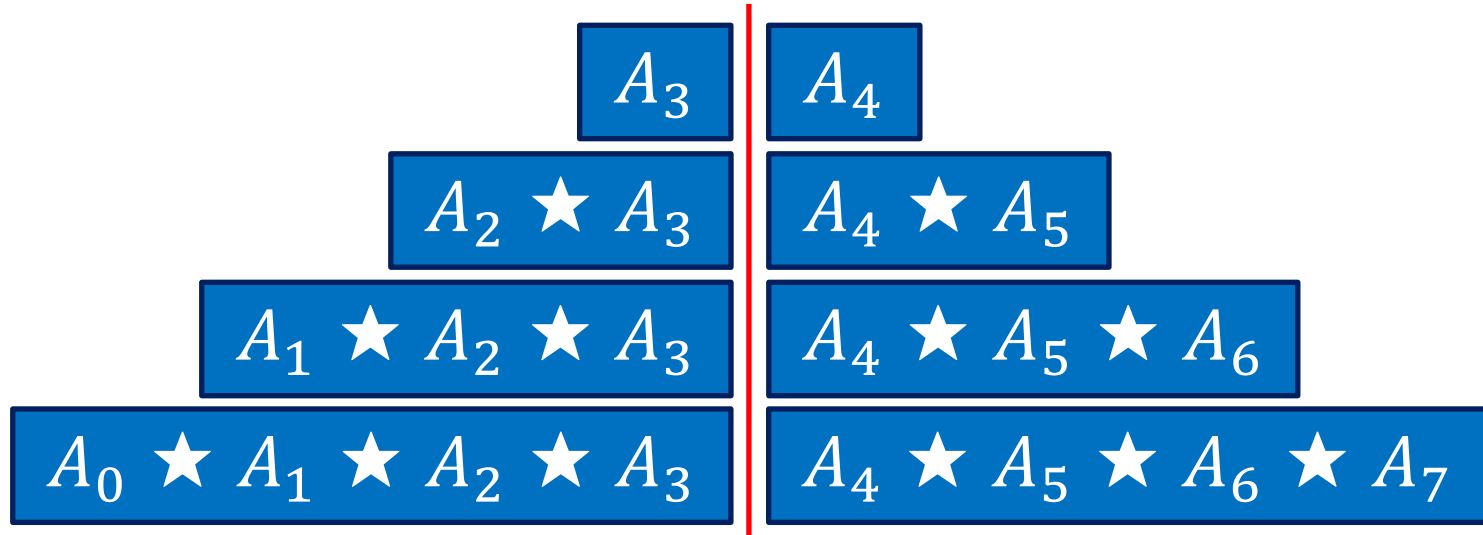
Segment tree (30 点)



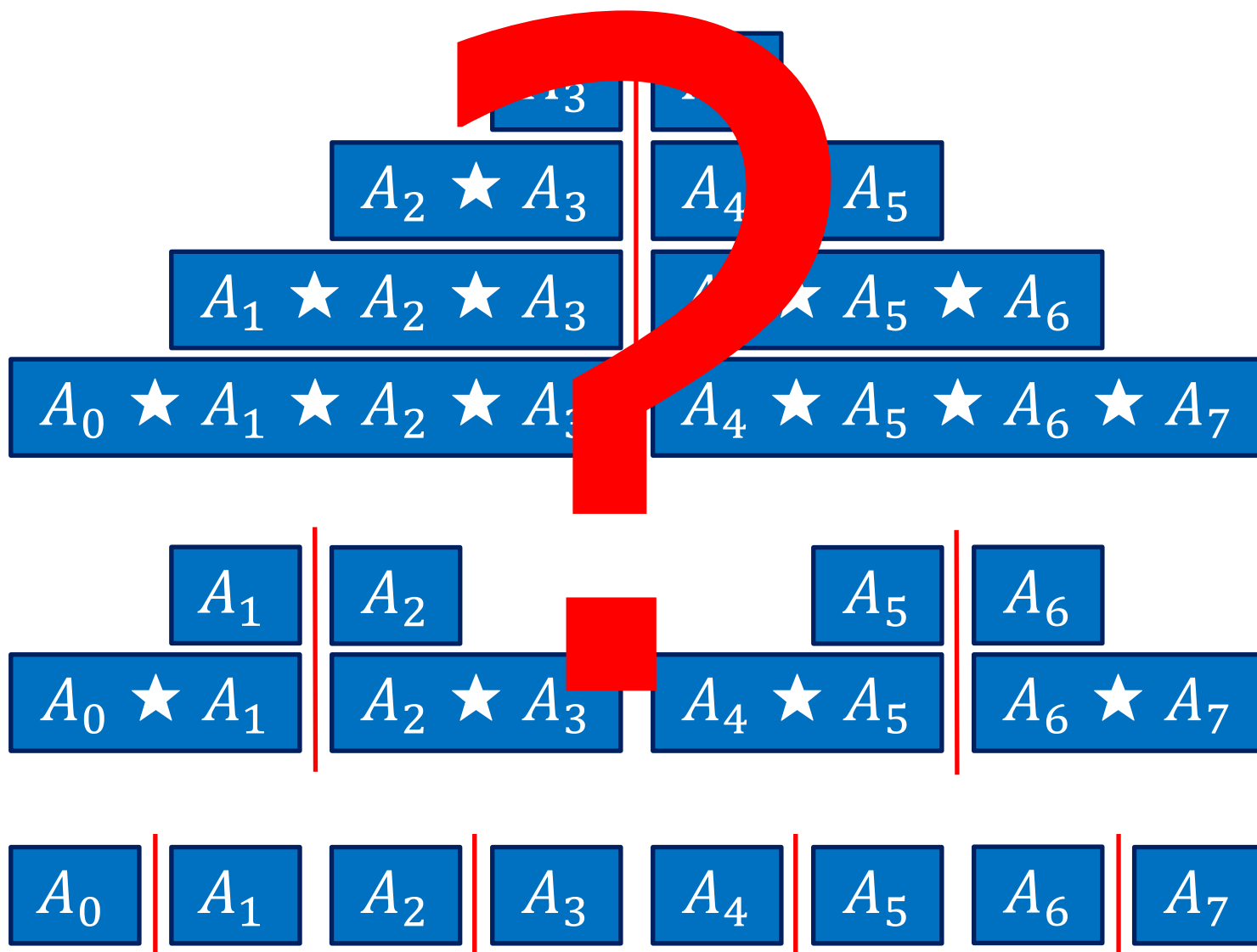
Segment tree (30 点)

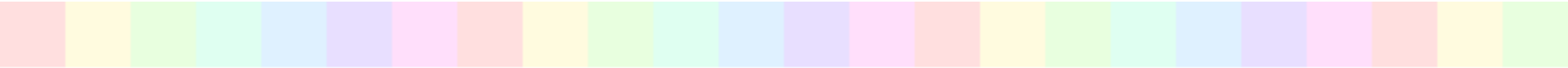
- k 段目は幅 2^{k-1} ごとに分割
 - N が 2 の冪でないときは, 上から再帰的に半々にしていてもよい
- **Init** で $N - 1$ 回
- **Query** の区間は, $2 \times (\text{段数})$ 個以下の区間たちに分割できる
 - $1000 < 2^{10}$ より 10 段以下 $\rightarrow$ 20 回以下

満点解法



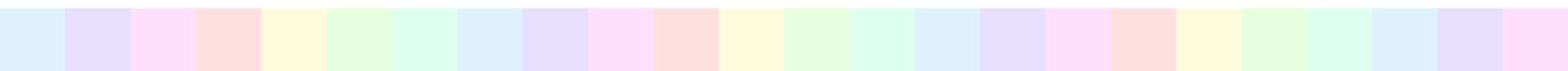
満点解法





分割統治法

–Divide and Conquer–



分割統治法

function 前処理(l, r)

if $l + 1 = r$ **then return**

$m := \lfloor (l + r) / 2 \rfloor$

前処理(l, m)

前処理(m, r)

$A_{m-1}, A_{m-2} \star A_{m-1}, \dots, A_l \star \dots \star A_{m-2} \star A_{m-1}$
を順に求める

$A_m, A_m \star A_{m+1}, \dots, A_m \star A_{m+1} \star \dots \star A_{r-1}$
を順に求める

分割統治法

区間 $[l, r - 1]$ について前処理

function 前処理(l, r)

if $l + 1 = r$ **then return**

$m := \lfloor (l + r) / 2 \rfloor$

前処理(l, m)

前処理(m, r)

m より左について
再帰的に前処理

m より右について
再帰的に前処理

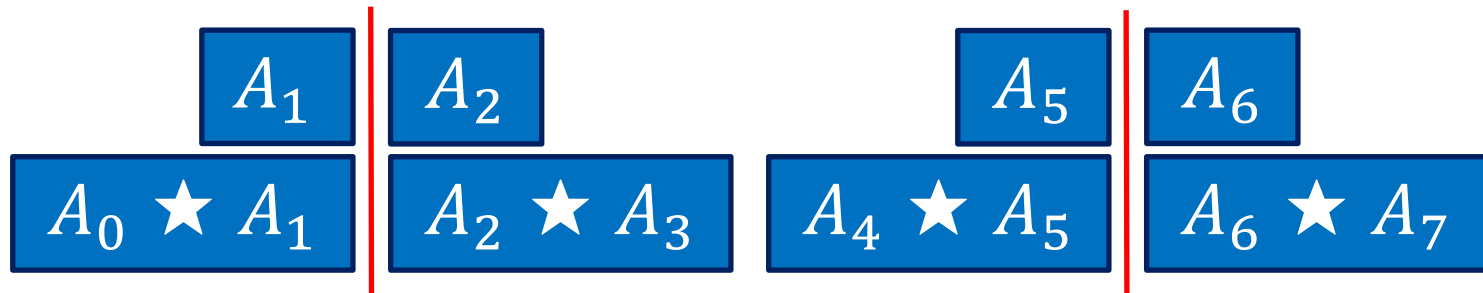
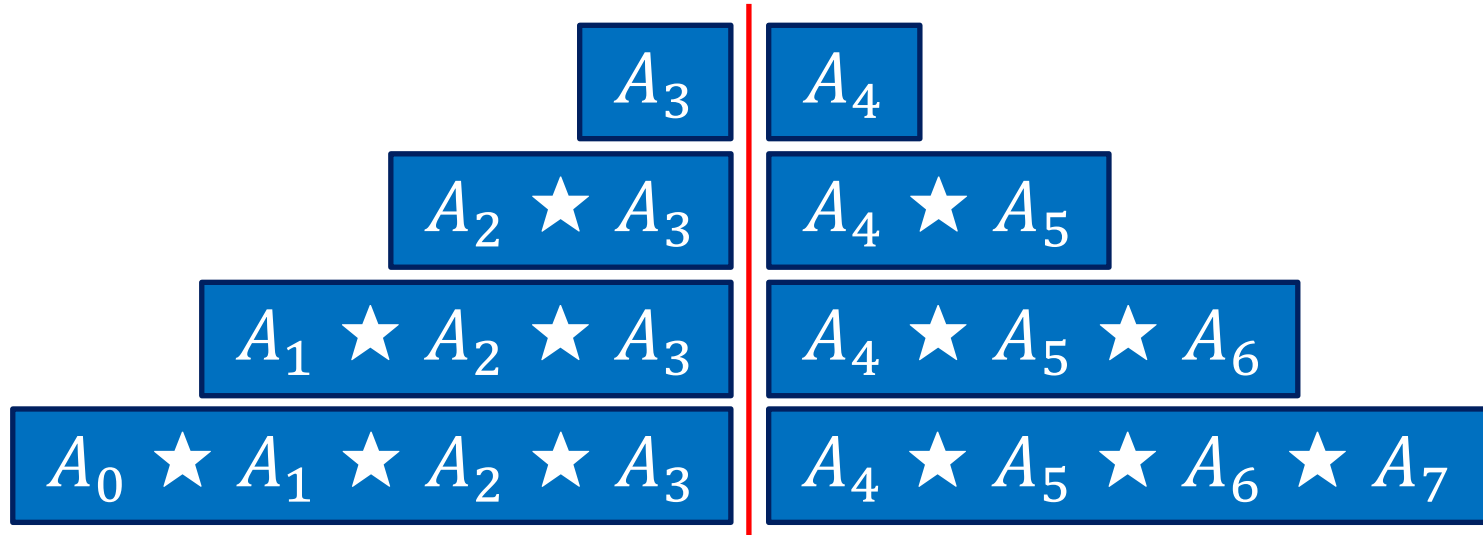
$A_{m-1}, A_{m-2} \star A_{m-1}, \dots, A_l \star \dots \star A_{m-2} \star A_{m-1}$
を順に求める

$A_m, A_m \star A_{m+1}, \dots, A_m \star A_{m+1} \star \dots \star A_{r-1}$
を順に求める

m でぶった切ったときの右側

m でぶった切ったときの左側

満点解法



満点解法

- **Init** で $O(N \log N)$ 回
 - 具体的に回数を数えると 8000 回で足りる
 - 1 段目は 0 回
 - 2 段目は 500 回
 - 3 段目は 500 回でできる (2 段目と同じものを計算しない)
 - 4 段目から 10 段目はそれぞれ 1000 回未満でできる
 - 再帰的に半々にすると 7000 回程度
- どの区間も 2 個以下の区間に分割できる